

Стюарт Рассел, Питер Норвиг

Искусственный интеллект: современный подход, 2-е изд.

Часть III. Знания и рассуждения

Содержание

Глава 7. Логические агенты	1
Глава 8. Логика первого порядка	63
Глава 9. Логический вывод в логике первого порядка	103
Глава 10. Представление знаний	167

Глава 7. Логические агенты

7 Логические агенты В данной главе описано, как проектировать агентов, которые обладают способностью формировать представления о мире, используют процесс логического вывода для получения новых представлений о мире, а также применяют эти новые представления для определения того, что следует делать.

В этой главе приведено вводное описание агентов, действующих на основе знаний (или просто агентов на основе знаний). Рассматриваемые здесь понятия (представление знаний и процессы рассуждения, которые связывают знания с действительностью) являются центральными во всей сфере искусственного интеллекта.

Вполне очевидно, что люди многое знают и обладают способностью рассуждать.

Кроме того, знания и рассуждения очень важны для искусственных агентов, поскольку обеспечивают формирование успешных способов поведения, которых было бы очень трудно добиться иным образом. В этой главе будет показано, что знания о результатах действий позволяют агентам, решающим задачи, успешно действовать в сложных вариантах среды. В отличие от них рефлексные агенты были способны найти путь от Арада до Бухареста только благодаря слепой удаче. Однако знания агентов, решающих задачи, являются очень специфичными и недостаточно гибкими. Шахматная программа способна рассчитать допустимые ходы для короля того цвета, за который она играет, но не обладает многими другими полезными сведениями, например, о том, что ни одна фигура не может стоять одновременно на двух разных клетках. Агенты, основанные на знаниях, способны воспользоваться знаниями, выраженными в очень общих формах, комбинируя и рекомбинируя информацию в соответствии с бесчисленным множеством внешних условий. Часто этот процесс может быть весьма далеким от потребностей текущего момента; это можно сравнить с тем, как математик доказывает абстрактную теорему или астроном вычисляет ожидаемую продолжительность существования Земли.

Кроме того, знания и рассуждения играют решающую роль, когда приходится действовать в частично наблюдаемых вариантах среды. Агент, основанный на знаниях, способен сочетать общие знания с результатами текущих восприятий, чтобы иметь возможность выявить скрытые аспекты текущего состояния, прежде чем выбирать действия. Например, терапевт диагностирует пациента (т.е. выявляет болезненное состояние, которое недоступно непосредственному наблюдению), прежде чем выбрать способ лечения. Некоторая часть знаний, используемых терапевтом,

находится в форме правил, полученных с помощью учебников и учителей, а еще одна часть представлена в форме ассоциативных образов, которые терапевт не всегда может описать словами. Но если эти ассоциации складываются в уме терапевта, они также относятся к области знаний.

Для понимания естественного языка требуется также выявление скрытых аспектов состояния, в частности намерений говорящего. Услышав фразу: "Джон увидел алмаз через окно и страстно пожелал его получить", мы знаем, что слово "его" относится к алмазу, а не к окну; мы рассуждаем, возможно даже подсознательно, с помощью имеющихся у нас знаний об относительной ценности этих предметов.

Аналогичным образом, услышав фразу: "Джон бросил кирпич в окно и разбил его", мы понимаем, что слово "его" относится к окну. Рассуждения позволяют нам справиться практически с бесконечным количеством форм выражения мысли, используя конечный запас обыденных знаний. Сталкиваясь с неоднозначностью подобного рода, агенты, решающие задачи, испытывают затруднения, поскольку применяемый в них способ представления задач с непредвиденными ситуациями обуславливает экспоненциальный рост количества рассматриваемых вариантов.

Не менее важная причина, по которой следует заниматься изучением агентов, основанных на знаниях, состоит в том, что такие агенты характеризуются значительной гибкостью. Они способны принимать к исполнению новые задачи, выраженные в форме явно поставленных целей, они могут быстро достигать компетентности, получая инструкции или усваивая новые знания, полученные из своей среды, кроме того, они способны приспосабливаться к изменениям в своей среде, обновляя соответствующие знания.

Изложение темы, рассматриваемой в данной главе, начинается с описания общего проекта агента, приведенного в разделе 7.1. В разделе 7.2 представлена новая простая среда (мир вымышленных существ — вампусов) и показано, как функционирует агент, основанный на знаниях, без углубления в какие-либо технические подробности. После этого в разделе 7.3 представлены основные принципы логики.

Логика будет служить основным средством представления знаний во всей части III данной книги. Знания логических агентов всегда являются определенными — каждое высказывание в этом мире является либо истинным, либо ложным, хотя агент может не знать о существовании некоторых высказываний.

С точки зрения изложения учебного материала логика обладает тем преимуществом, что служит простым примером одного из способов представления для агентов, основанных на знаниях, но логика имеет некоторые серьезные ограничения.

Очевидно, что значительная часть рассуждений, осуществляемых людьми и другими агентами в частично наблюдаемых вариантах среды, основана на оперировании знаниями, которые являются неопределенными. Логика не позволяет должным образом представить такую неопределенность, поэтому в части V рассматривается теория вероятностей, которая дает такую возможность. В частях VI и VII рассматриваются многие другие способы представления (в том числе основанные на континуальной математике), такие как сочетания гауссовых распределений, нейронных сетей и других представлений.

В разделе 7.4 этой главы представлена простая логика, называемая пропозициональной логикой. Несмотря на то что она является гораздо менее выразительной по сравнению с тем, что требуется для иллюстрации всех основных понятий логики. Кроме того, существует

хорошо разработанная технология формирования рассуждений в пропозициональной логике, которая будет описана в разделах 7.5 и 7.6. Наконец, в разделе 7.7 рассматривается сочетание понятия логических агентов с технологией пропозициональной логики, применяемое для создания некоторых простых агентов, действующих в мире вампуса. В этом разделе указаны некоторые ограничения пропозициональной логики, которые послужили стимулом к разработке более мощных версий логики, рассматриваемых в следующих главах.

7.1. Агенты, основанные на знаниях Центральным компонентом любого агента, основанного на знаниях, является его $\wedge$ база знаний, или сокращенно KB (Knowledge Base). Неформально базу знаний можно определить как множество $\wedge$ высказываний. (Здесь слово "высказывание" используется в качестве формального термина. Смысл этого термина близок, но не идентичен понятию высказывания в английском и других естественных языках.) Каждое высказывание выражено на языке, называемом $\wedge$ языком представления знаний, и представляет некоторое утверждение о мире.

Должен существовать определенный способ добавления новых высказываний к базе знаний, а также способ извлечения из этой базы тех знаний, которые в ней содержатся. Стандартными названиями для этих операций являются соответственно

Tell и Ask. Обе такие операции могут быть связаны с проведением $\wedge$ логического вывода, т.е. могут потребовать получения новых высказываний из старых. В $\wedge$ логических агентах, которые служат основной темой исследования данной главы, логический вывод должен подчиняться тому фундаментальному требованию, что ответ на запрос к базе знаний, переданный с помощью операции Ask, должен следовать из того, что было сообщено базе знаний (или точнее, введено с помощью операции Tell) до сих пор.

Ниже будет дано более точное определение важного понятия "логического следствия". А на данный момент будем считать, что в соответствии с этим понятием в процессе логического вывода не должны выполняться операции, не подчиняющиеся строгим правилам.

Общая схема программы агента, основанного на знаниях, приведена в листинге 7.1.

Как и все агенты, описанные в данной книге, этот агент принимает на входе результаты акта восприятия *percept* и возвращает действие *action*. Агент поддерживает базу знаний, KB, которая может первоначально содержать некоторые $\wedge$ фоновые знания.

После каждого вызова программа агента выполняет три этапа. Во-первых, программа вводит в базу знаний с помощью операции Tell результаты акта восприятия, во-вторых, передает в базу знаний с помощью операции Ask запрос о том, какое действие следует предпринять. В процессе поиска ответа на этот запрос могут быть проведены исчерпывающие рассуждения в отношении текущего состояния мира, результатов возможных последовательностей действий и т.д. В-третьих, агент регистрирует свой выбор с помощью операции Tell и выполняет действие. Вторая операция Tell необходима для передачи в базу знаний информации о том, что гипотетическое действие *action* действительно было выполнено.

Подробные сведения о языке представления скрыты в трех функциях, которые реализуют интерфейс между датчиками и исполнительными механизмами, а также между основным представлением и системой формирования рассуждений. Функция *Make-Percept-Sentence* формирует высказывание, подтверждающее, что агент получил результаты данного конкретного акта в текущий момент времени. Функция *Make-Action-Query* формирует высказывание, представляющее собой запрос о том, какое действие должно быть выполнено в текущий момент времени. Наконец, функция *Make-Action-Sentence* формирует высказывание, подтверждающее, что выбранное действие было выполнено. Подробные сведения о механизме логического вывода скрыты внутри функций Tell и Ask. Эти подробные сведения будут представлены в следующих разделах.

Листинг 7.1. Универсальный агент, основанный на знаниях function KB-Agent(percept) returns действие action static: KB, база знаний t, счетчик, обозначающий время, первоначально равный 0 Tell(KB, Make-Percept-Sentence(percept, t)) action <— Ask(KB, Make-Action-Query(t)) Tell(KB, Make-Action-Sentence(action, t)) t <- t + 1 return action Алгоритм агента, приведенный в листинге 7.1, внешне кажется весьма аналогичным алгоритмам агентов с поддержкой внутреннего состояния, описанных в главе 2.

Однако в силу приведенных выше определений операций Tell и Ask агента, основанного на знаниях, уже нельзя рассматривать как произвольную программу для вычисления действий. Он уже больше подходит для описания на Λ уровне знаний, в котором для фиксации его поведения требуется указать лишь то, что известно агенту и каковы его цели. Например, автоматизированный агент — водитель такси может иметь своей целью доставку пассажира в район Марин Каунти и знать, что этот район находится в г. Сан-Франциско и что мост "Золотые ворота" является единственным возможным связующим звеном между пунктом отправления и пунктом назначения. В таком случае вполне можно рассчитывать на то, что автоматизированный водитель поедет через мост "Золотые ворота", поскольку ему известно, что именно так он достигнет цели. Следует отметить, что данный анализ не зависит от того, как будет действовать водитель такси на Λ уровне реализации. Не имеет никакого значения, реализованы ли его знания географии в виде связанных списков или растровых изображений карт, а также проводит ли он рассуждения, манипулируя строками символов, хранящимися в регистрах, или распространяя зашумленные сигналы по сетям из нейронов.

Как уже упоминалось во введении к этой главе, Λ -агент, основанный на знаниях, может быть создан путем передачи ему с помощью операции Tell той информации, которую ему требуется знать. Первоначальная программа агента, применяемая до того, как агент начнет получать результаты актов восприятия, создается путем добавления одного за другим тех высказываний, которые представляют знания проектировщика о данной среде. А если удастся спроектировать язык представления, позволяющий легко выражать эти знания в форме высказываний, то задача создания агента чрезвычайно упрощается. Описанный выше подход называется Λ -декларативным

подходом к созданию системы. В отличие от этого процедурный подход требует представления желаемых правил поведения непосредственно в виде кода программы.

Но сведение к минимуму роли явных форм представления и формирования рассуждений позволяет добиться создания гораздо более эффективных систем. Агенты обоих этих типов рассматриваются в разделе 7.7. В 1970-1980-х годах приверженцы этих двух подходов вступили в ожесточенные дебаты. Однако теперь

стало ясно, что успешно действующий агент должен сочетать в своем проекте и декларативные, и процедурные элементы.

Кроме передачи основанному на знаниях агенту с помощью операции Tell той информации, которую ему требуется знать, может также потребоваться предоставить ему механизмы, позволяющие обучаться самостоятельно. Эти механизмы, которые рассматриваются в главе 18, обеспечивают получение общих знаний о среде с помощью ряда актов восприятия. Такие знания могут быть включены в базу знаний агента и использоваться для принятия решений. Благодаря этому агент может стать полностью автономным.

Все эти возможности по представлению, формированию рассуждению и обучению опираются на результаты в области теории и практики логики как отрасли математики, накопленные в течение многих столетий. Но прежде чем перейти к описанию теории и практики, создадим простой мир, на примере которого сможем проиллюстрировать их применение.

7.2. Мир вампуса ^ Мир вампуса — это пещера, состоящая из залов, соединенных проходами. Где-то по этой пещере бродит вампос — страшный зверь, который поедает всех, кто входит в зал, где он находится. Вампос может быть убит агентом, но у агента есть только одна стрела. В некоторых залах есть бездонные ямы, в которые попадают все, кто проходит через эти залы (все кроме вампоса, который слишком велик для того, чтобы в них провалиться). Единственное, что утешает живущих в этой среде, — это возможность найти кучу золота. Хотя мир вампоса является довольно скромным с точки зрения современных стандартов компьютерных игр, он представляет собой превосходную испытательную среду для интеллектуальных агентов. Первым, кто предложил использовать мир вампоса для такой цели, был Майкл Генезерет.

Пример мира вампоса показан на рис. 7.1. Точное определение среды этой задачи, соответствующее требованиям, которые приведены в главе 2, дано с помощью следующего описания PEAS. • Показатели производительности. За то, что агент находит золото, он получает +1000 очков, за то, что агент попадает в яму или его съедает вампос, ему назначается -1000 очков, он отдает -1 очко за каждое выполненное действие и -10 очков — за использование стрелы. • Среда. Залы расположены в виде решетки 4x4. Агент всегда начинает движение с квадрата, обозначенного как [1,1], и смотрит вправо.

Местонахождения золота и вампоса выбираются случайным образом с равномерным распределением из числа квадратов, отличных от начального квадрата. Кроме того, в

каждом квадрате, отличном от начального, с вероятностью 0,2 может находиться яма.

12 3 4 Рис. 7.1. Пример мира вампуса. Агент находится в нижнем левом углу •

Исполнительные механизмы. Агент может двигаться вперед, поворачиваться влево и вправо на 90° . Агент погибает жалкой смертью, если входит в квадрат, где имеется яма или живой вампус. (Входить в квадрат с мертвым вампусом безопасно, но при этом приходится чувствовать неприятный запах.) Попытка продвижения вперед остается безрезультатной, если перед агентом находится стена. Чтобы схватить объект, находящийся в том же квадрате, где находится агент, можно воспользоваться действием Grab. С помощью действия Shoot можно пустить стрелу по прямой линии в том направлении, куда смотрит агент. Стрела продолжает движение до тех пор, пока она либо не попадет в вампуса (и убьет его), либо не ударится в стену. У агента имеется только одна стрела, поэтому какой-либо эффект оказывает лишь первое действие Shoot.

• Датчики. У агента имеется пять датчиков, каждый из которых сообщает только один элемент информации:

- в квадрате, где находится вампус, а также в квадратах, непосредственно (а не по диагонали) соседних по отношению к этому квадрату, агент чувствует неприятный запах (восприятие Stench);
- в квадратах, непосредственно соседних с ямой, агент будет чувствовать ветерок (восприятие Breeze);
- в квадрате, где находится золото, агент видит блеск (восприятие Glitter);
- когда агент наталкивается на стену, он чувствует удар (восприятие Bump).

• перед тем как пораженный стрелой вампус умирает, он испускает жалобный крик, который разносится по всей пещере (восприятие Scream).

Результаты актов восприятия передаются агенту в форме списка из пяти символов; например, если есть неприятный запах и ветерок, но нет блеска, удара или крика, агент воспринимает результаты акта восприятия [Stench, Breeze, None, None, None].

В упр. 7.1 предлагается определить среду вампуса в соответствии с различными измерениями, приведенными в главе 2. Основная сложность для данного агента состоит в том, что он с самого начала не знает конфигурацию своей среды; повидимому, для преодоления этого незнания нельзя обойтись без логических рассуждений. В большинстве экземпляров мира вампуса для агента существует возможность безопасно получить золото. Но иногда агенту приходится выбирать: вернуться ли домой с пустыми руками или рисковать жизнью, чтобы найти золото. Около 21% вариантов среды являются совершенно неблагоприятными, поскольку золото находится в яме или окружено ямами.

Проследим за агентом для мира вампуса, действующим на основе знаний, который изучает среду, показанную на рис. 7.1. Первоначальная база знаний агента содержит правила существования в этой среде, которые были описаны выше; в частности, агент знает, что находится в квадрате [1,1] и что квадрат [1,1] является безопасным. Мы увидим, как расширяются знания агента по мере того, как поступают результаты новых актов восприятия и выполняются действия.

Первым восприятием является [None, None, None, None, None], на основании которого агент может сделать вывод, что соседние по отношению к нему квадраты являются безопасными. На рис. 7.2, а показано состояние знаний агента в этот момент.

Мы будем перечислять (некоторые) высказывания из базы знаний с использованием буквенных обозначений, таких как В (чувствуется ветерок) и ОК (безопасно — ни ямы, ни вампуса), проставленных в соответствующих квадратах. С другой стороны, на рис. 7.1 изображен сам этот мир.

1,4 1,3 1,2 ОК 1,1 Ш [ОК 2,4 2,3 2,2 2,1 ОК 3,4 3,3 3,2 3,1 4,4 4,3 4,2 4,1 Ш В G ОК Р S V
W - агент - ветерок - блеск, золото - безопасный квадрат - яма - запах - посещенный
квадрат - вампус [1,4 1,3 1,2 ОК 1,1 V ОК 2,4 2,3 2,2 Р?

21 ш в ОК 3,4 3,3 3,2 1 1 4,4 4,3 4,2 4,1 а) б) Рис. 7.2. Первый шаг, выполненный агентом в мире вампуса: первоначальная ситуация, возникшая после восприятия /"None, None, None, None, None 7 (а); ситуация после одного хода, в котором получено восприятие /"None, Breeze, None, None, None] F) На основании того факта, что в квадрате [1,1] не было ни неприятного запаха, ни ветерка, агент может сделать вывод, что квадраты [1,2] и [2,1] свободны от опасности. Для указания этого они отмечены обозначением ОК. Осторожный агент переходит только в такой квадрат, о котором известно, что в нем есть отметка ОК.

Предположим, что агент решил двинуться вперед, в квадрат [2,1], и была создана сцена, показанная на рис. 7.2, б.

Агент обнаруживает ветерок в квадрате [2,1], поэтому в одном из соседних квадратов должна быть яма. По правилам игры яма не может находиться в квадрате [1,1], поэтому она должна быть в квадрате [2,2], или [3,1], или в том и другом.

Обозначение Р? на рис. 7.2, б указывает на возможность наличия ямы в этих квадратах. В данный момент известен только один квадрат с отметкой ОК, который еще не был посещен. Поэтому благоразумный агент поворачивается кругом и возвращается в квадрат [1,1], а затем переходит в квадрат [1,2].

Новым восприятием в квадрате [1,2] является [Stench, None, None, None, None], что приводит к состоянию знаний, показанному на рис. 7.3, а. Наличие неприятного запаха в квадрате [1,2] означает, что где-то рядом есть вампус.

Но вампус не может находиться в квадрате [1,1] по правилам игры и не может быть в квадрате [2,2] (поскольку агент обнаружил бы неприятный запах, находясь в квадрате [2,1]). Поэтому агент может сделать вывод, что вампус находится в квадрате [1,3]. На это указывает обозначение W!. К тому же из отсутствия восприятия Breeze в квадрате [1,2] следует, что в квадрате [2,2] нет ямы. Тем не менее мы уже пришли к выводу, что яма должна быть в квадрате [2,2] или [3 , 1], а это означает, что она

действительно находится в квадрате [3,1]. Это — весьма сложный логический вывод, поскольку в нем объединяются знания, полученные в разное время в различных местах, и в нем решение о выполнении важного шага сделано на основании отсутствия определенного восприятия. Такой логический вывод превосходит способности большинства животных, но является типичным для рассуждений такого рода, которые выполняются логическими агентами.

И в G OK P S V W - агент - ветерок - блеск, золото - безопасный квадрат - яма - запах - посещенный квадрат - вампус 1,4 1 3 ' W! '2Ш S OK 1,1 V OK 2,4 2,3 2,2 OK 2,1 в V OK 3,4 3,3 3,2 зд P!

4,4 4,3 4,2 4,1 1,4 1,3 W! к V OK 1,1 V OK 2,4 P? 23 ш S G в 2,2 V OK 2,1 в V OK 3,4 3,3 p9 3,2 3,1 P!

4,4 4,3 4,2 4,1 а) б) Рис. 7.3. Два последних этапа в ходе деятельности агента: после третьего хода, когда было получено восприятие /"Stench, None, None, None, None] (а); после пятого хода, когда было получено восприятие /"Stench,Breeze,Glitter,None,None] F) Теперь агент доказал сам себе, что в квадрате [2,2] нет ни ямы, ни вампуса, поэтому может обозначить этот квадрат меткой OK, чтобы в него перейти. Мы не показываем состояние знаний агента в квадрате [2, 2], а просто предполагаем, что агент повернулся и перешел в квадрат [2, 3], в результате чего было получено состояние, показанное на рис. 7.3, б. В квадрате [2,3] агент обнаруживает блеск, поэтому должен схватить золото и тем самым закончить игру.

<Ж* В каждом случае, когда агент выводит заключение из доступной ему информации, гарантируется правильность этого заключения, если доступная информация является правильной. В этом состоит фундаментальное свойство логических рассуждений. В оставшейся части данной главы будет показано, как создавать логических агентов, которые могут представлять необходимую информацию и делать выводы, описанные в предыдущих абзацах.

7.3. Логика В этом разделе приведен краткий обзор всех фундаментальных понятий, касающихся логических представлений и рассуждений. Описание конкретных сведений о какой-либо определенной форме логики откладывается до следующего раздела.

Вместо этого в данном разделе используются неформальные примеры из мира вампуса и из знакомой всем области арифметики. Авторы приняли такой довольно необычный подход, поскольку идеи логики являются гораздо более универсальными и привлекательными, чем обычно предполагается.

В разделе 7.1 было указано, что базы знаний состоят из высказываний. Эти высказывания выражаются в соответствии с $\wedge$ синтаксисом языка представления,

который определяет форму всех высказываний, рассматриваемых как построенные правильно. Понятие синтаксиса является достаточно очевидным в обычной арифметике: " $x+y=4$ " — правильно построенное высказывание, а " $x^2y+=$ " — нет.

Синтаксис логических языков (и арифметики в том числе) обычно формируется так, чтобы с его помощью было удобно писать статьи и книги. Существуют буквально десятки различных форм представления синтаксиса; в некоторых из них используются греческие буквы и сложные математические символы, а в других — довольно привлекательные на вид схемы со стрелками и кружками. Однако во всех случаях высказывания в базе знаний агента представляют собой реальные физические конфигурации (части) агента. Формирование рассуждений сводится к выработке этих конфигураций и манипулированию ими.

Логика должна также определять $\wedge$ семантику языка. Говоря неформально, семантика касается "смысла" высказываний. В логике это определение становится более точным. Семантика языка определяет $\wedge$ истинность каждого высказывания применительно к каждому из $\wedge$ возможных миров. Например, обычная семантика, принятая в арифметике, определяет, что высказывание " $x+y=4$ " истинно в мире, где x равно 2 и y равно 2, но ложно в мире, где x равно 1 и y равно 1. В стандартной логике каждое высказывание должно быть либо истинным, либо ложным в каждом из возможных миров — в ней не может быть² каких-либо "промежуточных состояний".

В дальнейшем, если потребуется уточнить какое-то определение, то вместо выражения один из "возможных миров" будет использоваться термин $\wedge$ модель.

(Кроме того, для указания на то, что высказывание a является истинным в модели t ,¹ Несомненно, читатель заметил аналогию между понятием истинности высказываний и понятием удовлетворения ограничений, приведенным в главе 5. Это не случайно — языки ограничений действительно представляют собой языки логики, а решение ограничений — это одна из форм проведения логических рассуждений.

² В нечеткой логике, которая рассматривается в главе 14, допускается определение степени истинности.

будет использоваться фраза " t является моделью a ".) Возможные миры могут рассматриваться как (потенциально) реальные варианты среды, в которых может находиться или не находиться агент, а модели представляют собой математические абстракции, каждая из которых устанавливает, является ли истинным или ложным каждое высказывание, относящееся к данной модели. Неформально, например, можно рассматривать x и y как количество мужчин и женщин, которые сидят за столом для игры в бридж, а высказывание $x+y=A$ становится истинным, когда общее количество игроков равно четырем. В таком случае формально допустимые модели

представляют собой все возможные присваивания целочисленных значений переменным x и y . Каждое такое присваивание устанавливает истинность любого высказывания в арифметике, переменными которого являются x и y .

Теперь, после рассмотрения понятия истинности, мы можем перейти к теме, касающейся логических рассуждений. Для этого необходимо определить понятие логического $\wedge$ следствия между высказываниями — идею о том, что одно высказывание может логически следовать из другого высказывания. В математических обозначениях применяется следующая запись: $a \models C$ которая означает, что высказывание a влечет за собой высказывание C . Формальное определение логического следствия является таковым: $a \models C$ тогда и только тогда, когда в любой модели, в которой высказывание a является истинным, C также истинно. Другой способ выразить эту мысль состоит в том, что если a является истинным, высказывание C также является истинным. Неформально выражаясь, истинность высказывания C "содержится" в истинности высказывания a . Понятие следствия применяется и в арифметике, например, из выражения $x+y=4$ следует выражение $4=x+y$. Очевидно, что в любой модели, в которой $x+y=4$ (например, в такой модели, в которой x равно 2 и y равно 2), справедливо также, что $4=x+y$.

Вскоре будет показано, что любая база знаний может рассматриваться как некоторое утверждение, и в этой книге будет часто идти речь о том, что из некоторой базы знаний следует некоторое высказывание.

Анализ такого же типа можно применить и к примеру рассуждений о мире вампуса, приведенному в предыдущем разделе. Рассмотрим ситуацию, показанную на рис. 7.2, б: агент ничего не обнаружил в квадрате $[1,1]$, а в квадрате $[2,1]$ почувствовал ветерок. Эти восприятия в сочетании со знаниями агента о правилах существования мира вампуса (описание PEAS на с. 286) составляют его базу знаний.

Агенту необходимо узнать (кроме всего прочего), имеются ли ямы в соседних квадратах, $[1,2]$, $[2,2]$ и $[3,1]$. В каждом из этих трех квадратов может находиться или не находиться яма, поэтому (в данном конкретном примере) существуют $2^3=8$ возможных моделей. Они показаны на рис. 7.4.

В моделях, которые противоречат тому, что знает агент, база знаний становится ложной; например, база знаний ложна в любой модели, в которой квадрат $[1,2]$ имеет яму, поскольку в квадрате $[1,1]$ не чувствуется ветерок. Существуют факты - 3 Хотя на данном рисунке модели показаны как фрагменты мира вампуса, в действительности они представляют собой не что иное, как варианты присваивания значений $true$ и $false$ высказываниям "в квадрате $[1,2]$ имеется яма" и т.д. Для определения модели, в математическом смысле этого понятия, не нужно описывать в ней "страшных, покрытых шерстью" вампусов.

чески только три модели, в которых база знаний является истинной, и они показаны на рис. 7.4 как подмножество моделей. Теперь рассмотрим два возможных вывода:

$OC_1 = \text{"В квадрате [1,2] нет ямы"}$ $OC_2 = \text{"В квадрате [2,2] нет ямы"}$ а) б) Рис. 7.4. Возможные модели, описывающие наличие ям в квадратах [1,2], [2,2] и [3,1], которые основаны на тех наблюдениях, что в квадрате [1,1] нет ничего, а в квадрате [2,1] чувствуется ветерок: модели с базой знаний и высказыванием a_1 (в квадрате [1,2] нет ямы) (а); модели с базой знаний и высказыванием a_2 (в квадрате [2,2] нет ямы) (б) Эти модели на рис. 7.4, а и б обозначены соответственно как a_1 и a_2 .

Проанализировав все возможные ситуации, мы обнаружили следующее: в каждой модели, в которой база знаний является истинной, высказывание a_1 также является истинным.

Поэтому $KB \wedge OC_1$: в квадрате [1,2] нет ямы. Кроме того, можно убедиться в следующем: в некоторых моделях, в которых база знаний является истинной, высказывание OC_2 ложно.

Поэтому $KB \wedge a_2$: агент не может прийти к заключению, что в квадрате [2,2] нет ямы. (К тому же он не может сделать и вывод⁴, что в квадрате [2,2] есть яма.) В предыдущем примере не только проиллюстрирован процесс формирования логических следствий, но и показано, как можно применять определение логического следствия для формирования заключений, т.е. для проведения $\wedge$ логического вывода. Алгоритм логического вывода, проиллюстрированный на рис. 7.4, называется $\wedge$ проверкой по моделям, поскольку в нем осуществляется перебор всех возможных моделей для проверки того, что высказывание a является истинным во всех моделях, в которых истинна база знаний.

Чтобы лучше понять, что такое логическое следствие и логический вывод, можно представить себе, что множество всех логических заключений, полученных из базы данных, — это стог сена, а высказывание a — это иголка. Логическое следствие на⁴ Агент может лишь вычислить вероятность того, что в квадрате [2,2] есть яма; в главе 13 показано, как это сделать.

поминает утверждение о том, что в стоге сена есть иголка, а логический вывод можно сравнить с ее поиском. Это различие между понятиями логического следствия и логического вывода отражено и в формальных обозначениях: если некоторый алгоритм логического вывода i позволяет вывести логическим путем высказывание a из базы знаний KB , то можно записать следующее: $KB \vdash_i a$ Эта формула читается так: "высказывание a получено путем логического вывода из базы знаний KB с помощью алгоритма i " или "алгоритм i позволяет вывести логическим путем высказывание a из базы знаний KB ".

Алгоритм логического вывода, позволяющий получить только такие высказывания, которые действительно следуют из базы знаний, называется $\wedge$ непротиворечивым, или $\wedge$ сохраняющим истинность. Непротиворечивость — это в высшей степени желательное свойство. Противоречивая процедура логического вывода в ходе своей работы по сути создает то, чего нет на самом деле: объявляет об обнаружении несуществующих иголок. Можно легко показать, что алгоритм проверки по моделям, если он применим⁵, является непротиворечивым.

Желательным является также свойство $\wedge$ полноты: алгоритм логического вывода называется полным, если позволяет вывести любое высказывание, которое следует из базы знаний. Когда речь идет о настоящих стогах сена, имеющих конечный объем, то представляется вполне очевидным, что систематическое исследование всегда позволяет определить, есть ли иголочка в данном стоге сена. Но со многими базами знаний связан бесконечно большой стог сена, состоящий из следствий, поэтому обеспечение полноты становится важной проблемой⁶. К счастью, существуют полные процедуры логического вывода для многих форм логики, которые являются достаточно выразительными для того, чтобы они могли справиться со многими базами знаний.

Выше фактически был описан процесс формирования рассуждений, выводы которого гарантированно являются истинными в любом мире, в котором истинны предпосылки; в частности, $\&$ если база знаний является истинной в реальном мире, то любое высказывание a , полученное логическим путем из этой базы знаний с помощью непротиворечивой процедуры логического вывода, является также истинным в реальном мире.

Итак, несмотря на то, что процесс логического вывода оперирует с "синтаксисом"

(а фактически с внутренними физическими конфигурациями, такими как биты в регистрах или картины электрических возбуждений в мозговых структурах), этот процесс соответствует связям реального мира, согласно которым некоторые аспекты реального мира имеют место⁷ благодаря тому, что имеют место другие аспекты реального мира. Это соответствие между миром и его представлением продемонстрировано на рис. 7.5.

⁵ Алгоритм проверки по моделям может применяться, если пространство моделей является конечным, например в мирах вампуса с постоянными размерами. В арифметике, с другой стороны, пространство моделей является бесконечным: даже если мы ограничимся целыми числами, то количество пар значений для x и y в выражении $x+y=4$ является бесконечным.

⁶ Сравните эту ситуацию со случаем бесконечных пространств поиска, описанным в главе 3, где поиск в глубину является неполным.

7 Виттгенштейн [1607] выразил эту мысль в своей знаменитой книге *Tractatus* (Трактат) таким образом: "Мир — это все, что имеет место".

Представление Q Мир Предложения > Предложение Влечет за собой o Аспекты Аспект реального мира Следует реального мира Рис. 7.5. Высказывания представляют собой элементы физической конфигурации агента, а формирование рассуждений — это процесс создания новых элементов физической конфигурации из существующих. Процесс формирования логических рассуждений должен гарантировать, что новые элементы конфигурации будут представлять те аспекты мира, которые действительно следуют из аспектов, представленных существующими элементами конфигурации. Последняя проблема, которая должна быть решена применительно к логическим агентам, — это проблема $\wedge$ обоснования: установления связи между процессами логических рассуждений и реальной средой, в которой существует агент, если эта связь действительно имеется. В частности, эта проблема касается вопроса о том, <ж* как узнать, что база знаний является истинной в реальном мире? (В конце концов, ведь база знаний — просто "синтаксические конструкции" в голове агента.) Это — философская проблема, которой посвящено очень много книг (см. главу 26). Простой ответ состоит в том, что указанная связь создается с помощью датчиков агента. Например, наш агент для мира вампуса имеет датчик запаха. Обнаружив неприятный запах, программа агента создает соответствующее высказывание. Это означает, что если данное высказывание действительно находится в базе знаний, то является истинным и в реальном мире. Поэтому смысл и истинность высказываний, в которых выражаются результаты восприятий, определяются с помощью процессов формирования ощущений и составления высказываний, которые ими порождаются. А что можно сказать об остальных знаниях агента, который, например, считает, что вампуса можно обнаруживать из соседних квадратов, куда доносится его неприятный запах? Это — не прямое представление единственного акта восприятия, а общее правило, которое, возможно, было выведено из опыта восприятий, но не идентично утверждениям с описанием этого опыта. Выработка подобных общих правил в процессе формирования высказываний называется обучением и является темой части VI данной книги. Обучение чревато ошибками. Может оказаться, что вампусы распространяют неприятный запах во все дни, кроме 29 февраля високосного года, поскольку в эти дни вампусы принимают ванну. Поэтому в реальном мире база знаний может оказаться не истинной, но если агент оснащен хорошими процедурами обучения, у него есть повод для оптимизма.

7.4. Пропозициональная логика: очень простая логика В этом разделе будет представлена очень простая логика, называемая $\wedge$ пропозициональной логикой⁸. Здесь рассматривается синтаксис пропозициональной лог⁸ Пропозициональная логика называется также булевой логикой в честь логика Джорджа Буля A815-1864).

гики и ее семантика— способ определения истинности высказываний. Затем мы рассмотрим понятие следствия (отношения между одним высказыванием и другим высказыванием, которое следует из него) и определим, как с помощью этого понятия можно получить простой алгоритм для логического вывода. Безусловно, все это происходит в мире вампуса.

Синтаксис Синтаксис пропозициональной логики определяет допустимые высказывания. $\wedge$ Атомарные высказывания (неделимые синтаксические элементы) состоят из одного $\wedge$ пропозиционального символа. Каждый такой символ обозначает высказывание, которое может быть либо истинным, либо ложным. Для обозначения подобных символов в данном разделе используются прописные буквы: P, Q, R и т.д. Эти обозначения являются произвольными, но часто выбираются таким образом, чтобы они имели для читателя какое-то мнемоническое значение. Например, символ $W_{1,3}$ может использоваться для обозначения высказывания, согласно которому вампус находится в квадрате [1,3]. (Напомним, что символы, подобные $W_{1,3}$, являются атомарными; это означает, что $P_{1,}$, 1 и 3 не следует рассматривать как осмысленные части этого символа.) Существуют два пропозициональных символа, имеющих постоянный смысл: True — тождественно истинное высказывание, а False — тождественно ложное высказывание. $\wedge$ Сложные высказывания формируются из более простых высказываний с помощью $\wedge$ логических связок. Широко применяются пять описанных ниже логических связок. • $\neg$ (нет). Такое высказывание, как $\neg W_{1,3}$, называется $\wedge$ отрицанием высказывания $W_{1,3}$. $\wedge$ Литерал представляет собой либо атомарное высказывание (положительный литерал), либо отрицаемое атомарное высказывание (отрицательный литерал). • л (и). Высказывание, основной связкой которого является л, такое как $W_{1,3} \wedge P_3 \wedge R_3$, называется $\wedge$ конъюнкцией; его части называются конъюнктами.

(Символ л напоминает букву "A" в слове "And" — "И"). • v (или). Высказывание, в котором используется связка v, такое как $(W_{1,3} \wedge P_3) \vee W_{2,2}$, называется $\wedge$ дизъюнкцией дизъюнктов $\{W_{1,3} \wedge P_3\}$ и $W_{2,2}$. (Исторически обозначение v произошло из латинского слова "vel", которое означает "или". Большинство людей находят, что форму этой связки проще всего запомнить как перевернутый символ л.) • $\Rightarrow$ (влечет за собой). Такое высказывание, как $(W_{1,3} \wedge P_3) \Rightarrow \neg W_{2,2}$, называется $\wedge$ импликацией (или условным высказыванием). Его $\wedge$ предпосылкой, или антецедентом, является $(W_{1,3} \wedge P_3)$, а его $\wedge$ заключением, или консеквентом, является $\neg W_{2,2}$. Импликации называют также правилами, или утверждениями if—then (если—то). В других книгах символ импликации иногда записывается как D или $\rightarrow$. • $\Leftrightarrow$ (если и только если). Высказывание наподобие $W_{1,3} \Leftrightarrow W_{2,2}$ называется $\wedge$ двухсторонней импликацией.

Формальная грамматика пропозициональной логики показана в листинге 7.2; те, кто не знаком с системой обозначений BNF, должны обратиться за дополнительными сведениями на с. 1.

Листинг 7.2. Грамматика высказываний пропозициональной логики в форме BNF (Backus-Naur Form — форма Бэкуса-Наура) Sentence $\rightarrow$ AtomicSentence | ComplexSentence AtomicSentence $\rightarrow$ True | False | Symbol Symbol $\rightarrow$ P | Q | R | ...

ComplexSentence $\rightarrow$ UISentence | (Sentence a Sentence) | (Sentence v Sentence) | (Sentence => Sentence) | (Sentence <=> Sentence) Обратите внимание на то, что эта грамматика предъявляет строгие требования к использованию круглых скобок: каждое высказывание, сформированное с помощью бинарных связок, должно быть заключено в круглые скобки. Это гарантирует полную непротиворечивость синтаксиса. Такое требование также означает, что следует писать, например, ({A л B} => C), например, вместо A л B => C. Но для удобства чтения мы будем часто опускать круглые скобки, полагаясь вместо них на использование порядка предшествования связок. Это аналогично правилам предшествования, используемым в арифметике, например, выражение ab+c читается как ((ab) +c), а не как a{B+c}, поскольку операция умножения имеет более высокий приоритет, чем сложение. Порядок предшествования в пропозициональной логике (от высшего к низшему) состоит в следующем: ¬, л, v, => и <=>. Поэтому высказывание ¬P v Q л R => S эквивалентно высказыванию ((¬P) v (Q л R)) => S Определение порядка приоритета не позволяет устранить неоднозначность при чтении таких высказываний, как A л B л C, которое может быть прочитано как ((A л B) а C) или {A а {B а C} }. Но поскольку эти два прочтения, согласно семантике, описанной в следующем разделе, означают одно и то же, допускаются высказывания, подобные A а B а C. Разрешаются также высказывания наподобие A v B в v C и A л B л C. А такие высказывания, как A => B => C, не допускаются, поскольку для них соответствующие два прочтения имеют разный смысл; мы настаиваем на том, что в этом случае должны использоваться круглые скобки.

Наконец, в данной книге иногда вместо круглых скобок используются квадратные, если это позволяет немного упростить понимание данного высказывания.

Семантика Определив синтаксис пропозициональной логики, мы можем приступить к определению ее семантики. Семантика диктует правила выявления истинности высказывания по отношению к конкретной модели. В пропозициональной логике любая

модель просто фиксирует истинностное значение (true или false) для каждого пропозиционального символа. Например, если в высказываниях некоторой базы знаний используются пропозициональные символы P1, P2 и P3, то одна из возможных моделей состоит в следующем: $\mu = \{P1 = \text{false}, P2 = \text{false}, P3 = \text{true}\}$ При

наличии трех пропозициональных символов существует $2^3=8$ возможных моделей; именно столько моделей показано на рис. 7.4. Однако следует отметить, что с тех пор, как мы определили синтаксис, модели стали чисто математическими объектами, которые не обязательно должны быть связаны с миром вампуса.

Например, $P_{i,2}$ — это просто символ; он может означать, что "в квадрате [1,2] есть яма" или "я буду в Париже сегодня и завтра".

Семантика пропозициональной логики должна определять, как следует вычислять истинностное значение любого высказывания при наличии модели. Эта процедура выполняется рекурсивно. Все высказывания формируются из атомарных высказываний и пяти связок, поэтому необходимо указать, как следует вычислять истинность атомарных высказываний, а затем — как вычислять истинность высказываний, сформированных с помощью каждой из этих пяти связок. Задача вычисления истинности атомарных высказываний, как показано ниже, является простой. • Высказывание True истинно в любой модели, а высказывание False ложно в любой модели. • Истинностное значение любого другого пропозиционального символа должно быть указано непосредственно в модели. Например, в модели t_1 приведенное выше высказывание $P_{1/2}$ является ложным.

Для определения истинности сложных высказываний применяются правила, подобные приведенному ниже. • Для любого высказывания s и любой модели t высказывание $\neg s$ является истинным в модели t тогда и только тогда, когда s является ложным в модели t .

Эти правила позволяют свести задачу определения истинности сложных высказываний к задаче определения истинности более простых высказываний.

Правила определения истинности для каждой связки могут быть подытожены в виде "tSk истинностной таблицы, которая определяет истинностное значение сложного высказывания для каждого возможного присваивания значений истинности его компонентам. Истинностные таблицы для рассматриваемых пяти логических связок приведены в табл. 7.1. С использованием этих таблиц истинностное значение любого высказывания s применительно к любой модели t может быть вычислено с помощью простого процесса рекурсивной оценки. Например, высказывание $\neg P_{1/2} \wedge (P_{2/2} \vee P_{3/i})$, оцениваемое в модели m_9 приводит к получению $\text{true} \wedge \{\text{false} \vee \text{true}\} = \text{true} \wedge \text{true} = \text{true}$. В упр. 7.3 предложено написать алгоритм PL-True? (s, t), который вычисляет истинностное значение любого высказывания s пропозициональной логики в модели t .

Выше было сказано, что любая база знаний состоит из множества высказываний.

Теперь можно показать, что логическая база знаний представляет собой конъюнкцию этих высказываний. Это означает, что, начиная с пустой базы знаний KB и применяя операции Tell (KB, $S_{\pm}$) ,..., Tell (KB, S_n), мы получим: $KB = S_1 a \dots a S_n$.

Таким образом, базы знаний и высказывания могут рассматриваться как взаимозаменяемые понятия.

Таблица 7.1. Истинностные таблицы для пяти логических связок. Чтобы воспользоваться этой таблицей, например для вычисления значения $P \vee Q$, когда P является истинным, а Q — ложным, вначале необходимо найти в левой части таблицы строку, в которой P имеет значение true, а Q — false (третья строка). Затем нужно искать запись в этой строке, соответствующую столбцу $P \vee Q$, чтобы определить результат — true. Еще один способ поиска истинностного значения состоит в том, что каждая строка может рассматриваться как модель, а записи под каждым столбцом для этой строки — как утверждение о том, является ли соответствующее высказывание истинным в данной модели

	P	Q	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
1	true	true	true	true	true	true
2	true	false	false	true	false	false
3	false	true	false	true	true	false
4	false	false	false	false	true	true

Истинностные таблицы для связок $\wedge$ ("и"), $\vee$ ("или") и $\neg$ ("нет") почти полностью соответствуют интуитивным представлениям о смысле таких же слов естественного языка. Основным источником возможной путаницы является то, что высказывание $P \vee Q$ истинно, если истинными являются P, или Q, или оба эти высказывания. Есть также другая связка, называемая "исключительное ИЛИ" (сокращенно "XOR" — exclusive OR), которая принимает ложное значение, если оба дизъюнкта являются истинными⁹. В отношении того, какое обозначение следует применять для связки "исключительное ИЛИ", нет общего согласия; двумя возможными вариантами являются $\vee$ и $\oplus$.

Истинностная таблица для связки $\Rightarrow$ на первый взгляд может показаться озадачивающей, поскольку не совсем соответствует интуитивному пониманию выражений "P влечет за собой Q", или "если P, то Q". Но прежде всего необходимо отметить, что пропозициональная логика не требует, чтобы между высказываниями P и Q устанавливались какие-либо причинно-следственные отношения или отношения, определяющие их релевантность применительно к друг другу. Высказывание "то, что 5 — нечетное число, влечет за собой то, что Токио — столица Японии", — это истинное высказывание пропозициональной логики (при нормальной интерпретации), даже несмотря на то, что в естественном языке определенно кажется странным. Еще один источник путаницы состоит в том, что любая импликация является истинной, если ее антецедент ложен. Например, высказывание "то, что 5 — четное число, влечет за собой то, что Сэм умен", является истинным, независимо от того, умен ли Сэм. Это может показаться странным, но приобретает смысл, если рассматривать высказывание в форме " $P \Rightarrow Q$ " как утверждение: "Если P истинно, то я утверждаю, что Q истинно. В противном

случае я не высказываю никаких утверждений". Единственный вариант, при котором это высказывание может принять значение false, состоит в том, чтобы высказывание P было истинным, а Q — ложным.

Истинностная таблица для двухсторонней импликации, $P \Leftrightarrow Q$, показывает, что это высказывание является истинным, если истинны и $P \Rightarrow Q$, и $Q \Rightarrow P$. В словесной латинском языке для обозначения понятия "исключительного или" имеется отдельное слово, *aut*.

ной форме соответствующее высказывание часто записывают следующим образом:

" P тогда и только тогда, когда Q ", или сокращенно " P тогда Q " (ттогда — не опечатка). Правила для мира вампуса лучше всего записывать с помощью связки $\Leftrightarrow$.

Например, в некотором квадрате чувствуется ветерок, если в соседнем квадрате имеется яма, а ветерок в некотором квадрате может чувствоваться, только если в одном из соседних квадратов имеется яма. Поэтому нам потребуются двухсторонние импликации, такие как $B_{i,l} \Leftrightarrow (P_{i,2} \vee P_{2,l})$ где $B_{i,l}$ означает, что в квадрате $[i,l]$ чувствуется ветерок. Обратите внимание на то, что следующая односторонняя импликация:

$B_{i,l} \Rightarrow (P_{i,2} \vee P_{2,l})$ в мире вампуса является истинной, но неполной. Она не позволяет исключить из рассмотрения модели, в которых высказывание $B_{i,l}$ ложно и $P_{i,2}$ истинно и которые нарушают правила мира вампуса. Эту мысль можно иначе выразить таким образом, что импликация требует наличия ям, если чувствуется ветерок, а двухсторонняя импликация требует также отсутствия ям, если ветерок не чувствуется.

Простая база знаний Теперь, после определения семантики пропозициональной логики, мы можем сформировать базу знаний для мира вампуса. Для упрощения будем рассматривать только ямы; случай, в котором рассматривается также сам вампус, оставляем читателю в качестве упражнения. Мы предоставим агенту достаточный объем знаний, чтобы он мог сам формировать те логические выводы, которые были описаны неформально в разделе 7.3.

Вначале необходимо определить словарь пропозициональных символов. Для каждого i, y .

- допустим, что высказывание $P_{i,j}$ является истинным, если в квадрате $[i, j]$ имеется яма;
- допустим, что $B_{i,j}$ является истинным, если в квадрате $[i, j]$ чувствуется ветерок.

База знаний включает перечисленные ниже высказывания, каждому из которых для удобства присвоено отдельное обозначение.

- В квадрате $[1,1]$ отсутствует яма:

$R_1: \neg P_{1,1}$

- В квадрате чувствуется ветерок тогда и только тогда, когда в соседнем квадрате имеется яма. Такое высказывание должно быть сформулировано для

Общий алгоритм определения логитrue true true true true true true false false true true true true true true true true false false false true true true

ческого следствия в пропозициональной логике приведен в листинге 7.3. Как и в алгоритме Backtracking-Search, приведенном на с. 131, функция TT-Entails? осуществляет рекурсивный перебор конечного пространства вариантов присваивания значений переменным. Этот алгоритм является непротиворечивым, поскольку он непосредственно реализует определение логического следствия, и полным, поскольку может применяться для любой базы знаний KB и любого высказывания а и всегда заканчивает свою работу, при условии, что количество моделей, подлежащих проверке, является конечным.

Листинг 7.3. Алгоритм перебора истинностной таблицы для получения пропозициональных логических следствий, tt обозначает истинностную таблицу, функция PL-True? возвращает истинное значение, если некоторое высказывание является истинным в рамках некоторой модели.

Переменная model представляет частично заданную модель — присваивание только некоторым переменным. Вызов функции Extend(P, true,model) возвращает новую частично заданную модель, в которой высказывание P имеет значение true function TT-Entails?(KB, a) returns значение true или false inputs: KB, база знаний - высказывание в пропозициональной логике a, запрос - высказывание в пропозициональной логике symbols <— список пропозициональных символов в KB и a return TT-Check-All(KB, a, symbols, []) function TT-Check-All(KB, a, symbols, model) returns значение true или false if Empty?(symbols) then if PL-True?(KB, model) then return PL-True?(a, model) else return true else do P <— First(symbols); rest <— Rest(symbols) return TT-Check-All(KB, a, rest, Extend(P, true, model) and TT-Check-All(KB, a, rest, Extend(P, false, model) Безусловно, выражение "является конечным" не всегда означает то же, что и выражение "является небольшим". Если KB и a содержат в целом n символов, то количество моделей равно 2^n . Таким образом, временная сложность этого алгоритма составляет $O(2^n)$. (Пространственная сложность составляет только $O(n)$, поскольку перебор вариантов присваивания происходит по принципу поиска в глубину.) Ниже будут показаны алгоритмы, которые являются гораздо более эффективными на практике. К сожалению, ³ каждый известный алгоритм логического вывода для пропозициональной логики в худшем случае имеет сложность, экспоненциально зависящую от размера ввода. Мы не можем рассчитывать на то, что наши алгоритмы будут действовать лучше, поскольку задача получения пропозиционального логического следствия является co-NP-полной (см. приложение A).

Эквивалентность, допустимость и выполнимость Прежде чем перейти к изложению подробных сведений об алгоритмах логического вывода, необходимо рассмотреть некоторые дополнительные понятия, ка-

сающиеся логического следствия. Как и само понятие следствия, эти понятия относятся ко всем формам логики, но их лучше всего показать на примере какого-то конкретного варианта логики, например пропозициональной логики.

Первым понятием является $\wedge$ логическая эквивалентность: два высказывания, a и C , являются логически эквивалентными, если они истинны в одном и том же множестве моделей. Это утверждение записывается как $a \equiv C$. Например, можно легко показать (с помощью истинностных таблиц), что высказывания $P \wedge Q$ и $Q \wedge P$ логически эквивалентны; другие эквивалентности приведены в листинге 7.4. Они играют в логике практически такую же роль, какую арифметические равенства играют в обычной математике. Альтернативное определение эквивалентности является следующим: для любых двух высказываний a и C $a \equiv C$ тогда и только тогда, когда $a \models \neg C$ и $\neg C \models a$ (Еще раз напоминаем, что $\models$ означает логическое следствие.)

Листинг 7.4. Стандартные логические эквивалентности. Символы a , C и y обозначают произвольные высказывания пропозициональной логики

$(a \wedge C) \equiv (C \wedge a)$ коммутативность связки $\wedge$
 $(a \vee P) \equiv (P \vee a)$ коммутативность связки $\vee$
 $((a \wedge P) \wedge y) \equiv (a \wedge (P \wedge y))$ ассоциативность связки $\wedge$
 $((a \vee P) \vee y) \equiv (a \vee (P \vee y))$ ассоциативность связки $\vee$
 $\neg \neg a \equiv a$ устранение двойного отрицания
 $(a \Rightarrow P) \equiv (\neg C \Rightarrow \neg a)$ контрапозиция
 $(a \Rightarrow P) \equiv (\neg a \vee P)$ устранение импликации
 $(a \Leftrightarrow P) \equiv ((a \Rightarrow P) \wedge (P \Rightarrow a))$ устранение двухсторонней импликации
 $\neg(a \wedge P) \equiv (\neg a \vee \neg P)$ правило де Моргана
 $\neg(a \vee P) \equiv (\neg a \wedge \neg P)$ правило де Моргана
 $(a \wedge (P \vee y)) \equiv ((a \wedge P) \vee (a \wedge y))$ дистрибутивность связки $\wedge$ по $\vee$
 $(a \vee (P \wedge y)) \equiv ((a \vee P) \wedge (a \vee y))$ дистрибутивность связки $\vee$ по $\wedge$

Вторым понятием, которое нам потребуется, является $\wedge$ допустимость.

Высказывание допустимо, если оно истинно во всех моделях. Например, высказывание $P \vee \neg P$ является допустимым. Допустимые высказывания известны также под названием $\wedge$ тавтологий — они обязательно истинны и поэтому избыточны.

Поскольку высказывание True является истинным во всех моделях, то любое допустимое высказывание логически эквивалентно True.

Для чего могут применяться допустимые высказывания? Из определения логического следствия можно вывести $\wedge$ теорему дедукции, которая была известна еще в древней Греции:

CF* Для любых высказываний a и P , $a \vdash P$, если и только если высказывание $(a \Rightarrow P)$ является допустимым.

(В упр. 7.4 предлагается доказать эту теорему.) Алгоритм логического вывода, приведенный в листинге 7.3, может рассматриваться как проверка допустимости высказывания $\{KB \Rightarrow a\}$. И наоборот, каждое допустимое высказывание в форме импликации описывает приемлемый вариант логического вывода.

Последним понятием, которое нам потребуется, является $\wedge$ выполнимость.

Некоторое высказывание выполнимо тогда и только тогда, когда оно истинно в неко-

торой модели. Например, приведенная выше база знаний, $(i \wedge l R2 \wedge j R3 \wedge j R4 \wedge j R5)$, выполнима, поскольку существует даже не одна, а три модели, в которых она истинна (см. табл. 7.2). Если некоторое высказывание a является истинным в модели t , то для обозначения этого применяется выражение, что " t Ъ. выполняет a ", или что " t является моделью a ". Выполнимость можно проверить, перебирая все возможные модели до тех пор, пока не будет найдена хотя бы одна из них, которая выполняет данное высказывание. Первой задачей в пропозициональной логике, для которой было доказано, что она является NP-полной, была задача определения выполнимости высказываний.

Многие задачи в компьютерных науках в действительно представляют собой задачи определения выполнимости. Например, во всех задачах удовлетворения ограничений, приведенных в главе 5, по сути требовалось найти ответ на вопрос о том, выполнимы ли данные ограничения при некотором присваивании. Кроме того, с помощью соответствующих преобразований по методу проверки выполнимости можно решать и задачи поиска. Безусловно, что понятия допустимости и выполнимости связаны друг с другом: высказывание a является допустимым тогда и только тогда, когда высказывание $\neg a$ невыполнимо, и наоборот, высказывание a является выполнимым тогда и только тогда, когда высказывание $\neg a$ недопустимо. На этом основании может быть также получен следующий полезный результат: $a \wedge \neg a$ истинно, если и только если высказывание a ложно. $a \wedge \neg a$ невыполнимо.

Доказательство истинности высказывания C на основании истинности высказывания a путем проверки невыполнимости выражения $(a \wedge \neg C)$ точно соответствует стандартному математическому методу доказательства путем $\wedge$ приведения к абсурду (буквально, "доведение до абсурда" — *reductio ad absurdum*). Его также называют доказательством путем Ж опровержения, или доказательством с помощью выявления противоречия. В этом методе доказательства принимается предположение, что высказывание C ложно, и демонстрируется, что такое предположение приводит к противоречию с известными аксиомами a . Подобное противоречие точно соответствует тому, что подразумевается под утверждением, что выражение $(a \wedge \neg C)$ невыполнимо.

7.5. Шаблоны формирования рассуждений в пропозициональной логике В данном разделе рассматриваются стандартные шаблоны логического вывода, которые могут применяться для формирования цепочек заключений, ведущих к желаемой цели. Эти шаблоны логического вывода называются $\wedge$ правилами логического вывода. Наиболее широко известное правило называется $\wedge$ правилом отделения (Modus Ponens — модус поненс) и записывается следующим образом: $a \Rightarrow P, a \vdash P$ Эта запись означает, что если даны любые высказывания в форме $a \Rightarrow P$ и a , то из них можно вывести высказывание P . Например, если дано ($WumpusAhead \wedge$

$WumpusAlive$) $\Rightarrow$ Shoot и ($WumpusAhead \wedge WumpusAlive$), то можно вывести высказывание Shoot.

Еще одним полезным правилом логического вывода является $\wedge$ правило удаления связки "И", в котором утверждается, что из конъюнкции можно вывести любой из ее конъюнктов: $a \wedge P \vdash a$ Например, из высказывания ($WumpusAhead \wedge WumpusAlive$) можно вывести высказывание $WumpusAlive$.

Рассматривая возможные истинностные значения a и C , можно легко показать, что правило отделения и правило удаления связки "И" являются непротиворечивыми не только применительно к данным примерам, но и ко всем возможным высказываниям. Это означает, что данные правила могут использоваться во всех конкретных случаях, в которых они могут потребоваться, вырабатывая непротиворечивые логические выводы без необходимости перебирать все модели.

В качестве правил логического вывода можно также применять все логические эквивалентности, приведенные в листинге 7.4. Например, из эквивалентности двухсторонней импликации и двух импликаций можно получить следующие два правила логического вывода (первое из них называется правилом удаления двухсторонней импликации): $a \Leftrightarrow P \vdash (a \Rightarrow P) \wedge (P \Rightarrow a)$ $(a \Rightarrow P) \wedge (P \Rightarrow a) \vdash a \Leftrightarrow P$ Но не все правила логического вывода действуют в обоих направлениях, как это.

Например, нельзя применить правило отделения в противоположном направлении, чтобы получить высказывания $a \Rightarrow P$ и a из высказывания P .

Рассмотрим, как можно использовать эти правила логического вывода и эквивалентности в мире вампуса. Начнем с базы знаний, содержащей высказывания $R_1 \dots R_5$, и покажем, как доказать высказывание $\neg P_{1,2}$, т.е. утверждение, что в квадрате [1,2] нет ямы. Вначале применим правило удаления двухсторонней импликации к высказыванию R_2 , чтобы получить следующее:

Re: $(B_{1,i} \Rightarrow (P_{1,2} \vee P_{2,i})) \wedge ((P_{1,2} \vee P_{2,i}) \Rightarrow B_{1,i})$ Затем применим к высказыванию R_6 правило удаления связки "И" и получим:

Rt. $((P_{1,2} \vee P_{2,i}) \Rightarrow B_{1,i})$ Из логической эквивалентности отрицаний следует:

R8: $(\neg B1.1 \Rightarrow \neg (P_{i,2} \vee P_{2,i}))$ Теперь можно применить правило отделения к высказыванию R8 и к высказыванию Я4 с данными восприятия (т.е. $\neg B1.1$), чтобы получить следующее:

P9: $\neg (P_{i,2} \vee P_{2,i})$ Наконец, применим правило де Моргана, позволяющее получить такое заключение: $\neg R10: \neg P_{i,2} \wedge \neg P_{2,i}$ Это означает, что ни в квадрате [1,2], ни в квадрате [2,1] нет ямы.

Приведенный выше процесс логического вывода (последовательность применения правил логического вывода) называется $\wedge$ доказательством. Процесс формирования доказательств полностью аналогичен процессу обнаружения решений в задачах поиска. В действительности, если можно было бы составить функцию определения приемника для выработки всех возможных вариантов применения правил логического вывода, это позволило бы использовать для формирования доказательств все алгоритмы поиска, приведенные в главах 3 и 4. Таким образом, вместо трудоемкого перебора моделей может применяться более эффективный способ логического вывода — формирование доказательств. Поиск может проходить в прямом направлении, от первоначальной базы знаний, и осуществляться путем использования правил логического вывода для получения целевого высказывания, или же этот поиск может проходить в обратном направлении, от целевого высказывания, в попытке найти цепочку правил логического вывода, ведущую от первоначальной базы знаний. Ниже в этом разделе рассматриваются два семейства алгоритмов, в которых используются оба эти подхода.

Из того факта, что задача логического вывода в пропозициональной логике является NP-полной, следует, что в наихудшем случае поиск доказательств может оказаться не более эффективным по сравнению с перебором моделей. Однако во многих практических случаях с $\wedge$ поиск доказательства может быть чрезвычайно эффективным просто потому, что в нем могут игнорироваться не относящиеся к делу (нерелевантные) высказывания, независимо от того, насколько велико их количество.

Например, в приведенном выше доказательстве, ведущем к высказыванию $\neg R1/2 \wedge \neg P2,i$ не упоминались высказывания $B2il$, $P1tl$, $P2,2$ или $r3l$. Их можно было не рассматривать, поскольку целевое высказывание, $R1/2$, присутствует только в высказывании RA; остальные высказывания из состава Я4 присутствуют только в Я4 и R2, поэтому Ru R3 и R5 не имеют никакого отношения к доказательству. Те же рассуждения останутся справедливыми, если в базу знаний будет введено на миллион больше высказываний; с другой стороны, в этом случае простой алгоритм перебора строк в истинностной таблице был бы буквально подавлен из-за экспоненциального взрыва, вызванного стремительным увеличением количества моделей.

Это свойство логических систем фактически вытекает из гораздо более фундаментального свойства, называемого $\wedge$ монотонностью. Согласно этому свойству, множество высказываний, которые могут быть получены путем логического вывода, возрастает лишь по мере добавления к базе знаний новой информации¹⁰. Для любых высказываний a и C справедливо следующее: если КБ И a , то КБ $\cup \{a, C\}$. Например, предположим, что база знаний содержит дополнительное утверждение C , согласно которому в этом экземпляре мира вампуса имеется точно восемь ям.

Эти знания могут помочь агенту прийти к дополнительным заключениям, но не способны поставить под сомнение какое-либо уже сделанное заключение a , в частности вывод о том, что в квадрате $[1,2]$ нет ямы. Монотонность означает, что правила логического вывода могут применяться каждый раз, когда в базе знаний обнаружены новые факты. Немонотонные логики, в которых нарушается свойство монотонности, отражают типичную черту человеческого мышления — способность менять свое мнение. Эти варианты логики рассматриваются в разделе 10.7.

Важно отметить, что для того, чтобы из данных предпосылок можно было вывести заключение, должны существовать подходящие предпосылки, — полученное заключение будет следствием из данного правила, независимо от того, что еще находится в базе знаний.

Резолюция Выше было показано, что все рассматривавшиеся до сих пор правила логического вывода являются непротиворечивыми, но вопрос об их полноте применительно к алгоритмам логического вывода, в которых они используются, еще не обсуждался. Алгоритмы поиска, такие как поиск с итеративным углублением (с. 133), являются полными в том смысле, что позволяют найти любую достижимую цель, но если доступные правила логического вывода неадекватны, то цель становится недостижимой; это означает, что не существует доказательства, в котором могли бы применяться только эти правила логического вывода. Например, если мы откажемся от использования правила удаления двухсторонней импликации, то не сможем довести до конца доказательство, изложенное в предыдущем разделе. А в настоящем разделе представлено единственное правило логического вывода, правило резолюции, позволяющее получить алгоритм логического вывода, который становится полным в сочетании с любым полным алгоритмом поиска.

Начнем с использования простой версии правила резолюции в мире вампуса.

Рассмотрим шаги, ведущие вверх на рис. 7.3, а: агент возвращается из квадрата $[2,1]$ в квадрат $[1,1]$, а затем переходит в квадрат $[1,2]$, где он чувствует неприятный запах, но не ощущает ветерка. Введем следующие дополнительные факты в базу знаний:

$R_n: \neg i_{i,2} \vee R_{12}: B_{i,2} \Leftrightarrow (P_{i,1} \vee P_{2,2} \vee P_{1f3})$ С помощью того же процесса, который привел к получению высказывания R_{10} , приведенного выше, мы теперь можем сделать

заключение об отсутствии ям в квадратах [2,2] и [1,3] (напомним, что в отношении квадрата [1,1] уже известно, что в нем нетям):

Я1з: $\neg \exists x, y (P(x,y) \wedge \neg Q(x,y))$ Кроме того, можно применить к высказыванию R3 правило удаления двухсторонней импликации, после чего применить к полученному результату в сочетании с высказыванием R5 правило отделения, чтобы выяснить тот факт, что по меньшей мере в одном из квадратов [1,1], [2,2] или [3,1] есть яма:

Ris: $P(1,1) \vee P(2,2) \vee P(3,1)$ Именно здесь возникают условия, позволяющие впервые использовать правило резолюции (правило устранения противоречия): литерал $\neg P(2,2)$ в высказывании R13, противоположный литералу $P(2,2)$ в высказывании R15, устраняется, что приводит к получению следующего высказывания:

Riei $P(1,1) \vee P(3,1)$ Соответствующий ход рассуждения можно выразить словами таким образом: если по меньшей мере в одном из квадратов [1,1], [2,2] или [3,1] имеется яма, но ее нет в квадрате [2,2], то яма должна быть, по крайней мере, в квадрате [1,1] или [3,1]. Аналогичным образом, устраняется литерал $\neg P(1,1)$ в высказывании R19 противоположный литералу $P(1,1)$ в высказывании д1б, что приводит к получению следующего высказывания:

Rn: Ръ.1 Соответствующий ход рассуждений можно выразить словами таким образом: если по меньшей мере в одном из квадратов [1,1] или [3,1] есть яма, но ее нет в квадрате [1,1], то она находится в квадрате [3,1]. Эти два последних этапа логического вывода представляют собой примеры применения $\wedge$ правила логического вывода, называемого правилом единичной резолюции: $li \vee \dots \vee lk / \neg li \vee \dots \vee \neg lk \vdash li \vee \dots \vee lk$ где каждое из выражений P представляет собой литерал, а выражения $\neg li$ и $\neg lk$ являются $\wedge$ взаимно обратными литералами (т.е. такими литералами, что один из них является отрицанием другого). Таким образом, в правиле единичной резолюции берется $\wedge$ выражение (дизъюнкция литералов) и еще один литерал, после чего формируется новое выражение. Обратите внимание на то, что этот единственный литерал может рассматриваться как дизъюнкция из одного литерала, называемая также $\wedge$ единичным выражением.

Правило единичной резолюции может быть также обобщено до правила полной $\wedge$ резолюции: $li \vee \dots \vee lk, \neg li \vee \dots \vee \neg lk \vdash li \vee \dots \vee lk$ где li и $\neg li$ -- взаимно обратные литералы. Если бы мы имели дело только с выражениями, имеющими длину два, то могли бы записать данное правило следующим образом: $h \vee \neg h \vdash h$ Это означает, что в правиле резолюции берутся два выражения и вырабатывается новое выражение, содержащее все литералы двух первоначальных выражений, за исключением двух взаимно обратных литералов. Например, может иметь место такой логический вывод:

Конъюнктивная нормальная форма Как было описано выше, правило резолюции применяется только к дизъюнкциям литералов, поэтому на первый взгляд оно распространяется только на базы знаний и запросы, состоящие из таких дизъюнкций. На каком же основании мы утверждаем, что это правило может служить основой процедуры полного логического вывода для всей пропозициональной логики? Ответ на этот вопрос состоит в том, что с⁵ каз/сдое

высказывание пропозициональной логики логически эквивалентно конъюнкции дизъюнкций литералов. Любое высказывание, представленное как конъюнкция дизъюнкций литералов, называется высказыванием, находящимся в Σ^0 конъюнктивной нормальной форме, или CNF (Conjunctive Normal Form). Кроме того, ниже будет показано, что целесообразно определить также ограниченное семейство высказываний в конъюнктивной нормальной форме, называемое высказываниями в форме $\wedge k$ -CNF.

Высказывание в форме k -CNF имеет точно 7^k литералов в расчете на каждое выражение:

$D, 1 \vee \dots \vee 4, k) \wedge \dots \wedge D, 1 \vee \dots \vee 4, k)$ Как оказалось, любое высказывание может быть преобразовано в высказывание в форме 3-CNF, которое имеет эквивалентное множество моделей.

Вместо доказательства этих утверждений (см. упр. 7.10) опишем простую процедуру преобразования. Проиллюстрируем эту процедуру, преобразовав высказывание $\neg 2$, или В1Л « $(P_x, 2 \vee p_{2,1})$ », в форму CNF. Ниже описаны соответствующие этапы.

1. Устранить связку $\Leftrightarrow$, заменив высказывание $\text{ос} \Leftrightarrow P$ высказыванием $(\text{ос} \Rightarrow P) \wedge (P \Rightarrow \text{ос})$:

$(B_{i,1} \Rightarrow (P_{i,2} \vee P_{2,1})) \wedge ((P_{i,2} \vee P_{2,1}) \Rightarrow B_{i,1})$ 2. Устранить связку $\Rightarrow$, заменив высказывание $a \Rightarrow P$ высказыванием $\neg a \vee P$:

$(\neg B_{i,1} \vee P_{i,2} \vee P_{2,1}) \wedge (\neg (P_{i,2} \vee P_{2,1}) \vee B_{i,1})$ 3. Конъюнктивная нормальная форма требует, чтобы связка $\neg$ появлялась только перед литералами, поэтому, как принято называть эту операцию, "введем связку $\neg$ внутрь выражения", повторяя операцию применения следующих эквивалентностей из листинга 7.4: $\neg(\neg \text{ос}) = \text{ос}$ (устранение двойного отрицания) $\neg(\text{ос} \wedge P) = (\neg \text{ос} \vee \neg P)$ (правило де Моргана) $\neg(a \vee P) = (\neg a \wedge \neg P)$ (правило де Моргана) В данном примере требуется применение только одного, последнего правила:

$(\neg B_{i,1} \vee P_{i,2} \vee P_{2,1}) \wedge (\neg(\neg P_{i,2} \wedge \neg P_{2,1}) \vee B_{i,1})$ 4. В результате получено высказывание, содержащее вложенные связки $\wedge$ и $\vee$, которые применяются к литералам. Используем закон дистрибутивности, приведенный в листинге 7.4, распределяя связки $\vee$ по связкам $\wedge$ везде, где это возможно.

$(\neg B_{i,1} \vee P_{i,2} \vee P_{2,1}) \wedge (\neg P_{i,2} \vee B_{i,1}) \wedge (\neg P_{2,1} \vee B_{i,1})$ Теперь первоначальное высказывание представлено в форме CNF, как конъюнкция трех выражений. Это высказывание стало более сложным для чтения, но зато его можно использовать в качестве входных данных для процедуры резолюции.

Алгоритм резолюции Процедуры логического вывода, основанные на правиле резолюции, действуют с использованием принципа доказательства путем установления противоречия, который описывался в конце раздела 7.4. Таким

образом, чтобы показать, что кв $\models a$, мы покажем, что высказывание $\{KB \wedge \neg a\}$ является невыполнимым. Это можно сделать, доказав, что имеет место противоречие.

Алгоритм резолюции приведен в листинге 7.5. Вначале высказывание $\{KB \wedge \neg a\}$ преобразуется в форму CNF. Затем к результирующим выражениям применяется правило резолюции. В каждой паре выражений, содержащих взаимно противоположные литералы, происходит удаление этих противоположных друг другу литералов для получения нового выражения, которое добавляется к множеству существующих выражений, если в этом множестве еще нет такого выражения. Указанный процесс продолжается до тех пор, пока не происходит одно из следующих двух событий: • перестают вырабатываться какие-либо новые выражения, которые могли быть добавлены к существующим, и в этом случае из базы знаний кв не следует высказывание a ; • два противоположных друг другу выражения устраняются, в результате чего создается пустое выражение; в этом случае из базы знаний кв следует высказывание a .

Листинг 7.5. Простой алгоритм резолюции для пропозициональной логики. Функция PLResolve возвращает множество всех возможных выражений, полученных путем устранения противоположных друг другу литералов из двух высказываний, которые поступают на ее вход function PL-Resolution(KB, ce) returns значение true или false inputs: KB , база знаний - высказывание в пропозициональной логике a , запрос - высказывание в пропозициональной логике clauses $\leftarrow$ множество выражений, полученное после преобразования высказывания $KB \wedge \neg a$ в форму CNF new $\leftarrow \{\}$ loop do for each d, C_j in clauses do resolvents $\leftarrow$ PL-Resolve (C_i, C_j) if множество resolvents содержит пустое выражение then return true new $\leftarrow$ new $\cup$ resolvents if new с clauses then return false clauses $\leftarrow$ clauses $\cup$ new Пустое выражение (дизъюнкция без дизъюнктов) эквивалентно высказыванию False, поскольку дизъюнкция является истинной, только если истинен по меньшей мере один из ее дизъюнктов. Еще один способ убедиться в том, что пустое выражение служит свидетельством противоречия, состоит в том, что, вернувшись к описанному выше процессу логического вывода, можно заметить, что пустое выражение возникает только после устранения двух взаимно противоположных единичных выражений, таких как P и $\neg P$.

Эту процедуру резолюции можно применить для формирования очень простого логического вывода в мире вампуса. Когда агент находится в квадрате $[1, 1]$, он не чувствует ветерка, поэтому в соседних квадратах не может быть ям.

Соответствующая база знаний является следующей: $KB = R_2 \wedge \neg a = (\neg P_1 \vee P_2) \wedge (\neg P_2 \vee P_1) \wedge \neg a$. P_i и требуется доказать высказывание a , которое, скажем, имеет вид $P_1 \vee P_2$. После преобразования высказывания $\{KB \wedge \neg a\}$ в форму CNF получим выражения, показанные в верхнем ряду на рис. 7.6. В нижнем ряду на этом рисунке

показаны все выражения, полученные путем устранения противоположных друг другу литералов из всех пар выражений, приведенных в верхнем ряду. Затем после устранения литерала $P_i, 2$, противоположного литералу $\neg P_1 \# 2$, будет получено пустое выражение, показанное в виде небольшого квадрата. Анализ результатов, приведенных на рис. 7.6, позволяет обнаружить, что многие этапы резолюции были бессмысленными.

Например, выражение $B \vee \neg B_1; 1 \vee P_1 \# 2$ эквивалентно выражению $\text{True} \vee P_1 \# 2$, которое эквивалентно True . Логический вывод, согласно которому выражение True является истинным, не очень полезен. Поэтому любое выражение, в котором присутствуют два взаимно дополнительных литерала, может быть отброшено.

Полнота резолюции Чтобы завершить обсуждение правила резолюции в этом разделе, покажем, почему алгоритм PL-Resolution является полным. Для этого целесообразно ввести

понятие $\ast$ резолюционного замыкания $RC(S)$ множества выражений S , представляющего собой множество всех выражений, которые могут быть получены путем повторного применения правила резолюции к выражениям из множества S или к их производным. Резолюционным замыканием является множество выражений, которое вычисляется алгоритмом PL-Resolution в качестве окончательного значения переменной $clauses$. Можно легко показать, что множество $RC(S)$ должно быть конечным, поскольку количество различных выражений, которые могут быть сформированы из символов $\neg, \vee, \wedge, \rightarrow, \leftrightarrow$, присутствующих в S , является конечным. (Следует отметить, что это утверждение не было бы истинным, если бы не применялся этап факторизации, в котором уничтожаются дополнительные копии литералов.) Поэтому алгоритм PL-Resolution всегда оканчивает свою работу.

Рис. 7.6. Часть блок-схемы, показывающей процесс применения функции PL-Resolution для формирования простого логического вывода в мире вампуса. Здесь показано, что из первых четырех выражений, приведенных в верхнем ряду, следует выражение-19. Теорема полноты для правила резолюции в пропозициональной логике называется $\wedge$ основной теоремой резолюции.

Если множество выражений является невыполнимым, то резолюционное замыкание этих выражений содержит пустое выражение.

Докажем эту теорему, показав, что справедливо противоположное ей утверждение: если замыкание $RC(S)$ не содержит пустое выражение, то множество S выполнимо. В действительности для множества S можно создать модель с подходящими истинностными значениями для $p_1, \dots, p_k$. Процедура создания такой модели описана ниже.

Для i от 1 до k : • если в множестве $RC(S)$ имеется выражение, содержащее литерал $\neg iPi$, такое, что все другие его литералы являются ложными при данном присваивании, выбранном для $Pi / \dots \# Pi-1$, то присвоить литералу Pi значение false; • в противном случае присвоить литералу Pi значение true.

Остается показать, что это присваивание значений литералам $P_1, \dots, P_k$ представляет собой модель множества выражений S , при условии, что множество $RC(S)$ является замкнутым согласно правилу резолюции и не содержит пустого выражения.

Доказательство этого утверждения оставляем читателю в качестве упражнения.

Прямой и обратный логический вывод Благодаря такой полноте алгоритм резолюции становится очень важным методом логического вывода. Однако во многих практических ситуациях вся мощь правила

резолюции не требуется. Реальные базы знаний часто содержат только выражения в ограниченной форме, называемые $\wedge$ хорновскими выражениями. Хорновское выражение представляет собой дизъюнкцию литералов, среди которых положительным является не больше чем один. Например, выражение $(\neg iLi, i \vee \neg Breeze \vee Blt)$, где Llt означает, что местонахождением агента является квадрат $[1,1]$, представляет собой хорновское выражение, тогда как выражение $\{\neg iBlt \vee p1\#2 \vee P2\#i\}$ таковым не является.

Ограничение, согласно которому только один литерал выражения должен быть положительным, может на первый взгляд показаться немного надуманным и бесперспективным, но фактически является очень важным по трем описанным ниже причинам.

1. Каждое хорновское выражение может быть записано как импликация, предпосылкой которой является конъюнкция положительных литералов, а заключением — один положительный литерал (см. упр. 7.12). Например, хорновское выражение $(\neg l1/l \vee \neg iBreeze \vee B1l)$ может быть записано как импликация $(L1;l \wedge \neg Breeze) \Rightarrow B1l$. В этой последней форме данное высказывание становится более легким для чтения: в нем утверждается, что если агент находится в квадрате $[1,1]$ и чувствует ветерок, то ветерок чувствуется в квадрате $[1,1]$. Людям проще читать и писать в такой форме высказывания, касающиеся многих областей знаний.

Хорновские выражения, подобные этому, имеющие точно один положительный литерал, называются $\wedge$ определенными выражениями. Такой положительный литерал называется $\wedge$ головой выражения, а отрицательные литералы образуют $\wedge$ тело выражения. Определенное выражение без отрицательных литералов просто утверждает справедливость некоторого высказывания; такую конструкцию иногда называют $\wedge$ фактом. Определенные выражения образуют основу для логического программирования, которое рассматривается в главе 9. Хорновское выражение без

положительных литералов может быть записано как импликация, заключением которой является литерал False. Например, выражение $(\neg i \wedge i \vee \neg 1 \wedge 1, 2)$, согласно которому ванпус не может находиться одновременно в квадратах $[1, 1]$ и $[1, 2]$, эквивалентно выражению $w1 \vee l1 \vee l2 \Rightarrow \text{False}$.

В мире баз данных такие высказывания называются $\wedge$ ограничениями целостности и используются для обнаружения ошибок в данных. В приведенных ниже алгоритмах для простоты предполагается, что база знаний содержит только определенные выражения и не содержит ограничений целостности. Такие базы знаний мы будем называть находящимися в хорновской форме.

2. Логический вывод с использованием хорновских выражений может осуществляться с помощью алгоритма $\wedge$ прямого логического вывода и $\wedge$ обратного логического вывода, которые рассматриваются ниже. Оба эти алгоритма являются очень естественными в том смысле, что этапы логического вывода являются для людей очевидными и за ними можно легко проследить.

3. Получение логических следствий с помощью хорновских выражений может осуществляться за время, линейно зависящее от размера базы знаний.

Этот последний факт становится особенно приятным сюрпризом. Он означает, что процедура логического вывода оказывается весьма недорогостоящей применительно ко многим пропозициональным базам знаний, которые встречаются на практике.

Алгоритм прямого логического вывода PL-FC-Entails? $\{KB, q\}$ определяет, следует ли единственный пропозициональный символ q (запрос) из базы знаний, представленной в форме хорновских выражений. Он начинает работу с известных фактов (положительных литералов) в базе знаний. Если известны все предпосылки некоторой импликации, то ее заключение добавляется к множеству известных фактов. Например, если известно, что имеют место факты $b1 \vee l$ и $Breeze$, а в базе знаний имеется выражение $(b1 \vee l \wedge Breeze) \Rightarrow B1 \# 1$, то к ней можно добавить факт $B1 \vee l$. Этот процесс продолжается до тех пор, пока к базе знаний не добавляется запрос q или становится невозможным осуществление дальнейшего логического вывода. Этот подробный алгоритм приведен в листинге 7.6; главное, что следует о нем помнить, — то, что он действует за время, определяемое линейной зависимостью.

Листинг 7.6. Алгоритм прямого логического вывода для пропозициональной логики. В списке agenda ("повестка дня") отслеживаются символы, в отношении которых известно, что они истинны, но которые еще не "обработаны". В таблице count отслеживается информация о том, какое количество предпосылок каждой импликации еще не известно. Каждый раз, когда обрабатывается новый символ p из списка символов agenda, "стоящих на повестке дня", количество предпосылок в

таблице count сокращается на единицу для каждой импликации, в которой появляется предпосылка p . (Такие импликации могут обнаруживаться за постоянное время, если индексация базы знаний KB выполнена должным образом.) После того как количество неизвестных предпосылок для некоторой импликации достигает нуля, все эти предпосылки становятся известными, поэтому заключение этой импликации может быть добавлено к списку agenda. Наконец, необходимо следить за тем, какие символы уже были обработаны; символ, выведенный логическим путем, не следует добавлять к списку agenda, если он уже был обработан ранее. Это позволяет избежать излишней работы, а также предотвратить возникновение бесконечных циклов, которые могут быть вызваны наличием таких импликаций, как $p \Rightarrow Q \wedge Q \Rightarrow P$ function PL-FC-Entails?(KB, q) returns значение true или false inputs: KB, база знаний - множество пропозициональных хорновских выражений q, запрос - пропозициональный символ local variables: count, таблица, индексированная по высказываниям, первоначально имеющая размер, соответствующий количеству предпосылок inferred, таблица, индексированная по символам, в которой каждая запись первоначально имеет значение false agenda, список символов, первоначально включает символы, известные как истинные в базе знаний KB while список agenda не пуст do $p \leftarrow \text{Pop}(\text{agenda})$ unless inferred[p] do inferred[p] $\leftarrow$ true for each хорновское выражение c в котором присутствует предпосылка p do уменьшить на единицу значение count[c] if count[c] = 0 then do if Head[c] = q then return true Push(Head[c], agenda) return false

Лучший способ понять этот алгоритм состоит в том, чтобы рассмотреть пример и иллюстрацию. На рис. 7.7, а показана простая база знаний из хорновских выражений, содержащая выражения Айв как известные факты. На рис. 7.7, б приведена та же база знаний, изображенная в виде $\wedge$ графа AND-OR. В графах AND-OR кратные дуги, соединенные кривой линией, обозначают конъюнкцию (в них необходимо доказать истинность каждой дуги), а кратные дуги, не соединенные друг с другом, обозначают дизъюнкцию (достаточно доказать истинность любой из этих дуг). На этом графе можно легко проверить, как действует алгоритм прямого логического вывода. Устанавливаются значения известных листовых узлов (в данном случае Л и в), а процесс логического вывода распространяется вверх по графу до тех пор, пока это возможно. При появлении любой конъюнкции процесс распространения останавливается и ожидает до тех пор, пока все конъюнкты не становятся известными, и только после этого продолжается дальше. Рекомендую читателю подробно проработать этот пример.

$Q \Rightarrow P \Rightarrow Q$ $LaM \Rightarrow P$ $BaL \Rightarrow M$ $AaP \Rightarrow L$ $AaB \Rightarrow L$ $A \vee B$ а) б) Рис. 7.7. Пример применения алгоритма прямого логического вывода: простая база знаний, состоящая из хорновских выражений (а); соответствующий граф AND-OR (б) Можно легко убедиться в том, что алгоритм прямого логического вывода является

непротиворечивым: каждый этап логического вывода по сути представляет собой применение правила отделения. Кроме того, алгоритм прямого логического вывода является полным: в нем может быть получено каждое атомарное высказывание, которое следует из базы знаний. Проще всего в этом можно убедиться, рассмотрев заключительное состояние таблицы *inferred* (после того, как этот алгоритм достигает $\wedge$ фиксированной точки, в которой становятся невозможными какие-либо новые этапы логического вывода). Эта таблица содержит значение *true* для каждого символа, выведенного логическим путем в течение этого процесса, и *false*— для всех других символов. Эта таблица может рассматриваться как логическая модель; более того, $\wedge$ каждое определенное выражение в первоначальной базе знаний KB является истинным в данной модели. Чтобы убедиться в этом, предположим обратное, а именно, что некоторое выражение $a_1 \wedge \dots \wedge a_n \Rightarrow b$ из базы знаний является

ложным в этой модели. Тогда в этой модели выражение $a_1 \wedge \dots \wedge a_n$ должно быть истинным и в ней же выражение b должно быть ложным. Но это противоречит нашему предположению о том, что алгоритм достиг фиксированной точки! Поэтому можно сделать вывод, что множество атомарных высказываний, полученное логическим путем к моменту достижения фиксированной точки, определяет модель первоначальной базы знаний KB. Более того, любое атомарное высказывание d , которое следует из базы знаний KB, должно быть истинным во всех ее моделях, а также, в частности, в данной модели. Итак, любое высказывание d , которое следует из базы знаний, должно быть выведено логически данным алгоритмом.

Прямой логический вывод представляет собой пример применения общего понятия формирования логических рассуждений, $\wedge$ управляемых данными, т.е. рассуждений, в которых фокус внимания вначале сосредоточен на известных данных.

Прямой логический вывод может использоваться в любом агенте для получения заключений на основе поступающих результатов восприятия, часто даже без учета какого-либо конкретного запроса. Например, агент для мира вампуса может сообщать с помощью операции *Tell* результаты своих восприятий базе знаний с использованием инкрементного алгоритма прямого логического вывода, в котором новые факты могут добавляться к "повестке дня" для инициализации новых процессов логического вывода. У людей формирование рассуждений, управляемых данными, в определенной степени происходит по мере поступления новой информации. Например, находясь дома и услышав, что пошел дождь, человек, который собирался на пикник, может решить, что придется его отменить. Но такое решение вряд ли будет принято, если он узнает, что оказался мокрым семнадцатый лепесток самой крупной розы в саду его соседа; люди держат процессы прямого логического вывода под тщательным контролем, поскольку в противном случае их просто затопил бы поток логических заключений, не имеющих ничего общего с реальностью.

Алгоритм обратного логического вывода, как указывает само его название, действует в обратном направлении от запроса. Если удастся сразу же узнать, что высказывание, содержащееся в запросе q , является истинным, то не нужно выполнять никакой работы. В противном случае алгоритм находит те импликации в базе знаний, из которых следует q . Если можно доказать, что все предпосылки одной из этих импликаций являются истинными (с помощью обратного логического вывода), то высказывание q также является истинным. Будучи применен к запросу Q , показанному на рис. 7.7, этот алгоритм будет проходить вниз по графу до тех пор, пока не достигнет множества известных фактов, которые образуют основу для доказательства.

Разработку подробного алгоритма оставляем читателю в качестве упражнения; как и в случае алгоритма прямого логического вывода, эффективная реализация этого алгоритма выполняет свою работу за линейное время.

Обратный логический вывод представляет собой одну из форм $\wedge$ рассуждения, направляемого целями. Такая форма является полезной при получении ответов на конкретные вопросы, наподобие следующих: "Что теперь мне следует делать?" и "Где же находятся мои ключи?" Зачастую стоимость обратного логического вывода намного меньше по сравнению со стоимостью, линейно зависящей от размера базы знаний, поскольку в этом процессе затрагиваются только факты, непосредственно относящиеся к делу. Вообще говоря, агент должен разделять работу между процессами прямого и обратного формирования рассуждений, ограничивая прямое формирование рассужде-

ний выработкой фактов, которые, по всей вероятности, будут относиться к запросам, подлежащим решению с помощью обратного логического вывода.

7.6. Эффективный пропозициональный В данном разделе рассматриваются два семейства эффективных алгоритмов пропозиционального логического вывода, основанного на проверке по модели: один подход основан на поиске с возвратами, а другой — на поиске с восхождением к вершине. Указанные алгоритмы входят в состав основного "инструментария" пропозициональной логики. Этот раздел может быть пропущен при первом чтении настоящей главы.

Рассматриваемые здесь алгоритмы предназначены для проверки выполнимости.

Выше уже отмечалась связь между поиском модели, обеспечивающей выполнимость логического высказывания, и поиском решения задачи удовлетворения ограничения, поэтому, скорее всего, нет ничего удивительного в том, что эти два семейства алгоритмов весьма напоминают алгоритмы поиска с возвратами, описанные в разделе 5.2, и алгоритмы локального поиска, которые представлены в разделе 5.3. Тем не менее приведенные здесь алгоритмы являются чрезвычайно важными сами по себе, поскольку в компьютерных науках существует

очень много комбинаторных задач, которые можно свести к проверке выполнимости пропозиционального высказывания. Любое усовершенствование алгоритмов проверки выполнимости влечет за собой колоссальные последствия, связанные с повышением нашей способности справляться со всеми сложными задачами в целом.

Полный алгоритм поиска с возвратами Первый рассматриваемый здесь алгоритм часто называют $\wedge$ алгоритмом ДэвисаПатнем в честь авторов оригинальной статьи, в которой он был опубликован, Мартина Дэвиса и Хилари Патнем [336]. Затем этот алгоритм фактически стал одной из версий, описанных Дэвисом, Логеманом и Лавлендом [335], поэтому мы будем называть его DPLL по первым буквам фамилий всех четырех авторов. Алгоритм DPLL принимает на входе некоторое высказывание в конъюнктивной нормальной форме — высказывание, представленное как множество выражений. Как и алгоритмы Backtracking-Search и TT-Entails?, он фактически выполняет рекурсивный перебор в глубину всех возможных моделей. Но в этом алгоритме реализованы три описанных ниже усовершенствования, и этим он отличается от простой схемы, применяемой в алгоритме TT-Entails?.

- Раннее завершение. Алгоритм обнаруживает, должно ли данное высказывание быть истинным или ложным, даже с помощью частично завершенной модели.

Выражение является истинным, если истинен любой его литерал, даже притом что для других литералов еще не определены истинностные значения; поэтому об истинности всего высказывания в целом можно судить еще до того, как модель будет составлена полностью. Например, высказывание $\{A \vee B\} \wedge (A \vee C)$ является истинным, если истинен литерал A , незави-

симо от значений литералов B и C . Аналогичным образом, высказывание является ложным, если ложно любое его выражение, а это происходит, если каждый литерал этого выражения является ложным. Опять-таки, такая ситуация может возникнуть задолго до того, как модель будет полностью составлена.

Раннее завершение позволяет обойтись без исследования целых поддеревьев в пространстве поиска.

- Эвристика чистого символа. $\wedge$ Чистым символом называется символ, который всегда появляется с одним и тем же "знаком" во всех выражениях. Например, в трех выражениях $(A \vee \neg B)$, $(\neg B \vee \neg C)$ и $(C \vee A)$ символ A является чистым, поскольку он появляется только в виде положительных литералов, чистым можно также считать символ B , который появляется только в виде отрицательных литералов, а символ C считается нечистым. Можно легко показать, что если некоторое высказывание имеет модель, то это — модель с чистыми символами, значения которым присвоены так, чтобы их литералы приняли значение true, поскольку при таком условии ни одно выражение не может стать ложным. Следует отметить, что при определении чистоты символа алгоритм может игнорировать выражения, в отношении которых уже известно, что они истинны в модели,

составленной до сих пор. Например, если модель содержит присваивание $B \text{ false}$, то выражение $(\neg B \vee \dots C)$ уже является истинным, а символ C становится чистым, поскольку присутствует только в выражении $(C \vee A)$.

- Эвристика единичного выражения. Единичное выражение было определено ранее как выражение только с одним литералом. В контексте алгоритма DPLL оно также относится к выражениям, в которых в данной модели всем литералам, кроме одного, уже было присвоено значение false . Например, если модель содержит присваивание $B = \text{false}$, то выражение $(B \vee \dots C)$ становится единичным выражением, поскольку оно эквивалентно выражению $\{\text{False} \vee \neg C\}$, или просто $\neg C$. Очевидно, для того, чтобы это выражение приняло истинное значение, литералу C должно быть присвоено значение false . Эвристика единичного выражения предусматривает присваивание значений всем таким символам до того, как происходит переход к обработке оставшейся части высказывания. Одним из важных следствий из этой эвристики является то, что любая попытка доказать (путем опровержения) истинность литерала, который уже находится в базе знаний, немедленно приводит к успеху (упр. 7.16). Следует также отметить, что присваивание значения одному единичному выражению может привести к созданию еще одного единичного выражения; например, после присваивания символу C значения false единичным становится выражение $(C \vee A)$, что влечет за собой присваивание истинного значения символу A . Такое "каскадное" распространение форсированных присваиваний называется "& распространением единичных выражений". Оно напоминает процесс прямого логического вывода с применением хорновских выражений, а в действительности, если рассматриваемое высказывание в конъюнктивной нормальной форме содержит только хорновские выражения, то алгоритм DPLL по сути сводится к алгоритму прямого логического вывода (см. упр. 7.17).

Алгоритм DPLL приведен в листинге 7.7. В этом листинге показана самая важная структурная часть алгоритма, которая описывает сам процесс поиска, но не представлены структуры данных, которые необходимо сопровождать для обеспечения эффективности каждого этапа поиска, а также исключены все программистские "хитрости", которые можно было бы ввести для повышения производительности: изучение выражений, эвристики выбора переменных и операции рандомизированного перезапуска. После включения всех этих усовершенствований алгоритм DPLL, несмотря на свой почтенный возраст, становится одним из самых быстрых алгоритмов проверки выполнимости, которые когда-либо были разработаны. В частности, реализация Chaff этого алгоритма используется для решения задач проверки качества аппаратного обеспечения с миллионом переменных.

Листинг 7.7. Алгоритм DPLL для проверки выполнимости высказывания в пропозициональной логике. Принципы работы функций Find-Pure-Symbol и Find-

Unit-Clause описаны в тексте; каждая из них возвращает символ (или неопределенное значение), а также истинностное значение, которое должно быть присвоено этому символу. Как и алгоритм `tt-Entails?`, этот алгоритм работает с частично составленными моделями `function DPLL-Satisfiable?(s) returns` значение `true` или `false` `inputs: s`, высказывание в пропозициональной логике `clauses` $\leftarrow$ множество выражений высказывания `s`, преобразованного в форму представления CNF `symbols` $\leftarrow$ список пропозициональных символов в высказывании `s` `return DPLL{clauses,s symbols, []}` `function DPLL(clauses, symbols, model) returns` значение `true` или `false` `if` каждое выражение в множестве `clauses` имеет значение `true` в модели `model` `then return true` `if` какое-то выражение в множестве `clauses` имеет значение `false` в модели `model` `then return false` `P, value` $\leftarrow$ `Find-Pure-Symbol{symbols, clauses, model}` `if` значение `P` не является пустым `then return DPLL(clauses,symbols-P,Extend(P,value,model))` `P, value` $\leftarrow$ `Find-Unit-Clause(clauses, model)` `if` значение `P` не является пустым `then return DPLL(clauses,symbols-P,Extend(P,value,model))` `P` $\leftarrow$ `rFirst(symbols)` ; `rest` $\leftarrow$ `Rest(symbols)` `return DPLL(clauses, rest, Extend(P, true, model))` or `DPLL(clauses, rest, Extend(P, false, model))` Алгоритмы локального поиска До сих пор в данной книге было представлено несколько алгоритмов локального поиска, включая алгоритмы `Hill-Climbing` (с. 176) и `Simulated-Annealing` (с. 181).

Эти алгоритмы могут непосредственно применяться для решения задач проверки выполнимости, при условии, что будет выбрана правильная функция оценки.

Поскольку цель состоит в том, чтобы найти присваивание, обеспечивающее выполне-

ние каждого выражения, для этой цели может использоваться любая функция оценки, которая подсчитывает количество невыполненных выражений. Фактически именно этот критерий и применяется в алгоритме `Min-Conflicts` для задач CSP (с. 227). Все эти алгоритмы делают шаги в пространстве полных присваиваний, каждый раз меняя на противоположное (инвертируя) истинностное значение одного символа. Это пространство обычно содержит много локальных минимумов, для выхода из которых требуются различные формы рандомизации. В последние годы было проведено множество экспериментов с тем, чтобы можно было найти приемлемый компромисс между стремлением к "жадному" выбору наилучшего приемника и необходимостью выходить из локальных минимумов с помощью случайного выбора очередного приемника.

Одним из простейших и наиболее эффективных алгоритмов, которые были созданы в результате всей этой работы, является алгоритм, получивший название `WalkSAT` (листинг 7.8). При каждой итерации этот алгоритм выбирает одно невыполненное выражение, а в этом выражении выбирает один символ для инвертирования. В данном алгоритме происходит случайным образом переключение между двумя способами выбора символа для инвертирования его истинностного значения:

вопервых, этап с "минимизацией конфликтов", в котором минимизируется количество невыполненных выражений в новом состоянии, и, во-вторых, этап "случайного передвижения", в котором один из символов выбирается случайным образом.

Листинг 7.8. Алгоритм WalkSAT для проверки выполнимости путем инвертирования значений переменных случайным образом. Существует много версий этого алгоритма `function WalkSAT(clauses, p, max_flips) returns` выполняющую высказывание модель `model` или индикатор отказа `failure` `inputs: clauses`, множество выражений в пропозициональной логике `p`, вероятность выбора решения сделать ход со "случайным перемещением", которая, как правило, составляет около 0,5 `max_flips`, количество инверсий, которые разрешено выполнить, прежде чем отказаться от дальнейших попыток `model` $\leftarrow$ случайно выбранное присваивание значений `true/false` символам в выражениях `for i = 1 to max_flips do if` в модели `model` выполняется множество `clauses` `then return model` `clause` $\leftarrow$ выбранное случайным образом выражение из множества `clauses`, которое имеет значение `false` в модели `model` `with probability p` инвертировать в модели `model` значение случайно выбранного символа из выражения `clause` `else` инвертировать значение того символа из выражения `clause`, который максимизирует количество выполненных выражений в множестве `clauses` `return failure` Действительно ли такой алгоритм, как WalkSAT, способен выполнять производительную работу? Очевидно, что если он возвращает некоторую модель, то входное высказывание в самом деле выполнимо. А что если он возвращает индикатор неудачи `failure`? К сожалению, в этом случае невозможно определить, верно ли, что

высказывание является невыполнимым, или просто этому алгоритму требуется предоставить больше шансов добиться успеха. При этом можно попытаться присвоить параметру с определением максимального количества инверсий `max_flips` бесконечно большое значение. В данном случае можно легко показать, что алгоритм Walks AT в конечном итоге вернет какую-то модель (если она существует), при условии, что вероятность этого $p > 0$. Это связано с тем, что всегда существует последовательность инверсий, ведущая к присваиванию, которое обеспечивает выполнение высказывания, и такая последовательность в конечном итоге вырабатывается в результате случайного выбора этапов перемещения. К сожалению, если параметр `max_flips` имеет бесконечно большое значение, а высказывание является невыполнимым, данный алгоритм никогда не заканчивает свою работу!

Из этого следует, что алгоритмы локального поиска, подобные Walks AT, являются наиболее полезными, когда можно действительно рассчитывать на то, что решение существует; например, задачи, которые рассматривались в главах 3 и 5, обычно имеют решения. С другой стороны, локальный поиск не всегда может обнаружить

невыполнимость, а именно это требуется, когда задача состоит в том, чтобы определить, следует ли какое-то высказывание из базы знаний. Например, агент не может надежно использовать локальный поиск для доказательства того, что некоторый квадрат в мире вампуса безопасен. Вместо этого он может лишь утверждать:

"Я размышлял об этом в течение часа, но так и не мог найти хотя бы один из возможных миров, в котором данный квадрат не является безопасным". А поскольку данный алгоритм локального поиска обычно позволяет действительно быстро найти модель, если она существует, то агент, использующий этот алгоритм, должен учитывать, что неудача в попытке найти модель высказывания с помощью данного алгоритма скорее всего свидетельствует о невыполнимости данного высказывания.

Безусловно, такой результат — нечто иное, чем доказательство, и агент должен трижды подумать, прежде чем рискнуть своей жизнью, основываясь на подобных результатах.

Трудные задачи определения выполнимости Теперь рассмотрим, какие показатели производительности демонстрируют алгоритмы DPLL и WalkSAT на практике. Нас особенно интересуют трудные задачи, поскольку легкие могут быть решены с помощью любого старого алгоритма. В главе 5 мы ознакомились с некоторыми удивительными открытиями в области решения задач определенного типа. Например, задача с n ферзями (которая когда-то рассматривалась как чрезвычайно сложная) при ее решении с помощью алгоритмов поиска с возвратами оказалась тривиально простой для методов локального поиска, таких как метод с минимальными конфликтами. Это связано с тем, что решения этой задачи распределены в пространстве присваиваний очень плотно и для любого начального присваивания гарантируется, что решение находится недалеко. Таким образом, задача с n ферзями является простой потому, что она $\wedge$ недостаточно ограничена.

Если речь идет о решении задач проверки выполнимости, представленных в конъюнктивной нормальной форме, то среди них недостаточно ограниченными можно считать такие задачи, которые содержат относительно немного выражений, ограничиваю-

щих значения переменных. Например, ниже представлено выработанное случайным образом¹² высказывание в форме 3-CNF с пятью символами и пятью выражениями.

$(\neg iD \vee \neg iB \vee C) \wedge (B \vee \neg A \vee \neg C) \wedge (\neg iC \vee \neg iB \vee E) \wedge (E \vee \neg iD \vee B) \wedge (B \vee E \vee \neg iC)$ Моделями этого высказывания являются 16 из 32 возможных вариантов присваивания, поэтому в среднем для поиска модели потребуются только две случайно выбранных гипотезы.

Так где же найти сложные задачи? Можно предположить, что если количество выражений будет увеличено, притом что количество символов останется постоянным, то задача станет более ограниченной и поиск решений окажется более затруднительным. Допустим, что t — количество выражений, а p — количество символов.

На рис. 7.8, а показана вероятность того, что случайно сформированное высказывание в форме 3-CNF является выполнимым, как функция от отношения "выражение/символ", t/p , при постоянном значении p , равном 50. Как и следовало ожидать, при малых значениях t/p эта вероятность близка к 1, а при больших значениях t/p вероятность близка к 0. На кривой вероятности начинается довольно резкий спад приблизительно после достижения значения $t/p=4$. 3. Высказывания в форме CNF, приближающиеся к этой критической точке, можно назвать "почти выполнимыми", или "почти невыполнимыми". Можно ли считать, что именно здесь находятся трудные задачи?

ГГ сП Я Л Х О С СQ 1 ^ 0,8 \ 0,6 0,2 I II I I I I I 0 12 3 4 5 6 7 8 Отношение "выражение/символ", t/p а) б) Рис. 7.8. Анализ производительности алгоритмов DPLL и WalkSAT при решении трудных задач: график, на котором показана вероятность того, что случайно сформированное высказывание в форме 3-CNF с количеством символов $p=50$ окажется выполнимым, как функция от отношения "выражение/символ", t/p (а); график усредненного времени прогона алгоритмов DPLL и WalkSAT применительно к 100 выполнимым сформированным случайным образом высказываниям в форме 3-CNF с $p=50$, который показывает наличие узкого диапазона значений t/p вокруг критической точки (б) На рис. 7.8, б показано время прогона алгоритмов DPLL и WalkSAT на участке вокруг этой критической точки, где наше внимание ограничивалось только выпол0 12 3 4 5 6 7 Отношение "выражение/символ", t/p 12 Каждое выражение содержит три случайно выбранных различных символа, к каждому из которых применяется отрицание с вероятностью 50%.

нимыми задачами. Изучение приведенных результатов позволяет сделать три вывода: во-первых, задачи, приближающиеся к критической точке, являются гораздо более трудными по сравнению с другими случайно сформированными задачами; во-вторых, даже при решении самых сложных задач алгоритм DPLL является весьма эффективным — он требует выполнения в среднем нескольких тысяч шагов по сравнению со значением количества шагов 250~1015, которое требуется для перебора истинностной таблицы; в-третьих, во всем этом диапазоне характеристик задач алгоритм WalkSAT работает намного быстрее, чем алгоритм DPLL.

Безусловно, эти результаты относятся только к случайно сформированным задачам. Реальные задачи не обязательно имеют такую же структуру, как случайно

сформированные задачи (они могут характеризоваться разными значениями относительного количества положительных и отрицательных литералов, разными плотностями соединений между выражениями и т.д.). Тем не менее на практике алгоритм WalkSAT и подобные ему алгоритмы очень хорошо справляются также с решением реальных задач и часто не уступают самым лучшим алгоритмам специального назначения для этих задач. Такие решатели задач, как Chaff, легко справляются с задачами, состоящими из тысяч символов и миллионов выражений. На основании этих наблюдений можно сделать вывод, что некоторые комбинации эвристики с минимальными конфликтами и поведения со случайным блужданием предоставляют универсальную способность находить решения в большинстве ситуаций, в которых требуется комбинаторное формирование рассуждений.

7.7. Агенты, основанные на пропозициональной логике В данном разделе соберем воедино все, что было описано до сих пор, для того, чтобы приступить к созданию агентов, действующих с использованием пропозициональной логики. Здесь будут рассматриваться два вида агентов: во-первых, агенты, в которых применяются алгоритмы логического вывода и база знаний, подобные показанному в листинге 7.1 универсальному агенту, основанному на знаниях, и, во-вторых, агенты, которые вычисляют значения логических высказываний непосредственно с помощью логических схем. В этом разделе будет также продемонстрировано функционирование агентов обоих типов в мире вампуса, а также показано, что оба они страдают от серьезных недостатков.

Поиск ям и вампусов с помощью логического вывода Начнем с описания агента, который рассуждает логически о том, где находятся ямы, вампусы и безопасные квадраты. Агент начинает свою работу с базы знаний, в которой описаны "законы" мира вампуса. Он знает, что квадрат $[1,1]$ не содержит яму или вампуса; это означает, что $\neg P_{1,1}$ и $\neg W_{1,1}$. Для каждого квадрата $[x,y]$ агенту известно высказывание с указанием того, как возникает ветерок:

$V_{x,y} \Leftrightarrow (P_{x,y+1} \vee P_{x,y-1} \vee P_{x+1,y} \vee P_{x-1,y})$ G.1) Для каждого квадрата $[x, y]$ агенту известно высказывание с указанием того, как возникает неприятный запах:

$S_{x,y} \Leftrightarrow (W_{x,y+i} \vee W_{x,y-i} \vee W_{x+1,y} \vee W_{x-1,y})$ G.2)

Наконец, он знает, что в мире вампуса существует точно один вампус.

Соответствующее высказывание состоит из двух частей. Прежде всего необходимо указать, что имеется самое меньшее один вампус:

$W_{1,1} \vee W_{1,2} \vee \dots \vee W_{4,3} \vee W_{4,4}$ Затем необходимо указать, что существует самое большее один вампус. Один из способов формулировки этого утверждения состоит в том, что при наличии любых двух квадратов один из них обязательно должен быть свободным от вампуса. При наличии p квадратов мы получаем $p(p-1)/2$ таких

высказываний, которые подобны по форме высказыванию $\neg \text{Wli} \vee \neg \text{Wli}^2$. Таким образом, для мира с размерами 4x4 описание начинается с общего количества 155 высказываний, содержащих 64 различных символа.

Программа агента, приведенная в листинге 7.9, сообщает в свою базу знаний с помощью операции Tell о каждом новом восприятии ветерка и неприятного запаха. (Она также обновляет некоторые обычные программные переменные для слежения за тем, где находится агент и где он побывал; дополнительная информация об этом приведена ниже.) Затем программа выбирает среди периферийных квадратов (т.е. квадратов, являющихся соседними по отношению к тем, которые уже посетил агент) такой квадрат, который должен быть проверен в следующую очередь. Агент может доказать, что периферийный квадрат $[i,j]$ безопасен, если из базы знаний следует высказывание $(\neg \text{Pif } j \wedge \neg \text{Wi} \# j)$. На втором месте по своей привлекательности находится квадрат, который, возможно, является безопасным; таковым является квадрат, применительно к которому агент не может доказать, что в нем имеется яма или вампус, т.е. квадрат, для которого из базы знаний не следует высказывание $(\text{Pi}, j \vee \text{Wit}, j)$.

Листинг 7.9. Агент для мира вампуса, использующий пропозициональную логику для определения ям, вампусов и безопасных квадратов. Процедура Route-Problem формирует задачу поиска, решением которой является последовательность действий, ведущих от $[x,y]$ до $[i,j]$ и проходящих только через ранее посещенные квадраты

```
function PL-Wumpus-Agent(percept) returns действие
action inputs: percept,
список результатов восприятия в форме [Stench,Breeze,Glitter]
(["Неприятный запах", "Ветерок", "Блеск"])
static: KB, база знаний, первоначально содержащая лишь
определения "законов функционирования" мира вампуса
x, y, orientation, позиция агента (первоначально 1,1) и его ориентация
(первоначально Right - смотрит вправо)
visited, массив, показывающий, какие квадраты были посещены,
первоначально содержащий значения false
action, последнее по времени действие агента, первоначально не определено
plan, намеченная последовательность действий, первоначально пустая
обновить значения x, y, orientation, visited с учетом действия
action if stench then Tell (KB,  $S^x, y$ ) else if breeze then Tell (KB,  $\neg S^x, y$ )
else Tell (KB,  $\neg S^x, y$ )
```

```
if glitter then action  $\leftarrow$  grab else if plan не пуст then action  $\leftarrow$  Pop (plan)
else if для некоторого периферийного квадрата  $[i,j]$  результат Ask(KB,  $(\neg \text{Pif } j \wedge \neg \text{Wi} \# j)$ )
имеет значение true or для некоторого периферийного квадрата  $[i,j]$  результат Ask(KB,  $(\text{Pi}, j \vee \text{Wit}, j)$ )
имеет значение false then do plan  $\leftarrow$  A*-Graph-Search (Route-Problem( $[x,y]$ ,
orientation,  $[i,j]$ , visited))
action  $\leftarrow$  Pop(plan) else action  $\leftarrow$  случайно выбранный шаг
return action
```

Вычисление логического следствия в процедуре Ask может быть реализовано с использованием любого из методов, описанных выше в этой главе. Очевидно, что алгоритм TT-Entails? (см. листинг 7.3) является практически не

применимым, поскольку в нем приходится выполнять перебор 264 строк. Алгоритм DPLL (см. листинг 7.7) осуществляет требуемый логический вывод за несколько миллисекунд в основном благодаря использованию эвристики с распространением единичных выражений. Может также использоваться алгоритм Walks AT с учетом обычных предостережений о его неполноте. В рассматриваемых экземплярах мира вампуса неудачи в поиске модели при использовании 10 000 инверсий неизменно соответствуют невыполнимости, поэтому вероятность ошибок, вызванных неполнотой этого алгоритма, весьма мала.

В малом мире вампуса алгоритм PL-Wumpus-Agent действует весьма неплохо.

Тем не менее некоторые особенности базы знаний агента остаются крайне неудовлетворительными. База знаний KB содержит "буквальные" высказывания в общей форме, приведенной в уравнениях 7.1 и 7.2, для каждого отдельного квадрата. Чем больше среда, тем крупнее должна быть такая начальная база знаний. Было бы гораздо лучше иметь только два высказывания, которые сообщают, из-за чего возникают ветерки и неприятные запахи в любых квадратах. Но такие выразительные возможности выходят за рамки пропозициональной логики. В следующей главе будет показан более выразительный логический язык, в котором можно легко представить подобные высказывания.

Слежение за местонахождением и ориентацией Программа агента, приведенная в листинге 7.9, немного "жульничает", поскольку в ней слежение за местонахождением осуществляется за пределами базы знаний, тогда как вместо этого следует использовать логические рассуждения¹³. Для выполнения этих функций "должным образом" потребуются высказывания, касающиеся местонахождения. Одна из первых попыток решения этой задачи может состоять в использовании некоторого символа наподобие $\gamma_{1\#1}$ для обозначения того, что агент находится в квадрате [1,1]. В таком случае начальная база знаний могла бы включать высказывания, подобные следующим:

13 Наблюдательный читатель должен был заметить, что это позволило бы нам определить связь между чистыми восприятиями, такими как Breeze, и высказываниями, касающимися местонахождения, такими как γ_i , γ_j

$L_{i,j}$ л FacingRight л Forward $\Rightarrow L_{2,i}$ Однако сразу же обнаруживается, что такой способ является неприменимым.

Если агент начинает движение с квадрата [1,1], глядя вправо, и передвигается вперед, то из базы знаний будут следовать высказывания и $L_{1\#1}$ (о его первоначальном местонахождении), и $L_{1\#2}$ (о его новом местонахождении). Тем не менее эти высказывания не могут одновременно быть истинными! Проблема состоит в том, что эти два высказывания, касающиеся местонахождения, должны ссылаться на два различных промежутка времени. Нам требуется ввести

обозначение наподобие $\langle x, y \rangle$ которое означало бы, что агент находился в квадрате $[1,1]$ в момент времени 1, $L22, i$ — для обозначения того, что агент находился в квадрате $[2,1]$ в момент времени 2, и т.д. Высказывания, касающиеся ориентации и действий, также должны зависеть от времени. Поэтому правильными высказываниями являются следующие:

$L1, i \wedge \text{FacingRight}1 \wedge \text{Forward}1 \Rightarrow L22, i \wedge \text{FacingRight} \wedge \text{TurnLeft}1 \Rightarrow \text{FacingUp}2$ и тому подобные высказывания. Как оказалось, задача создания полной и правильной базы знаний, которая позволяла бы следить за всем, что происходит в мире вампуса, является весьма сложной; отложим полное обсуждение этой темы до главы 10.

А в данном разделе достаточно лишь подчеркнуть такую мысль, что начальная база знаний должна содержать высказывания, подобные приведенным в двух предыдущих примерах, для каждого момента времени t , а также для каждого местонахождения. Это означает, что для каждого момента времени t и местонахождения $[x,y]$ база знаний содержит некоторое высказывание примерно в такой форме:

$\langle x, y \rangle \wedge \text{FacingRight}^i \wedge \text{Forward}^j \Rightarrow L_{t+1x+i,y} G.3$) Но даже если мы установим верхний предел количества допустимых интервалов времени (допустим, 100), в конечном итоге база знаний заполнится десятками тысяч высказываний. Та же самая проблема возникает, если для каждого нового интервала времени добавление высказываний происходит "по мере необходимости". Из-за такого стремительного увеличения количества высказываний база знаний становится неудобной для чтения людьми, но быстрые пропозициональные алгоритмы — решатели задач все еще способны легко справляться с задачами в мире вампуса с размерами 4×4 (они достигают предела своих возможностей в мире, размеры которого составляют приблизительно 100×100). Агенты, основанные на логических схемах, которые описаны в следующем подразделе, предоставляют частичное решение этой проблемы стремительного увеличения количества высказываний, но нам придется подождать полного решения до того, как будет разработана логика первого порядка в главе 8.

Агенты на основе логических схем ^ Агент на основе логической схемы представляет собой особую разновидность рефлексных агентов с поддержкой состояния, которые определены в главе 2. В этом агенте результаты восприятия становятся входными данными для ^ последовательной логической схемы — сети из ^ логических элементов, каждый из которых реализует какую-то логическую связку, и ^ регистров, каждый из которых хранит истинностное значение одного высказывания. Выходами этой логической схемы являются регистры, соответствующие действиям, например, выход `Grab` принимает значение `true`, если агент

хочет что-то схватить. Если вход Glitter (блеск) связан непосредственно с выходом Grab, то агент хватается свою цель, где бы он ее не обнаружил (рис. 7.9).

Вычисления значений с помощью логических схем осуществляются по принципу " $\wedge$ " обработки потока данных: на каждом временном интервале устанавливаются значения входных данных и по логической схеме распространяются сигналы. Каждый раз, когда некоторый логический элемент получает на входе все необходимые данные, он вырабатывает на выходе некоторое значение. Этот процесс весьма напоминает процесс прямого логического вывода в таком графе AND—OR, какой приведен на рис. 7.7, б.

Breeze I I Forward Stench ? Glitter DTurnLeft Bump II Turn Right Scream I h $\wedge$ T Grab A Alive -DShoot Рис. 7.9. Часть структурной схемы агента на основе логической схемы для мира вампуса, на которой показаны входы, выходы, схема для захвата золота и схема для определения того, жив ли вампус. Регистры показаны в виде прямоугольников, а задержки на один шаг обозначаются небольшими треугольниками В предыдущем разделе было указано, что агенты на основе логической схемы позволяют учитывать временн $\bar{y}$ е соотношения намного более успешно, чем пропозициональные агенты на основе логического вывода. Это связано с тем, что значение, хранящееся в каждом регистре, показывает истинностное значение соответствующего пропозиционального символа в текущий момент времени t , поэтому регистр не хранит отдельные копии этого значения для каждого отдельного интервала времени. Например, можно предусмотреть регистр Alive, который должен содержать значение true, если вампус жив, и false, если он мертв. Этот регистр соответствует пропозициональному символу Alive1, поэтому в каждом временном интервале он ссылается на разные высказывания. Внутреннее состояние этого агента (т.е. его память) поддерживается путем обратного подключения выхода некоторого регистра к той же схеме через $\wedge$ линию задержки. Это позволяет доставить в схему информацию о состоянии регистра в предыдущем интервале времени. Один из примеров такой конструкции приведен на рис. 7.9. Значение регистра Alive формируется с помощью конъюнкции отрицания восприятия Scream и пропущенного через линию задержки значения самого регистра Alive. Действие, выполняемое этой схемой применительно к регистру Alive, можно представить в форме высказывания как следующую двухстороннюю импликацию:

$Alive^* \Leftrightarrow \neg Scream^* \wedge Alive^*_{t-1}$ (G.4) Это высказывание означает, что вампус в момент времени t жив тогда и только тогда, когда в момент времени t не был слышен его жалобный предсмертный крик (такое состояние нельзя определить по крику в момент времени $t-1$), и он был жив в момент времени $t-1$. Предполагается, что при инициализации этой логической схемы значение регистра Alive устанавливается равным true. Поэтому значение регистра Alive остается истинным до тех пор, пока

не раздается жалобный крик вампуса, после чего это значение становится ложным и остается таковым. Это — именно то, что требуется.

За местонахождением агента можно следить в основном с использованием такого же способа, с помощью которого мы следим за здоровьем вампуса. Для этого требуется иметь отдельный регистр BX/Y для каждого значения x и y ; его значение должно быть равным true, если агент находится в квадрате $[x,y]$. Однако логическая схема, которая присваивает значение регистру Lx,y , является гораздо более сложной по сравнению с логической схемой для регистра Alive. Например, агент находится в квадрате $[1,1]$ в момент времени t , если он, во-первых, был здесь в момент времени $t-1$ и либо не двинулся вперед, либо попытался двинуться, но ударился об стену; во-вторых, находился в квадрате $[1,2]$, смотря вниз, и двинулся вперед; в-третьих, находился в квадрате $[2,1]$, смотря влево, и двинулся вперед. Эти три условия можно описать с помощью следующего высказывания:

$B_{1,1} \Leftrightarrow (L_{t-1} \wedge \neg (Forward \vee Bump)) \vee (FacingDown \wedge Forward) \vee (FacingLeft \wedge Forward) \vee (FacingRight \wedge Forward)$ Схема для регистра $B_{1,1}$ показана на рис. 7.10. К каждому регистру местонахождения подключена аналогичная схема. В упр. 7.13, б предлагается спроектировать логическую схему для высказываний, касающихся ориентации.

Forward $\vee$ Turn Left Breeze $\vee$ Stench ? Glitter ?

Bump $\vee$ r Scream Рис. 7.10. Схема определения того, находится ли агент в квадрате $[1,1]$.

К каждому регистру с определением местонахождения и ориентации подключена аналогичная схема

Логические схемы, показанные на рис. 7.9 и 7.10, постоянно поддерживают правильные истинностные значения для регистров Alive и Lx,y . Однако необычным свойством логических высказываний, соответствующих этим схемам и регистрам, является то, что их правильные истинностные значения всегда можно проверить.

С другой стороны, рассмотрим высказывание $B_{4,4}$ о том, что в квадрате $[4,4]$ чувствуется ветерок. Несмотря на то что истинностное значение этого высказывания остается постоянным, агент не сможет узнать это истинностное значение до тех пор, пока не посетит квадрат $[4,4]$ (или не выведет логическим путем, что рядом с этим квадратом есть яма). Пропозициональная логика и логика первого порядка предназначены для того, чтобы с их помощью можно было автоматически представлять высказывания с истинными, ложными и неизвестными значениями, а логические схемы таким свойством не обладают: регистр для высказывания $B_{4,4}$ обязан содержать хоть какое-то значение, либо true, либо false, даже несмотря на то, что истинные данные о том, каково его значение, еще не получены. Это означает,

что значение в регистре вполне может оказаться неправильным, и это введет агента в заблуждение. Иными словами, в регистре должны быть представлены три возможных состояния (высказывание $v_{4/4}$ является заведомо истинным, заведомо ложным или имеющим неизвестное значение), но в нашем распоряжении для этой цели имеется только один бит.

Решение такой проблемы состоит в использовании двух битов вместо одного.

Высказывание $B_{4/4}$ будет представлено с помощью двух регистров, которые мы будем называть $x(v_{4/4})$ и $i=C(-i_{4/4})$, где K обозначает "known" (известный). (Напомним, что это все еще просто символы со сложными именами, даже несмотря на то, что они выглядят как структурированные выражения!) Если оба регистра, $i=C(B_{4/4})$ и $i=C(-i_{B_{4/4}})$, содержат ложное значение, то истинностное значение $v_{4/4}$ неизвестно. (А если оба они содержат истинное значение, то в базе знаний есть ошибка!) С этого времени, каждый раз, когда в некоторой части логической схемы нужно будет использовать высказывание $v_{4/4}$, вместо него будет применяться высказывание $i=C(B_{4/4})$, а когда потребуется использовать $-i_{B_{4/4}}$, вместо него будет служить $i=C(-i_{B_{4/4}})$. Вообще говоря, каждое потенциально неопределенное высказывание можно представить двумя " $\wedge$ " высказываниями с оценкой знаний (knowledge proposition), которые позволяют определить, известно ли о том, что соответствующее высказывание является истинным, или известно, что оно ложно.

Примеры того, как использовать высказывания с оценкой знаний, будут вскоре приведены. Но вначале необходимо провести определенную работу, чтобы найти способ выяснения истинностных значений самих высказываний с оценкой знаний.

Обратите внимание на то, что высказывание $B_{4/4}$ имеет постоянное истинностное значение, а значения высказываний $J_{v_{4/4}}$ и $i=C(-i_{B_{4/4}})$ изменяются по мере того, как агент больше узнает об этом мире. Например, высказывание $k(v_{4/4})$ вначале является ложным, а затем становится истинным, как только появляется возможность определить, что высказывание $v_{4/4}$ является истинным, т.е. после того, как агент переходит в квадрат $[4,4]$ и обнаруживает ветерок. С тех пор оно продолжает оставаться истинным. Таким образом, справедливо следующее уравнение:

$X(B_{4/4})_t \Leftrightarrow X(B_{4/4})_{t-L} \vee (L_{t_{4/4}} \wedge \text{Breeze}^*) \wedge G.6$ Аналогичное уравнение может быть составлено для высказывания $K(-i_{B_{4/4}})_t$.

Теперь, после того как агент узнал, в каких квадратах чувствуется ветерок, он может заняться обнаружением ям. В отсутствии ямы в некотором квадрате можно быть уве-

ренным тогда и только тогда, когда в отношении одного из соседних квадратов известно, что в нем не чувствуется ветерок. Например, имеет место следующее уравнение:

$K(-nP^*, *)^* \Leftrightarrow K(-nB3,4)t \vee K(-,B4.3) \text{ в } G.7)$ Задача определения того, что в некотором квадрате есть яма, является более сложной — для этого в одном из соседних квадратов должен чувствоваться ветерок, который нельзя приписать наличию других ям, как показывает следующее уравнение:

$X(P4,4)t \Leftrightarrow (K(BZiA)t \wedge K(-nP2iA)t \wedge \text{Л } X\text{Б}P3.3)^* \vee (K(B^{\wedge},3)t \wedge K(-iP4,2)t \wedge \text{Ж-п}P3.3)^* \text{ в } G.8)$
 Хотя логические схемы для определения наличия или отсутствия ям являются довольно сложными, $s^{\wedge} =$ в них имеется лишь постоянное количество логических элементов в расчете на каждый квадрат. Это свойство является существенным, если мы занимаемся созданием агентов на основе логической схемы, предназначенных для решения задач, масштабы которых могут увеличиваться в разумных пределах. А фактически это — свойство самого мира вампуса; среда называется проявляющей свойство локальности, если истинность каждого интересующего нас высказывания можно определить, рассматривая лишь постоянное количество других высказываний.

Обоснованность классификации по признаку локальности очень сильно зависит от того, насколько точно определена "физическая структура" данной среды. Например, область определения задачи игры в минного тральщика (Minesweeper) (упр. 7.11) является нелокальной, поскольку, чтобы узнать, есть ли мина в данном квадрате, может потребоваться проверить квадраты, находящиеся от него на произвольном расстоянии. Поэтому агенты на основе логических схем не всегда практически применимы для нелокальных областей определения.

Осталась еще одна проблема, которую мы до сих пор тщательно обходили, — вопрос об $s^{\wedge}$ ацикличности. Логическая схема является ациклической, если каждый путь, соединяющий выход некоторого регистра в обратном направлении с его входом, содержит промежуточный элемент задержки. Мы требуем, чтобы все схемы были ациклическими, поскольку логические схемы, являющиеся циклическими, не работают как физические устройства! Они могут переходить в режим неустойчивых колебаний, что приводит к появлению неопределенных значений. В качестве примера циклической схемы рассмотрим следующий дополненный вариант уравнения 7.6:

$K(B4,4)t \Leftrightarrow i=C(-B4,4)t-1 \vee (LV4 \wedge \text{Breeze}^*) \vee K(P3/4)t \vee K(Pii3)t \text{ в } G.9)$ Дополнительные дизъюнкты, $ЖP3/4)t$ и $ЖP4/3)t$, позволяют агенту определить, чувствуется ли ветерок, на основании известных данных о наличии ям в соседних квадратах, и, на первый взгляд, в этом нет ничего предосудительного. Но, к сожалению, появление ветерка зависит от наличия ям в соседних квадратах, а наличие ям зависит от того, чувствуется ли ветерок в соседних квадратах, и такая связь устанавливается с помощью уравнений, подобных уравнению 7.8. Поэтому полная логическая схема будет содержать циклы.

Сложность здесь заключается не в том, что дополненное уравнение 7.9 стало неправильным. Скорее всего, проблема состоит в том, что промежуточные зависимости, представленные подобными уравнениями, невозможно разрешить с помощью распространения истинностных значений по соответствующим логическим схемам.

Ациклическая версия схемы, составленной с использованием уравнения 7.6, которая определяет наличие ветерка только с помощью прямого наблюдения, является неполной в том смысле, что могут возникать такие ситуации, когда агент на основе логической схемы окажется менее осведомленным, чем агент на основе логического вывода, использующий полную процедуру логического вывода. Например, если чувствуется ветерок в квадрате [1,1], то агент на основе логического вывода может прийти к заключению, что ветерок чувствуется также в квадрате [2,2], а агент на основе ациклической логической схемы, использующий уравнение 7.6, на это не способен. Задача создания полной логической схемы выполнима (в конце концов, логические схемы дают возможность эмулировать любой цифровой компьютер), но такая схема окажется значительно более сложной по сравнению с ациклической логической схемой.

Сопоставление двух описанных типов агентов Агент на основе логического вывода и агент на основе логической схемы представляют собой выражение двух противоположных подходов к проектированию агентов: декларативного и процедурного. Их сравнение может быть проведено по нескольким описанным ниже параметрам. • Краткость. Агент на основе логической схемы, в отличие от агента на основе логического вывода, не обязан иметь отдельные копии своих "знаний", относящиеся к каждому временному интервалу. Вместо этого он обращается только к данным, касающимся текущего и предыдущего временных интервалов.

Для обоих агентов требуются копии описаний "физической структуры"

(представленные в виде высказываний или логических схем) для каждого квадрата, и поэтому они не могут хорошо приспособляться к более крупным вариантам среды. В тех вариантах среды, характеризующихся наличием многочисленных объектов, между которыми установлены сложные связи, количество необходимых высказываний превосходит возможности любого пропозиционального агента. Для подобных вариантов среды требуется выразительная мощь логики первого порядка (см. главу 8). Кроме того, пропозициональные агенты обоих типов плохо приспособлены для представления или решения задачи поиска пути к ближайшему безопасному квадрату. (По этой причине в алгоритме PL-Wumpus-Agent приходится прибегать к использованию алгоритма поиска.) • Вычислительная эффективность. В наихудшем случае логический вывод может потребовать времени,

экспоненциально зависящего от количества символов, тогда как вычисление логического значения с помощью логической схемы требует времени, линейно зависящего от размера этой схемы (или линейно зависящего от глубины этой схемы, если она реализована в виде физического устройства). Однако на практике (как было показано выше) алгоритм DPLL выполняет требуемый логический вывод очень быстро¹⁴.

14 Фактически все логические выводы, выполняемые с помощью логической схемы, могут быть сделаны с использованием алгоритма DPLL за линейное время! Это связано с тем, что в алгоритме DPLL вычисление по схеме (как и прямой логический вывод) может быть эмулировано с помощью правила распространения единичного выражения.

- Полнота. Выше было указано, что агент на основе логической схемы может быть неполным из-за ограничений, связанных с ацикличностью. Но фактически причины неполноты могут оказаться более фундаментальными. Прежде всего напомним, что продолжительность выполнения задания логической схемы линейно зависит от размера данной схемы. Это означает, что логическая схема для некоторых вариантов среды, являющаяся полной (т.е. позволяющая вычислить истинностное значение любого высказывания с определенным значением), должна быть экспоненциально больше, чем база знаний агента на основе логического вывода. В противном случае такая схема представляла бы собой реализацию некоего способа решения задачи поиска логического следствия в пропозициональной логике за время меньше экспоненциального, а это весьма маловероятно. Еще одна причина состоит в том, каков характер внутреннего состояния агента. Агент на основе логического вывода запоминает результаты каждого восприятия и обладает знаниями, либо явными, либо неявными, о каждом высказывании, которое следует из этих восприятий и начальной базы знаний. Например, получив результат восприятия $B1;1$, этот агент знает о наличии дизъюнкции $P1/2 \vee p2\#1$, из которой следует высказывание $B2$. Агент на основе логической схемы, с другой стороны, забывает все полученные ранее результаты восприятия и помнит только отдельные высказывания, хранящиеся в регистрах. Поэтому высказывания $p1\#2$ и $P2$, отдельно взятые, становятся для него неизвестными после получения результатов первого восприятия, т.е. он не может сделать вывод о том, что истинно высказывание $B2$.
- Простота конструирования. Это — очень важный вопрос, на который нелегко найти точный ответ. Безусловно, авторы данной книги считают, что намного проще сформулировать высказывание о "физической структуре" декларативно, тогда как задача создания небольших, ациклических, не слишком неполных логических схем для непосредственного обнаружения ям представляется для них весьма сложной.

Подводя итог, можно прийти к выводу, что для согласования требований вычислительной эффективности, краткости, полноты и простоты конструирования

необходимо прийти к определенным компромиссам. Если связь между восприятиями и действиями является простой (такой как связь между Glitter и Grab), оптимальным решением можно считать логическую схему, а для реализации более сложных связей может оказаться лучшим декларативный подход. Например, в такой проблемной области, как шахматы, декларативные правила являются краткими и простыми для кодирования (по крайней мере, в логике первого порядка), а логическая схема для вычисления ходов непосредственно по данным о позиции на доске была бы невообразимо огромной.

В царстве животных часто можно обнаружить различные проявления таких компромиссов. Низшие животные с очень простыми нервными системами, повидимому, основаны на логических схемах, тогда как высшие животные, включая людей, очевидно, обладают способностью выполнять логический вывод на основе явных представлений. Это позволяет им вычислять гораздо более сложные функции агента. Люди имеют также логические схемы для реализации рефлексов, кроме того, возможно, обладают также способностью $\wedge$ компилировать декларатив-

ные представления для дальнейшего их использования в виде логических схем после того, как некоторые логические выводы становятся рутинными. Таким образом, проект гибридного агента (см. главу 2) может обладать лучшими способностями, взятыми из обоих миров.

7.8. Резюме В данной главе приведены вводные сведения об агентах на основе знаний, а также показано, как сформулировать логическое определение, с помощью которого такие агенты могут формировать рассуждения о мире, в котором они существуют.

Основные идеи этой главы перечислены ниже.

- Интеллектуальным агентам требуются знания о мире для того, чтобы они могли вырабатывать хорошие решения.
- Знания содержатся в агентах в форме высказываний на языке представления знаний, которые хранятся в базе знаний.
- Агент на основе знаний состоит из базы знаний и механизма логического вывода. Он действует путем сохранения высказываний о мире в своей базе знаний, использования механизма логического вывода для получения новых высказываний и применения этих высказываний для принятия решения о том, какое действие следует выполнить.
- Язык представления определяется с помощью его синтаксиса, который задает структуру высказываний, и его семантики, которая определяет истинность каждого высказывания в каждом из возможных миров, или модель этого высказывания.
- Для понимания процесса формирования логических рассуждений крайне важно определить, как связаны логические следствия разных высказываний.

Из высказывания a следует другое высказывание $|3$, если $|3$ является истинным во всех мирах, где истинно a . Эквивалентные определения основаны на понятии

допустимости высказывания $a \Rightarrow b$ и невыполнимости высказывания $a \wedge \neg b$. • Логический вывод — это процесс получения новых высказываний из старых.

Непротиворечивые алгоритмы логического вывода обеспечивают получение только таких высказываний, которые являются логическими следствиями; полные алгоритмы обеспечивают получение всех высказываний, являющихся логическими следствиями. • Пропозициональная логика — это очень простой язык, состоящий из пропозициональных символов и логических связок. Этот язык позволяет рассматривать высказывания, в отношении которых известно, являются ли они истинными, ложными или имеют полностью неизвестное логическое значение. • При наличии постоянного словаря пропозициональных символов множество возможных моделей остается конечным, поэтому логическое следствие можно проверить, перебирая модели. Эффективные алгоритмы логического вывода на основе проверки по модели для пропозициональной логики включают мето-

ды поиска с возвратами и локального поиска и часто позволяют очень быстро решать крупные задачи. • Правила логического вывода представляют собой шаблоны непротиворечивого логического вывода, которые могут использоваться для поиска доказательств.

Правило резолюции позволяет создать полный алгоритм логического вывода для баз знаний, которые представлены в конъюнктивной нормальной форме.

Прямой логический вывод и обратный логический вывод — это алгоритмы логического вывода, которые чрезвычайно естественно подходят для баз знаний, представленных в форме хорновских выражений. • На основе пропозициональной логики могут быть созданы агенты двух типов: в агентах на основе логического вывода для слежения за миром и выявления его скрытых свойств используются алгоритмы логического вывода, тогда как в агентах на основе логических схем высказывания представлены в виде битов в регистрах, а обновление содержимого регистров происходит в результате распространения сигналов по логическим схемам. • Пропозициональная логика является достаточно эффективной для решения некоторых задач в отдельно взятом агенте, но не позволяет распространить свое действие в масштабах таких вариантов среды, которые обладают неограниченными размерами, поскольку не имеет достаточной выразительной мощности, которая позволила бы кратко выразить временные, пространственные и другие универсальные шаблоны связей между объектами.

Библиографические и исторические заметки Понятие агента, который использует процесс формирования логических рассуждений для установления связи между восприятиями и действиями, было выдвинуто в статье Джона Маккарти "Programs with Common Sense" [1009], [1011]. Кроме того, Маккарти стал основоположником декларативного подхода, указав, что очень изящным является такой способ

создания программного обеспечения, который предусматривает передачу агенту указаний о том, что он должен знать. В статье Аллена Ньюэлла "The Knowledge Lever [1124] подчеркнуто, что рациональные агенты могут быть описаны и проанализированы на абстрактном уровне, определяемом знаниями, которым они обладают, а не программами, которые в них выполняются.

Сравнение декларативного и процедурного подходов к искусственному интеллекту было проведено Боденом [145]. Незатухающие дебаты между приверженцами этих двух подходов снова оживились, в частности, после публикации [192] и [1145].

Сама логика имеет свои истоки в древнегреческой философии и математике.

Фрагменты с изложением различных логических принципов (связывающих синтаксическую структуру высказываний с их истинным или ложным значением, с их смыслом или с допустимостью доводов, в которых они фигурируют) встречаются во многих трудах Платона. Первое известное систематическое исследование логики было проведено Аристотелем, работы которого были собраны его учениками после его смерти в 322 до н.э. в виде трактата, называемого Органон. ^ Силлогизмы Аристотеля представляли собой то, что мы теперь называем правилами логического вывода. Хотя силлогизмы включали элементы и пропозициональной логики, и логики

первого порядка, сама система Аристотеля в целом была очень слабой с точки зрения современных стандартов. В ней не было места для шаблонов логического вывода, которые могли бы применяться к высказываниям с произвольной сложностью, как в современной пропозициональной логике.

Тесно связанные со школой Аристотеля мегарская и стоическая школы (которые зародились в пятом веке до н.э. и продолжали свою работу в течение нескольких столетий) ввели в научный обиход принципы систематического исследования импликации и других основных конструкций, до сих пор используемых в современной пропозициональной логике. Применение истинностных таблиц для определения логических связей было впервые предложено в трудах Филона из Мегары. Стоики приняли к использованию пять основных правил логического вывода как допустимые без доказательства, в том числе правило, которое теперь принято называть правилом отделения (Modus Ponens). Из этих пяти правил они вывели множество других правил, используя, кроме других принципов, теорему дедукции (с. 302), и имели гораздо более четкое представление о таком понятии, как доказательство, чем Аристотель. Стоики утверждали, что их логика была полной в том смысле, что они смогли описать все допустимые правила логического вывода, но от их трудов остались лишь отдельные фрагменты, по которым трудно судить об их правоте. Хорошее описание истории развития логики на примере мегарской и

стоической школ, в той степени, в какой эти школы нам известны, приведено в работе Бенсона Мэйтса [1000].

Идеи о том, что логический вывод можно свести к чисто механическому процессу, применяемому к формальному языку, были высказаны Готфридом Вильгельмом Лейбницем (1646-1716). Однако собственные работы Лейбница в области математической логики обладали серьезными недостатками, поэтому он запомнился скорее как человек, выдвинувший эти идеи как цели, которые должны быть достигнуты, а не действительно предпринявший результативную попытку их достичь.

Джордж Буль [149] впервые представил полную и работоспособную систему формальной логики в своей книге *The Mathematical Analysis of Logic*. Логика Буля была почти полностью промоделирована на основе обычной алгебры действительных чисел, и в ней в качестве основного метода логического вывода использовалась подстановка логически эквивалентных выражений. Хотя систему Буля еще нельзя было считать полной пропозициональной логикой, она оказалась настолько близкой к таковой, что другие математики сумели быстро заполнить все недостающие части.

Шредер [1368] описал конъюнктивную нормальную форму, тогда как хорновская форма была введена намного позднее Альфредом Хорном [675]. Первое полное описание современной пропозициональной логики (и логики первого порядка) можно найти в книге *Begriffsschrift* ("Система обозначения понятий") Готтлоба Фреге [496].

Первое механическое устройство для формирования логических выводов было сконструировано потомственным графом Стенхоупом (1753-1816). Машина Стенхоупа, *Demonstrator*, была способна обрабатывать силлогизмы и некоторые логические выводы в теории вероятностей. Уильям Стэнли Джевонс, один из тех, кто усовершенствовал и расширил результаты Буля, сконструировал в 1869 году "логическое фортепьяно" для формирования выводов в булевой логике.

Занимательная и поучительная история этих и других ранних механических устройств для формирования рассуждений описана Мартином Гарднером [519]. Первой опубликованной компьютерной программой для формирования логического вывода была разработанная Ньюэллом, Шоу и Саймоном программа *Logic Theorist* [1127]. Эта

программа была предназначена для моделирования мыслительных процессов человека. Программа, позволяющая сформировать доказательство, была фактически спроектирована в 1954 году Мартином Дэвисом [334], но результаты работ в области создания программы *Logic Theorist* были опубликованы немного раньше. И программа Дэвиса от 1954 года, и программа *Logic Theorist* были

основаны на достаточно произвольно выбранных методах, которые не оказали существенного влияния на дальнейшие работы в области автоматизированного дедуктивного вывода.

Истинностные таблицы как метод проверки допустимости или невыполнимости высказываний в языке пропозициональной логики были введены в арсенал ученых независимо Людвигом Виттгенштейном [1607] и Эмилем Постом [1231]. В 1930-х годах большие успехи были достигнуты в области создания методов логического вывода для логики первого порядка. В частности, Гёдель [565] показал, что полная процедура логического вывода в логике первого порядка может быть получена путем сведения к пропозициональной логике с использованием теоремы Эрбрана [650].

Мы снова вернемся к этой теме в главе 9, а здесь необходимо сделать важное замечание о том, что разработка эффективных пропозициональных алгоритмов в 1960-х годах мотивировалась в основном стремлением математиков создать эффективные средства доказательства теорем для логики первого порядка. Алгоритм Дэвиса-Патнем [336] был первым эффективным алгоритмом логического вывода в пропозициональной логике, но в большинстве случаев обладал меньшей эффективностью по сравнению с алгоритмом поиска с возвратами DPLL, который был введен двумя годами позже [335]. Полное правило резолюции и доказательство его полноты было опубликовано в оригинальной статье Дж.Э. Робинсона [1298], который также показал, как может осуществляться формирование рассуждений в логике первого порядка без обращения к пропозициональным методам.

Стефен Кук [289] показал, что задача определения выполнимости высказывания в пропозициональной логике является NP-полной. Поскольку определение того, является ли высказывание логическим следствием, эквивалентно задаче определения его невыполнимости, эта задача является co-NP-полной. Известны многие подмножества пропозициональной логики, для которых задача проверки выполнимости решается за полиномиальное время; одним из таких подмножеств являются хорновские выражения. Алгоритм прямого логического вывода для хорновских выражений с линейными затратами времени предложен Доулингом и Галльером [407], которые описали свой алгоритм в виде процесса обработки потока данных, подобного распространению сигналов в логической схеме. Задача проверки выполнимости стала одним из канонических примеров сведения к NP-полным задачам; например, Кайе [782] показал, что игра Minesweeper (минный тральщик) (см. упр. 7.11) является NP-полной.

Попытки применения алгоритмов локального поиска для решения задач проверки выполнимости предпринимались различными авторами на протяжении всех 1980-х годов; все эти алгоритмы были основаны на идее минимизации количества невыполненных выражений [614]. Особенно эффективный алгоритм был разработан

Гу [601] и независимо от него Селманом и др. [1382], которые назвали этот алгоритм GSAT и показали, что он способен решать широкий перечень очень трудных задач сочень большой скоростью. Алгоритм WalkSAT, описанный в этой главе, также предложен Селманом и др. [1380].

"Фазовый переход" в процессе определения выполнимости сформированных случайным образом задач k-SAT впервые был обнаружен Саймоном и Дюбуа [1421].

Эмпирические результаты, полученные Кроуфордом и Отоном [307], свидетельствуют о том, что при решении крупных сформированных случайным образом задач 3-SAT это резкое изменение характеристик возникает в диапазоне значений отношения "высказывание/переменная", приближающихся к 4,24; кроме того, в этой статье описана очень эффективная реализация алгоритма DPLL. Байардо и Шрэг [87] описали еще одну эффективную реализацию алгоритма DPLL с использованием методов из области удовлетворения ограничений, а в [1090] описан алгоритм Chaff, который решает задачи проверки аппаратного обеспечения с миллионом переменных, ставший победителем конкурса SAT 2002. Ли и Энбулаган [926] описали эвристики, основанные на распространении единичных выражений, позволяющие создавать быстрые решатели задач. Чизман и др. [244] предоставили данные о многих задачах, связанных с задачами проверки выполнимости, и пришли к выводу, что все NP-трудные задачи обнаруживают фазовый переход. Кёркпатрик и Селман [800] показали, каким образом можно использовать методы статистической физики для получения представления о том, какова точная "форма" фазового перехода. Теоретический анализ его местонахождения все еще довольно неудовлетворителен: до сих пор удалось доказать лишь то, что для случайно сформированных задач 3-SAT фазовый переход находится в диапазоне [3.003, 4.598].

Кук и Митчелл [290] составили превосходный обзор результатов по этой и некоторым другим темам, касающимся выполнимости.

Самые ранние теоретические исследования показали, что алгоритм DPLL применительно к некоторым естественным распределениям задач характеризуется в среднем полиномиальной сложностью. Этот факт, который мог бы, в принципе, стать предметом восхищения, начал казаться менее восхитительным, когда Франко и Паулл [494] показали, что те же задачи можно было бы решить за постоянное время, просто проверяя случайно выбранные варианты присваивания. Метод случайной выработки вариантов, описанный в данной главе, приводит к получению гораздо более трудных задач. Заинтересовавшись достигнутым на практике успехом в использовании локального поиска при решении таких задач, Кутсупиас и Пападимитриу [848] показали, что простой алгоритм восхождения к вершине способен решать почти все экземпляры задачи проверки выполнимости очень быстро, а это свидетельствует о том, что трудные задачи встречаются редко. Более того, Шёнинг [1365] продемонстрировал рандомизированный вариант алгоритма

GSAT, для которого ожидаемое время прогона в худшем случае при решении задач 3-SAT составляет 1. 333п; этот предел все еще является экспоненциальным, но гораздо более низким по сравнению с достигнутыми ранее пределами для худшего случая. Алгоритмы проверки выполнимости все еще остаются очень активной областью исследования; хорошей отправной точкой для изучения этой темы может служить сборник статей под редакцией Дю и др. [418].

Истоки идей по созданию агентов на основе логических схем можно проследить до оригинальной статьи Мак-Каллока и Питтса [1017], которая послужила началом исследований в области нейронных сетей. Вопреки сложившемуся общему мнению, эта статья касалась реализации проекта агента на основе логической схемы в мозгу.

Однако агенты на основе логической схемы долго не привлекали значительного внимания

исследователей в области искусственного интеллекта. Самым заметным исключением из этого правила явились работы Стена Розеншайна [760], [1308], который разработал способы компиляции агентов на основе логических схем из декларативных описаний проблемной среды. Логические схемы для обновления высказываний, хранящихся в регистрах, тесно связаны с аксиомой состояния-преемника, разработанной для логики первого порядка Рейтером [1277]. В работах Рода Брукса [189], [190] продемонстрирована эффективность использования проектов на основе логической схемы для управления роботами; к этой теме мы вернемся в главе 25. Брукс [192] утверждает, что для искусственного интеллекта необходимы лишь проекты на основе логических схем, а методы представления и формирования логических рассуждений являются громоздкими, дорогостоящими и ненужными. Но, по мнению авторов, ни один из этих подходов, отдельно взятых, не является достаточным.

Мир вампуса был придуман Грегори Йобом [1633]. Любопытно то, что Йоб разработал этот мир, поскольку ему надоели игры, которые проводятся в мире, представленном в виде квадратной решетки: топология его первоначального мира вампуса представляла собой додекаэдр, а мы снова поместили вампуса в старую скучную квадратную решетку. Майкл Генезерет был первым, кто указал, что мир вампуса можно использовать как испытательную площадку для агентов.

Упражнения 7.1. Опишите мир вампуса в соответствии со свойствами проблемной среды, перечисленными в главе 2.

7.2. Предположим, что агент достиг пункта, показанного на рис. 7.3, я, получив такие результаты восприятия: в квадрате [1,1] — ничего, в квадрате [2,1] — ветерок, а в квадрате [1,2] — неприятный запах. В настоящий момент агента интересует содержимое квадратов [1,3], [2,2] и [3,1]. Каждый из них может содержать яму, и

самое большее в одном из них может находиться вампус. На основе примера, приведенного на рис. 7.4, сконструируйте множество возможных миров.

(Должно быть найдено 32 таких возможных мира.) Отметьте миры, в которых существующая база знаний является истинной, а также те, в которых истинным является каждое из следующих высказываний:

ОС2 = "В квадрате [2,2] нет ямы" аз = "В квадрате [1,3] есть вампус"

На основании этого покажите, что KB И а2икв И ос3.

7.3. Рассмотрите задачу определения того, является ли некоторое высказывание пропозициональной логики истинным в данной модели. а) Напишите рекурсивный алгоритм PL-True? (s,m), который возвращает true тогда и только тогда, когда высказывание s является истинным в модели t (где t присваивает истинностное значение каждому символу в s). Этот алгоритм должен заканчивать свою работу за время, линейно зависящее от размера высказывания. (Еще один вариант состоит в том, чтобы воспользоваться версией этой функции из оперативного репозитория кода.)

б) Приведите три примера высказываний, в отношении которых можно определить, являются ли они истинными или ложными, в частичной модели, в которой не заданы истинностные значения для некоторых из символов. в) Покажите, что в общем случае истинностное значение высказывания (если оно имеется) невозможно эффективно определить в частичной модели. г) Доработайте свой алгоритм PL-True? так, чтобы он иногда позволял судить об истинностном значении по частичным моделям, сохраняя вместе с тем свою рекурсивную структуру и линейную зависимость времени прогона от размера высказывания. Приведите три примера высказываний, истинность которых в частичной модели не обнаруживается вашим алгоритмом. д) Проведите исследование того, позволяет ли этот модифицированный алгоритм повысить эффективность алгоритма TT-Entails?.

7.4. Докажите каждое из приведенных ниже утверждений. а) Высказывание a является допустимым тогда и только тогда, когда $\text{True} \models a$. б) Для любого высказывания a истинно, что $\text{False} \models a$. в) Истинно, что $a \models \exists z$ тогда и только тогда, когда высказывание $(a \Rightarrow \exists z)$ допустимо. г) Истинно, что $a = p$ тогда и только тогда, когда высказывание $(a \Leftrightarrow P)$ допустимо. д) Истинно, что $a \models P$ тогда и только тогда, когда высказывание $(\text{ос } l \text{ - } l \text{ } P)$ недостижимо.

7.5. Рассмотрите словарь, состоящий только из четырех высказываний, A, B, C и D.

Сколько существует моделей для каждого из следующих высказываний? а) $\{A \vee B\} \models C$ б) $A \vee B \models A \Leftrightarrow B \wedge C$ 7.6. В этой главе мы определили четыре различных бинарных логических связки. а) Имеются ли какие-либо другие логические связки,

которые могут оказаться полезными? б) Сколько может быть определено разных бинарных связей? в) Почему некоторые из них не очень полезны?

7.7. Используя любой предпочитаемый вами метод, проверьте каждую из эквивалентностей, приведенных в листинге 7.4.

7.8. Определите, является ли каждое из приведенных ниже высказываний действительным, невыполнимым или ни тем ни другим. Проверьте полученные вами результаты с помощью истинностных таблиц или правил эквивалентности, приведенных в листинге 7.4. а) $\text{Smoke} \Rightarrow \text{Smoke}$ б) $\text{Smoke} \Rightarrow \text{Fire}$ в) $(\text{Smoke} \Rightarrow \text{Fire}) \Rightarrow \neg \text{Smoke} \Rightarrow \neg \text{Fire}$

г) $\text{Smoke} \vee \text{Fire} \vee \neg \text{Fire}$ д) $\{(\text{Smoke} \wedge \text{Heat}) \Rightarrow \text{Fire}\} \Leftrightarrow \{(\text{Smoke} \Rightarrow \text{Fire}) \vee (\text{Heat} \Rightarrow \text{Fire})\}$ е) $(\text{Smoke} \Rightarrow \text{Fire}) \Rightarrow \{(\text{Smoke} \wedge \text{Heat}) \Rightarrow \text{Fire}\}$ ж) $?\text{ig} \vee \text{Dumb} \vee \{\text{Big} \Rightarrow \text{Dumb}\}$ з) $\{\text{Big} \wedge \text{Dumb}\} \vee \neg \text{Dumb}$ 7.9. (Адаптировано из [78].) Можете ли вы доказать на основании приведенных ниже рассуждений, что единорог — мифическое существо? А что насчет того, что это волшебное существо? Существо, вооруженное рогом?

Если единорог — мифическое существо, то он бессмертен, а если не мифическое, то он — смертное млекопитающее. Если единорог либо бессмертен, либо является млекопитающим, то он вооружен рогом. Единорог является волшебным существом, если он вооружен рогом.

7.10. Любое высказывание пропозициональной логики логически эквивалентно утверждению, что любой возможный мир, в котором это высказывание было бы ложным, не имеет места. На основании этого наблюдения докажите, что любое высказывание может быть записано в конъюнктивной нормальной форме.

7.11. Minesweeper (минный тральщик)— широко известная компьютерная игра, которая тесно связана с миром вампуса. Мир минного тральщика представляет собой прямоугольную решетку из N квадратов с M разбросанными по ним невидимыми минами. Агент может проверить любой квадрат; если он проверяет заминированный квадрат, его ждет немедленная смерть. В игре Minesweeper наличие мин раскрывается путем показа в каждом проверенном квадрате количества мин в квадратах, соседних по горизонтали, вертикали или диагонали. Цель игры состоит в том, чтобы проверить каждый незаминированный квадрат. а) Допустим, что высказывание $x_{li} j$ является истинным тогда и только тогда, когда в квадрате $[i, j]$ находится мина. Составьте утверждение, согласно которому в квадратах, соседних по отношению к квадрату $[1, 1]$, имеется точно две мины, в виде высказывания, включающего определенную логическую комбинацию высказываний $x_{li} j$. б) Обобщите ваше утверждение, составленное при выполнении упр. 7.11, я, и объясните, как следует формировать высказывание CNF с утверждением, что k из p соседних квадратов содержат мины. в) Точно объясните, как агент может использовать алгоритм DPLL для доказательства того, что данный

квадрат содержит (или не содержит мину), игнорируя глобальное ограничение, согласно которому всего имеется точном мин. г) Предположим, что глобальное ограничение конструируется с помощью метода, созданного вами при выполнении упр. 7.11, б. Как зависит количество выражений в этом ограничении от m и n ? Предложите способ осуществления такой модификации алгоритма DPLL, чтобы в нем не требовалось явно представлять это глобальное ограничение.

д) Становятся ли недействительными какие-либо выводы, полученные при создании метода, о котором речь идет в задании 7.11, в, если учитывается это глобальное ограничение? е) Приведите примеры конфигураций проверочных значений, которые порождают такие далеко идущие зависимости, что содержимое любого непроверенного квадрата может предоставить информацию о содержимом какого-то далеко отстоящего от него квадрата. {Подсказка. Рассмотрите доску с размерами $n \times m$.} 7.12. В данном упражнении рассматривается связь между выражениями и импликативными высказываниями. а) Покажите, что выражение $(\neg P_1 \vee \dots \vee \neg P_m \vee Q)$ логически эквивалентно импликативному высказыванию $(P_1 \wedge \dots \wedge P_m) \Rightarrow Q$. б) Покажите, что каждое выражение (независимо от количества положительных литералов) может быть записано в форме $(P_1 \wedge \dots \wedge P_m) \Rightarrow (Q_1 \vee \dots \vee Q_n)$, где P и Q — пропозициональные символы. База знаний, состоящая из таких высказываний, находится в $\wedge$ импликативной нормальной форме, или в форме Ковальского. в) Составьте полное правило резолюции для высказываний в импликативной нормальной форме.

7.13. В этом упражнении речь идет о проектировании агента для мира вампуса на основе логической схемы. а) Запишите уравнение, аналогичное уравнению 7.4, для высказывания Arrow, которое должно быть истинным, если у агента все еще есть стрела. Составьте соответствующую логическую схему. б) Повторите задание 7.13, а для высказывания FacingRight (агент смотрит вправо), используя в качестве образца уравнение 7.5. в) Создайте версии уравнений 7.7 и 7.8 для поиска вампуса и начертите логическую схему.

7.14. Обсудите понятие оптимального поведения в мире вампуса. Покажите, что приведенное в этой главе определение алгоритма PL-Wumpus-Agent не является оптимальным, и предложите способы его усовершенствования.

7.15. (#) Дополните алгоритм PL-Wumpus-Agent таким образом, чтобы он обеспечивал отслеживание всех относящихся к делу фактов с помощью только базы знаний.

7.16. Сколько времени потребуется, чтобы доказать высказывание KB И a с помощью алгоритма DPLL, если a — литерал, который уже содержится в базе знаний кв! Объясните полученные результаты.

7.17. Проследите за поведением алгоритма dpll при обработке приведенной на рис.

7.7 базы знаний в попытке доказать высказывание 0 и сравните это поведение с тем, которое демонстрируется алгоритмом прямого логического вывода.

Глава 8. Логика первого порядка

8 Логика первого ПОРЯДКА В данной главе мы обращаем внимание на то, что мир состоит из множества объектов, причем некоторые из них связаны с другими объектами, а также предпринимаем попытку рассуждать об этих объектах.

В главе 7 было показано, каким образом агент, основанный на знаниях, может представлять мир, в котором он действует, и определять с помощью логического вывода, какие действия следует ему предпринять. В этой главе в качестве языка представления использовалась пропозициональная логика, которая вполне позволяет проиллюстрировать основные понятия логики и принципы функционирования агентов, основанных на знаниях. Но, к сожалению, язык пропозициональной логики слишком слаб для того, чтобы с его помощью можно было кратко представить знания о сложных вариантах среды. В настоящей главе рассматривается ¹ логика первого порядка¹, которая является достаточно выразительной для того, чтобы с ее помощью можно было представить значительную часть наших общих знаний. Кроме того, логика первого порядка либо становится итогом развития, либо образует основу многих других языков представления, и ей посвящены многие десятилетия интенсивных исследований. Начнем изложение этой темы с описания в разделе 8.1 всех языков представления в целом; в разделе 8.2 рассматриваются синтаксис и семантика логики первого порядка; в разделах 8.3 и 8.4 иллюстрируется использование логики первого порядка для простых представлений.

8.1. Дополнительные сведения о представлении В данном разделе рассматривается общий характер языков представления. Здесь будет показано, чем была вызвана необходимость разработки логики первого порядка — гораздо более выразительного языка по сравнению с пропозициональной логикой, представленной в главе 7. Мы рассмотрим язык пропозициональной логики и 1 Логику первого порядка называют также исчислением предикатов первого порядка, и эти области знаний иногда сокращенно обозначают как FOL — First-Order Logic или FOPC — First-Order Predicate Calculus.

языки других типов, чтобы понять, в чем состоят их преимущества и недостатки.

Приведенное здесь описание будет кратким, и в нем результаты многовековых размышлений, проб и ошибок сведутся всего лишь к нескольким абзацам.

Среди наиболее широко распространенных формальных языков наиболее крупным классом являются языки программирования (такие как C++, Java или Lisp).

Сами программы представляют, в прямом смысле этого понятия, только вычислительные процессы, а структуры данных, применяемые в программах, могут представлять факты; например, в программе для представления содержимого мира вампуса может использоваться массив 4x4. Поэтому оператор $\text{World}[2,2] < r\text{-Pi } t$ языка программирования представляет собой довольно естественный способ формирования утверждения о том, что в квадрате [2,2] имеется яма. (Такие способы представления могут рассматриваться как выбранные произвольным образом; системы баз данных были разработаны именно для предоставления более общего способа хранения и выборки фактов, независимого от проблемной области.) В языках программирования недостает какого-то общего механизма логического вывода фактов на основе других фактов; каждое обновление в структуре данных осуществляется с помощью процедуры, характерной для данной проблемной области, подробности устройства которой уточняются программистом на основании его знаний о проблемной области. Такой процедурный подход представляет собой резкий контраст с декларативным характером пропозициональной логики, в которой не смешиваются знания и методы логического вывода, а логический вывод осуществляется полностью независимо от проблемной области.

Еще одним недостатком применения структур данных в программах (а также, в этом отношении, и баз данных) является отсутствие удобного способа сформировать, например, такое утверждение: "В квадрате в [2,2] или [3,1] имеется яма" или "Если вампус находится в квадрате [1,1], то его нет в квадрате [2,2]".

Программы позволяют хранить для каждой переменной единственное значение, а некоторые системы допускают использование "неопределенных" значений, но в них не хватает выразительности, которая требуется для обработки частичной информации.

Пропозициональная логика — это декларативный язык, поскольку ее семантика основана на истинностных отношениях между высказываниями и возможными мирами.

Кроме того, она имеет достаточную выразительную мощь для того, чтобы с ее помощью можно было обрабатывать частично заданную информацию с использованием дизъюнкции и отрицания. Пропозициональная логика обладает еще одним свойством, которое является желательным для языков представления, а именно $\wedge$ композициональностью.

В композициональном языке смысл высказывания представляет собой функцию от смысла его частей. Например, смысл высказывания " $\text{Sli}_4 \text{ л } \text{Sli}_2$ " связан со смыслами высказываний " S1_4 " и " Si_2 ". Было бы очень странным, если бы " $\text{S1}\#4$ " означало, что в квадрате [1,4] чувствуется неприятный запах, " $\text{S};_2$ " — что неприятный запах

чувствуется в квадрате [1,2], а выражение " $S_{1/4}$ л $S_{1/2}$ " означало бы, что квалификационный матч по хоккею с шайбой между Францией и Польшей, проходивший на прошлой неделе, закончился со счетом 1:1. Безусловно, что при отсутствии в языке представления свойства композициональности функционирование системы формирования рассуждений значительно затрудняется. Как было показано в главе 7, пропозициональная логика не обладает достаточной выразительной мощностью, которая позволяла бы кратко описывать среду с многочисленными объектами. Например, мы были вынуждены записывать отдельное правило, которое связывает между собой наличие ветерка и ям для каждого квадрата, например, таким образом: $?i,l \Leftrightarrow (P_{i,2} \vee P_{2,i})$. С другой стороны, на естественном языке, по-видимому, совсем несложно раз и навсегда сформулировать правило: "В квадратах, непосредственно примыкающих к ямам, чувствуется ветерок". Синтаксис и семантика естественного языка каким-то образом позволяют кратко описать эту среду.

Даже не очень глубокие размышления наводят на мысль, что естественные языки (такие как английский или русский) действительно являются очень выразительными. Например, авторы смогли написать почти всю эту книгу на естественном языке, лишь время от времени прибегая к использованию формальных языков (включая логику, математику и язык схем). В лингвистике и философии языка давно существует традиция, в которой естественный язык по сути рассматривается как декларативный язык представления знаний и предпринимаются попытки однозначно определить его формальную семантику. Если бы такая программа исследований завершилась успехом, это имело бы большое значение для искусственного интеллекта, поскольку позволило бы непосредственно использовать естественный язык (или какую-то его производную) в системах представления и формирования рассуждений.

В соответствии с современными взглядами естественный язык выполняет немного иное назначение: служит в качестве средства общения, а не исключительно как средство представления. Когда один приятель указывает пальцем и говорит другому: "Смотри!", он хочет этим сказать, что, допустим, в фильме Супермен наконец-то взлетел над крышами. Тем не менее нельзя утверждать, что в предложении "Смотри!" закодирован именно этот факт. Вместо этого смысл предложения на естественном языке зависит и от самого предложения, и от контекста, в котором оно было высказано. Очевидно, что невозможно сохранить такое предложение, как "Смотри!", в базе знаний и надеяться на то, что удастся восстановить его смысл, если в базе знаний не будет также представлен контекст. А из этого следует вопрос — как же можно представить сам контекст? К тому же естественные языки являются некомпозиционными — смысл предложения, такого как "Потом она это увидела", может зависеть от контекста, состоящего из многих предшествующих и последующих предложений. Наконец, недостатком естественных языков является

неоднозначность, которая может стать причиной затруднений при формировании рассуждений. Например, Пинкер [1212] выразил эту мысль таким образом: "Когда люди говорят о косе, то их, безусловно, не затрудняет определение того, идет ли речь об элементе женской прически, или о форме береговой полосы, или о чем-то другом, а поскольку одно и то же слово может соответствовать двум идеям, сами идеи не могут выражаться отдельными словами".

ЯЗЫК МЫШЛЕНИЯ Философы и психологи уже в течение долгих столетий размышляют над тем, как люди и другие животные представляют знания. Очевидно, что в развитии этой способности у людей важную роль играла эволюция естественного языка. С другой стороны, многие психологические факты свидетельствуют о

том, что люди не используют язык непосредственно в своих внутренних представлениях. Например, кто сейчас может вспомнить, с какой именно из этих двух фраз начинался раздел 8.1? • "В данном разделе рассматривается общий характер языков представления ..." • "В данном разделе рассматривается тема, касающаяся языков представления знаний ..."

Ваннер [1553] обнаружил, что участники его экспериментов делали правильный выбор в подобных тестах на уровне случайности (выбирали правильный вариант примерно в 50% попыток), но помнили содержание того, что было ими прочитано с точностью больше чем 90%. Эти данные свидетельствуют о том, что люди обрабатывают слова, а затем формируют своего рода несловесное представление, которое мы называем памятью.

Вопрос о том, какими именно являются механизмы, с помощью которых люди используют язык для обеспечения и формирования представлений различных идей, продолжает привлекать значительный интерес. В знаменитой гипотезе Сапира-Уорфа утверждается, что язык, на котором мы говорим, оказывает глубокое влияние на тот способ, с помощью которого мы мыслим и принимаем решения, в частности, используемый язык влияет на формирование структуры категорий, с помощью которых мы подразделяем мир на объекты разных типов. Уорф [1585] утверждал, что эскимосы имеют в своем языке много слов для описания снега и поэтому снег вызывает у них более разнообразные восприятия по сравнению с людьми, говорящими на других языках.

Некоторые лингвисты оспаривали фактографическую основу для этого утверждения (например, Паллем [1242] показал, что в таких эскимосских языках, как юпик и инупик, и в других родственных языках северных народов, повидимому, имеется такое же количество слов для выражения понятий, касающихся снега, как и в английском языке), а другие выступили в его поддержку [485]. Однако, по-видимому, нельзя оспаривать ту мысль, что популяции, имеющие более глубокое знакомство с

некоторыми аспектами мира, разрабатывают гораздо более подробные словари для описания этих аспектов.

Например, энтомологи, проводящие полевые исследования, подразделяют то, что большинство из нас называет просто жуками, на сотни тысяч видов и лично знакомы с многими из них. (Биолог Дж.Б.С. Холдейн, изучающий эволюцию насекомых, когда-то пожаловался на то, что Создатель проявляет "невероятную заботу о жуках".) Более того, множеством терминов для обозначения снега пользуются и опытные лыжники (различая такие его состояния, как пудра, каша, пюре, простокваша, крупа, цемент, наст, сахар, асфальт, вельвет, пух, жижа и т.д.), которые в основном не известны обычному человеку. Однако остается неясным направление развития причинноследственных отношений — узнают ли лыжники об этих различиях, только изучая свои профессиональные термины, или эти различия выявляются из индивидуального опыта и становятся согласованными с теми их обозначениями, которые в данное время приняты в этом сообществе спортсменов?

Данный вопрос становится очень важным в исследованиях развития детей. До сих пор мы слишком мало знаем о том, насколько связаны друг с другом обучение языку и обучение мышлению. Например, позволяют ли знания о том,

как именуется некоторое понятие, такое как бакалавр, упростить формирование идей и размышление с помощью более сложных понятий, которые включают имя исходного понятия, например претендент на звание бакалавра!

Принятый авторами подход состоит в том, что в качестве основы следует выбрать пропозициональную логику, обладающую декларативной, композиционной семантикой, которая является независимой от контекста и непротиворечивой, и создать на этой основе более выразительную логику, заимствуя идеи представления из естественного языка и избегая вместе с тем его недостатков. Изучение синтаксиса естественного языка показывает, что наиболее очевидными его элементами являются существительные и именные конструкции, которые обозначают $\wedge$ объекты (квадраты, ямы, вампусы), а также глаголы и глагольные конструкции, которые обозначают $\wedge$ отношения между объектами (чувствовать ветерок, быть соседним, стрелять). Некоторые из этих отношений являются $\wedge$ функциями — отношениями, в которых в ответ на каждое "входное" значение формируется только одно "выходное" значение. Такой подход позволяет сразу же приступить к составлению перечня примеров объектов, отношений и функций, как показано ниже.

- Объекты: люди, дома, числа, теории, Рональд Макдональд, цвета, бейсбольные соревнования, войны, столетия ...
- Отношения: могут быть унарными отношениями, или $\wedge$ свойствами, такими как красный, круглый, поддельный, первичный, многоэтажный и т.д., либо более общими n-арными отношениями, такими как "быть братьями", "быть больше", "находиться внутри", "входить в состав", "иметь цвет", "произойти позже", "принадлежать", "находиться между" и т.д.
- Функции: "быть отцом", "быть

лучшим другом", "быть третьей подачей мяча", "быть на единицу больше", "быть началом" и т.д.

Безусловно, почти каждое утверждение может рассматриваться как обозначающее объекты, а также их свойства или отношения. Ниже приведены некоторые примеры.

- "Один плюс два равняется трем".

Объекты: один, два, три, один плюс два; отношение: равняется; функция: плюс. ("Один плюс два" представляет собой название объекта, полученного путем применения функции "плюс" к объектам "один" и "два". "Три" — другое название для этого объекта.) • "В квадратах, соседних с тем квадратом, где находится вампус, чувствуется неприятный запах".

Объекты: вампус, квадраты; свойство: неприятный запах; отношение: быть соседним.

- "Злой король Джон управлял Англией в 1200 году".

Объекты: Джон, Англия, 1200 год; отношение: управлял; свойства: злой, король.

Язык логики первого порядка, синтаксис и семантику которого мы определим в следующем разделе, основан на понятиях объектов и отношений. Он стал чрезвычайно важным для математики, философии и искусственного интеллекта именно потому, что эти области знаний (а фактически основная часть повседневного человеческого существования) могут вполне продуктивно рассматриваться как касающиеся объектов

и отношений между ними. Логика первого порядка позволяет также выражать факты о некоторых или обо всех объектах во Вселенной. Это дает возможность представлять общие законы, или правила, такие как следующее утверждение: "В квадратах, соседних с тем квадратом, где находится вампус, чувствуется неприятный запах".

Основное различие между пропозициональной логикой и логикой первого порядка заключается в том, что каждый из этих языков вносит различный $\wedge$ онтологический вклад в описание действительности, т.е. они по-разному представляют характер действительности. Например, в пропозициональной логике предполагается, что существуют лишь факты, которые относятся или не относятся к данному миру. Каждый факт может находиться в одном из двух состояний: быть истинным или ложным². В логике первого порядка приняты более широкие предположения, а именно, что мир состоит из объектов, между которыми могут быть или не быть некоторые отношения. Некоторые варианты логики специального назначения позволяют внести еще больший онтологический вклад; например, во $\wedge$ временной логике предполагается, что факты имеют место в конкретные интервалы времени и что эти интервалы (которые могут рассматриваться как бесконечно малые или конечные) являются упорядоченными.

Поэтому в вариантах логики специального назначения придается первоклассный статус некоторым видам объектов особого рода (и аксиомам об этих объектах), а не просто вводятся в базу знаний их определения. В $\wedge$ логике высокого порядка как объекты трактуются сами отношения и функции, рассматриваемые в логике первого порядка. Это позволяет формировать утверждения обо всех отношениях, например, если потребуется точно определить, что означает понятие транзитивного отношения. В отличие от большинства вариантов логики специального назначения, логика высокого порядка является строго более выразительной, чем логика первого порядка, в том смысле, что некоторые высказывания логики высокого порядка не могут быть выражены с помощью любого конечного количества высказываний логики первого порядка.

Логику можно также охарактеризовать по ее $\wedge$ эпистемологическому вкладу в познание; под этим подразумеваются возможные состояния знаний, которые она позволяет выразить в отношении каждого факта. И в пропозициональной логике, и в логике первого порядка любое высказывание представляет собой факт, и агент либо доверяет утверждению, что это высказывание истинно, либо доверяет утверждению, что оно ложно, либо не имеет мнения на этот счет. Поэтому в таких вариантах логики имеются три возможных состояния знаний, касающихся любого высказывания.

С другой стороны, в тех системах, где используется теория вероятностей, может рассматриваться любая степень доверия³, начиная от 0 (полное недоверие) и заканчивая 1 (полное доверие). Например, в вероятностном мире вампуса агент может доверять утверждению о том, что вампус находится в квадрате [1,3], с вероятностью 0,75. Онтологический и эпистемологический вклады пяти различных вариантов логики показаны в табл. 8.1.

2 В отличие от этого, в нечеткой логике факты имеют определенную степень истинности от 0 до 1. Например, в рассматриваемом мире высказывание "Вена — большой город" может считаться истинным только со степенью 0.6.

3 Важно не путать степень доверия в теории вероятностей со степенью истинности в нечеткой логике. В действительности в некоторых системах нечеткой логики допускается выражать степень недоверия (или степень доверия) в отношении степеней истинности.

Таблица 8.1. Формальные языки и показатели их онтологического и эпистемологического вкладов в познание

Язык	Онтологический вклад (что существует в мире)	Эпистемологический вклад (какую степень доверия может выражать агент в отношении фактов)
Пропозициональная логика	Факты	Факты, объекты, отношения
Первый порядок логики	Факты, объекты, отношения	Факты, объекты, отношения, интервалы времени
Теория вероятностей	Факты	Факты со
Нечеткая логика	Факты, объекты, отношения	Факты со

степень истинности G $[0,1]$ Истинно/ложно/неизвестно
Истинно/ложно/неизвестно Истинно/ложно/неизвестно Степень доверия e $[0,1]$
Известное интервальное значение B

В следующем разделе мы приступим к изучению подробных сведений о логике первого порядка. Так же как студенту-физику требуется определенное знакомство с высшей математикой, так и студент, изучающий искусственный интеллект, должен развить у себя способность работать с логическими обозначениями. С другой стороны, важно также не слишком увлекаться изучением специфики отдельной логической системы обозначений, поскольку в конечном итоге количество различных версий таких систем исчисляется десятками. Главное, не упустить из виду то, благодаря чему данный язык обеспечивает формирование кратких представлений и каким образом его семантика приводит к созданию непротиворечивых процедур формирования рассуждений.

8.2. Синтаксис и семантика логики первого порядка Начнем изложение темы этого раздела с более точного определения того способа, с помощью которого возможные миры логики первого порядка отражают ее онтологический вклад в познание объектов и отношений. Затем опишем различные элементы языка этой логики, в ходе этого объясняя их семантику.

Модели для логики первого порядка Как было указано в главе 7, моделями для любого логического языка служат формальные структуры, из которых состоят возможные рассматриваемые миры. Модели для пропозициональной логики представляют собой множества истинностных значений для пропозициональных символов. Модели для логики первого порядка являются более интересными. Прежде всего, в них имеются объекты! $\wedge$ Проблемной областью модели является множество объектов, которые она содержит; эти объекты иногда называют $\wedge$ элементами проблемной области. На рис. 8.1 показана модель с пятью объектами: Ричард Львиное Сердце, король Англии, который правил с 1189 по 1199 годы; его младший брат, злой король Джон, который правил с 1199 по 1215; левые ноги Ричарда и Джона; корона.

корона лицо король Рис. 8.1. Модель, состоящая из пяти объектов, двух бинарных отношений, трех унарных отношений (обозначенных метками на объектах) и одной унарной функции, "быть левой ногой"

Отношения между объектами в этой модели могут быть определены различными способами. На данном рисунке показано, что Ричард и Джон — братья. Выражаясь формально, отношение — это просто множество $\wedge$ кортежей объектов, связанных друг с другом. (Кортеж — это коллекция объектов, расположенных в установленном порядке и записанных в угловых скобках, окружающих эти объекты.) Таким образом, отношение, обозначающее родство на уровне братьев в этой модели, представляет собой следующее множество: $\{ \langle \text{Ричард Львиное Сердце, король Джон} \rangle, \langle \text{король}$

Джон, Ричард Львиное Сердце>} (8.1) (Здесь эти объекты были указаны на естественном языке, но читатель при желании может мысленно вставить вместо имен этих королей их портреты.) Корона находится на голове короля Джона, поэтому отношение "быть на голове" содержит только один кортеж, <корона, король Джон>. Отношения "быть братом" и "быть на голове" являются бинарными, т.е. они устанавливают связь между парами объектов.

Кроме того, эта модель содержит унарные отношения, или свойства: свойство "быть человеком" является истинным и для Ричарда, и для Джона; свойство "быть королем" истинно только для Джона (предположительно потому, что к этому моменту Ричард уже был мертв); а свойство "быть короной" истинно только для короны.

Некоторые виды связей удобнее рассматривать как функции, в том смысле, что указанным образом данный конкретный объект должен быть связан только с одним объектом. Например, каждый человек имеет только одну левую ногу, поэтому в данной модели имеется унарная функция "быть левой ногой", которая включает следующие отображения: <Ричард Львиное Сердце> —> левая нога Ричарда <король Джон> —> левая нога Джона (8.2)

Строго говоря, для модели в логике первого порядка требуются $\wedge$ полностью определенные функции, т.е. функции, в которых должно быть предусмотрено значение для каждого входного кортежа. Таким образом, левую ногу должна иметь и корона, а также, безусловно, каждая из левых ног. Предусмотрено некоторое формальное решение этой неприятной проблемы, возникающей из-за того, что каждый объект, который в действительности не имеет левую ногу, включая саму левую ногу, в результате применения функции получает дополнительный "невидимый" объект, представляющий собой левую ногу. К счастью, при условии, что никто не будет составлять утверждений о левых ногах объектов, не имеющих левой ноги, применять эти формальные решения не обязательно.

Символы и интерпретации Вернемся к синтаксису языка. Нетерпеливый читатель может найти полное описание формальной грамматики логики первого порядка в листинге 8.1.

Листинг 8.1. Синтаксис логики первого порядка с оператором равенства, заданный в форме Бэкуса—Наура. (Для ознакомления с этой системой обозначений см. с. 1297.) В этом синтаксисе предусмотрены строгие правила применения круглых скобок; примечания, касающиеся круглых скобок и предшествования операторов, приведенные на с. 296, равным образом относятся и к логике первого порядка

$$\text{Sentence} \rightarrow \text{AtomicSentence} \mid (\text{Sentence} \text{ Connective } \text{Sentence}) \mid \text{Quantifier Variable, ... Sentence} \mid \text{---}(\text{Sentence} \text{ AtomicSentence} \rightarrow \text{Predicate } \{ \text{Term, ...} \} \mid \text{Term} = \text{Term} \text{ Term} \rightarrow$$

Function (Term, ...) | Constant | Variable
Connective $\rightarrow$ $=>$ | a | v | $<=>$ Quantifier $\rightarrow$ V
| $\exists$ Constant $\rightarrow$ A | $X \setminus$ | John | ...

Variable $\rightarrow$ a | $x \setminus s$ | ... Predicate $\rightarrow$ Before \ HasColor \ Raining | ...

Function $\rightarrow$ Mother | LeftLeg | ... Основными синтаксическими элементами логики первого порядка являются символы, которые обозначают объекты, отношения и функции. Поэтому сами символы подразделяются на три типа: $\wedge$ константные символы, которые обозначают объекты; $\wedge_k$ предикатные символы, которые обозначают отношения, и $\wedge_f$ функциональные символы, которые обозначают функции. Примем соглашение, что имена этих символов будут начинаться с прописных букв. Например, могут использоваться константные символы Richard и John; предикатные символы Brother, OnHead, Person, King и Crown и функциональный символ LeftLeg. Как и применительно к пропозициональным символам, выбор имен этих символов полностью предоставляется пользова-

телю. Каждый предикатный и функциональный символ характеризуется $\wedge$ арностью, которая определяет количество формальных параметров.

Семантика должна связывать высказывания с моделями, для того чтобы можно было определить истинность. Чтобы иметь возможность решить такую задачу, требуется $\wedge$ интерпретация, которая определяет, на какие именно объекты, отношения и функции ссылаются те или иные константные, предикатные и функциональные символы. Одна из возможных интерпретаций для рассматриваемого примера (которую мы будем называть $\wedge$ намеченной интерпретацией) состоит в следующем:

- Символ Richard обозначает Ричарда Львиное Сердце, а символ John — злого короля Джона.
- Символ Brother обозначает отношение родства между братьями, т.е. множество кортежей объектов, приведенное в уравнении 8.1; символ OnHead обозначает отношение "быть на голове", которое установлено между короной и королем Джоном; символы Person, King и Crown относятся к множествам объектов, представляющих собой людей, королей и короны.
- Символ LeftLeg относится к функции "быть левой ногой", т.е. к отображению, приведенному в уравнении 8.2.

Может быть также много других возможных интерпретаций, связывающих эти символы с данной конкретной моделью. Например, одна интерпретация отображает символ Richard на корону, а символ John — на левую ногу короля Джона. В этой модели имеется пять объектов, поэтому существует 25 возможных интерпретаций только для константных символов Richard и John. Обратите внимание на то, что не все объекты имеют имя, например, в данной намеченной интерпретации не предусмотрены имена для короны или для ног. Возможно также, чтобы один объект имел несколько имен; примером такой интерпретации была бы интерпретация, в которой и символ Richard, и символ John относились бы к короне. Если читатель находит, что такая возможность приводит к путанице, напомним, что

в пропозициональной логике вполне допустимо иметь модель, в которой высказывания Cloudy (пасмурно) и Sunny (солнечно) одновременно являются истинными; задача исключения из рассмотрения моделей, несовместимых с нашими знаниями, возлагается на базу знаний.

Истинность любого высказывания определяется с помощью некоторой модели и некоторой интерпретации символов этого высказывания. Поэтому логическое следствие, допустимость и другие свойства высказываний определяются в терминах всех возможных моделей и всех возможных интерпретаций. Важно отметить, что количество элементов проблемной области в каждой модели может быть неограниченным, например, элементами проблемной области могут быть целые числа или действительные числа. Поэтому не ограничено количество возможных моделей, как и количество интерпретаций. Проверка логического следствия путем перебора всех возможных моделей, которая была осуществимой в пропозициональной логике, в логике первого порядка больше не может применяться. Даже если количество рассматриваемых объектов ограничено, количество их комбинаций может быть очень большим. Например, при использовании символов, рассматриваемых в данном примере, существует приблизительно 1025 комбинаций для проблемной области с пятью объектами (см. упр. 8.5).

Термы $\wedge$ < Терм — это логическое выражение, которое относится к некоторому объекту.

Поэтому константные символы также представляют собой термы, но не всегда удобно иметь отдельный символ для каждого отдельного объекта. Например, в естественном языке можно воспользоваться обозначением "левая нога короля Джона", а не присваивать ноге короля имя. Именно для этого и нужны функциональные символы: вместо использования константного символа мы можем написать LeftLeg(John). В общем случае сложный терм формируется с помощью функционального символа, за которым следует заключенный в круглые скобки список формальных параметров данного функционального символа. Важно помнить, что любой сложный терм — это некоторое подразумеваемое имя, выраженное в сложной форме. Это — не "вызов процедуры", которая "возвращает значение". Не существует такой процедуры LeftLeg, которая принимает на входе какого-то человека и возвращает его левую ногу. Мы можем рассуждать о левых ногах (например, формулировать общее правило, что каждый человек имеет таковую, и на основании этого делать вывод, что таковую должен иметь Джон), даже не предусматривая определение символа LeftLeg. Это — такая логическая операция, которая не может быть выполнена с помощью процедур в языках программирования⁴.

Формальное определение семантики термов является несложным. Рассмотрим терм $f(t_1, \dots, t_n)$. Функциональный символ f относится к некоторой функции в модели (назовем ее F); термы с обозначением формальных параметров относятся к объектам данной проблемной области (назовем их $d_1, \dots, d_n$); а сам терм в целом относится к объекту, представляющему собой значение функции F , применяемой к объектам $d_1, \dots, d_n$. Например, предположим, что функциональный символ `LeftLeg` относится к функции, показанной в уравнении 8.2, а символ `John` относится к королю Джону; в таком случае `LeftLeg(John)` относится к левой ноге короля Джона. Таким образом, интерпретация определяет для каждого терма соответствующий референт (объект, к которому относится данный терм).

Атомарные высказывания Как было указано выше, термы позволяют ссылаться на объекты, предикатные символы — на отношения, а при их совместном использовании формируются атомарные высказывания, позволяющие констатировать факты. Атомарное высказывание состоит из предикатного символа, за которым следует заключенный в круглые скобки список термов:

`Brother(Richard, John)` 4 X-выражения представляют собой удобные обозначения, которые дают возможность формировать новые функциональные символы "динамически". Например, функция, которая формирует квадрат своего формального параметра, может записываться как $(\lambda x \text{ xxx})$ и применяться к формальным параметрам точно так же, как и любой другой функциональный символ. X-выражение можно также определить и использовать как предикатный символ (см. главу 22). Точно такую же роль играет оператор `lambda` на языке Lisp. Следует отметить, что использование X-выражений в такой форме формально не увеличивает выразительную мощь логики первого порядка, поскольку любое высказывание, которое включает λ -выражение, может быть преобразовано путем "вставки" в него соответствующих формальных параметров для получения эквивалентного высказывания.

В соответствии с намеченной интерпретацией, приведенной выше, это атомарное высказывание констатирует тот факт, что Ричард Львиное Сердце — брат короля Джона⁵. Атомарные высказывания могут включать в качестве фактических параметров сложные термы. Поэтому в высказывании `Married(Father(Richard), Mother(John))` утверждается, что отец Ричарда Львиное Сердце был женат на матери короля Джона (опять-таки при использовании подходящей интерпретации).

Любое $s^{\wedge}$ атомарное высказывание является истинным в данной конкретной модели при данной конкретной интерпретации, если отношение, на которое ссылается его предикатный символ, соблюдается среди объектов, на которые сыплются его параметры.

Сложные высказывания Для формирования более сложных высказываний, как и в пропозициональном исчислении, могут использоваться логические связки. Семантика высказываний, сформированных с помощью логических связок, идентична семантике, которая рассматривается в пропозициональной логике. Ниже приведены четыре высказывания, которые являются истинными в модели, показанной на рис. 8.1, при использовании рассматриваемой намеченной интерпретации — $\wedge \text{Brother}(\text{LeftLeg}(\text{Richard}), \text{John}) \text{Brother}(\text{Richard}, \text{John}) \wedge \text{Brother}(\text{John}, \text{Richard}) \text{King}(\text{Richard}) \vee \text{King}(\text{John}) \neg \text{King}(\text{Richard}) \Rightarrow \text{King}(\text{John})$ Кванторы После определения логики, которая допускает использование объектов, становится вполне естественным стремление к созданию средств, позволяющих выражать свойства целых коллекций объектов, а не перебирать эти объекты по именам. Это позволяют сделать $\wedge$ кванторы. Логика первого порядка включает два стандартных квантора, называемых кванторами всеобщности и существования.

Применение квантора всеобщности (V) Напомним, с какими трудностями мы сталкивались в главе 7, пытаясь выразить общие правила в пропозициональной логике. С другой стороны, в логике первого порядка квинтэссенцией становятся такие правила, как: "В квадратах, соседних с тем квадратом, где находится вампус, чувствуется неприятный запах" и "Все короли являются людьми". Первое из этих правил будет рассматриваться в разделе 8.3, а второе правило, "Все короли являются людьми", записывается в логике первого порядка следующим образом:

$\forall x \text{King}(x) \Rightarrow \text{Person}(x)$ Квантор V обычно произносится как "для всех ...". (Следует помнить, что перевернутая буква "A" обозначает "all" — все.) Таким образом, в этом высказывании утверждается следующее: "Для всех x, если x — король, то x — человек". Символ x на 5 Как правило, мы будем придерживаться такого соглашения об упорядочении параметров, что $P(x, y)$ интерпретируется как "x представляет собой P от y".

зывается $\wedge$ переменной. В соответствии с общепринятым соглашением в качестве переменных применяются строчные буквы. Переменная сама является термом и как таковая может также служить параметром функции, например $\text{LeftLeg}(x)$. Терм без переменных называется $\wedge$ базовым термом.

Интуитивно ясно, что в высказывании $\forall x P$, где P — любое логическое выражение, утверждается, что P является истинным для каждого объекта x. Точнее, высказывание $\forall x P$ истинно в данной модели при данной интерпретации, если выражение P истинно при всех возможных $\wedge$ расширенных интерпретациях, сформированных из данной интерпретации, где каждая расширенная интерпретация задает элемент проблемной области, на которую ссылается объект x На первый взгляд такое определение может показаться сложным, но оно фактически представляет собой лишь формальное определение интуитивного смысла применения квантора всеобщности. Рассмотрим модель, показанную на рис.

8.1, и намеченную интерпретацию, которая ее сопровождает. Эту интерпретацию можно расширить следующими пятью способами: $x \rightarrow$ Ричард Львиное Сердце $x \rightarrow$ король Джон $x \rightarrow$ левая нога Ричарда $x \rightarrow$ левая нога Джона $x \rightarrow$ корона. Высказывание с квантором всеобщности $\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$ является истинным при первоначальной интерпретации, если высказывание $\text{King}(x) \Rightarrow \text{Person}(x)$ истинно в каждой из пяти расширенных интерпретаций. Это означает, что данное высказывание с квантором всеобщности эквивалентно утверждению об истинности следующих пяти высказываний:

Ричард Львиное Сердце — король $\Rightarrow$ Ричард Львиное Сердце — человек король Джон — король $\Rightarrow$ король Джон — человек левая нога Ричарда — король $\Rightarrow$ левая нога Ричарда — человек левая нога Джона — король $\Rightarrow$ левая нога Джона — человек корона — король $\Rightarrow$ корона — человек. Рассмотрим внимательно это множество утверждений. Поскольку в нашей модели единственным королем является король Джон, то во втором высказывании утверждается, что он — человек, как и следовало ожидать. А что же можно сказать об остальных четырех высказываниях, в частности о тех, в которых приведены утверждения о ногах и коронах? Являются ли они частью смысла утверждения "Все короли являются людьми"? Действительно, остальные четыре утверждения истинны в данной модели, но не позволяют ничего судить о том, можно ли считать людьми ноги, короны или даже Ричарда. Это связано с тем, что ни один из этих объектов не является королем. Рассматривая истинностную таблицу для связки $\Rightarrow$ (см. табл. 7.1), можно убедиться в том, что импликация истинна, даже если ее предпосылка ложна, независимо от того, является ли истинным заключение. Таким образом, утверждая истинность высказывания с квантором всеобщности, что эквивалентно утверждению об истинности целого списка отдельных импликаций, мы в конечном итоге утверждаем об истинности правила, выраженного в виде этого высказывания, только для тех объектов, для которых предпосылка является истинной, и вообще ничего не говорим о тех объектах, для которых предпосылка ложна. Поэтому, как оказалось, записи истинностной таблицы, относящиеся к связке $\Rightarrow$, являются идеальным способом формулировки общих правил с кванторами всеобщности.

Распространенная ошибка, которую часто допускают даже внимательные читатели, которые прочли предыдущий абзац несколько раз, состоит в том, что они используют конъюнкцию вместо импликации. Тогда следующее высказывание:

$\forall x \text{ King}(x) \wedge \text{Person}(x)$ становится эквивалентным таким утверждениям:

Ричард Львиное Сердце — король л Ричард Львиное Сердце — человек король Джон — король л король Джон — человек левая нога Ричарда — король л левая нога Ричарда — человек и т.д. Очевидно, что такой ряд утверждений не передает желаемый смысл.

Применение квантора существования С) Квантор всеобщности позволяет формировать утверждения о каждом объекте.

Аналогичным образом, мы можем формировать утверждение о некотором объекте во вселенной без его именованности с помощью квантора существования. Например, чтобы выразить мысль, что на голову короля Джона возложена корона, можно записать следующее:

$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$ Квантор существования с переменной $\exists x$ читается как: "Существует x , такой, что ...", или "Для некоторого x ...".

Интуитивно ясно, что в высказывании $\exists x P$ утверждается, будто выражение P истинно по меньшей мере для одного объекта x . Точнее, высказывание $\exists x P$ истинно в данной конкретной модели при данной конкретной интерпретации, если выражение P истинно по меньшей мере в одной расширенной интерпретации, в которой присваивается x одному из элементов проблемной области. В данном примере это означает, что должно быть истинным по меньшей мере одно из приведенных ниже утверждений.

Ричард Львиное Сердце — корона $\wedge$ Ричард Львиное Сердце находится на голове Джона
король Джон — корона $\wedge$ король Джон находится на голове Джона
левая нога Ричарда — корона $\wedge$ левая нога Ричарда находится на голове Джона
левая нога Джона — корона $\wedge$ левая нога Джона находится на голове Джона
корона — корона $\wedge$ корона находится на голове Джона
В рассматриваемой модели истинно пятое утверждение, поэтому в ней является истинным само первоначальное утверждение с квантором существования. Обратите внимание на то, что в соответствии с приведенным выше определением квантора существования это высказывание будет также истинным и в такой модели, в которой на короля Джона возложены две короны. Такая ситуация является полностью совместимой с первоначальным высказываниемб: "На голову короля Джона возложена корона".

6 Применяется также определенный вариант квантора существования, обычно записываемый как $\exists!$ или $\exists^1$, который означает: "Существует только один...". Как будет показано в разделе 8.2, тот же смысл можно выразить с использованием утверждений, содержащих знак равенства.

Итак, логическая связка $\Rightarrow$ может рассматриваться как наиболее подходящая для использования с квантором $\forall$, а логическая связка $\wedge$ естественным образом подходит для использования с квантором $\exists$. В примере, который рассматривался в предыдущем разделе, применение $\wedge$ в качестве основной связки в сочетании с квантором $\forall$ приводило к формированию слишком сильного утверждения, а использование связки $\Rightarrow$ в сочетании с квантором $\exists$ обычно приводит к формированию действительно очень слабых утверждений. Рассмотрим следующее высказывание:

$\exists x \text{ Crown}(x) \Rightarrow \text{OnHead}(x, \text{John})$ На первый взгляд может показаться, что в этом высказывании вполне успешно передана мысль о том, что на голову короля Джона возложена корона. Применяя соответствующее определение семантики, можно убедиться в том, что данное высказывание декларирует истинность по меньшей мере одного из следующих утверждений:

Ричард Львиное Сердце — корона $\Rightarrow$ Ричард Львиное Сердце находится на голове Джона
король Джон — корона $\Rightarrow$ король Джон находится на голове Джона
левая нога Ричарда — корона $\Rightarrow$ левая нога Ричарда находится на голове Джона и т.д.
Итак, импликация истинна, если и предпосылка, и заключение являются истинными, или если ложна ее предпосылка. Поэтому, если Ричард Львиное Сердце — не корона, то первое утверждение истинно и выполняется высказывание с квантором существования. Таким образом, высказывание в форме импликации с квантором существования истинно в любой модели, содержащей объект, для которого предпосылка импликации является ложной, поэтому подобные высказывания фактически не несут почти никакой информации.

Вложенные кванторы Часто возникает необходимость сформировать более сложные высказывания с использованием нескольких кванторов. Простейшим является случай, когда кванторы относятся к одному и тому же типу. Например, утверждение: "Братья — это люди, связанные братскими родственными узами", может быть записано следующим образом:

$\forall x \forall y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$ Последовательно применяемые кванторы могут быть записаны как один квантор с несколькими переменными. Например, чтобы выразить мысль о том, что родственные отношения между людьми, связанными братскими узами, являются симметричным, можно составить следующее высказывание:

$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow \text{Sibling}(y, x)$ В других случаях возникает необходимость в использовании сочетания разных кванторов. Например, строка из песни "Everybody loves somebody" (Каждый кого-то любит) означает, что для каждого человека существует кто-то, кого этот человек любит:

$\forall x \exists y \text{ Loves}(x, y)$ С другой стороны, чтобы сформулировать утверждение "Есть некто, кого любят все", можно записать следующее:

$\exists y \forall x \text{ Loves}(x, y)$

Таким образом, порядок расположения кванторов очень важен. Он становится очевиднее после вставки круглых скобок. В высказывании $\forall x (\exists y \text{ Loves}(x, y))$ утверждается, что каждый имеет конкретное свойство, а именно то свойство, что его кто-то любит. С другой стороны, в высказывании $\exists x (\forall y \text{ Loves}(x, y))$ утверждается,

что некто в мире имеет конкретное свойство, а именно свойство быть любимым всеми.

Если два квантора используются с одним и тем же именем переменной, может возникнуть некоторая путаница. Рассмотрим следующее высказывание:

$\forall x [\text{Crown}(x) \vee (\exists x \text{ Brother}(\text{Richard}, x))]$ Здесь к переменной x в атомарном высказывании $\text{Brother}(\text{Richard}, x)$ применяется квантор существования. Общее правило состоит в том, что переменная принадлежит к самому внутреннему квантору, в котором она упоминается; это означает, что такая переменная не может стать субъектом действия любого другого квантора⁷. Еще один способ анализа приведенного выше высказывания состоит в следующем: $\exists x \text{ Brother}(\text{Richard}, x)$ — это высказывание о Ричарде (о том, что у него есть брат), а не о переменной x , поэтому размещение квантора $\forall x$ за пределами данного высказывания не оказывает на него никакого действия, и оно могло быть равным образом записано как $\exists z \text{ Brother}(\text{Richard}, z)$. Но поскольку такая ситуация может стать источником путаницы, в подобных обстоятельствах мы всегда будем использовать разные переменные.

Связь между кванторами $\forall$ и $\exists$ Кванторы $\forall$ и $\exists$ фактически тесно связаны друг с другом через отрицание.

Утверждение о том, что никто не любит пастернак, равносильно утверждению о том, что не существует никого, кто бы его любил, и наоборот:

$\forall x \neg \text{Likes}(x, \text{Parsnips})$ эквивалентно высказыванию $\neg \exists x \text{ Likes}(x, \text{Parsnips})$ По такому же принципу может быть сформирована более сложная конструкция; например, выражение "Все любят мороженое" означает, что нет никого, кто не любил бы мороженое:

$\forall x \text{ Likes}(x, \text{Icecream})$ эквивалентно высказыванию $\neg \exists x \neg \text{Likes}(x, \text{Icecream})$ Поскольку квантор $\forall$ фактически определяет в универсуме объектов конъюнкцию, а квантор $\exists$ определяет дизъюнкцию, нет ничего удивительного в том, что они подчиняются правилам де Моргана. Правила де Моргана для высказываний с кванторами и без кванторов приведены ниже.

$\forall x \neg P \equiv \neg \exists x P$ $\neg \forall x P \equiv \exists x \neg P$ $\neg \exists x P \equiv \forall x \neg P$ $\neg \neg(P \vee Q) \equiv P \vee Q$ Тем не менее сохраняется возможность обеспечить взаимодействие кванторов с использованием переменной с одним и тем же именем, которая послужила побудительной причиной разработки несколько замысловатого механизма применения расширенных интерпретаций в семантике высказываний с кванторами. В данном примере более интуитивно очевидный подход, при котором выполняется подстановка объектов вместо каждого вхождения переменной x , становится неосуществимым, поскольку значение переменной x в атомарном высказывании $\text{Brother}(\text{Richard}, x)$ в результате такой подстановки будет "закреплено". Расширенные интерпретации позволяют справиться с этой ситуацией

правильно, поскольку присваивание значения переменной x во внутреннем кванторе перекрывает присваивание, выполняемое во внешнем кванторе.

$\rightarrow \forall x P = \exists x \neg (P \wedge Q) = \neg P \vee \neg Q \forall x P = \neg \exists x \neg P \wedge 0 = \neg (\neg P \vee \neg 0) \exists x P \equiv \neg \forall x \neg P \vee 0 = \neg (\neg P \wedge \neg 0)$ Таким образом, в действительности нет необходимости иметь одновременно кванторы $\forall$ и $\exists$, так же как фактически не нужны обе связки $\wedge$ и $\vee$. Тем не менее удобство для чтения важнее, чем экономия выразительных средств, поэтому мы будем пользоваться обоими этими кванторами.

Равенство в логике первого порядка предусмотрен еще один способ составления атомарных высказываний, отличный от использования предикатных символов и термов, как было описано выше. Для составления утверждений о том, что два терма ссылаются на один и тот же объект, может использоваться $=$ символ равенства. Например, следующее утверждение:

$\text{Father}(\text{John}) = \text{Henry}$ говорит о том, что объект, на который ссылается высказывание $\text{Father}(\text{John})$, и объект, указанный под именем Henry , представляет собой одно и то же.

Поскольку в любой интерпретации за каждым термом закрепляется референт, т.е. объект, на который он ссылается, определение истинности любого высказывания с символом равенства сводится к проверке того, представляют ли собой референты двух термов, соединенных символом равенства, один и тот же объект.

Символ равенства может использоваться для констатации фактов, касающихся данной конкретной функции, как было только что сделано применительно к функциональному символу Father . Он может также применяться с отрицанием как указание на то, что два терма не представляют собой один и тот же объект. Чтобы выразить мысль о том, что Ричард имеет по меньшей мере двух братьев, можно записать следующее:

$\exists x, y \text{ Brother}(x, \text{Richard}) \wedge \text{Brother}(y, \text{Richard}) \wedge \neg (x = y)$ С другой стороны, высказывание:

$\exists x, y \text{ Brother}(x, \text{Richard}) \wedge \text{Brotherly}(y, \text{Richard})$ не имеет такого намеченного смысла. В частности, оно истинно в модели, приведенной на рис. 8.1, где у Ричарда имеется только один брат. Чтобы убедиться в этом, рассмотрим расширенную интерпретацию, в которой обоим переменным, x и y , присваивается объект — король Джон. Но в результате добавления выражения $\neg (x = y)$ такие модели становятся недействительными. В качестве сокращения для $\neg (x = y)$ иногда используется обозначение $x \neq y$.

8.3. Использование логики первого порядка Теперь, после того как мы определили выразительный логический язык, можно приступить к изучению способов его

использования. Лучше всего можно сделать это с помощью примеров. Выше уже были приведены некоторые простые высказы-

ния, иллюстрирующие различные аспекты этого логического синтаксиса; в данном разделе будут показаны более систематические представления некоторых простых ^ проблемных областей. В проблематике представления знаний проблемной областью считается некоторая часть мира, о которой необходимо выразить некоторые знания.

Начнем с краткого описания интерфейса Tell/Ask для базы знаний в логике первого порядка. Затем рассмотрим проблемную область семейных отношений, чисел, множеств и списков, а также мира вампуса. В следующем разделе приведен более развернутый пример (электронные схемы), а в главе 10 рассматривается все, что касается данной области науки.

Утверждения и запросы в логике первого порядка Высказывания вводятся в базу знаний с помощью операции Tell, точно так же, как и в пропозициональной логике. Такие высказывания называются ^ утверждениями.

Например, можно ввести утверждения, что Джон — король и что короли — люди:

$\text{Tell}\{\text{KB}, \text{King}(\text{John})\}$ $\text{Tell}(\text{KB}, \forall x \text{King}(x) \Rightarrow \text{Person}(x))$ Мы можем задавать вопросы о содержимом базы знаний с использованием операции Ask. Например, следующее выражение:

$\text{Ask}(\text{KB}, \text{King}(\text{John}))$ возвращает true. Вопросы, заданные с помощью операции Ask, называются " ^ запросами, или ^ целями (которые не следует путать с целями, используемыми при описании желаемых состояний агента). Вообще говоря, на любой запрос, который логически следует из базы знаний, должен быть получен утвердительный ответ.

Например, если в ней содержатся два утверждения, приведенные в предыдущем абзаце, то следующий запрос:

$\text{Ask}\{\text{KB}, \text{Person}(\text{John})\}$ должен также вернуть true. Кроме того, можно задавать запросы с кванторами, такие как следующий:

$\text{Ask}(\text{KB}, \exists x \text{Person}(x))$ На этот запрос должен быть получен ответ true, но этот ответ — ни полезный, ни забавный. (Его можно сравнить с получением ответа "Да" на вопрос: "Можете ли вы сказать мне, который час?") Запрос с переменными, на которые распространяется квантор существования, имеет смысл: "Существует ли такое значение x , что...", и мы решаем его, предоставляя соответствующее значение x . Стандартная форма для ответа такого рода представляет собой ^ подстановку, или ^ список связывания, который является множеством пар "переменная—терм". В данном конкретном случае, при наличии только двух утверждений, ответом должно

быть {x/ John}. А если имеется больше одного возможного ответа, может быть возвращен список подстановок.

Проблемная область родства В качестве первого примера рассмотрим проблемную область семейных отношений, или родства. Эта проблемная область включает такие факты, как "Элизабет —

мать Чарльза" и "Чарльз — отец Уильяма", и правила наподобие того, что "Бабушка — это мать родителя".

Очевидно, что объектами в этой проблемной области являются люди. В ней будут применяться два унарных предиката, Male (Мужчина) и Female (Женщина).

Отношения родства (связи между родителями и детьми, братьями и сестрами, мужем и женой и т.д.) будут представлены с помощью бинарных предикатов: Parent (Родитель), Sibling (Брат или сестра), Brother (Брат), Sister (Сестра), Child (Ребенок), Daughter (Дочь), Son (Сын), Spouse (Супруг или супруга), Wife (Жена), Husband (Муж), Grandparent (Дедушка или бабушка), Grandchild (Внук или внучка), Cousin (Двоюродный брат или двоюродная сестра), Aunt (Тетя) и Uncle (Дядя). В качестве предикатов Mother (Мать) и Father (Отец) мы будем использовать функции, поскольку каждый человек имеет по одному из этих объектов (по крайней мере, в соответствии с законами природы).

Рассмотрим каждую из функций и предикатов, записывая все, что мы знаем о них, в терминах других символов. Например, мать — это родитель женского рода: $\forall t, c \text{ Mother } \{c\} - t \Leftrightarrow \text{Female } \{t\}$ л Parent (t, c) Муж — это супруг мужского пола: $\forall w, h \text{ Husband} \{h, w\} \Leftrightarrow \text{Male} \{h\}$ л Spouse{h,w} Мужчины и женщины — непересекающиеся категории людей:

$\forall x \text{ Male}(x) \Leftrightarrow \neg \text{Female}(x)$ Отношения между родителями и детьми являются взаимно противоположными: $\forall p, c \text{ Parent } (p, c) \Leftrightarrow \text{Child}(cp)$ Дедушка или бабушка — это родитель родителя: $\forall d, c \text{ Grandparent } (d, c) \Leftrightarrow \forall p \text{ Parent } \{d, p\}$ л Parent (p, c) Брат или сестра — это еще один ребенок тех же родителей:

$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow x \neq y \wedge \forall p \text{ Parent } (p, x) \wedge \text{Parent } (p, y)$ Формулируя подобные сведения, можно заполнить еще несколько страниц, и в упр. 8.11 предлагается сделать именно это.

Каждое из этих высказываний может рассматриваться как одна из $\wedge$ аксиом в проблемной области родства. Аксиомы обычно принято связывать чисто с математическими проблемными областями (и мы вскоре рассмотрим некоторые аксиомы для чисел), но они нужны во всех проблемных областях. Аксиомы предоставляют основную фактическую информацию, на основании которой могут быть получены логическим путем полезные заключения. Кроме того, приведенные выше аксиомы родства имеют $\wedge$ определения; последние представлены в форме $\forall x, y$

$P(x,y) \Leftrightarrow \dots$. Аксиомы определяют функцию Mother и предикаты Husband, Male, Parent, Grandparent и Sibling в терминах других предикатов. Но приведенные выше определения можно "свести" к базовому множеству предикатов (Child, Spouse и Female), в терминах которых в конечном итоге определяются все остальные предикаты. Это — очень естественный способ, с помощью которого создается представление некоторой проблемной области, и он аналогичен способу, применяемому при создании пакетов программного обеспечения путем последовательного определения процедур из при-

митивных библиотечных функций. Обратите внимание на то, что множество примитивных базовых предикатов не обязательно должно быть уникальным; с таким же успехом можно использовать множество Parent, Spouse и Male. Как будет показано ниже, в некоторых проблемных областях нельзя найти четко определенное базовое множество.

Не все логические высказывания о некоторой проблемной области являются аксиомами. Некоторые из них представляют собой $\wedge$ теоремы, т.е. следуют из аксиом.

Например, рассмотрим утверждение, что родственные отношения между братьями и сестрами являются симметричными:

$\forall x,y \text{ Sibling}(x,y) \Leftrightarrow \text{Sibling}(y, x)$ Это — аксиома или теорема? В действительности это — теорема, которая логически следует из аксиомы, определяющей понятие родственных отношений между братьями и сестрами. Если в базу знаний с помощью операции Ask будет введен запрос в виде этого высказывания, то должно быть получено значение true.

С чисто логической точки зрения в базе знаний должны содержаться только аксиомы, но не теоремы, поскольку теоремы не увеличивают множество заключений, которые следуют из базы знаний. Но с практической точки зрения важным свойством теорем является то, что они уменьшают вычислительные издержки на логический вывод новых высказываний. Без них системе формирования рассуждений приходится всякий раз начинать с самых фундаментальных принципов, во многом аналогично тому, что математику приходилось бы снова выводить правила исчисления, приступая к решению каждой новой задачи.

Не все аксиомы имеют определения. Некоторые из них предоставляют более общую информацию об определенных предикатах без формулировки определений.

И действительно, некоторые предикаты не имеют полного определения, поскольку мы не знаем достаточно для того, чтобы их полностью охарактеризовать. Например,

нельзя найти очевидного способа закончить следующее высказывание с определением понятия человек:

$\forall x \text{ Person}(x) \Leftrightarrow \dots$ К счастью, логика первого порядка позволяет использовать предикат *Person*, не определяя его полностью. Вместо этого можно записывать частичные спецификации свойств, которыми обладает каждый человек, и свойств, которые идентифицируют некоторый объект как человека:

$\forall x \text{ Person}(x) \Rightarrow \dots \forall x \dots \Rightarrow \text{Person}(x)$ Аксиомы могут также представлять собой просто "голые факты", такие как *Male* (Jim) и *Spouse* (Jim, Laura). Такие факты формируют описания конкретных экземпляров задачи, что дает возможность находить ответы на конкретные вопросы. Ответы на эти вопросы представляют собой теоремы, которые следуют из аксиом. Но часто приходится убеждаться в том, что ожидаемые ответы получить не удастся, например, можно было бы ожидать, что удастся вывести логическим путем факт *Female*(Laura) из аксиом *Male*(George) и *Spouse* (George, Laura), но данный факт не следует как теорема из этих аксиом. Такая ситуация свидетельствует о том, что в базе знаний не хватает какой-то аксиомы. В упр. 8.8 предлагается предоставить эту аксиому.

Числа, множества и списки По-видимому, числа представляют собой наиболее яркий пример того, как может быть построена крупная теория из крошечного ядра аксиом. В этом разделе будет описана теория $\wedge$ натуральных чисел, или неотрицательных целых чисел. Для этого потребуется предикат *NatNum*, который будет принимать истинное значение при использовании параметра, представляющего собой натуральное число; кроме того, требуется один константный символ, 0, и один функциональный символ, *S* (successor — преемник). $\wedge$ Аксиомы Пеано определяют натуральные числа и операцию сложения⁸.

Натуральные числа определяются рекурсивно: $\text{NatNum}\{0\} \forall n \text{ NatNum}\{n\} \Rightarrow \text{NatNum}\{S(n)\}$ Это означает, что 0 — натуральное число и для каждого объекта *л*, если *п* — натуральное число, *S*(*п*) — натуральное число. Поэтому натуральными числами являются 0, *S*(0), *S*(*S*(0)) и т.д. Необходимы также аксиомы для ограничения действия функции определения преемника:

$\forall n \text{ } 0 \neq S(n) \forall t, p \text{ } t \neq p \Rightarrow S(t) \neq S(p)$ Теперь мы можем определить операцию сложения в терминах функции определения преемника: $\forall t \text{ NatNum}(m) \Rightarrow +@, t) = t \forall t, p \text{ NatNum}(m) \wedge \text{NatNum}(n) \Rightarrow +(S(m), n) = S(+ (m, n))$ В первой из этих аксиом утверждается, что сложение числа 0 с любым натуральным числом *т* приводит к получению самого числа *т*. Обратите внимание на использование бинарного функционального символа "+" в терме + @, *т*); в обычной математике такой терм принято записывать в виде 0+*т* с использованием $\wedge$ инфиксной системы обозначений. (Система обозначений, которая используется в этом разделе для логики первого порядка, называется $\wedge$ префиксной.) Чтобы было удобнее читать

высказывания, касающиеся чисел, здесь будет также разрешено использовать инфиксные обозначения. Кроме того, мы можем записывать как $s(n)$ как $n+1$, поэтому вторая аксиома принимает следующий вид: $\forall t, n \in \text{NatNum}(m) \quad n \in \text{NatNum}(n) \Rightarrow (t+n)+1 = t+(n+1)$ Эта аксиома сводит сложение к повторному применению функции определения преемника.

Такое использование инфиксных обозначений представляет собой пример применения $\wedge$ синтаксического упрощения, т.е. расширения или сокращения стандартного синтаксиса, при котором семантика не изменяется. Любое высказывание, в котором используются такие сокращения, может быть "разупрощено" для получения эквивалентного высказывания в обычной логике первого порядка.

После определения понятия сложения становится простой задачей определения умножения как повторного сложения, возведения в степень как повторного умножения. Аксиомы Пеано включают также принцип индукции, который представляет собой высказывание логики второго порядка, а не логики первого порядка. Важность этого различия объясняется в главе 9.

жения, целочисленного деления и определения остатков от деления, формулировки понятия простых чисел и т.д. Таким образом, вся теория чисел (включая такие ее приложения, как криптография) может быть сформирована на основе одной константы, одной функции, одного предиката и четырех аксиом.

Проблемная область $\wedge$ множеств также является фундаментальной не только для математики, но и для рассуждений на уровне здравого смысла. (В действительности на основе теории множеств может быть также построена теория чисел.) Необходимо иметь возможность представлять отдельные множества, включая пустое множество.

Кроме того, требуется способ составления множеств с помощью добавления элемента к множеству или применения операции объединения или пересечения к двум множествам. К тому же необходимо знать, входит ли некоторый элемент в состав множества, и иметь возможность отличать множества от объектов, не являющихся множествами.

В качестве синтаксического упрощения в данном разделе будет использоваться обычный словарь теории множеств. Пустое множество представляет собой константу, которая записывается как $\{\}$. Применяется также один унарный предикат, Set , который принимает истинное значение, если его фактическим параметром является множество. Бинарными предикатами являются $x \in s$ (x — элемент множества s) и $S \subseteq s$ (S — подмножество, не обязательно строгое, множества s). В качестве бинарных функций применяются $s \cap s_2$ (пересечение двух множеств), $s \cup s_2$ (объединение двух множеств) и $\{x \mid s\}$ (множество, полученное в результате

присоединения элемента x к множеству s). Один из возможных наборов аксиом приведен ниже.

1. Единственными множествами являются пустое множество и множества, полученные путем присоединения некоторого элемента к множеству.

$\forall s \text{ Set}(s) \Leftrightarrow (s = \{\}) \vee (\forall x, s2 \text{ Set}(s2) \wedge s = \{x \setminus s2\})$ 2. Пустое множество не имеет присоединенных к нему элементов, иными словами, не существует способа разложения множества Empty Set на меньшее множество и еще один элемент: $\neg \forall x, s \{x \setminus s\} = \{\}$ 3. Присоединение к множеству элемента, уже имеющегося в этом множестве, не оказывает никакого эффекта:

$\forall x, s \ x \in s \Leftrightarrow s = \{x \setminus s\}$ 4. Единственными элементами множества являются элементы, которые были к нему присоединены. Мы выразим эту мысль рекурсивно, утверждая, что x — элемент множества s тогда и только тогда, когда s эквивалентно некоторому множеству $s2$, к которому присоединен некоторый элемент y , где либо y совпадает с x , либо является элементом $s2$:

$\forall x, s \ x \in s \Leftrightarrow [\forall y, s2 (s = \{y \setminus s2\} \wedge (x = y \vee x \in s2))]$ 5. Множество является подмножеством другого множества тогда и только тогда, когда все элементы первого множества являются элементами второго множества:

$\forall s1, s2 \ s1 \subseteq s2 \Leftrightarrow (\forall x \ x \in s1 \Rightarrow x \in s2)$ 6. Два множества равны тогда и только тогда, когда каждое из них является подмножеством другого:

$\forall s1, s2 (s1 = s2) \Leftrightarrow (s1 \subseteq s2 \wedge s2 \subseteq s1)$

7. Некоторый объект принадлежит к пересечению двух множеств тогда и только тогда, когда он является элементом обоих этих множеств: $\forall x, s1, s2 \ x \in (s1 \cap s2) \Leftrightarrow (x \in s1 \wedge x \in s2)$ 8. Некоторый объект принадлежит к объединению двух множеств тогда и только тогда, когда он является элементом того или другого множества:

$\forall x, s1, s2 \ x \in (s1 \cup s2) \Leftrightarrow (x \in s1 \vee x \in s2)$ ^ Списки подобны множествам. Различия между ними состоят в том, что списки упорядочены, а один и тот же элемент может появляться в списке несколько раз. Для списков может использоваться словарь ключевых слов Lisp: Nil — это список-константа без элементов; Cons , Append , First и Rest — функции; Find — предикат, который выполняет для списков такие же функции, какие Member выполняет для множеств. List? — предикат, принимающий истинное значение только при получении параметра, представляющего собой список. Как и в случае множеств, обычно принято использовать синтаксические упрощения в логических высказываниях, в которых применяются списки. Пустой список обозначается как $[]$. Терм $\text{Cons}(x, y)$, где y — непустой список, записывается как $[x \setminus y]$. Терм $\text{Cons}(x, \text{Nil})$ (т.е. список, содержащий только элемент x) записывается как $[x]$. Список из нескольких элементов, такой как $[A, B, C]$, соответствует вложенному терму

Cons (A, Cons(B, Cons (C, Nil))). В упр. 8.14 предлагается составить аксиомы для списков.

Мир вампуса Некоторые аксиомы пропозициональной логики для мира вампуса были приведены в главе 7. Аксиомы первого порядка, рассматриваемые в этом разделе, являются гораздо более краткими и выражают совершенно естественным способом именно то, что требуется описать в этом мире.

Напомним, что агент в мире вампуса получает вектор восприятий с пятью элементами. Соответствующее высказывание в логике первого порядка, хранящееся в базе знаний, должно включать данные о результатах восприятия и о времени, в которое они были получены; в противном случае у агента возникнет путаница в отношении того, когда и что он воспринимал. Для обозначения интервалов времени будут использоваться целые числа. Типичным высказыванием с данными о восприятии является следующее:

Percept([Stench,Breeze,Glitter, None,None],5) где Percept— бинарный предикат; Stench и т.д.— константы, помещенные в список. Действия в мире вампуса могут быть представлены с помощью логических термов следующим образом:

Тит (Right) , Turn {Left} , Forward, Shoot, Grab, Release, Climb Чтобы определить, какое действие является наилучшим, программа агента составляет примерно такой запрос:

За BestAction{a,5) Функция Ask должна разрешить этот запрос и вернуть список связывания, такой как {a/Grab}. Затем программа агента может вернуть Grab как действие, которое

должно быть выполнено, но вначале должна ввести в свою собственную базу знаний данные о том, что будет выполнено действие Grab, с помощью операции Tell.

Из исходных данных о восприятии следуют некоторые факты о текущем состоянии, например:

$\forall t, s, d, t, c \text{ Percept}([s, \text{Breeze}, d, t, c], t) \Rightarrow \text{Breeze}(t) \vee t, s, b, m, c \text{ Percept}([s, b, \text{Glitter}, m, c], t) \Rightarrow \text{Glitter}(t)$ и т.д. Эти правила являются проявлением простейшей формы процесса формирования рассуждений, называемого восприятием, который будет подробно рассматриваться в главе 24. Обратите внимание на то, что квантификация происходит по переменной t с обозначением времени. А в пропозициональной логике приходилось создавать копии каждого высказывания для каждого интервала времени.

Кроме того, в этой логике могут быть реализованы простые "рефлекторные" варианты поведения с помощью импликационных высказываний с кванторами.

Например, может быть предусмотрено следующее правило:

$\forall t \text{ Glitter}(t) \Rightarrow \text{BestAction}(\text{Grab}, t)$ При наличии результатов восприятия и правил, приведенных в предыдущих абзацах, применение данного правила привело бы к желаемому заключению $\text{BestAction}(\text{Grab}, 5)$ о том, что в данный момент следует выполнить действие Grab . Обратите внимание на соответствие между этим правилом и прямым соединением "восприятие/действие" в агенте на основе логической схемы, приведенной на рис. 7.9; соединение в этой логической схеме неявно предусматривает применение квантора к переменной, обозначающей время.

До сих пор в этом разделе все высказывания, касающиеся времени, были $\wedge$ синхронными (т.е. "одновременными") высказываниями; это означает, что они связывали свойства некоторого состояния мира с другими свойствами того же состояния мира. А высказывания, которые допускают формирование "разновременных" рассуждений, называются $\wedge$ диахронными; например, агенту требуется знать, как комбинировать информацию о его предыдущем местонахождении с информацией о только что выполненном действии, чтобы определить свое текущее местонахождение. Отложим обсуждение диахронных высказываний до главы 10, а пока будем просто предполагать, что в отношении предикатов, касающихся изменения местонахождения, и других предикатов, зависящих от времени, выполняется требуемый логический вывод.

Выше были представлены восприятия и действия; теперь настало время представить саму среду. Начнем с объектов. Очевидными кандидатами являются квадраты, ямы и вампус. Можно было бы присвоить имя каждому квадрату (Squareli2 и т.д.), но тогда тот факт, что Squareli2 и Squareli3 являются соседними, пришлось бы оформить как "дополнительный" факт и нам потребовалось бы по одному такому факту для каждой пары квадратов. Поэтому лучше использовать сложный терм, в котором строка и столбец показаны в виде целых чисел; например, списковый терм $[1,2]$.

В таком случае определение понятия соседства любых двух квадратов можно представить следующим образом:

$\forall x, y, a, b \text{ Adjacent}([x, y], [a, b]) \Leftrightarrow [a, b] \in \{[x+1, y], [x-1, y], [x, y+1], [x, y-1]\}$ Кроме того, можно было бы присвоить имя каждой яме, но такое решение является неподходящим по другой причине: нам нет смысла проводить различия между

ямами⁹. Гораздо проще использовать унарный предикат Pit , который принимает истинное значение в квадратах, содержащих ямы. Наконец, поскольку имеется точно один вампус, для его представления равным образом подходят и константа Wumpus , и унарный предикат (а последний способ обозначения с точки зрения вампуса может даже оказаться более почетным). Вампус проживает точно в одном квадрате, поэтому для именования этого квадрата целесообразно использовать функцию, такую как $\text{Home}(\text{Wumpus})$. Это позволяет полностью избежать

необходимости применения громоздкого множества высказываний, которые требовались в пропозициональной логике для обозначения того, что вампус находится точно в одном квадрате. (А при наличии двух вампусов ситуации в пропозициональной логике стала бы еще хуже.) Местонахождение агента меняется со временем, поэтому мы будем применять запись $At \{Agent, s, t\}$ для указания на то, что агент находится в квадрате s во время t . Зная свое текущее местонахождение, агент сможет выявлять путем логического вывода свойства текущего квадрата на основании данных о свойствах его текущего восприятия. Например, если агент находится в некотором квадрате и чувствует ветерок, то в этом квадрате чувствуется ветерок:

$\forall s, t \ At\{Agent, s, t\} \wedge Breeze(t) \Rightarrow Breezy(s)$ Нам требуется знать не то, что агент вообще почувствовал ветерок, а то, что ветерок чувствуется в определенном квадрате, поскольку известно, что ямы не могут менять своего местонахождения. Обратите внимание на то, что предикат $Breezy$ ("в квадрате чувствуется ветерок") не имеет параметра с обозначением времени.

Обнаружив, в каких местах чувствуется ветерок (или неприятный запах), а также, что очень важно, в каких местах не чувствуется ветерок (или неприятный запах), агент получает возможность определять логическим путем, где находятся ямы (и где находится вампус). Существуют два описанных ниже типа синхронных правил, которые позволяют делать такие логические выводы. • $\wedge$ Диагностические правила. Диагностические правила ведут от наблюдаемых эффектов к раскрытию скрытых причин. В очевидных диагностических правилах, касающихся поиска ям, утверждается, что если в квадрате чувствуется ветерок, то в некотором соседнем квадрате должна находиться яма, таким образом:

$\forall s \ Breezy(s) \Rightarrow \exists r \ Adjacent(r, s) \wedge Pit(r)$ С другой стороны, если в некотором квадрате не чувствуется ветерок, то ни в одном из соседних квадратов не находится яма¹⁰:

$\forall s \ \neg Breezy(s) \Rightarrow \neg \exists r \ Adjacent(r, s) \wedge Pit(r)$ 9 Аналогичным образом, большинство из нас не присваивают имя каждой птице, которая пролетает у нас над головой или переселяется в теплые края на зиму. С другой стороны, орнитологи, желающие изучить пути миграции, показатели выживания и т.д., присваивают имя каждой отловленной птице, закрепив кольцо на ее ноге, поскольку им приходится следить за отдельными птицами.

¹⁰ Люди имеют естественную склонность забывать записывать отрицательную информацию, подобную этой. В обычной беседе такая склонность вполне оправдана. Было бы странно услышать слова: "На столе — две чашки, а не три или больше", даже несмотря на то, что, формально говоря, утверждение: "На столе — две чашки" остается истинным, даже если на столе их три. Мы вернемся к этой теме в главе 10.

Объединяя эти два правила, мы получим следующее высказывание в форме двусторонней импликации:

$\forall s \text{ Breezy}(s) \Leftrightarrow \exists r \text{ Adjacent}(r, s) \wedge \text{Pit}(r)$ (8.3) • $\wedge$ Причинные правила Причинные правила отражают предполагаемую направленность причинноследственных отношений в мире: появление некоторых восприятий вызывается определенными скрытыми свойствами мира. Например, наличие ямы вызывает появление ветерка во всех соседних квадратах:

$\forall r \text{ Pit}(r) \Rightarrow [\forall s \text{ Adjacent}(r, s) \Rightarrow \text{Breezy}(s)]$ А если во всех квадратах, соседних по отношению к данному конкретному квадрату, нет ям, то в данном квадрате не чувствуется ветерка:

$\forall s [\forall r \text{ Adjacent}(r, s) \Rightarrow \neg \text{Pit}(r)] \Rightarrow \neg \text{Breezy}(s)$ Приложив определенные усилия, можно показать, что эти два высказывания, вместе взятые, логически эквивалентны высказыванию с двусторонней импликацией, приведенному в уравнении 8.3. Это высказывание с двусторонней импликацией само может также рассматриваться как причинное, поскольку показывает, каким образом вырабатывается истинностное значение предиката Breezy по данным о состоянии мира.

Системы, в которых логические рассуждения формируются с помощью причинных правил, называются системами $\wedge$ формирования рассуждений на основе модели, поскольку причинные правила образуют модель того, как действует среда. Различие между рассуждениями на основе модели и диагностическими рассуждениями является важным для многих областей искусственного интеллекта. В частности, одной из активных областей исследования остается медицинская диагностика, и в ней подходы, основанные на прямых ассоциациях между симптомами и заболеваниями (диагностический подход), постепенно вытесняются подходами, в которых используются явно заданные модели заболеваний и данные о проявлениях этих заболеваний в виде симптомов. Мы вернемся к этой теме в главе 13.

Какой бы тип представления не использовался в агенте, с $\wedge$ если аксиомы правильно и полно описывают способ функционирования мира и способ выработки восприятий, то любая полная процедура логического вывода позволяет получить наиболее сильное возможное описание состояния мира при наличии всех доступных восприятий. Благодаря этому проектировщик агента получает возможность сосредоточиться на решении задачи получения правильных знаний, не слишком заботясь о том, как должны быть организованы процессы логического вывода. Кроме того, в данном разделе было показано, что логика первого порядка позволяет представить мир вампуса не менее кратко, чем первоначальное описание на естественном языке, приведенное в главе 7.

8.4. Инженерия знаний с применением логики В предыдущем разделе иллюстрировалось использование логики первого порядка для представления знаний в трех простых проблемных областях. В этом разделе рас-

сматривается общий процесс конструирования базы знаний, называемый $\wedge$ инженерией знаний. Инженером по знаниям называется специалист, который исследует конкретную проблемную область, определяет, какие понятия важны в этой проблемной области, и создает формальное представление объектов и отношений в этой проблемной области. В данном разделе процесс инженерии знаний будет проиллюстрирован на проблемной области проектирования электронных схем, которая должна быть уже довольно знакомой читателю, поэтому мы можем сосредоточиться на самих относящихся к этой теме проблемах представления знаний. Применяемый в этом разделе подход является приемлемым для разработки баз знаний специального назначения, проблемная область которых тщательно очерчена и спектр запросов известен заранее.

Базы знаний общего назначения, которые предназначены для поддержки запросов, касающихся полного спектра человеческих знаний, обсуждаются в главе 10.

Процесс инженерии знаний Безусловно, проекты в области инженерии знаний во многом отличаются друг от друга по своему содержанию, охвату и сложности, но все эти проекты включают перечисленные ниже этапы.

1. Идентификация задания. Инженер по знаниям должен очертить круг вопросов, которые должна поддерживать база знаний, и виды фактов, которые будут доступными применительно к каждому конкретному экземпляру задачи.

Например, должна ли база знаний о кампусе предоставлять возможность выбирать действия, или от нее требуется только поиск ответов на вопросы о содержании различных компонентов среды? Должны ли факты, полученные от датчиков, включать данные о текущем местонахождении? Само задание определяет, какие знания должны быть представлены в базе, чтобы можно было связать экземпляры задачи с ответами. Этот этап аналогичен процессу PEAS проектирования агентов, описанному в главе 2.

2. Сбор относящихся к делу знаний. Инженер по знаниям может уже быть экспертом в рассматриваемой проблемной области или ему может потребоваться общаться с настоящими экспертами для выявления всего, что они знают — этот процесс называется $\wedge$ приобретением знаний. На этом этапе знания еще не представлены формально. Его назначение состоит в том, чтобы понять, каким должен быть спектр знаний в базе знаний, определяемый самим заданием, а также разобраться в том, как фактически функционирует рассматриваемая проблемная область.

Относящиеся к делу (релевантные) знания для мира вампуса, который определен с помощью искусственного набора правил, выявить несложно. (Но следует отметить, что определение понятия соседства квадратов не формулировалось явно в правилах функционирования мира вампуса.) Тем не менее для реальных проблемных областей задача выявления релевантных знаний может оказаться весьма сложной; например, система для эмуляции работы спроектированных СБИС может требовать или не требовать учета паразитных емкостей и поверхностных эффектов.

3. Определение словаря предикатов, функций и констант. В иной формулировке этот этап можно определить как преобразование важных понятий уровня

проблемной области в имена логического уровня. Для этого необходимо ответить на многие вопросы в стиле инженерии знаний. Как и от стиля программирования, от стиля инженерии знаний может существенным образом зависеть окончательный успех проекта. Например, должны ли ямы быть представлены с помощью объектов или с помощью унарного предиката, определенного на квадратах? Должна ли ориентация агента быть задана в виде функции или предиката? Должно ли местонахождение вампуса зависеть от времени? Результатом выбора наиболее подходящих средств представления становится словарь, известный под названием $\wedge$ онтологии проблемной области. Само слово онтология в данном контексте означает конкретную теорию пребывания в определенном состоянии, или теорию существования.

Онтология определяет, какого рода объекты существуют, но не определяет их конкретные свойства и взаимосвязи.

4. Регистрация общих знаний о проблемной области. Инженер по знаниям записывает аксиомы для всех терминов словаря. Тем самым он закрепляет (в возможной степени) смысл этих терминов, позволяя эксперту проверить их содержание. На этом этапе часто обнаруживаются неправильные трактовки или пропуски в словаре, который необходимо исправить, возвратившись на этап 3 и снова пройдя данную итерацию в текущем процессе проектирования.

5. Составление описания данного конкретного экземпляра задачи. Если онтология хорошо продумана, этот этап будет несложным. Он сводится к написанию простых атомарных высказываний об экземплярах понятий, которые уже являются частью онтологии. Для логического агента определения экземпляров задачи поставляются датчиками, а сама "бестелесная" база знаний снабжается дополнительными высказываниями таким же образом, как традиционные программы снабжаются входными данными.

6. Передача запросов процедуре логического вывода и получение ответов. Именно здесь нас ожидает награда: теперь мы можем применить процедуру логического

вывода к аксиомам и фактам о конкретной задаче для получения фактов, которые нам хочется узнать.

7. Отладка базы знаний. К сожалению, ответы на запросы при первой попытке редко оказываются правильными. Точнее, ответы будут правильными для базы знаний в том виде, в каком она написана, при условии, что процедура логического вывода является непротиворечивой, но они не будут такими, каких ожидает пользователь. Например, если недостает какой-то аксиомы, то на некоторые запросы из этой базы знаний нельзя будет найти ответ. В этом может помочь продуманный процесс отладки. Недостающие или слишком слабые аксиомы могут быть легко выявлены путем обнаружения участков, на которых неожиданно обрывается цепочка этапов логического вывода. Например, если база знаний содержит одну из диагностических аксиом, касающихся ям, $\forall s \text{ Breezy}(s) \Rightarrow \exists r \text{ Adjacent}(r, s) \wedge \neg \exists t(r) \text{ но не содержит другую, то агент не сможет доказать отсутствие ям.}$

Неправильные аксиомы могут быть выявлены на основании того, что они представляют собой ложные утверждения о мире. Например, аксиома $\forall x \text{ NumOfLegs}(x, 4) \Rightarrow \text{Mammal}(x)$

является ложной, поскольку относит к млекопитающим рептилий, амфибий и, что еще важнее, столы с четырьмя ножками. cf= Ложность этого высказывания может быть определена независимо от остальной части базы знаний.

В отличие от этого, типичная ошибка в программе выглядит примерно таким образом: $\text{offset} = \text{position} + 1$ Невозможно определить, является ли этот оператор правильным, не изучив остальную часть программы для определения того, что, например, переменная offset используется для ссылки на текущую позицию или на позицию, которая следует за текущей позицией, а также происходит ли изменение значения переменной position в другом операторе и поэтому возникает необходимость снова изменять значение переменной offset .

Чтобы лучше понять этот процесс, состоящий из семи этапов, мы теперь применим его к расширенному примеру — к проблемной области электронных схем.

Проблемная область электронных схем Разработаем онтологию и базу знаний, которые позволят нам рассуждать об электронных схемах такого типа, как показано на рис. 8.2. Мы будем руководствоваться описанным семиэтапным процессом инженерии знаний.

Рис. 8.2. Цифровая схема C1} предназначенная для использования в качестве однобитового полного сумматора. На первые два входа подаются два бита, подлежащие сложению, а на третий вход подается бит переноса. На первом выходе находится сумма, а на втором — бит переноса для следующего сумматора. Эта схема включает два логических элемента XOR, два логических элемента AND и один

логический элемент OR Идентификация задания С цифровыми схемами связано много заданий, требующих логического рассуждения. На самом высоком уровне требуется проанализировать функциональное назначение схемы. Например, действительно ли приведенная на рис. 8.2 схема выполняет сложение должным образом? Если на все входы подаются сигналы с высоким потенциалом, то каким является выход логического элемента A2? Интерес представляют также вопросы о структуре схемы. Например, каковыми являются все логические элементы, подключенные к первой входной клемме? Содержит ли эта схема

петли обратной связи? В этом и состоят задания, рассматриваемые в данном разделе. Существуют также более детализированные уровни анализа, включая те, которые относятся к определению продолжительности задержек, площади схемы, потреблению энергии, стоимости производства и т.д. Для каждого из этих уровней могут потребоваться дополнительные знания.

Сбор относящихся к делу знаний Что мы знаем о цифровых схемах? Для наших целей достаточно знать, что они состоят из проводов и логических элементов. Сигналы распространяются по проводам к входным клеммам логических элементов, а каждый логический элемент вырабатывает на выходной клемме сигнал, который распространяется по другому проводу. Чтобы определить, какими должны быть эти сигналы, нам необходимо знать, как логические элементы преобразуют свои входные сигналы. Существует четыре основных типа логических элементов: логические элементы AND, OR и XOR имеют две входные клеммы, а логический элемент NOT имеет одну входную клемму. Все логические элементы имеют одну выходную клемму. Схемы, как и логические элементы, имеют входные и выходные клеммы.

Для того чтобы рассуждать о функциональных назначениях и связях в электронной схеме, не нужно вести речь о самих проводах, о путях, по которым проложены провода, или о соединениях, в которых встречаются два провода. Играть роль только соединения между клеммами — можно утверждать, что одна выходная клемма соединена с другой, входной клеммой, не упоминая о том, что они фактически соединены проводами. В этой проблемной области имеется также множество других факторов, не имеющих отношения к нашему анализу, таких как размеры, форма, цвет или стоимость различных компонентов.

Если бы нашей целью было нечто иное, чем проверка проектов на уровне логических элементов, то онтология была бы другой. Например, если бы нас интересовала отладка неисправных электронных схем, то, по-видимому, целесообразно было бы включить в онтологию провода, поскольку неисправный провод может исказить проходящий по нему сигнал. С другой стороны, для устранения ошибок синхронизации потребовалось бы включить в рассмотрение задержки логических элементов. А если бы мы были заинтересованы в

проектировании продукта, который оказался бы прибыльным, то имели бы значение такие данные, как стоимость производства электронной схемы и ее быстроедействие в сравнении с другими продуктами на рынке.

Определение словаря Теперь нам известно, что нужно вести речь о схемах, клеммах, сигналах и логических элементах. На следующем этапе необходимо выбрать функции, предикаты и константы для их представления. Начнем с отдельных логических элементов и перейдем к схемам.

Прежде всего необходимо иметь возможность отличать один логический элемент от других логических элементов. Это можно обеспечить, присвоив логическим элементам имена с помощью констант: x_1 x_2 и т.д. Хотя каждый логический элемент подключен к схеме своим индивидуальным способом, его поведение (способ преобразования входных сигналов в выходные) зависит только от типа. Для ссылки на тип

логического элемента может использоваться функция¹¹. Например, можно написать $Type(x) = XOR$. Тем самым вводится константа XOR для логического элемента конкретного типа; другие константы будут называться OR, AND и NOT. Функция Type не является единственным способом регистрации онтологических различий. Для этого можно было бы использовать бинарный предикат, $Type(x) XOR$, или несколько отдельных предикатов типов, таких как XOR (x_1). Любой из этих вариантов вполне подходит, но выбирая функцию Type, мы избегаем необходимости определять аксиому, в которой указано, что каждый отдельный логический элемент может иметь только один тип, поскольку это уже гарантирует сама семантика функций.

Затем рассмотрим клеммы. Логический элемент или цифровая схема может иметь одну или несколько входных клемм и одну или несколько выходных клемм.

Можно было бы просто присвоить каждой из них имя с помощью константы, точно так же, как мы именовали p_i логические элементы. Таким образом, логический элемент x_i мог бы иметь клеммы с именами x_{i1} , x_{i2} и x_{i0} . Но следует избегать тенденции к созданию длинных составных имен. Назвав нечто x_{11} , мы не сделаем его первой входной клеммой x_1 ; все равно потребуется указать это с помощью явного утверждения. По-видимому, лучше именовать клеммы логического элемента с помощью функции, так же, как мы именовали левую ногу короля Джона как $LeftLeg(John)$. Таким образом, примем, что J_i обозначает первую входную клемму для логического элемента x_i . Аналогичная функция Out используется для выходных клемм.

Связь между логическими элементами может быть представлена с помощью предиката *Connected*, который принимает в качестве параметров имена двух клемм, как в выражении $Connected(Out(A), In(X))$.

Наконец, необходимо знать, включен или выключен сигнал. Одна из возможностей состоит в использовании унарного предиката On, который принимает истинное значение, когда сигнал на клемме включен. Тем не менее при этом затрудняется постановка таких вопросов, как: "Каковы возможные значения сигналов на выходных клеммах схемы Cг?" Поэтому введем в качестве объектов два "значения сигнала", 1 и 0, и функцию Signal, которая принимает имя клеммы в качестве параметра и указывает значение сигнала для этой клеммы.

Регистрация общих знаний о проблемной области Одним из признаков наличия хорошей онтологии является то, что с ее использованием требуется определить очень немного общих правил, а признаком наличия хорошего словаря служит то, что каждое правило может быть сформулировано четко и кратко. В рассматриваемом примере нам потребуется только семь приведенных ниже простых правил для описания всего, что нужно знать о цифровых схемах.

1. Если две клеммы соединены, то на них присутствует один и тот же сигнал:

$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2)$ 11 Обратите внимание на то, что в этом разделе имена, начинающиеся с соответствующих букв (A_i , x_i и т.д.), используются исключительно для того, чтобы данный пример стал более удобным для чтения. База знаний все еще должна содержать информацию о типе для всех этих логических элементов.

2. Сигнал на каждой клемме равен либо 1, либо 0 (но не имеет оба значения одновременно):

$\forall t \text{ Signal}(t) = 1 \vee \text{Signal}(t) = 0$ 1*0 3. Предикат Connected является коммутативным:

$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Leftrightarrow \text{Connected}(t_2, t_1)$ На выходе логического элемента OR присутствует 1 тогда и только тогда, когда на любом из его входов присутствует 1:

$\forall g \text{ Type}(g) = \text{OR} \Rightarrow \text{Signal}(\text{Out}(g)) = 1 \Leftrightarrow \forall n \text{ Signal}(\text{In}(n, g)) = 1$ На выходе логического элемента AND присутствует 0 тогда и только тогда, когда на любом из его входов присутствует 0:

$\forall g \text{ Type}(g) = \text{AND} \Rightarrow \text{Signal}(\text{Out}(g)) = 0 \Leftrightarrow \forall n \text{ Signal}(\text{In}(n, g)) = 0$ На выходе логического элемента XOR присутствует 1 тогда и только тогда, когда на его входах присутствуют разные сигналы:

$\forall g \text{ Type}(g) = \text{XOR} \Rightarrow \text{Signal}(\text{Out}(g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(A, g)) \neq \text{Signal}(\text{In}(B, g))$ Выход логического элемента NOT противоположен его входу:

$\forall g \text{ Type}(g) = \text{NOT} \Rightarrow \text{Signal}(\text{Out}(g)) = \neg \text{Signal}(\text{In}(A, g))$ Составление конкретного экземпляра задачи Схема, показанная на рис. 8.2, представлена как схема d со следующим описанием. Прежде всего определим типы всех логических элементов:

$Type(X_{\pm}) = XOR\ Type(X_2) = XOR\ Type(A_i) = AND\ Type(A_2) - AND\ Type(O_i) = OR$ Затем покажем связи между ними:

$Connected\{Out\ A,\ X_i\},\ In\{l,\ X_2\}\} Connected\ (In\ A,\ d)\ ,\ In\ \{1,\ X_i\}\} Connected\{Out\ A,\ X_i\},\ In\ B,\ A_2\}\} Connected\{In\{l,\ d\}\ ,\ Jn\{l,\ A_i\}\} Connected\{Out\ \{1,\ A_2\}\ ,\ In\{l,\ O_i\}\} Connected\{In\{2fC1\},\ In\ B,\ X_1\}\} Connected\{Out\ A,\ A_i\}\ ,\ Jn\ B,\ O_i\}\} Connected\{Jn\ B,\ C_i\}\ ,\ Jn\ B,\ A_i\}\} Connected\{Out\ (If-X)\ ,\ Out\ A,\ C_i\}\} Connected\{Jn\ C,\ C_i\}\ ,\ In\ B,\ X_2\}\} Connected\ (Out\ A,\ O_i)\ ,\ Out\ B\ ,\ C_i)\} Connected\{In\ \{3\ ,\ C_i\}\ ,\ In\ A,\ A_2\}\}$

Передача запросов процедуре логического вывода Какая комбинация входов вызовет появление 0 на первом выходе схемы $C_{\pm}$ (бит суммы) и появление 1 на втором выходе схемы $C_{\pm}$ (бит переноса)?

$3ii, i_2, i_3\ Signal\ (In\{l,\ d\}\) = i_{\pm}\ л\ Signal\ (In\ B\ ,\ d)\) = i_2\ л\ Signal\ (In\ C,\ d)\) = i_3\ л\ Signal\ (Out\ A,\ d)\) = 0\ л\ Signal\ (Out\ B,\ d)\) = 1$ Ответами являются такие подстановки значений вместо переменных $i_1?$ i_2 и i_3 , что результирующее высказывание следует из базы знаний. Существуют три такие подстановки: $\{ii/l,\ i_2/l,\ i_3/0\}\ \{ii/1,\ i_2/0,\ i_3/l\}\ \{ii/0,\ i_2/l,\ i_3/l\}$ Каковы возможные множества значений сигналов на всех клеммах этой схемы сумматора?

$3ii,\ i_2/\ i_3/\ o_i/\ O_2\ Signal\ (In\{l,\ d\}\) = ii\ л\ Signal\ (In\ B\ ,\ C_i)\) = i_2\ л\ Signal\ (In\ C,\ C_i)\) = i_3\ л\ Signal\ (Out\ A,\ C_i)\) = d\ л\ Signal\ (Out\ B,\ d)\) = o_2$ Этот последний запрос должен вернуть полную таблицу входов и выходов для данного устройства, которая может использоваться для проверки того, действительно ли эта схема правильно складывает свои входные данные. Это — простой пример ^ проверки схемы. Можно также применять приведенное здесь определение данной схемы для создания более крупных цифровых систем, для которых может осуществляться процедура проверки такого же рода (см. упр. 8.17). Для структурированной разработки базы знаний такого же рода подходят многие проблемные области, в которых более сложные понятия определяются на основе более простых понятий.

Отладка базы знаний Эту базу знаний можно испортить многими способами для определения того, какие виды ошибочного поведения будут с этим связаны. Например, допустим, что пропущено такое утверждение¹², как $1*0$. Тогда система внезапно потеряет способность доказывать наличие каких-либо входных сигналов для данной схемы, за исключением тех случаев, когда на входе присутствуют сигналы 000 и 110. Наличие этой проблемы можно выявить, запрашивая выводы каждого логического элемента.

Например, можно ввести следующий запрос: $3ii, i_2, o\ Signal\ (In\ A,\ d)\) = ii\ л\ Signal\ (In\ B\ ,\ d)\) = i_2\ л\ Signal\ (Out\ A,\ X_i)\)$ который позволит обнаружить, что выходы x_i становятся неизвестными в случае входов 10 и 01. Затем мы рассмотрим аксиому для логических элементов XOR применительно к $x_{\pm}$:

$\text{Signal}(\text{Out } A \text{ f } X_{\pm}) = 1 \Leftrightarrow \text{Signal}(\text{In } A, X_r) \Phi \text{Signal}(\text{In } B, X_{\pm})$ Если известно, что входы, скажем, равны 1 и 0, то эта аксиома сводится к следующей:

12. Такого рода упущения встречаются очень часто, поскольку люди обычно предполагают, что разные имена относятся к разным объектам. Аналогичное предположение принято также в системах логического программирования, которые рассматриваются в главе 9.

$\text{Signal}(\text{Out } A, X_i) = 1 \Leftrightarrow 1 \Phi O$ Теперь проблема становится очевидной: система неспособна получить заключение, что $\text{Signal}(\text{Out } A, X_r) = 1$, поэтому ей необходимо сообщить, что 1^0 .

8.5. Резюме В этой главе приведены вводные сведения о логике первого порядка — языке представления, который является гораздо более мощным по сравнению с пропозициональной логикой. Ниже перечислены наиболее важные понятия, представленные в настоящей главе.

- Языки представления знаний должны быть декларативными, композиционными, выразительными, независимыми от контекста и непротиворечивыми.
- Различные варианты логики отличаются друг от друга по своему онтологическому вкладу и эпистемологическому вкладу. Пропозициональная логика вносит вклад только в общий объем сведений, касающихся существования фактов, а логика первого порядка позволяет наращивать объем сведений, касающихся существования объектов и отношений, поэтому приобретает значительную выразительную мощь.
- Возможный мир, или модель, для логики первого порядка определяется множеством объектов, отношениями между ними и функциями, которые могут к ним применяться.
- Константные символы именуют объекты, предикатные символы именуют отношения, а функциональные символы именуют функции. Интерпретация задает отображение между символами и моделью. В сложных термах функциональные символы применяются к термам для именования объекта. Истинность высказывания определяется при наличии интерпретации и модели.
- Атомарное высказывание состоит из предиката, применяемого к одному или нескольким термам; оно становится истинным тогда и только тогда, когда имеет место отношение, обозначенное предикатом, между объектами, обозначенными его термами. В сложных высказываниях, так же как и в пропозициональной логике, используются связки, а высказывания с кванторами позволяют выражать общие правила.
- Для разработки базы знаний в логике первого порядка требуется тщательно провести процесс анализа проблемной области, выбора словаря и составления аксиом, необходимых для поддержки желаемых процедур логического вывода.

Библиографические и исторические заметки Даже в логике Аристотеля предпринимались попытки применять обобщения к объектам, но датой рождения подлинной логики первого порядка стало появление книги Готтлоба Фреге

Begriffsschrift ("Система обозначения понятий") [496], в которой были впервые введены кванторы. Предложенная в этой книге возможность вкладыва-

вать кванторы была большим шагом вперед, но Фреге использовал громоздкую систему обозначений. (Один из примеров таких обозначений приведен на первой странице обложки данной книги.) Современная система обозначений для логики первого порядка появилась в основном благодаря Джузеппе Пеано [1185], но ее семантика практически идентична семантике, предложенной в работе Фреге. Хотя сейчас это кажется довольно странным, но аксиомы Пеано в значительной степени обязаны своим появлением работам Грассмана [588] и Дедекинда [375].

Основным барьером на пути разработки логики первого порядка была полная концентрация усилий ученых на одноместных предикатах за счет исключения из рассмотрения многоместных реляционных предикатов. Такое сосредоточение на одноместных предикатах в логических системах было почти повсеместным, начиная от Аристотеля и включая Буля. Первая систематическая трактовка отношений была выполнена Аугустусом де Морганом [351], который цитировал следующий пример, чтобы показать, какого рода логические выводы не позволяет выполнять логика Аристотеля: "Все лошади — животные; поэтому голова лошади — это голова животного". Такой логический вывод неосуществим в логике Аристотеля, поскольку в любом допустимом правиле, способном поддерживать этот логический вывод, вначале необходимо проанализировать данное высказывание с использованием двухместного предиката "хголова у". Логика отношений была глубоко исследована Чарльзом Сандерсом Пирсом [1196], который также разработал логику первого порядка независимо от Фреге, хотя и немного позднее [1197].

Леопольд Лёвенгейм дал систематическую трактовку теории моделей для логики первого порядка в 1915 году [951]. В его статье рассматривался также символ равенства как неотъемлемая часть логики. Результаты Лёвенгейма были дополнительно расширены Торальфом Сколемом [1424]. Альфред Тарский [1490], [1491] дал явное определение понятий истинности и модельно-теоретического выполнения в логике первого порядка с использованием теории множеств.

Первым, кто предложил использовать логику первого порядка в качестве инструментального средства для создания систем искусственного интеллекта, был Маккарти [1009]. Перспективы искусственного интеллекта на основе логики значительно расширились в результате разработки Робинсоном [1298] резолюции — описанной в главе 9 полной процедуры вывода для логики первого порядка. Подход, получивший название логицистского, зародился в Станфордском университете. Корделл Грин [591], [592] разработал первую систему формирования рассуждений в логике первого порядка, QA3, что привело к первым попыткам создания логического робота в институте SRI [466]. Логика первого порядка была применена Зохаром Манна и Ричардом Валдингером [976] для формирования

рассуждений о программах, а позднее — Майклом Генезеретом [536] для формирования рассуждений об электронных схемах.

В Европе для использования в лингвистическом анализе [285] и для создания общих декларативных систем [849] было разработано логическое программирование (ограниченная форма проведения рассуждений в логике первого порядка). Кроме того, в ходе разработки проекта LCF (Logic for Computable Functions) в Эдинбурге [580] был внесен большой вклад в вычислительную логику. История этих разработок будет описана более подробно в главах 9 и 10.

Существует целый ряд хороших современных вводных учебников по логике первого порядка. Одним из наиболее удобных для чтения является [1254]. В [438] эта область знаний рассматривается в перспективе, в большей степени ориентирован-

ной на математику. В высшей степени формальная трактовка логики первого порядка, наряду с описанием более сложных тем логики, предоставлена в [94]. В [977] приведено удобное для чтения введение в логику с точки зрения компьютерных наук.

В [515] дано чрезвычайно строгое математическое описание логики первого порядка, наряду со значительным объемом материала, который может использоваться в области автоматического формирования рассуждений. В книге *Logical Foundations of Artificial Intelligence* [537] приведено не только солидное введение в логику, но и первое систематическое описание логических агентов с восприятиями и действиями.

Упражнения 8.1. Логическая база знаний представляет мир с использованием множества высказываний без явной структуры. Аналогическое представление, с другой стороны, имеет физическую структуру, которая непосредственно соответствует структуре представляемого объекта. Возьмите дорожную карту своей страны в качестве образца аналогического представления фактов о стране. Двухмерная структура карты соответствует двухмерной поверхности данного региона. а) Приведите пять примеров символов на языке карты. б) Явным высказыванием называется высказывание, фактически формулируемое разработчиком представления. Неявным высказыванием называется высказывание, которое следует из явных высказываний в соответствии со свойствами аналогического представления. Приведите по три примера неявных и явных высказываний на языке карты. в) Приведите три примера сведений о физической структуре территории вашей страны, которые не могут быть представлены на языке карты. г) Приведите два примера фактов, которые гораздо легче выразить на языке карты, чем на языке логики первого порядка. д) Приведите еще два примера полезных аналогических представлений.

Каковы преимущества и недостатки каждого из этих языков, логического и аналогического?

8.2. Рассмотрите базу знаний, содержащую только два высказывания: $P(a)$ и $P(b)$. Следует ли из этой базы знаний высказывание $\forall x P(x)$? Объясните свой ответ в терминах моделей.

8.3. Является ли допустимым высказывание $\exists x, y x = y$? Объясните, почему.

8.4. Напишите такое логическое высказывание, что каждый мир, в котором оно является истинным, содержит точно один объект.

8.5. Рассмотрите словарь символов, который содержит s константных символов, rk предикатных символов с арностью k к каждому и fk функциональных символов с арностью k к каждому, где $1 < k < A$. Допустим, что размер проблемной области ограничен значением D . Для каждой конкретной комбинации "интерпретация/модель" каждый предикатный или функциональный символ отображается, соответственно, на отношение или функцию с той же арностью. Может быть принято предположение, что функции в этой модели допускают, чтобы некоторые входные кортежи не имели значения для этой

функции (т.е. чтобы значение было невидимым объектом). Выведите формулу определения количества возможных комбинаций "интерпретация/модель" для проблемной области с D элементами. Не учитывайте необходимость устранения избыточных комбинаций.

8.6. Представьте приведенные ниже высказывания в логике первого порядка, используя совместимый словарь (который вы должны определить). а) Некоторые студенты участвовали в экзамене по французскому языку весной 2001 года. б) Каждый студент, который участвует в экзамене по французскому языку, сдает его. в) Только один студент участвовал в экзаменах по греческому языку весной 2001 года. г) Лучшая оценка по греческому всегда выше чем лучшая оценка по французскому. д) Каждый человек, который покупает страховой полис, умен. е) Ни один человек не покупает дорогой страховой полис. ж) Существует агент, который продает страховой полис только людям, которые не застрахованы. з) Есть некий парикмахер, который бреет всех мужчин в городе, которые не бреются сами. и) Любой человек, рожденный в Великобритании, каждым из родителей которого является британский гражданин или британский резидент, является британским гражданином по рождению. к) Любой человек, рожденный вне Великобритании, одним из родителей которого является британский гражданин по рождению, является британским гражданином по происхождению. л) Политические деятели могут постоянно вводить некоторых людей в заблуждение, они также могут вводить в заблуждение всех людей некоторое время, но они не могут постоянно вводить в заблуждение всех людей.

8.7. Представьте высказывание: "Все немцы говорят на одном и том же языке" в исчислении предикатов. Используйте предикат $\text{Speaks}(x, 2)$, который означает, что лицо x говорит на языке 2.

8.8. Какая аксиома необходима, чтобы вывести логически факт $\text{Female}(\text{Laura})$, если даны факты $\text{Male}(\text{Jim})$ и $\text{Spouse}(\text{Jim}, \text{Laura})$?

8.9. Напишите общее множество фактов и аксиом, чтобы представить утверждение: "Веллингтон слышал о смерти Наполеона" и правильно ответить на вопрос: "Наполеон слышал о смерти Веллингтона?"

8.10. Запишите в логике первого порядка факты о мире вампуса, представленные в разделе 7.5 с помощью пропозициональной логики. Насколько более компактной является новая версия?

8.11. Составьте аксиомы с описанием предикатов Grandchild (Внук или внучка), GreatGrandparent (Прадедушка или прабабушка), Brother (Брат), Sister (Сестра), Daughter (Дочь), Son (Сын), Aunt (Тетя), Uncle (Дядя), BrotherInLaw (Брат мужа или жены), SisterInLaw (Сестра мужа или жены) и

FirstCousin (Двоюродный брат или двоюродная сестра). Найдите правильное определение m -го кузена или кузины в n -м колене и запишите это определение в логике первого порядка.

Теперь запишите основные факты, изображенные в генеалогическом дереве, приведенном на рис. 8.3. С использованием подходящей системы формирования логических рассуждений введите в базу знаний с помощью операции Tell все записанные вами высказывания, а затем определите с помощью операции Ask , кто является внуками или внучками Элизабет (Elizabeth), братьями мужа Дианы (Diana), а также прадедушкой и прабабушкой Сары (Sarah).

George M Mum Spencer M Kydd Elizabeth M Philip Margaret Diana X Charles Anne M Mark Andrew M Sarah Edward AAA William Harry Peter Zara Beatrice Eugenie Рис. 8.3. Типичное генеалогическое дерево. Символ x соединяет супругов, а стрелки указывают на детей 8.12. Запишите высказывание с утверждением, что функция $+$ является коммутативной. Следует ли это высказывание из аксиом Пеано? В случае положительного ответа объясните, почему; в случае отрицательного ответа приведите модель, в которой эти аксиомы являются истинными, а ваше высказывание — ложным.

8.13. Объясните, в чем ошибка в следующем предлагаемом определении предиката принадлежности к множеству, e :

$\forall x, s \ x \in \{x \mid s\} \iff \forall x, s \ x \in s \Rightarrow \forall y \ x \in G \{y \mid s\}$ 8.14. Используя в качестве примеров аксиомы множества, запишите аксиомы для проблемной области списков, включая все константы, функции и предикаты, упомянутые в данной главе.

8.15. Объясните, в чем ошибка в следующем предлагаемом определении соседних квадратов в мире вампуса:

$Ux, y \text{ Adjacent } ([x, y], [x+1, y]) \wedge \text{Adjacent } ([x, y], [x, y+1])$ 8.16. Запишите аксиомы, требуемые для формирования рассуждений о местонахождении вампуса, с использованием константного символа Wumpus и бинарного предиката In (Wumpus, Location). Не забудьте учесть, что существует только один вампус.

8.17. (§§D Дополните словарь, приведенный в разделе 8.4, чтобы определить правила сложения для n-битовых двоичных чисел. Затем сформулируйте описание четырехбитового сумматора, показанного на рис. 8.4, и составьте запросы, необходимые для проверки того, что это описание действительно правильно.

ИЗ X2 X1 Xq + Y3 Y2 Yx YQ Рис. 8.4. Четырехбитовый сумматор 8.18. Представление схемы, приведенное в этой главе, является более подробным, чем это необходимо, если нас интересуют только функциональные возможности этой схемы. В более простой формулировке рассматриваются любые логические элементы или цифровые схемы с t входами и p выходами с использованием предиката, имеющего t+p параметров, такого, что предикат принимает истинное значение тогда и только тогда, когда входы и выходы согласованы. Например, логические элементы NOT могут быть описаны с помощью бинарного предиката NOT(i,o), для которого известны значения NOT (0,0) и NOT (1,0). Композиции логических элементов определяются с помощью конъюнкций предикатов логических элементов, в которых наличие разделяемых переменных указывает на прямые соединения. Например, схема NAND может состоять из элементов AND и NOT:

$\forall i, i2, o, o \text{ NAND}(i, i2, o) \Leftrightarrow \text{AND}(i, i2, o) \wedge \text{NOT}(o, o)$ С использованием этого представления определите однобитовый сумматор, приведенный на рис. 8.2, и четырехбитовый сумматор, приведенный на рис. 8.4, и объясните, какими запросами вы бы воспользовались для проверки таких проектов. Какого рода запросы, поддерживаемые представлением, описанным в разделе 8.4, не поддерживаются данным представлением?

8.19. Получите бланк заявки на оформление паспорта для гражданина своей страны, выясните, какие правила определяют права человека на получение паспорта, и переведите эти правила на язык логики первого порядка в соответствии с этапами, описанными в разделе 8.4.

Глава 9. Логический вывод в логике первого порядка

9 Логический вывод ПОРЯДКА В этой главе будут определены эффективные процедуры получения ответов на вопросы, сформулированные в логике первого порядка.

В главе 7 определено понятие логического вывода и показано, как можно обеспечить непротиворечивый и полный логический вывод для пропозициональной логики. В данной главе эти результаты будут дополнены для получения алгоритмов, позволяющих найти ответ на любой вопрос, сформулированный в логике первого порядка и имеющий ответ. Обладать такой возможностью очень важно, поскольку в логике первого порядка можно сформулировать практически любые знания, приложив для этого достаточные усилия.

В разделе 9.1 представлены правила логического вывода для кванторов и показано, как можно свести вывод в логике первого порядка к выводу в пропозициональной логике, хотя и за счет значительных издержек. В разделе 9.2 описана идея унификации и показано, как эта идея может использоваться для формирования правил логического вывода, которые могут применяться непосредственно к высказываниям в логике первого порядка. После этого рассматриваются три основных семейства алгоритмов вывода в логике первого порядка: прямой логический вывод и его применение к дедуктивным базам данных и продукционным системам рассматриваются в разделе 9.3; процедуры обратного логического вывода и системы логического программирования разрабатываются в разделе 9.4; а системы доказательства теорем на основе резолюции описаны в разделе 9.5. Вообще говоря, в любом случае следует использовать наиболее эффективный метод, позволяющий охватить все факты и аксиомы, которые должны быть выражены в процессе логического вывода. Но следует учитывать, что формирование рассуждений с помощью полностью общих высказываний логики первого порядка на основе метода резолюции обычно является менее эффективным по сравнению с формированием рассуждений с помощью определенных выражений с использованием прямого или обратного логического вывода.

9.1. Сравнение методов логического вывода в пропозициональной логике и логике первого порядка В этом и следующих разделах будут представлены идеи, лежащие в основе современных систем логического вывода. Начнем описание с некоторых простых правил логического вывода, которые могут применяться к высказываниям с кванторами для получения высказываний без кванторов. Эти правила естественным образом приводят к идее, что логический вывод в логике первого порядка может осуществляться путем преобразования высказываний в логике первого порядка, хранящихся в базе знаний, в высказывания, представленные в пропозициональной логике, и дальнейшего использования пропозиционального логического вывода, а о том, как выполнять этот вывод, нам уже известно из предыдущих глав. В следующем разделе указано одно очевидное сокращение,

которое приводит к созданию методов логического вывода, позволяющих непосредственно манипулировать высказываниями в логике первого порядка.

Правила логического вывода для кванторов Начнем с кванторов всеобщности. Предположим, что база знаний содержит следующую стандартную аксиому, которая передает мысль, содержащуюся во многих сказках, что все жадные короли — злые:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$ В таком случае представляется вполне допустимым вывести из нее любое из следующих высказываний:

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$ $\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$ $\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John}))$ Согласно правилу $\wedge$ конкретизации высказывания с квантором всеобщности (сокращенно UI — Universal Instantiation), мы можем вывести логическим путем любое высказывание, полученное в результате подстановки базового терма (терма без переменных) вместо переменной, на которую распространяется квантор всеобщности¹. Чтобы записать это правило логического вывода формально, воспользуемся понятием подстановки, введенным в разделе 8.3. Допустим, что $\text{Subst}(\theta, \alpha)$ обозначает результат применения подстановки θ к высказыванию α . В таком случае данное правило для любой переменной v и базового терма d можно записать следующим образом:

W a $\text{Subst}(\{v/g\}, \alpha)$ 1 Эти подстановки не следует путать с расширенными интерпретациями, которые использовались для определения семантики кванторов. В подстановке переменная заменяется термом (синтаксической конструкцией) для получения нового высказывания, тогда как любая интерпретация отображает некоторую переменную на объект в проблемной области.

Например, три высказывания, приведенные выше, получены с помощью подстановок $\{x/\text{John}\}$, $\{x/\text{Richard}\}$ и $\{x/\text{Father}(\text{John})\}$.

Соответствующее правило $\wedge$ конкретизации высказывания с квантором существования (Existential Instantiation — EI) для квантора существования является немного более сложным. Для любых высказывания α , переменной v и константного символа k , который не появляется где-либо в базе знаний, имеет место следующее:

$\exists v \wedge \text{Subst}(\{v/k\}, \alpha)$ Например, из высказывания $\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$ можно вывести высказывание $\text{Crown}(c_i) \wedge \text{OnHead}(c_i, \text{John})$ при условии, что константный символ c_i не появляется где-либо в базе знаний.

По сути, в этом высказывании с квантором существования указано, что существует некоторый объект, удовлетворяющий определенному условию, а в процессе конкретизации просто присваивается имя этому объекту. Естественно, что это имя не должно уже принадлежать другому объекту. В математике есть прекрасный

пример: предположим, мы открыли, что имеется некоторое число, которое немного больше чем 2,71828 и которое удовлетворяет уравнению $d(xy)/dy = xy$ для x .

Этому числу можно присвоить новое имя, такое как e , но было бы ошибкой присваивать ему имя существующего объекта, допустим, π . В логике такое новое имя называется $\wedge$ сколемовской константой. Конкретизация высказывания с квантором существования — это частный случай более общего процесса, называемого сколемизацией, который рассматривается в разделе 9.5.

Конкретизация высказывания с квантором существования не только сложнее, чем конкретизация высказывания с квантором всеобщности, но и играет в логическом выводе немного иную роль. Конкретизация высказывания с квантором всеобщности может применяться много раз для получения многих разных заключений, а конкретизация высказывания с квантором существования может применяться только один раз, а затем соответствующее высказывание с квантором существования может быть отброшено. Например, после того как в базу знаний будет добавлено высказывание $\text{Kill}\{\text{Murderer}, \text{Victim}\}$, становится больше не нужным высказывание $\exists x \text{ Kill}\{x, \text{Victim}\}$. Строго говоря, новая база знаний логически не эквивалентна старой, но можно показать, что она $\wedge$ эквивалентна с точки зрения логического вывода, в том смысле, что она выполнима тогда и только тогда, когда выполнима первоначальная база знаний.

Приведение к пропозициональному логическому выводу. Получив в свое распоряжение правила вывода высказываний с кванторами из высказываний без кванторов, мы получаем возможность привести вывод в логике первого порядка к выводу в пропозициональной логике. В данном разделе будут изложены основные идеи этого процесса, а более подробные сведения приведены в разделе 9.5.

Основная идея состоит в следующем: по аналогии с тем, как высказывание с квантором существования может быть заменено одной конкретизацией, высказывание с квантором всеобщности может быть заменено множеством всех возможных конкретизаций. Например, предположим, что наша база знаний содержит только такие высказывания:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x) \quad \text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \wedge \text{Brother}(\text{Richard}, \text{John})$ (9.1)

Затем применим правило конкретизации высказывания с квантором всеобщности к первому высказыванию, используя все возможные подстановки базовых термов из словаря этой базы знаний — в данном случае $\{x/\text{John}\}$ и $\{x/\text{Richard}\}$.

Мы получим следующие высказывания: $\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$ и $\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$ и отбросим высказывание с квантором всеобщности. Теперь база знаний становится по сути пропозициональной, если базовые атомарные высказывания ($\text{King}(\text{John})$, Greedy

{John} и т.д.) рассматриваются как пропозициональные символы. Поэтому теперь можно применить любой из алгоритмов полного пропозиционального вывода из главы 7 для получения таких заключений, как Evil (John).

Как показано в разделе 9.5, такой метод $\wedge$ пропозиционализации (преобразования в высказывания пропозициональной логики) может стать полностью обобщенным; это означает, что любую базу знаний и любой запрос в логике первого порядка можно пропозиционализировать таким образом, чтобы сохранялось логическое следствие. Таким образом, имеется полная процедура принятия решения в отношении того, сохраняется ли логическое следствие... или, возможно, такой процедуры нет.

Дело в том, что существует такая проблема: если база знаний включает функциональный символ, то множество возможных подстановок базовых термов становится бесконечным! Например, если в базе знаний упоминается символ Father, то существует возможность сформировать бесконечно большое количество вложенных термов, таких как Father {Father {Father {John} } }). А применяемые нами пропозициональные алгоритмы сталкиваются с затруднениями при обработке бесконечно большого множества высказываний.

К счастью, имеется знаменитая теорема, предложенная Жаком Эрбраном [650], согласно которой, если некоторое высказывание следует из первоначальной базы знаний в логике первого порядка, то существует доказательство, которое включает лишь конечное подмножество этой пропозиционализированной базы знаний.

Поскольку любое такое подмножество имеет максимальную глубину вложения среди его базовых термов, это подмножество можно найти, формируя вначале все конкретизации с константными символами (Richard и John), затем все термы с глубиной 1 (Father (Richard) и Father (John)), после этого все термы с глубиной 2 и т.д. до тех пор, пока мы не сможем больше составить пропозициональное доказательство высказывания, которое следует из базы знаний.

Выше был кратко описан один из подходов к организации вывода в логике первого порядка с помощью пропозиционализации, который является полным, т.е. позволяет доказать любое высказывание, которое следует из базы знаний. Это — важное достижение, если учесть, что пространство возможных моделей является бесконечным. С другой стороны, до тех пор пока это доказательство не составлено, мы не

знаем, следует ли данное высказывание из базы знаний! Что произойдет, если это высказывание из нее не следует? Можем ли мы это определить? Как оказалось, для логики первого порядка это действительно невозможно. Наша процедура доказательства может продолжаться и продолжаться, вырабатывая все более и более глубоко вложенные термы, а мы не будем знать, вошла ли она в безнадежный

цикл или до получения доказательства остался только один шаг. Такая проблема весьма напоминает проблему останова машин Тьюринга. Алан Тьюринг [1518] и Алонсо Черч [255] доказали неизбежность такого состояния дел, хотя и весьма различными способами. $d^{\wedge}$ Вопрос о следствии для логики первого порядка является полу разрешимым; это означает, что существуют алгоритмы, которые позволяют найти доказательство для любого высказывания, которое следует из базы знаний, но нет таких алгоритмов, которые позволяли бы также определить, что не существует доказательства для каждого высказывания, которое не следует из базы знаний.

9.2. Унификация и поднятие В предыдущем разделе описан уровень понимания процесса вывода в логике первого порядка, который существовал вплоть до начала 1960-х годов. Внимательный читатель (и, безусловно, специалисты в области вычислительной логики, работавшие в начале 1960-х годов) должен был заметить, что подход на основе пропозиционализации является довольно неэффективным. Например, если заданы запрос $Evil(x)$ и база знаний, приведенная в уравнении 9.1, то становится просто нерациональным формирование таких высказываний, как $King\{Richard\}$ а $Greedy\{Richard\} \Rightarrow Evil\{Richard\}$. И действительно, для любого человека вывод факта $Evil\{John\}$ из следующих высказываний кажется вполне очевидным:

$\forall x King(x) \wedge Greedy(x) \Rightarrow Evil(x)$ $King\{John\}$ $Greedy\{John\}$ Теперь мы покажем, как сделать его полностью очевидным для компьютера.

Правило вывода в логике первого порядка Процедура вывода того факта, что Джон — злой, действует следующим образом: найти некоторый x , такой, что x — король и x — жадный, а затем вывести, что x — злой. Вообще говоря, если существует некоторая подстановка θ , позволяющая сделать предпосылку импликации идентичной высказываниям, которые уже находятся в базе знаний, то можно утверждать об истинности заключения этой импликации после применения θ . В данном случае такой цели достигает подстановка $\{x/John\}$.

Фактически можно обеспечить выполнение на этом этапе вывода еще больше работы. Предположим, что нам известно не то, что жаден Джон — $Greedy(John)$, а что жадными являются все:

$\forall y Greedy(y)$ (9.2) Но и в таком случае нам все равно хотелось бы иметь возможность получить заключение, что Джон злой — $Evil(John)$, поскольку нам известно, что Джон — ко-

роль (это дано) и Джон жаден (так как жадными являются все). Для того чтобы такой метод мог работать, нам нужно найти подстановку как для переменных в высказывании с импликацией, так и для переменных в высказываниях, которые должны быть согласованы. В данном случае в результате применения подстановки $\{x/John, y/John\}$ к предпосылкам импликации $King(x) \wedge Greedy(x) \Rightarrow Evil(x)$ и к высказываниям

из базы знаний King (John) и Greedy [y] эти высказывания становятся идентичными. Таким образом, теперь можно вывести заключение импликации.

Такой процесс логического вывода может быть представлен с помощью единственного правила логического вывода, которое будет именоваться $\wedge$ обобщенным правилом отделения (Generalized Modus Ponens): для атомарных высказываний p_i и d , если существует подстановка θ , такая, что $\text{Subst}(\theta, p_i) = \text{Subst}(\theta, p_j)$, то для всех i имеет место следующее:

$p_1', p_2', \dots, p_n', (p_i \wedge p_2 \wedge \dots \wedge p_n \Rightarrow d) \text{Subst}(\theta, d)$ В этом правиле имеется $n+1$ предпосылка: n атомарных высказываний p_i и одна импликация. Заключение становится результатом применения подстановки θ к следствию d . В данном примере имеет место следующее:

p_1 — это King(John) p_i — это King(x) p_2 — это Greedy(y) p_2 — это Greedy(x) θ — это $\{x/\text{John}, y/\text{John}\}$ d — это Evil(x) $\text{Subst}(\theta, g)$ — это Evil(John) Можно легко показать, что обобщенное правило отделения — непротиворечивое правило логического вывода. Прежде всего отметим, что для любого высказывания p (в отношении которого предполагается, что на его переменные распространяется квантор всеобщности) и для любой подстановки θ справедливо следующее правило: $p \wedge \text{Subst}(\theta, p)$ Это правило выполняется по тем же причинам, по которым выполняется правило конкретизации высказывания с квантором всеобщности. Оно выполняется, в частности, в любой подстановке θ , которая удовлетворяет условиям обобщенного правила отделения. Поэтому из $p_1', \dots, p_n'$ можно вывести следующее:

$\text{Subst}(\theta, p_1') \wedge \dots \wedge \text{Subst}(\theta, p_n')$ а из импликации $p \wedge \dots \wedge p_n \Rightarrow d$ — следующее:

$\text{Subst}(\theta, p_i) \wedge \dots \wedge \text{Subst}(\theta, p_n) \Rightarrow \text{Subst}(\theta, g)$ Теперь подстановка θ в обобщенном правиле отделения определена так, что $\text{Subst}(\theta, p_i) = \text{Subst}(\theta, p_j)$ для всех i , поэтому первое из этих двух высказываний точно совпадает с предпосылкой второго высказывания. Таким образом, выражение $\text{Subst}(\theta, g)$ следует из правила отделения.

Как принято выражаться в логике, обобщенное правило отделения представляет собой $\wedge$ поднятую версию правила отделения — оно поднимает правило отделения из пропозициональной логики в логику первого порядка. В оставшейся части этой главы будет показано, что могут быть разработаны поднятые версии алгоритмов прямого логического вывода, обратного логического вывода и резолюции, представленных в главе 7. Основным преимуществом применения поднятых правил логиче-

ского вывода по сравнению с пропозиционализацией является то, что в них предусмотрены только те подстановки, которые требуются для обеспечения дальнейшего выполнения конкретных логических выводов. Единственное соображение, которое может вызвать недоумение у читателя, состоит в том, что в

определенном смысле обобщенное правило вывода является менее общим, чем исходное правило отделения (с. 303): правило отделения допускает применение в левой части импликации любого отдельно взятого высказывания os , а обобщенное правило отделения требует, чтобы это высказывание имело специальный формат. Но оно является обобщенным в том смысле, что допускает применение любого количества выражений pL .

Унификация Применение поднятых правил логического вывода связано с необходимостью поиска подстановок, в результате которых различные логические выражения становятся идентичными. Этот процесс называется $\wedge$ унификацией и является ключевым компонентом любых алгоритмов вывода в логике первого порядка. Алгоритм Unify принимает на входе два высказывания и возвращает для них $\wedge$ унификатор, если таковой существует:

$Unify(p, g) = \exists$ где $Subst(e, p) = Subst(e, g)$ Рассмотрим несколько примеров того, как должен действовать алгоритм Unify.

Предположим, что имеется запрос $Knows(John, x)$ — кого знает Джон? Некоторые ответы на этот запрос можно найти, отыскивая все высказывания в базе знаний, которые унифицируются с высказыванием $Knows(John, x)$. Ниже приведены результаты унификации с четырьмя различными высказываниями, которые могут находиться в базе знаний.

$Unify(Knows(John, x), Knows(John, Jane)) = \{x/Jane\}$ $Unify(Knows(John, x), Knows(y, Bill)) = \{x/Bill, y/John\}$ $Unify(Knows(John, x), Knows(y, Mother(y))) = \{y/John, x/Mother(John)\}$ $Unify(Knows(John, x), Knows(x, Elizabeth)) = fail$ Последняя попытка унификации оканчивается неудачей (fail), поскольку переменная x не может одновременно принимать значения John и Elizabeth.

Теперь вспомним, что высказывание $Knows(x, Elizabeth)$ означает "Все знают Элизабет", поэтому мы обязаны иметь возможность вывести логически, что Джон знает Элизабет. Проблема возникает только потому, что в этих двух высказываниях, как оказалось, используется одно и то же имя переменной, x . Возникновения этой проблемы можно избежать, $\wedge$ стандартизируя отличие (standardizing apart) одного из этих двух унифицируемых высказываний; под этой операцией подразумевается переименование переменных в высказываниях для предотвращения коллизий имен.

Например, переменную x в высказывании $Knows(x, Elizabeth)$ можно переименовать в $z17$ (новое имя переменной), не меняя смысл этого высказывания. После этого унификация выполняется успешно:

$\text{Unify}(\text{Knows}(\text{John}, x), \text{Knows}(z_n, \text{Elizabeth})) = \{x/\text{Elizabeth}, z_n/\text{John}\}$ С дополнительными сведениями о том, с чем связана необходимость в стандартизации отличия, можно ознакомиться в упр. 9.7.

Возникает еще одна сложность: выше было сказано, что алгоритм `Unify` должен возвращать такую подстановку (или унификатор), в результате которой два параметра становятся одинаковыми. Но количество таких унификаторов может быть больше единицы. Например, вызов алгоритма `Unify(Knows{John, x}, Knows(y, z))` может вернуть $\{y/\text{John}, x/z\}$ или $\{y/\text{John}, x/\text{John}, z/\text{John}\}$. Первый унификатор позволяет получить в качестве результата унификации выражение `Knows{John, z}`, а второй дает `Knows(John, John)`. Но второй результат может быть получен из первого с помощью дополнительной подстановки $\{z/\text{John}\}$, в таком случае принято считать, что первый унификатор является более общим по сравнению со вторым, поскольку налагает меньше ограничений на значения переменных. Как оказалось, для любой унифицируемой пары выражений существует единственный ^наиболее общий унификатор (Most General Unifier— MGU), который является уникальным вплоть до переименования переменных. В данном случае таковым является $\{y/\text{John}, x/z\}$.

Алгоритм вычисления наиболее общих унификаторов приведен в листинге 9.1.

Процесс его работы очень прост: рекурсивно исследовать два выражения одновременно, "бок о бок", наряду с этим формируя унификатор, но создавать ситуацию неудачного завершения, если две соответствующие точки в полученных таким образом структурах не совпадают. При этом существует один дорогостоящий этап: если переменная согласуется со сложным термом, необходимо провести проверку того, встречается ли сама эта переменная внутри терма; в случае положительного ответа на данный вопрос согласование оканчивается неудачей, поскольку невозможно сформировать какой-либо совместимый унификатор. Из-за этой так называемой ^ проверки вхождения (occure check) сложность всего алгоритма становится квадратично зависимой от размера унифицируемых выражений. В некоторых системах, включая все системы логического программирования, просто исключается такая проверка вхождения и поэтому в результате иногда формируются противоречивые логические выводы, а в других системах используются более развитые алгоритмы со сложностью, линейно зависящей от времени.

Листинг 9.1. Алгоритм унификации. Алгоритм действует путем поэлементного сравнения структур входных высказываний. В ходе этого формируется подстановка 8, которая также является параметром функции `Unify` и используется для проверки того, что дальнейшие сравнения совместимы со связываниями, которые были определены ранее. В составном выражении, таком как `F(A, v)`, функция `Op` выбирает функциональный символ `F`, а функция `Args` выбирает список параметров `(A, B)`

```
function Unify(x, y, 0) returns подстановка, позволяющая сделать x и y идентичными
```

inputs: x , переменная, константа, список или составной терм y , переменная, константа, список или составной терм 0 , подстановка, подготовленная до сих пор (необязательный параметр, по умолчанию применяется пустой терм) if $\exists = \text{failure}$ then return failure else if $x = y$ then return 0 else if $\text{Variable?}(x)$ then return $\text{Unify-Var}(x, y, 0)$ else if $\text{Variable?}(y)$ then return $\text{Unify-Var}(y, x, 0)$ else if $\text{Compound?}(x)$ and $\text{Compound?}(y)$ then

return $\text{Unify}(\text{Args}[x], \text{Args}[y], \text{Unify}(\text{Op}[x], \text{Op}[y], 8))$ else if $\text{List?}(x)$ and $\text{List?}(y)$ then return $\text{Unify}(\text{Rest}[x], \text{Rest}[y], \text{Unify}(\text{First}[x], \text{First}[y], 0))$ else return failure function $\text{Unify-Var}\{\text{var}, x, 0\}$ returns подстановка inputs: var, переменная x , любое выражение 0 , подстановка, подготовленная до сих пор if $\{\text{var}/\text{val}\} \supseteq 0$ then return $\text{Unify}(\text{val}, x, 0)$ else if $\{x/\text{val}\} \supseteq 0$ then return $\text{Unify}(\text{var}, \text{val}, 0)$ else if $\text{Occur-Check?}(\text{var}, x)$ then return failure else return добавление $\{\text{var}/x\}$ к подстановке 0 Хранение и выборка В основе функций Tell и Ask, применяемых для ввода информации и передачи запросов в базу знаний, лежат более примитивные функции Store и Fetch. Функция Store (s) сохраняет некоторое высказывание s в базе знаний, а функция Fetch (q) возвращает все унификаторы, такие, что запрос q унифицируется с некоторым высказыванием из базы знаний. Описанная выше задача, служившая для иллюстрации процесса унификации (поиск всех фактов, которые унифицируются с высказыванием Knows (John, x)), представляет собой пример применения функции Fetch.

Проще всего можно реализовать функции Store и Fetch, предусмотрев хранение всех фактов базы знаний в виде одного длинного списка, чтобы затем, после получения запроса q , можно было просто вызывать алгоритм $\text{Unify}(g, s)$ для каждого высказывания s в списке. Такой процесс является неэффективным, но он осуществим, и знать об этом — это все, что нужно для понимания последней части данной главы. А в оставшейся части данного раздела описаны способы, позволяющие обеспечить более эффективную выборку, и он может быть пропущен при первом чтении.

Функцию Fetch можно сделать более эффективной, обеспечив, чтобы попытки унификации применялись только к высказываниям, имеющим определенный шанс на унификацию. Например, нет смысла пытаться унифицировать Knows (John, x) и Brother (Richard, John). Такой унификации можно избежать, ^индексируя факты в базе знаний. Самая простая схема, называемая ^ индексацией по предикатам, предусматривает размещение всех фактов Knows в одном сегменте, а всех фактов Brother — в другом. Сами сегменты для повышения эффективности доступа можно хранить в хэш-таблице².

Индексация по предикатам является удобной, когда имеется очень много предикатных символов, но лишь небольшое количество выражений в расчете на каждый символ. Однако в некоторых приложениях имеется много выражений в расчете на 2 Хэш-таблица — эта структура данных для хранения и выборки

информации, индексируемой с помощью фиксированных ключей. С точки зрения практики хэш-таблица может рассматриваться как имеющая постоянные временные показатели хранения и выборки, даже если эта таблица содержит очень большое количество элементов.

каждый конкретный предикатный символ. Например, предположим, что налоговые органы желают следить за тем, кто кого нанимает, с использованием предиката `Employs (x, y)`. Такой сегмент, возможно, состоящий из миллионов нанимателей и десятков миллионов наемных работников, был бы очень большим. Для поиска ответа на такой запрос, как `Employs (x, Richard)` ("Кто является нанимателем Ричарда?"), при использовании индексации по предикатам потребовался бы просмотр всего сегмента.

Поиск ответа на данный конкретный запрос стал бы проще при использовании индексации фактов и по предикату, и по второму параметру, возможно, с использованием комбинированного ключа хэш-таблицы. В таком случае существовала бы возможность просто формировать ключ из запроса и осуществлять выборку именно тех фактов, которые унифицируются с этим запросом. А для ответа на другие запросы, такие как `Employs (AIMA.org, y)`, нужно было бы индексировать факты, комбинируя предикат с первым параметром. Поэтому факты могут храниться под разными индексными ключами, что позволяет моментально сделать их доступными для разных запросов, с которыми они могли бы унифицироваться.

Если дано некоторое высказывание, которое подлежит хранению, то появляется возможность сформировать индексы для всех возможных запросов, которые унифицируются с ними. Применительно к факту `Employs (AIMA.org, Richard)` возможны следующие запросы:

`Employs(AIMA.org,Richard)` Является ли организация AIMA.org нанимателем Ричарда?

`Employs(x,Richard)` Кто является нанимателем Ричарда?

`Employs (AIMA.org,y)` Для кого является нанимателем организация AIMA.org?

`Employs(x,y)` Кто для кого является нанимателем? Как показано на рис. 9.1, а, эти запросы образуют $\wedge$ решетку обобщения. Такая решетка обладает некоторыми интересными свойствами. Например, дочерний узел любого узла в этой решетке может быть получен из его родительского узла с помощью единственной подстановки, а "наибольший" общий потомок любых двух узлов является результатом применения наиболее общего унификатора для этих узлов. Та часть решетки, которая находится выше любого базового факта, может быть сформирована систематически (упр. 9.5). Высказывание с повторяющимися константами имеет несколько иную решетку, как показано на рис. 9Л, б. Наличие

функциональных символов и переменных в высказываниях, подлежащих хранению, приводит к появлению еще более интересных структур решетки.

Employs(x,y) Employs(x, Richard) Employs(AIMA.org,y) Employs(xjohn) Employs(x,y)
Employs(John,y) Employs(AIMA.org, Richard) Employs(John,John) а) б) Рис. 9.1. Примеры
решеток обобщения: решетка обобщения, самым нижним узлом которой является
высказывание Employs fAIMA.org,Richard; (a); решетка обобщения для высказывания
Employs (John, John) (б)

Только что описанная схема применяется очень успешно, если решетка содержит небольшое количество узлов. А если предикат имеет p параметров, то решетка включает $O(p)$ узлов. Если же разрешено применение функциональных символов, то количество узлов также становится экспоненциально зависимым от размера термов в высказывании, подлежащем хранению. Это может вызвать необходимость создания огромного количества индексов. В какой-то момент затраты на хранение и сопровождение всех этих индексов перевесят преимущества индексации. Для выхода из этой ситуации можно применять какой-либо жесткий подход, например сопровождать индексы только на ключах, состоящих из некоторого предиката плюс каждый параметр, или адаптивный подход, в котором предусматривается создание индексов в соответствии с потребностями в поиске ответов на запросы того типа, которые встречаются наиболее часто. В большинстве систем искусственного интеллекта количество фактов, подлежащих хранению, является достаточно небольшим для того, чтобы проблему эффективной индексации можно было считать решенной.

А что касается промышленных и коммерческих баз данных, то эта проблема стала предметом значительных и продуктивных технологических разработок.

9.3. Прямой логический вывод Алгоритм прямого логического вывода для пропозициональных определенных выражений приведен в разделе 7.5. Его идея проста: начать с атомарных высказываний в базе знаний и применять правило отделения в прямом направлении, добавляя все новые и новые атомарные высказывания до тех пор, пока не возникнет ситуация, в которой невозможно будет продолжать формулировать логические выводы. В данном разделе приведено описание того, как можно применить этот алгоритм к определенным выражениям в логике первого порядка и каким образом он может быть реализован эффективно. Определенные выражения, такие как Situation=>Response, особенно полезны для систем, в которых логический вывод осуществляется в ответ на вновь поступающую информацию. Таким образом могут быть определены многие системы, а формирование рассуждений с помощью прямого логического вывода может оказаться гораздо более эффективным по сравнению с доказательством теорем с помощью резолюции. Поэтому часто имеет смысл попытаться сформировать базу

знаний с использованием только определенных выражений, чтобы избежать издержек, связанных с резолюцией.

Определенные выражения в логике первого порядка Определенные выражения в логике первого порядка весьма напоминают определенные выражения в пропозициональной логике (с. 312): они представляют собой дизъюнкции литералов, среди которых положительным является один и только один. Определенное выражение либо является атомарным, либо представляет собой импликацию, антецедентом (предпосылкой) которой служит конъюнкция положительных литералов, а консеквентом (следствием) — единственный положительный литерал. Ниже приведены примеры определенных выражений в логике первого порядка.

$\text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$ $\text{King}(\text{John}) \wedge \text{Greedy}(y)$ В отличие от пропозициональных литералов, литералы первого порядка могут включать переменные, и в таком случае предполагается, что на эти переменные распространяется квантор всеобщности. (Как правило, при написании определенных выражений кванторы всеобщности исключаются.) Определенные выражения представляют собой подходящую нормальную форму для использования в обобщенном правиле отделения.

Не все базы знаний могут быть преобразованы в множество определенных выражений из-за того ограничения, что положительный литерал в них должен быть единственным, но для многих баз знаний такая возможность существует. Рассмотрим приведенную ниже задачу.

Закон гласит, что продажа оружия недружественным странам, осуществляемая любым американским гражданином, считается преступлением. В государстве Ноуноу, враждебном по отношению к Америке, имеются некоторые ракеты, и все ракеты этого государства были проданы ему полковником Уэстом, который является американским гражданином.

Мы должны доказать, что полковник Уэст совершил преступление. Вначале все имеющиеся факты будут представлены в виде определенных выражений в логике первого порядка, а в следующем разделе будет показано, как решить эту задачу с помощью алгоритма прямого логического вывода. • "...продажа оружия враждебным странам, осуществляемая любым американским гражданином, является преступлением":

$\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$ (9.3) • "В государстве Ноуноу... имеются некоторые ракеты". Высказывание $\exists x \text{ Owns}(\text{Nono}, x) \wedge \text{Missile}(x)$ преобразуется в два определенных выражения путем устранения квантора существования и введения новой константы Mx !

$\text{Owns}\{\text{Nono}, \text{Mi}\}$ (9.4) $\text{Missile}\{\text{M1}\}$ (9.5) • "... все ракеты этого государства были проданы ему полковником Уэстом":

$\text{Missile}\{x\} \wedge \text{Owns}\{\text{Nono}, x\} \Rightarrow \text{Sells}\{\text{West}, x, \text{Nono}\}$ (9.6) Нам необходимо также знать, что ракеты — оружие:

$\text{Missile}\{x\} \Rightarrow \text{Weapon}\{x\}$ (9.7) Кроме того, мы должны знать, что государство, враждебное по отношению к Америке, рассматривается как "недружественное":

$\text{Enemy}\{x, \text{America}\} \Rightarrow \text{Hostile}\{x\}$ (9.8) • "... полковником Уэстом, который является американским гражданином":

$\text{American}\{\text{West}\}$ (9.9) • "В государстве Ноуноу, враждебном по отношению к Америке...":

$\text{Enemy}\{\text{Nono}, \text{America}\}$ (9.10)

Эта база знаний не содержит функциональных символов и поэтому может служить примером класса баз знаний языка $\wedge$ Datalog, т.е. примером множества определенных выражений в логике первого порядка без функциональных символов.

Ниже будет показано, что при отсутствии функциональных символов логический вывод становится намного проще.

Простой алгоритм прямого логического вывода Как показано в листинге 9.2, первый рассматриваемый нами алгоритм прямого логического вывода является очень простым. Начиная с известных фактов, он активизирует все правила, предпосылки которых выполняются, и добавляет заключения этих правил к известным фактам. Этот процесс продолжается до тех пор, пока не обнаруживается ответ на запрос (при условии, что требуется только один ответ) или больше не происходит добавление новых фактов. Следует отметить, что факт не является "новым", если он представляет собой $\wedge$ переименование известного факта.

Одно высказывание называется переименованием другого, если оба эти высказывания идентичны во всем, за исключением имен переменных. Например, высказывания $\text{Likes}(x, \text{IceCream})$ и $\text{Likes}(y, \text{IceCream})$ представляют собой переименования по отношению друг к другу, поскольку они отличаются лишь выбором имени переменной, x или y ; они имеют одинаковый смысл — все любят мороженое.

Листинг 9.2. Концептуально простой, но очень неэффективный алгоритм прямого логического вывода. В каждой итерации он добавляет к базе знаний кв все атомарные высказывания, которые могут быть выведены за один этап из импликационных высказываний и атомарных высказываний, которые уже находятся в базе знаний `function FOL-FC-Ask(KB, a) returns подстановка или значение false`

inputs: KB, база знаний - множество определенных выражений a , запрос - атомарное высказывание $local\ variables: new$, новые высказывания, выводимые в каждой итерации $repeat\ until$ множество new не пусто $new \leftarrow \{\}$ for each высказывание g in KB do $(P_1 \wedge \dots \wedge P_n \Rightarrow q) \leftarrow rStandardize-Apart(r)$ for each подстановка θ , такая что $Subst(F, p_1 \theta, \dots, p_n \theta) = Subst(Q, p_1 \theta, \dots, p_n \theta)$ для некоторых $p_1, \dots, p_n$ в базе знаний KB $d' \leftarrow Subst(a, \theta)$ if выражение d' не является переименованием некоторого высказывания, которое уже находится в KB, или рассматривается как элемент множества new then do добавить d' к множеству new $\phi \leftarrow Unify(g, a)$ if значение ϕ не представляет собой fail then return ϕ добавить множество new к базе знаний KB return false

Для иллюстрации работы алгоритма FOL-FC-Ask воспользуемся описанной выше задачей доказательства преступления. Импликационными высказываниями являются высказывания, приведенные в уравнениях 9.3, 9.6-9.8. Требуются следующие две итерации. • В первой итерации правило 9.3 имеет невыполненные предпосылки.

Правило 9.6 выполняется с подстановкой $\{x/M\}$ и добавляется высказывание $Sells(West, Mlt\ Nono)$.

Правило 9.7 выполняется с подстановкой $\{x/M\}$ и добавляется высказывание $Weapon(Mi)$.

Правило 9.8 выполняется с подстановкой $\{x/Nono\}$ и добавляется высказывание $Hostile(Nono)$. • На второй итерации правило 9.3 выполняется с подстановкой $\{x/West, y/Mlt\ z/Nono\}$ и добавляется высказывание $Criminal(West)$.

Сформированное дерево доказательства показано на рис. 9.2. Обратите внимание на то, что в этот момент невозможны какие-либо новые логические выводы, поскольку каждое высказывание, заключение которого можно было бы найти с помощью прямого логического вывода, уже явно содержится в базе знаний KB. Такое состояние базы знаний называется фиксированной точкой (fixed point) в процессе логического вывода. Фиксированные точки, достигаемые при прямом логическом выводе с использованием определенных выражений первого порядка, аналогичны фиксированным точкам, возникающим при пропозициональном прямом логическом выводе (с. 315); основное различие состоит в том, что фиксированная точка первого порядка может включать атомарные высказывания с квантором всеобщности.

$Criminal(West) Hostile(Nono) Owns(Nono, Mj) Enemy(Nono, America)$ Рис. 9.2. Дерево доказательства, сформированное путем прямого логического вывода в примере доказательства преступления. Первоначальные факты показаны на нижнем уровне, факты, выведенные логическим путем в первой итерации, — на среднем уровне, а факт, логически выведенный во второй итерации, — на верхнем уровне Свойства алгоритма FOL-FC-Ask проанализировать несложно. Во-первых, он является

непротиворечивым, поскольку каждый этап логического вывода представляет собой применение обобщенного правила отделения, которое само является непротиворечивым. Во-вторых, он является полным применительно к базам знаний с определенными выражениями; это означает, что он позволяет ответить на любой запрос, ответы на который следуют из базы знаний с определенными выражениями.

Для баз знаний Datalog, которые не содержат функциональных символов, доказательство полноты является довольно простым. Начнем с подсчета количества возможных фактов, которые могут быть добавлены, определяющего максимальное количество итераций. Допустим, что k — максимальная арифность (количество параметров) любого предиката, p — количество предикатов и n — количество константных символов. Очевидно, что может быть не больше чем $p \cdot k^n$ различных базовых фактов, поэтому алгоритм должен достичь фиксированной точки именно после стольких итераций. В таком случае можно применить обоснование приведенного выше утверждения, весьма аналогичное доказательству полноты пропозиционального прямого логического вывода (см. с. 315). Подробные сведения о том, как осуществить переход от пропозициональной полноты к полноте первого порядка, приведены применительно к алгоритму резолюции в разделе 9.5.

При его использовании к более общим определенным выражениям с функциональными символами алгоритм FOL-FC-Ask может вырабатывать бесконечно большое количество новых фактов, поэтому необходимо соблюдать исключительную осторожность. Для того случая, в котором из базы знаний следует ответ на высказывание запроса d , необходимо прибегать к использованию теоремы Эрбрана для обеспечения того, чтобы алгоритм мог найти доказательство (случай, касающийся резолюции, описан в разделе 9.5). А если запрос не имеет ответа, то в некоторых случаях может оказаться, что не удастся нормально завершить работу данного алгоритма. Например, если база знаний включает аксиомы Пеано:

$\text{NatNum}\{0\} \cup \text{NatNum}\{n\} \Rightarrow \text{NatNum}\{S(n)\}$ то в результате прямого логического вывода будут добавлены факты $\text{NatNum}(S(0))$, $\text{NatNum}(S(S(0)))$, $\text{NatNum}(S(S(S(0))))$ и т.д. Вообще говоря, избежать возникновения этой проблемы невозможно. Как и в общем случае логики первого порядка, задача определения того, следуют ли высказывания из базы знаний, сформированной с использованием определенных выражений, является полурешимой.

Эффективный прямой логический вывод Алгоритм прямого логического вывода, приведенный в листинге 9.2, был спроектирован не с целью обеспечения эффективного функционирования, а, скорее, с целью упрощения его понимания. Существуют три возможных источника осложнений в его работе. Во-первых, "внутренний цикл" этого алгоритма предусматривает поиск всех возможных унификаторов, таких, что предпосылка некоторого правила унифицируется с

подходящим множеством фактов в базе знаний. Такая операция часто именуется $\wedge$ согласованием с шаблоном и может оказаться очень дорогостоящей. Во-вторых, в этом алгоритме происходит повторная проверка каждого правила в каждой итерации для определения того, выполняются ли его предпосылки, даже если в базу знаний в каждой итерации вносится лишь очень немного дополнений. В-третьих, этот алгоритм может вырабатывать много фактов, которые не имеют отношения к текущей цели.

Устраним каждый из этих источников неэффективности по очереди.

Согласование правил с известными фактами Проблема согласования предпосылки правила с фактами, хранящимися в базе знаний, может показаться достаточно простой. Например, предположим, что требуется применить следующее правило:

$\text{Missile}(x) \Rightarrow \text{Weapon}(x)$ Для этого необходимо найти все факты, которые согласуются с выражением $\text{Missile}(x)$; в базе знаний, индексируемой подходящим образом, это можно выполнить за постоянное время в расчете на каждый факт. А теперь рассмотрим правило, подобное следующему:

$\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$ Найти все объекты, принадлежащие государству Ноуноу, опять-таки можно за постоянное время в расчете на каждый объект; затем мы можем применить к каждому объекту проверку, является ли он ракетой. Но если в базе знаний содержится много сведений об объектах, принадлежащих государству Ноуноу, и лишь немного данных о ракетах, то было бы лучше вначале найти все ракеты, а затем проверить, какие из них принадлежат Ноуноу. Это — проблема $\wedge$ упорядочения конъюнктов: поиск упорядочения, позволяющего решать конъюнкты в предпосылке правила таким образом, чтобы общая стоимость решения была минимальной. Как оказалось, задача поиска оптимального упорядочения является NP-трудной, но имеются хорошие эвристики. Например, эвристика с наиболее ограниченной переменной, применявшаяся при решении задач CSP в главе 5, подсказывает, что необходимо упорядочить конъюнкты так, чтобы вначале проводился поиск ракет, если количество ракет меньше по сравнению с количеством всех известных объектов, принадлежащих государству Ноуноу.

Между процедурами согласования с шаблоном и удовлетворения ограничений действительно существует очень тесная связь. Каждый конъюнкт может рассматриваться как ограничение на содержащиеся в нем переменные; например, $\text{Missile}(x)$ — это унарное ограничение на x . Развивая эту идею, можно прийти к выводу, что $\wedge$ существует возможность представить любую задачу CSP с конечной областью определения как единственное определенное выражение наряду с некоторыми касающимися ее базовыми фактами. Рассмотрим приведенную на рис. 5.1 задачу раскраски карты, которая снова показана на рис. 9.3, а. Эквивалентная

формулировка в виде одного определенного выражения приведена на рис. 9.3, б. Очевидно, что заключение Colorable () можно вывести из этой базы знаний, только если данная задача CSP имеет решение. А поскольку задачи CSP, вообще говоря, включают задачи 3-SAT в качестве частных случаев, на основании этого можно сделать вывод, что $\wedge$ задача согласования определенного выражения с множеством фактов является NP-трудной.

То, что во внутреннем цикле алгоритма прямого логического вывода приходится решать NP-трудную задачу согласования, может показаться на первый взгляд довольно неприятным. Тем не менее, есть следующие три фактора, благодаря которым эта проблема предстает немного в лучшем свете. • Напомним, что большинство правил в базах знаний, применяемых на практике, являются небольшими и простыми (подобно правилам, используемым в примере доказательства преступления), а не большими и сложными (как в формулировке задачи CSP, приведенной на рис. 9.3). В мире пользователей

баз данных принято считать, что размеры правил и арности предикатов не превышают некоторого постоянного значения, и принимать во внимание только $\wedge$ сложность данных, т.е. сложность логического вывода как функции от количества базовых фактов в базе данных. Можно легко показать, что обусловленная данными сложность в прямом логическом выводе определяется полиномиальной зависимостью.

Diff(wa, nt) $\wedge$ Diff(wa, sa) $\wedge$ Diff(nt,q) $\wedge$ Diff(nt,sa) $\wedge$ Diff(q, nsw) $\wedge$ Diff(q, sa) $\wedge$ Diff(nsw, v) $\wedge$ Diff(nsw, sa) $\wedge$ Diff(v, sa) $\Rightarrow$ ColorableQ Dig (Red, Blue) Diff(Red, Green) Diff (Green, Red) Diff (Green, Blue) Diff(Blue, Red) Diff (Blue, Green) а) 6) Рис. 9.3. Иллюстрация связи между процессами согласования с шаблоном и удовлетворения ограничений: граф ограничений для раскрашивания карты Австралии (см. рис. 5.1) (а); задача CSP раскрашивания карты, представленная в виде единственного определенного выражения (б). Обратите внимание на то, что области определения переменных заданы неявно с помощью констант, приведенных в базовых фактах для предиката Diff • Могут рассматриваться подклассы правил, для которых согласование является наиболее эффективным. По сути, каждое выражение на языке Datalog может рассматриваться как определяющее некоторую задачу CSP, поэтому согласование будет осуществимым только тогда, когда соответствующая задача CSP является разрешимой. Некоторые разрешимые семейства задач CSP описаны в главе 5. Например, если граф ограничений (граф, узлами которого являются переменные, а дугами — ограничения) образует дерево, то задача CSP может быть решена за линейное время. Точно такой же результат остается в силе для согласования с правилами. Например, если из карты, приведенной на рис. 9.3, будет удален узел SA, относящийся к Южной Австралии, то результирующее выражение примет следующий вид:

$\text{Diff}\{wa, nt\} \sqcup \text{Diff}(nt, q) \sqcup \text{Diff}(q, nsw) \sqcup \text{Diff}\{nsw, v\} \Rightarrow \text{ColorableO}$ что соответствует сокращенной задаче CSP, показанной на рис. 5.7. Для решения задачи согласования с правилами могут непосредственно применяться алгоритмы решения задач CSP с древовидной структурой. • Можно приложить определенные усилия по устранению излишних попыток согласования с правилами в алгоритме прямого логического вывода, что является темой следующего раздела.

Инкрементный прямой логический вывод Когда авторы демонстрировали в предыдущем разделе на примере доказательства преступления, как действует прямой логический вывод, они немного схитрили; в частности, не показали некоторые из согласований с правилами, выполняемые алгоритмом, приведенным в листинге 9.2. Например, во второй итерации правило $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$ согласуется с фактом $\text{Missile}(Mx)$ (еще раз), и, безусловно, при этом ничего не происходит, поскольку заключение $\text{Weapon}(M)$ уже известно. Таких излишних согласований с правилами можно избежать, сделав следующее наблюдение: $\wedge$ каз/сдый новый факт, выведенный в итерации t , должен быть получен по меньшей мере из одного нового факта, выведенного в итерации $t-1$. Это наблюдение соответствует истине, поскольку любой логический вывод, который не требовал нового факта из итерации $t-1$, уже мог быть выполнен в итерации $t-1$.

Такое наблюдение приводит естественным образом к созданию алгоритма инкрементного прямого логического вывода, в котором в итерации t проверка правила происходит, только если его предпосылка включает конъюнкты, который унифицируется с фактом P_i' , вновь выведенным в итерации $t-1$. Затем на этапе согласования с правилом значение $p^{\wedge}$ фиксируется для согласования с $p^{\pm}$, но при этом допускается, чтобы остальные конъюнкты в правиле согласовывались с фактами из любой предыдущей итерации. Этот алгоритм в каждой итерации вырабатывает точно такие же факты, как и алгоритм, приведенный в листинге 9.2, но является гораздо более эффективным.

При использовании подходящей индексации можно легко выявить все правила, которые могут быть активизированы любым конкретным фактом. И действительно, многие реальные системы действуют в режиме "обновления", при котором прямой логический вывод происходит в ответ на каждый новый факт, сообщенный системе с помощью операции Tell. Операции логического вывода каскадно распространяются через множество правил до тех пор, пока не достигается фиксированная точка, а затем процесс начинается снова, вслед за поступлением каждого нового факта.

Как правило, в результате добавления каждого конкретного факта в действительности активизируется лишь небольшая доля правил в базе знаний. Это означает, что при повторном конструировании частичных согласований с

правилами, имеющими некоторые невыполненные предпосылки, выполняется существенный объем ненужной работы.

Рассматриваемый здесь пример доказательства преступления слишком мал, чтобы на нем можно было наглядно показать такую ситуацию, но следует отметить, что частичное согласование конструируется в первой итерации между следующим правилом:

American{x} л Weaponry) л Sells{x,y, z) л Hostileiz) => Criminal (x) и фактом American (West). Затем это частичное согласование отбрасывается и снова формируется во второй итерации (в которой данное правило согласуется успешно). Было бы лучше сохранять и постепенно дополнять частичные согласования по мере поступления новых фактов, а не отбрасывать их.

В ^ rete-алгоритме³ была впервые предпринята серьезная попытка решить эту проблему. Алгоритм предусматривает предварительную обработку множества пра³ Здесь rete — латинское слово, которое переводится как сеть и читается по-русски "рете", а по-английски — "рити".

вил в базе знаний для формирования своего рода сети потока данных (называемой rete-сетью), в которой каждый узел представляет собой литерал из предпосылки какого-либо правила. По этой сети распространяются операции связывания переменных и останавливаются после того, как в них не удастся выполнить согласование с каким-то литералом. Если в двух литералах некоторого правила совместно используется какая-то переменная (например, Sells{x,y, z) л Hostileiz) в примере доказательства преступления), то варианты связывания из каждого литерала пропускаются через узел проверки равенства. Процессу связывания переменных, достигших узла n-арного литерала, такого как Sells{x,y, z), может потребоваться перейти в состояние ожидания того, что будут определены связывания для других переменных, прежде чем он сможет продолжаться. В любой конкретный момент времени состояние rete-сети охватывает все частичные согласования с правилами, что позволяет избежать большого объема повторных вычислений.

Не только сами rete-сети, но и различные их усовершенствования стали ключевым компонентом так называемых ^ продукционных систем, которые принадлежат к числу самых первых систем прямого логического вывода, получивших широкое распространение⁴. В частности, с использованием архитектуры продукционной системы была создана система Xcop (которая первоначально называлась R1) [1026].

Система Xcop содержала несколько тысяч правил и предназначалась для проектирования конфигураций компьютерных компонентов для заказчиков Digital Equipment Corporation. Ее создание было одним из первых очевидных успешных коммерческих проектов в развивающейся области экспертных систем. На основе

той же базовой технологии, которая была реализована на языке общего назначения Ops-5, было также создано много других подобных систем.

Кроме того, продукционные системы широко применяются в [^] когнитивных архитектурах (т.е. моделях человеческого мышления), в таких как ACT [31] и Soar [880]. В подобных системах "рабочая память" системы моделирует кратковременную память человека, а продукции образуют часть долговременной памяти. В каждом цикле функционирования происходит согласование продукции с фактами из рабочей памяти. Продукции, условия которых выполнены, могут добавлять или удалять факты в рабочей памяти. В отличие от типичных ситуаций с большим объемом данных, наблюдаемых в базах данных, продукционные системы часто содержат много правил и относительно немного фактов. При использовании технологии согласования, оптимизированной должным образом, некоторые современные системы могут оперировать в реальном времени больше чем с миллионом правил.

Не относящиеся к делу факты Последний источник неэффективности прямого логического вывода, повидимому, свойствен самому этому подходу и также возникает в контексте пропозициональной логики (см. раздел 7.5). Прямой логический вывод предусматривает выполнение всех допустимых этапов логического вывода на основе всех известных фактов, даже если они не относятся к рассматриваемой цели. В примере доказательства преступления не было правил, способных приводить к заключениям, не относящимся к делу, поэтому такое отсутствие направленности не вызывало каких-либо проблем. В других случаях (например, если бы в базу знаний было внесено несколько слов продукция в названии продукционная система обозначает правило "условие-действие".

ко правил с описанием кулинарных предпочтений американцев и цен на ракеты) алгоритм FOL-FC-Ask вырабатывал бы много нерелевантных заключений.

Один из способов предотвращения формирования нерелевантных заключений состоит в использовании обратного логического вывода, как описано в разделе 9.4.

Еще одно, второе решение состоит в том, чтобы ограничить прямой логический вывод избранным подмножеством правил; этот подход обсуждался в контексте пропозициональной логики. Третий подход сформировался в сообществе пользователей дедуктивных баз данных, для которых прямой логический вывод является стандартным инструментальным средством. Идея этого подхода состоит в том, чтобы перезаписывать множество правил с использованием информации из цели так, что в процессе прямого логического вывода рассматриваются только релевантные связывания переменных (принадлежащие к так называемому [^] магическому множеству).

Например, если целью является Criminal (West), то правило, приводящее к заключению Criminal (x), может быть перезаписано для включения дополнительного конъюнкта, ограничивающего значение x, следующим образом:

$\text{Magic}(x) \wedge \text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}\{x,y,z\} \wedge \text{Hostile}(z) \models \text{Criminal}(x)$ Факт Magic (West) также добавляется в базу знаний. Благодаря этому в процессе прямого логического вывода будет рассматриваться только факт о полковнике Уэсте, даже если база знаний содержит факты о миллионах американцев. Полный процесс определения магических множеств и перезаписи базы знаний является слишком сложным для того, чтобы мы могли заняться в этой главе его описанием, но основная идея состоит в том, что выполняется своего рода "универсальный" обратный логический вывод от цели для выяснения того, какие связывания переменных нужно будет ограничивать. Поэтому подход с использованием магических множеств может рассматриваться как гибридный между прямым логическим выводом и обратной предварительной обработкой.

9.4. Обратный логический вывод Во втором большом семействе алгоритмов логического вывода используется подход с обратным логическим выводом, представленный в разделе 7.5. Эти алгоритмы действуют в обратном направлении, от цели, проходя по цепочке от одного правила к другому, чтобы найти известные факты, которые поддерживают доказательство.

Мы опишем основной алгоритм, а затем покажем, как он используется в логическом программировании, представляющем собой наиболее широко применяемую форму автоматизированного формирования рассуждений. В этом разделе будет также показано, что обратный логический вывод имеет некоторые недостатки по сравнению с прямым логическим выводом, и описаны некоторые способы преодоления этих недостатков. Наконец, будет продемонстрирована тесная связь между логическим программированием и задачами удовлетворения ограничений.

Алгоритм обратного логического вывода Простой алгоритм обратного логического вывода, FOL-BC-Ask, приведен в листинге 9.3. Он вызывается со списком целей, содержащим единственный элемент

(первоначальный запрос) и возвращает множество всех подстановок, которые удовлетворяют этому запросу. Список целей можно рассматривать как "стек" целей, ожидающих отработки; если все они могут быть выполнены, то текущая ветвь доказательства формируется успешно. В алгоритме берется первая цель из списка и выполняется поиск в базе знаний всех выражений, положительный литерал которых (или голова) унифицируется с целью. При обработке каждого такого выражения создается новый рекурсивный вызов, в котором предпосылки (или тело) выражения добавляются к стеку целей. Напомним, что факты представляют собой выражения с головой, но без тела, поэтому, если какая-то цель унифицируется с известным

фактом, то к стеку не добавляются какие-то подцели, а сама эта цель считается получившей решение. На рис. 9.4 показано дерево доказательства для получения факта Criminal (West) из высказываний, приведенных в уравнениях 9.3—9.10.

American[^] West) Hostile(Nono) Missikiy) MissileWj) Owns(Nono,Mj)
Enemy{Nono,America) WM1} {} U {} Рис. 9.4. Дерево доказательства, сформированное путем обратного логического вывода для доказательства того, что полковник Уэст совершил преступление. Это дерево следует читать в глубину, слева направо. Чтобы доказать факт Criminal (West; необходимо доказать четыре конъюнкта, находящихся под ним. Некоторые из них находятся в базе знаний, а другие требуют дальнейшего обратного логического вывода. Связывания для каждой успешной унификации показаны после соответствующей подцели. Обратите внимание на, что после успешного достижения одной подцели в конъюнкции ее подстановка применяется для последующих подцелей. Таким образом, к тому времени, как алгоритм FOLBC-Ask достигает последнего конъюнкта, первоначально имевшего форму Hostile (z), переменная z уже связана с Nono Листинг 9.3. Простой алгоритм обратного логического вывода function FOL-BC-Ask(KB, goals, 6) returns множество подстановок inputs: KB, база знаний goals, список конъюнктов, образующих запрос (подстановка 0 уже применена) 0, текущая подстановка, первоначально пустая подстановка {} local variables: answers, ответы - множество подстановок, первоначально пустое if список goals пуст then return {} g' <- Subst@, First(goals))

for each высказывание r in KB, где Standardize-Apart(r) = (p₁ л ... л p_n => q) и 9' <rUnify (qr, g') является выполнимым new_goals <— [p₁,...,p_n, | Rest (goals)] answers <— FOL-BC-Ask(.FCB, new__goals, Compose (9', 9)) и answers return answers В этом алгоритме используется ^ композиция подстановок. Здесь Compose (Q1,92) — это подстановка, результат которой идентичен результату применения каждой подстановки по очереди, следующим образом:

Subst (Compose (9_i,92) ,p) = Subst (92, Subst (9_i,p)) В данном алгоритме текущие связывания переменных, которые хранятся в подстановке 9, komponуются со связываниями, возникающими в результате унификации цели с головой выражения, что приводит к получению нового множества текущих связываний для рекурсивного вызова.

Алгоритм обратного логического вывода в том виде, в каком он был приведен в этом разделе, безусловно, представляет собой алгоритм поиска в глубину. Это означает, что его потребности в пространстве линейно зависят от размера доказательства (если на данный момент пренебречь тем, какой объем пространства требуется для накопления решений). Это также означает, что обратный логический вывод (в отличие от прямого логического вывода) страдает от проблем, обусловленных наличием повторяющихся состояний и неполноты. Эти проблемы и некоторые потенциальные решения будут рассматриваться ниже, но вначале

покажем, как обратный логический вывод используется в системах логического программирования.

Логическое программирование Логическое программирование — это технология, позволяющая довольно близко приблизиться к воплощению декларативного идеала, описанного в главе 7, согласно которому системы должны конструироваться путем представления знаний на некотором формальном языке, а задачи решаться путем применения процессов логического вывода к этим знаниям. Такой идеал выражен в следующем уравнении Роберта Ковальского:

Алгоритм = Логика + Управление Одним из языков логического программирования, намного превосходящим все прочие по своей распространенности, является [^] Prolog. Количество его пользователей насчитывает сотни тысяч. Он используется в основном в качестве языка быстрой разработки прототипов, а также служит для решения задач символических манипуляций, таких как написание компиляторов [1536] и синтаксический анализ текстов на естественном языке [1208]. На языке Prolog было написано много экспертных систем для юридических, медицинских, финансовых и других проблемных областей.

Программы Prolog представляют собой множества определенных выражений, записанных в системе обозначений, немного отличающейся от используемой стандартной логики первого порядка. В языке Prolog прописные буквы применяются для обозначения переменных, а строчные — для обозначения констант. Выражения записываются с головой, предшествующей телу; символ : - служит для обозначения импликации, направленной влево, запятые разделяют литералы в теле, а точка обозначает конец высказывания, как показано ниже. `criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).`

Язык Prolog включает "синтаксические упрощения" (syntactic sugar) для обозначения списков и арифметических выражений. Например, ниже приведена программа Prolog для предиката `append (X, Y, Z)`, которая выполняется успешно, если список Z представляет собой результат дополнения списка Y списком X. `append([], Y, Y) . append([A|X],Y,[A|Z]) :- append(X,Y,Z).`

На естественном языке эти выражения можно прочесть так: во-первых, дополнение списка Y пустым списком приводит к получению того же списка Y, и, во-вторых, `[A | Z]` — это результат дополнения списка Y списком `[A | X]`, при условии, что Z — это результат дополнения списка Y списком X. Такое определение предиката `append` на первый взгляд кажется весьма подобным соответствующему определению на языке Lisp, но фактически является гораздо более мощным. Например, в систему можно ввести запрос `append (A, B, [1,2] )` — какие два списка можно дополнить один другим, чтобы получить `[1,2]`? Система возвратит следующие решения:

$A=[]$ $B=[1,2]$ $A=[1]$ $B=[2]$ $A=[1,2]$ $B=[]$ Выполнение программ Prolog осуществляется по принципу обратного логического вывода с поиском в глубину, при котором попытка применения выражений выполняется в том порядке, в каком они записаны в базу знаний. Но некоторые описанные ниже особенности языка Prolog выходят за рамки стандартного логического вывода. • В нем предусмотрено множество встроенных функций для выполнения арифметических операций. Литералы, в которых используются соответствующие функциональные символы, "доказываются" путем выполнения кода, а не осуществления дальнейшего логического вывода. Например, цель "x is 4+3" достигается успешно после связывания переменной X со значением 7. С другой стороны, попытка достижения цели is X+Y" оканчивается неудачей, поскольку эти встроенные функции не обеспечивают решения произвольных уравнений⁵. • В языке предусмотрены встроенные предикаты, вызывающие при их выполнении побочные эффекты. К ним относятся предикаты ввода-вывода и предикаты assert/retract для модификации базы знаний. Такие предикаты не имеют аналогов в логике и могут порождать некоторые эффекты, вызывающие путаницу, например, если факты подтверждаются (и вводятся в базу знаний) некоторой ветвью дерева доказательства, которая в конечном итоге оканчивается неудачей.

⁵ Следует отметить, что если бы в некоторой программе Prolog были предусмотрены аксиомы Пеано, то такие цели могли быть решены с помощью логического вывода.

- В языке Prolog допускается определенная форма отрицания —отрицание как недостижение цели. Отрицаемая цель not P считается доказанной, если системе не удастся доказать P. Таким образом, следующее высказывание: alive(X) :- not dead(X). можно прочесть так: "Любого следует считать живым, если нельзя доказать, что он мертв".
- В языке Prolog предусмотрен оператор равенства, "=", но он не обладает всей мощью логического равенства. Цель с оператором равенства достигается успешно, если в ней два терма являются унифицируемыми, а в противном случае попытка ее достижения оканчивается неудачей. Таким образом, цель $x+Y=2+3$ достигается успешно, после связывания переменной X со значением 2, а Y — со значением 3, а попытка достижения цели mornings tar=evenings tar оканчивается неудачей. (В классической логике последнее равенство может быть или не быть истинным.) Не могут быть подтверждены (введены в базу знаний) какие-либо факты или правила, касающиеся равенства.
- Из алгоритма унификации Prolog исключена проверка вхождения. Это означает, что могут быть сделаны некоторые противоречивые логические выводы; такая проблема возникает редко, за исключением тех ситуаций, когда язык Prolog используется для доказательства математических теорем.

Решения, принятые при проектировании языка Prolog, представляют собой компромисс между стремлениями обеспечить декларативность и вычислительную эффективность (по крайней мере, эффективность в той ее трактовке, которая

существовала в период разработки языка Prolog). Мы вернемся к этой теме после рассмотрения того, как реализована система Prolog.

Эффективная реализация логических программ Подготовка программ Prolog к выполнению может осуществляться в двух режимах: интерпретация и компиляция. Интерпретация по существу представляет собой применение алгоритма FOL-BC-Ask, приведенного в листинге 9.3, к программе, представленной в виде базы знаний. Предыдущее предложение включает слова "по существу", поскольку в интерпретаторах Prolog предусмотрены всевозможные усовершенствования, предназначенные для максимального повышения скорости работы. Здесь рассматриваются только два таких усовершенствования.

Во-первых, вместо формирования списка всех возможных ответов для каждой подцели перед переходом к следующей подцели интерпретаторы Prolog вырабатывают один ответ и "дают обещание" выработать остальные после того, как будет полностью исследован текущий ответ. Такое "обещание" оформляется как $\wedge$ точка выбора (choice point). После того как в процессе поиска в глубину завершается исследование возможных решений, вытекающих из текущего ответа, и происходит возврат к точке выбора, в этой точке используемые структуры данных дополняются, чтобы в них можно было включить новый ответ для данной подцели и сформировать новую точку выбора. Такой подход позволяет экономить и время, и пространство, а также обеспечивает создание очень простого интерфейса для отладки, поскольку постоянно существует лишь единственный путь решения, подлежащий рассмотрению.

Во-вторых, в приведенной в данной главе простой реализации алгоритма FOLBC-Ask много времени затрачивается на выработку и компоновку подстановок.

В языке Prolog подстановки реализуются с использованием логических переменных, позволяющих сохранить в памяти их текущее связывание. В любой момент времени каждая переменная в программе является либо несвязанной, либо связанной с некоторым значением. Эти переменные и значения, вместе взятые, неявно определяют подстановку для текущей ветви доказательства. Любая попытка продления пути позволяет лишь добавить новые связывания переменных, поскольку стремление ввести другое связывание для уже связанной переменной приводит к неудачному завершению унификации. После того как попытка продления пути поиска оканчивается неудачей, система Prolog возвращается к предыдущей точке выбора и только после этого получает возможность отменить связывания некоторых переменных.

Такая операция отмены выполняется благодаря тому, что данные обо всех переменных, для которых было выполнено связывание, отслеживаются в стеке, называемом $\wedge$ контрольным стеком (trail). По мере того как в функции Unify-Var

осуществляется связывание каждой новой переменной, эта переменная задвигается в контрольный стек. Если попытка достижения некоторой цели оканчивается неудачей и наступает время возвратиться к предыдущей точке пункта выбора, отменяется связывание каждой из этих переменных по мере их выталкивания из контрольного стека.

Но даже в самых эффективных интерпретаторах Prolog, в связи с издержками на поиск по индексу, унификацию и формирование стека рекурсивных вызовов, требуется выполнение нескольких тысяч машинных команд в расчете на каждый этап логического вывода. В действительности интерпретатор постоянно ведет себя так, как если бы он никогда до сих пор не видел данную программу; например, ему приходится каждый раз находить выражения, которые согласуются с целью. С другой стороны, откомпилированная программа Prolog представляет собой процедуру логического вывода для конкретного множества выражений, поэтому ей известно, какие выражения согласуются с целью. В процессе компиляции система Prolog по сути формирует миниатюрную программу автоматического доказательства теоремы для каждого отдельного предиката, устраняя тем самым основную часть издержек интерпретации. Эта система позволяет также применять ^ открытый код для процедуры унификации каждого отдельного вызова, что позволяет избежать необходимости проведения явного анализа структуры терма (подробные сведения об унификации с открытым кодом приведены в [1557]).

Наборы команд современных компьютеров плохо согласуются с семантикой Prolog, поэтому большинство компиляторов Prolog компилирует программу в промежуточный язык, а не непосредственно в машинный язык. Наиболее широко применяемым промежуточным языком является язык WAM (Warren Abstract Machine — абстрактная машина Уоррена) получивший название в честь Дэвида Г.Д. Уоррена, одного из создателей первого компилятора Prolog. Язык WAM представляет собой абстрактное множество команд, которое подходит для преобразования в него программ Prolog и может интерпретироваться или транслироваться в машинный язык.

Другие компиляторы транслируют программу Prolog в программу на языке высокого уровня, таком как Lisp или C, а затем используют компилятор этого языка для трансляции в машинный язык. Например, определение предиката Append может быть откомпилировано в код, показанный в листинге 9.4. Ниже приведено несколько замечаний, заслуживающих упоминания в этой связи.

Листинг 9.4. Псевдокод, представляющий собой результат компиляции предиката Append.

Функция New-Variable возвращает новую переменную, отличную от всех других переменных, использовавшихся до сих пор. Процедура Call (continuation)

продолжает выполнение с заданным продолжением continuation procedure

```
Append(ax, y, az, continuation) trail <— Global-Trail-Pointer() if ax = [] and Unify(y, az)
then Call(continuation) Reset-Trail(trail) a <— New-Variable(); x <— New-Variable(); z <—
New-Variable() if Unify(ax, [a \ x] ) and Unify(az, [a | z]) then Append(x, y, z, continuation)
```

- Выражения предиката Append преобразуются в процедуру, а этапы логического вывода осуществляются путем вызова этой процедуры, поэтому не приходится выполнять поиск соответствующих выражений в базе знаний.
- Как было описано выше, текущие связывания переменных хранятся в контрольном стеке. На первом этапе выполнения этой процедуры текущее состояние контрольного стека сохраняется в памяти, поэтому оно может быть восстановлено с помощью функции Reset-Trail, если попытка выполнения первого выражения окончится неудачей. Это приводит к отмене всех связываний, сформированных при первом вызове процедуры Unify.
- Сложнейшей частью этой программы является использование $\wedge$ продолжений для реализации точек выбора. Продолжение может рассматриваться как структура данных, в которой упакованы процедура и список параметров, вместе взятые, определяющая, что следует делать дальше, после успешного достижения текущей цели. Дело в том, что после достижения цели не было бы достаточно просто вернуть управление из процедуры, подобной Append, поскольку успех может быть достигнут несколькими способами, и каждый из них должен быть исследован. Параметр continuation, определяющий продолжение, позволяет решить эту проблему, поскольку он может быть вызван после каждого успешного достижения цели. В приведенном здесь коде процедуры Append, если первый параметр является пустым, то предикат Append достиг успеха. Затем вызывается продолжение с помощью процедуры Call (притом что в контрольном стеке находятся все подходящие связывания) для того, чтобы определить, что делать дальше. Например, если процедура Append вызвана на верхнем уровне дерева доказательства, то структура данных continuation будет использоваться для вывода информации о связывании переменных.

До того как Уоррен выполнил свою работу по внедрению компиляции в системы логического вывода на языке Prolog, средства логического программирования работали слишком медленно для того, чтобы они действительно могли найти широкое применение. Компиляторы, разработанные Уорреном и другими специалистами, позволили достичь скоростей обработки кода Prolog, способных конкурировать с языком C в самых различных стандартных эталонных тестах [1536]. Безусловно, тот факт, что теперь появилась возможность написать планировщик или синтаксиче-

ский анализатор текста на естественном языке, состоящий из нескольких десятков строк на языке Prolog, говорит о том, что этот язык становится более подходящим (по сравнению с языком C) для создания прототипов большинства

исследовательских проектов в области искусственного интеллекта небольших масштабов.

Значительное повышение скорости позволяет также обеспечить распараллеливание работы. Организация параллельного выполнения может осуществляться по двум направлениям. Первое направление, получившее название $\wedge$ ИЛИ-параллелизма, исходит из возможности унификации цели со многими различными выражениями в базе знаний. Каждая из этих операций унификации приводит к появлению независимой ветви в пространстве поиска, которая может привести к потенциальному решению, и поиск решений по всем таким ветвям может осуществляться параллельно.

Второе направление, называемое $\wedge$ И-параллелизмом, исходит из возможности параллельного решения каждого конъюкта в теле некоторой импликации. Задача достижения И-параллелизма является более сложной, поскольку для поиска решений всей конъюнкции требуются совместимые связывания для всех переменных. Поэтому при обработке каждой конъюнктивной ветви необходимо обеспечивать обмен данными с другими ветвями для гарантированного достижения глобального решения.

Избыточный логический вывод и бесконечные циклы Теперь рассмотрим, в чем состоит ахиллесова пята языка Prolog: несогласованность между организацией поиска в глубину и деревьями поиска, которые включают повторяющиеся состояния и бесконечные пути. Рассмотрим следующую логическую программу, позволяющую определить, существует ли путь между двумя точками в ориентированном графе: $\text{path}(X,Z) :- \text{link}(X,Z).$ $\text{path}(X,Z) :- \text{path}(X,Y), \text{link}(Y,Z).$

На рис. 9.5, а приведен простой граф, состоящий из трех узлов, который описан с помощью фактов $\text{link}(a, b)$ и $\text{link}(b, c)$. При использовании этой программы запрос $\text{path}(a, c)$ вырабатывает дерево доказательства, показанное на рис. 9.6, а.

С другой стороны, если эти два выражения расположены в таком порядке: $\text{path}(X,Z) :- \text{path}(X,Y) / \text{link}(Y,Z).$ $\text{path}(X,Z) :- \text{link}(X,Z).$ то система Prolog следует по бесконечному пути, как показано на рис. 9.6, б.

Поэтому система Prolog является неполной, как и программа автоматического доказательства теоремы для определенных выражений (как показано в этом примере, даже применительно к программам, соответствующим формату языка Datalog), поскольку для некоторых баз знаний эта система оказывается неспособной доказать высказывания, которые из них следуют. Следует отметить, что алгоритм прямого логического вывода не подвержен этой проблеме: сразу после вывода фактов $\text{path}(a, b)$, $\text{path}(b,c)$ и $\text{path}(a, c)$ процесс прямого логического вывода останавливается.

Обратный логический вывод с поиском в глубину сталкивается также с проблемами, обусловленными излишними вычислениями. Например, при поиске пути от узла A_x к узлу J_4 на рис. 9.5, б система Prolog выполняет 877 этапов логического вывода, большинство из которых связано с поиском всех возможных путей к узлам, не позволяющим достичь цели. Такая проблема аналогична проблеме повторяющихся состоя-

ний, которая описывалась в главе 3. Общее количество этапов логического вывода может определяться экспоненциальной зависимостью от количества базовых фактов, вырабатываемых в процессе вывода. А если бы вместо этого применялся прямой логический вывод, то можно было бы ограничиться выработкой, самое большее, n^2 фактов $path(X, Y)$, связывающих n узлов. При этом для решения задачи, приведенной на рис. 9.5, б, потребовалось бы только 62 этапа логического вывода.

а) Рис. 9.5. Иллюстрация недостатков языка Prolog: поиск пути от A до C может привести систему Prolog к созданию бесконечного цикла (а); граф, в котором каждый узел связан с двумя случайно выбираемыми преемниками на следующем уровне; для поиска пути от A_i до J_4 требуется 877 этапов логического вывода (б) $path(a,c) \text{ link}(a,c) \text{ fail link}(b,c) \text{ link}(Y,c) \{ \} \text{ link}(a,Y) \{ Y/b \}$ а) б) Рис. 9.6. Иллюстрация того, что работа программы Prolog зависит от расположения взаимосвязанных выражений: успешное доказательство того, что путь от A до C существует (а); бесконечное дерево доказательства, формируемое из-за того, что выражения находятся в "неправильном" порядке (б) Применение прямого логического вывода при решении задач поиска в графе представляет собой пример ^ динамического программирования, в котором решения подзадач формируются инкрементно из решений меньших подзадач и кэшируются для предотвращения повторного вычисления. Аналогичный эффект в системе обратного логического вывода может быть достигнут с помощью запоминания (memoization), т.е. кэширования решений, найденных для подцелей, по мере их обнаружения, а затем повторного применения этих решений после очередного появления той же подцели, вместо повторения предыдущих вычислений. Этот подход применяется в системах ^табулированного логического программирования (tabled logic programming), в которых для реализации метода запоминания используются

эффективные механизмы сохранения и выборки. В табулированном логическом программировании объединяется целенаправленность обратного логического вывода с эффективностью динамического программирования, присущей прямому логическому выводу. Кроме того, эти системы являются полными применительно к программам в формате Datalog, а это означает, что программисту приходится меньше беспокоиться о бесконечных циклах.

Логическое программирование в ограничениях В приведенном выше описании прямого логического вывода (раздел 9.3) было показано, как можно представить

задачи удовлетворения ограничений (Constraint Satisfaction Problem— CSP) в виде определенных выражений. Стандартный язык Prolog позволяет решать подобные задачи точно таким же способом, как и алгоритм поиска с возвратами, приведенный в листинге 5.1.

Поскольку поиск с возвратами предусматривает полный перебор областей определения переменных, он может применяться только для решения задач CSP с конечными областями определения. В терминах Prolog это можно перефразировать таким образом, что для любой цели с несвязанными переменными должно существовать конечное количество решений. (Например, цель `diff(q, sa)`, которая определяет, что штаты Квинсленд и Южная Австралия должны быть окрашены в разные цвета, имеет шесть решений, если допускается применение трех цветов.) Задачи CSP с бесконечными областями определения (например, с целочисленными или вещественными переменными) требуют применения совсем других алгоритмов, таких как распространение пределов или линейное программирование.

Приведенное ниже выражение выполняется успешно, если подставленные в него три числа удовлетворяют неравенству треугольника. `triangle(X,Y,Z) :- X>=0, Y>=0, Z>=0, X+Y>=Z, Y+Z>=X, X+Z>=Y.`

Если системе Prolog передан запрос `triangle C, 4, 5)`, он будет выполнен безукоризненно. С другой стороны, после передачи запроса `triangle C, 4, Z)` решение не будет найдено, поскольку подцель `Z>=0` не может быть обработана системой Prolog. Возникающая при этом сложность состоит в том, что переменные в системе Prolog должны находиться только в одном из двух состояний: несвязанные или связанные с конкретным термом.

Связывание переменной с конкретным термом может рассматриваться как крайняя форма ограничения, а именно как ограничение равенства. [^] Логическое программирование в ограничениях (Constraint Logic Programming — CLP) позволяет ограничивать, а не связывать переменные. Решением для программы в логике ограничений является наиболее конкретное множество ограничений, налагаемых на переменные запроса, которое может быть определено с помощью базы знаний. Например, решением запроса `triangle C, 4, Z)` является ограничение `7>=Z>=1`. Программы в стандартной логике представляют собой частный случай программ CLP, в которых ограничения решения должны быть не ограничениями сравнения, а ограничениями равенства, т.е. связываниями.

Системы CLP включают различные алгоритмы решения задач с ограничениями для таких вариантов ограничений, которые разрешены к использованию в языке.

Например, система, позволяющая использовать линейные неравенства с переменными, имеющими вещественные значения, может включать алгоритм линейного программирования для решения этих ограничений. Кроме того, в

системах CLP принят гораздо более гибкий подход к решению запросов стандартного логического программирования. Например, вместо использования поиска в глубину, слева направо, с возвратами, в них может применяться любой из более эффективных алгоритмов, описанных в главе 5, включая эвристическое упорядочение конъюнктов, обратный переход, определение условий формирования множества отсечения и т.д.

Поэтому в системах CLP сочетаются элементы алгоритмов удовлетворения ограничений, логического программирования и дедуктивных баз данных.

Кроме того, системы CLP позволяют воспользоваться преимуществами различных методов оптимизации поиска в задачах CSP, описанных в главе 5, таких как упорядочение переменных и значений, предварительная проверка и интеллектуальный возврат. В частности, разработаны проекты нескольких систем, позволяющих программисту получить больший контроль над порядком поиска для логического вывода. Например, язык MRS [539], [1321] позволяет программисту-пользователю записывать $\wedge$ метаправила для определения того, какие конъюнкты должны быть опробованы в первую очередь. Например, пользователь может сформулировать правило с указанием, что в первую очередь следует пытаться достичь цели с наименьшим количеством переменных, или оформить характерные для проблемной области правила, касающиеся конкретных предикатов.

9.5. Резолюция Последнее из трех рассматриваемых в данной главе семейств логических систем основано на резолюции. Как было показано в главе 7, пропозициональная резолюция — это полная процедура логического вывода для пропозициональной логики на основе опровержения. В этом разделе будет показано, как распространить резолюцию на логику первого порядка.

Проблема существования полных процедур доказательства всегда является предметом непосредственного внимания математиков. Если бы удалось найти полную процедуру доказательства для математических утверждений, это повлекло бы за собой два последствия: во-первых, вывод всех заключений мог бы осуществляться механически; во-вторых, всю математику можно было бы построить как логическое следствие некоторого множества фундаментальных аксиом. Поэтому вопрос о полноте доказательства стал в XX веке предметом наиболее важных математических работ. В 1930 году немецкий математик Курт Гёдель доказал первую $\wedge$ теорему о полноте для логики первого порядка, согласно которой любое высказывание, являющееся следствием заданных аксиом, имеет конечное доказательство. (Но никакая действительно применимая на практике процедура доказательства не была найдена до тех пор, пока Дж.Э. Робинсон не опубликовал в 1965 году алгоритм резолюции.) В 1931 году Гёдель доказал еще более знаменитую $\wedge$ теорему о неполноте. В этой теореме утверждается, что любая логическая система, которая включает принцип индукции (а без этого принципа удастся построить лишь

очень малую часть дискретной математики), обязательно является неполной. Поэтому существуют такие высказывания, которые следуют из заданных аксиом, но в рамках данной логиче-

ской системы для них невозможно найти конечное доказательство. Иголка действительно может быть в метафорическом стоге сена, но ни одна процедура не позволяет гарантировать, что она будет найдена.

Несмотря на то, что теорема Гёделя о неполноте устанавливает определенные пределы, программы автоматического доказательства теорем на основе резолюции широко применялись и применяются для обоснования математических теорем, включая несколько таких теорем, для которых до сих пор не было известно доказательство. Кроме того, программы автоматического доказательства теорем успешно использовались для проверки проектов аппаратных средств, формирования логически правильных программ, а также во многих других приложениях.

Конъюнктивная нормальная форма для логики первого порядка Как и в случае пропозициональной логики, для резолюции в логике первого порядка требуется, чтобы высказывания находились в конъюнктивной нормальной форме (Conjunctive Normal Form — CNF), т.е. представляли собой конъюнкцию выражений, в которой каждое выражение представляет собой дизъюнкцию литералов.

Литералы могут содержать переменные, на которые, согласно принятому предположению, распространяется квантор всеобщности. Например, высказывание $\forall x \text{ American}(x) \wedge \forall y \text{ Weapon}(y) \wedge \forall x,y,z \text{ Sells}(x,y,z) \wedge \forall z \text{ Hostile}(z) \Rightarrow \forall x \text{ Criminal}(x)$ принимает в форме CNF следующий вид: $\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x,y,z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x)$ ^ Каждое высказывание в логике первого порядка может быть преобразовано в эквивалентное с точки зрения логического вывода высказывание CNF. В частности, высказывание CNF является невыполнимым тогда и только тогда, когда невыполнимо первоначальное высказывание, поэтому мы получаем основу для формирования доказательств от противного с помощью высказываний CNF.

Процедура преобразования любого высказывания в форму CNF весьма подобна процедуре, применяемой в пропозициональной логике, которая показана на с. 308.

Принципиальное различие связано с необходимостью устранения кванторов существования. Проиллюстрируем эту процедуру на примере преобразования в форму CNF высказывания "Каждого, кто любит всех животных, кто-то любит", или $\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x,y)] \Rightarrow \exists y \text{ Loves}(y,x)$ Ниже приведены этапы этого преобразования. • Устранение импликаций:

$\forall x [\neg \exists y \neg \text{Animal}(y) \vee \text{Loves}(x,y)] \vee [\exists y \text{Loves}(y,x)]$ • Перемещение связок $\neg$ внутрь выражений. Кроме обычных правил для отрицаемых связок, нам нужны правила для отрицаемых кванторов. Поэтому получаем следующие правила:

6 Как показано в упр. 7.12, любое выражение может быть представлено как импликация с конъюнкцией атомов слева и дизъюнкцией атомов справа. Эта форма, иногда называемая формой Ковальского, при использовании записи с символом импликации, ориентированным справа налево [851], часто бывает намного более удобной для чтения по сравнению с другими формами.

$\neg \exists x p$ принимает вид $\exists x \neg p$ $\neg \forall x p$ принимает вид $\forall x \neg p$ Рассматриваемое высказывание проходит через такие преобразования:

$\forall x [\exists y \neg (\neg \text{Animal}(y) \vee \text{Loves}(x,y))] \vee [\exists y \text{Loves}(y,x)] \forall x [\exists y \neg \neg \text{Animal}(y) \wedge \neg \text{Loves}(x,y)] \vee [\exists y \text{Loves}(y,x)] \forall x [\exists y \text{Animal}(y) \wedge \neg \text{Loves}(x,y)] \vee [\exists y \text{Loves}(y,x)]$
Обратите внимание на то, что квантор всеобщности ($\forall y$) в предпосылке импликации стал квантором существования. Теперь это высказывание приобрело такое прочтение: "Либо существует какое-то животное, которого x не любит, либо (если это утверждение не является истинным) кто-то любит x ".

Очевидно, что смысл первоначального высказывания был сохранен. • Стандартизация переменных. В высказываниях наподобие $(\forall x P(x)) \vee (\exists x Q(x))$, в которых дважды используется одно и то же имя переменной, изменим имя одной из переменных. Это позволит в дальнейшем избежать путаницы после того, как будут удалены кванторы. Поэтому получим следующее:

$\forall x [\exists y \text{Animal}(y) \wedge \neg \text{Loves}(x,y)] \vee [\exists z \text{Loves}(z,x)]$ • Сколемизация. ^ Сколемизация — это процесс устранения кванторов существования путем их удаления. В данном простом случае этот процесс подобен применению правила конкретизации с помощью квантора существования, приведенного в разделе 9.1: преобразовать $\exists x P(x)$ в $P(A)$, где A — новая константа. Однако, если это правило будет непосредственно применено к высказыванию, рассматриваемому в качестве примера, то будет получено следующее высказывание:

$\forall x [\text{Animal}(A) \wedge \neg \text{Loves}(x,A)] \vee \text{Loves}(B,x)$ которое имеет полностью неправильный смысл: в нем утверждается, что каждый либо не способен любить какое-то конкретное животное A , либо его любит некоторая конкретная сущность B . В действительности первоначальное высказывание позволяет каждому человеку не быть способным любить какое-то другое животное или быть любимым другим человеком. Поэтому желательно, чтобы сущности, определяемые в процессе сколемизации, зависели от x :

$\forall x [\text{Animal}(F(x)) \wedge \neg \text{Loves}(x,F(x))] \vee \text{Loves}(G(x),x)$ где F и G — ^ сколемовские функции. Общее правило состоит в том, что все параметры сколемовской функции

должны быть переменными, на которые распространяются кванторы всеобщности, в область действия которых попадает соответствующий квантор существования. Как и при использовании конкретизации с помощью квантора существования, сколемизированное высказывание является выполнимым тогда и только тогда, когда выполнимо первоначальное высказывание. • Удаление кванторов всеобщности. В данный момент на все оставшиеся переменные должны распространяться кванторы всеобщности. Кроме того, данное высказывание эквивалентно тому, в котором все кванторы всеобщности перенесены влево. Поэтому кванторы всеобщности могут быть удалены следующим образом:

$[Animal(F(x)) \wedge \neg Loves(x, F(x))] \vee Loves(G(x), x) \bullet$ Распределение связки $\vee$ по л: $[Animal(F(x)) \vee Loves(G(x), x)] \wedge [\neg Loves(x, F(x)) \vee Loves(G(x), x)]$ На этом этапе может также потребоваться выполнить раскрытие скобок во вложенных конъюнкциях и дизъюнкциях.

Теперь рассматриваемое высказывание находится в форме CNF и состоит из двух выражений. Оно полностью недоступно для восприятия. (Помочь его понять может такое пояснение, что сколемовская функция $F(x)$ указывает на животное, которое потенциально может быть нелюбимым лицом x , а $G(x)$ указывает на кого-то, кто может любить лицо x .) К счастью, людям редко приходится изучать высказывания в форме CNF, поскольку показанный выше процесс преобразования может быть легко автоматизирован.

Правило логического вывода с помощью резолюции Правило резолюции для выражений в логике первого порядка представляет собой поднятую версию правила резолюции для пропозициональной логики, приведенного на с. 307. Два выражения, которые, согласно принятому предположению, должны быть стандартизованными таким образом, чтобы в них не было общих переменных, могут быть подвергнуты операции резолюции, если они содержат взаимно дополнительные литералы. Пропозициональные литералы являются взаимно дополнительными, если один из них представляет собой отрицание другого, а литералы в логике первого порядка являются взаимно дополнительными, если один из них унифицируется с отрицанием другого. Поэтому имеет место следующее правило: $li \vee \dots \vee lk; mi \vee \dots \vee mj$ Subst(θ , $li \vee \dots \vee mi$) $\vee$ $li + i \vee \dots \vee lk \vee ui + i \vee \dots \vee mj - i \vee irj + i \vee \dots \vee mn$ где $Uni f y (li, -im - j) = \theta$. Например, можно применить операцию резолюции к следующим двум выражениям: $[Animal(F(x)) \vee Loves(G(x), x)]$ и $[\neg Loves(u, v) \wedge Kills(u, v)]$ путем устранения взаимно дополнительных литералов $Loves(G(x), x)$ и $\neg Loves(u, v)$ с помощью унификатора $\theta = \{u/G(x), v/x\}$ для получения следующего выражения, называемого резольвентой: $[Animal(F(x)) \vee \neg Kills(G(x), x)]$ Только что приведенное правило называется правилом $\wedge$ бинарной резолюции, поскольку в нем происходит удаление с помощью резолюции двух и только двух взаимно дополнительных литералов. Но правило бинарной резолюции, отдельно взятое, не

позволяет получить полную процедуру логического вывода. С другой стороны, правило полной резолюции позволяет удалять в каждом выражении подмножества литералов, которые являются унифицируемыми. Альтернативный подход состоит в том, чтобы распространить операцию факторизации (удаления избыточных литералов) на логику первого порядка. В пропозициональной факторизации два литерала сводятся к одному, если они являются идентичными, а в факторизации первого порядка два литерала сводятся к одному, если они являются унифицируемыми.

Унификатор должен быть применен ко всему выражению. Сочетание бинарной резолюции и факторизации позволяет создать полную процедуру логического вывода. Примеры доказательств. При использовании резолюции доказательство того, что $\neg a$ (из базы знаний следует высказывание a) осуществляется путем доказательства невыполнимости выражения $a \vee \neg a$, т.е. путем получения пустого выражения. Алгоритмический подход, применяемый в логике первого порядка, идентичен подходу в пропозициональной логике, который показан в листинге 7.5, поэтому мы не будем здесь его повторять. Вместо этого приведем два примера доказательства. Первым из них является пример доказательства преступления, описанного в разделе 9.3. Соответствующие высказывания, преобразованные в форму CNF, выглядят следующим образом: $\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x) \vee \neg \text{Missile}(x) \vee \neg \text{Owns}(\text{Nono}, x) \vee \text{Sells}(\text{West}, x, \text{Nono}) \vee \neg \text{Enemy}(x, \text{America}) \vee \text{Hostile}(x) \vee \neg \text{Missile}(x) \vee \text{Weapon}(x) \vee \text{Owns}(\text{Nono}, \text{Mi}) \vee \text{Missile}(\text{Mi}) \vee \text{American}(\text{West}) \vee \neg \text{Enemy}(\text{Nono}, \text{America})$. В число этих высказываний должна быть включена также отрицаемая цель $\neg \text{Criminal}(\text{West})$. Процедура доказательства по методу резолюции показана на рис. 9.7. Обратите внимание на его структуру: единственный "хребет" начинается с целевого выражения, и операция резолюции применяется к выражениям из базы знаний до тех пор, пока не образуется пустое выражение. В этом состоит характерная особенность применения метода резолюции к базам знаний, представленным в виде хорновских выражений. В действительности выражения, расположенные вдоль главного хребта, точно соответствуют последовательным значениям целевых переменных в алгоритме обратного логического вывода, приведенном в листинге 9.3. Это связано с тем, что для резолюции всегда выбирается выражение, положительный литерал которого унифицируется с самым левым литералом "текущего" выражения в хребте; именно это происходит при обратном логическом выводе. Таким образом, обратный логический вывод в действительности представляет собой просто частный случай резолюции, в котором применяется конкретная стратегия управления для определения того, какая операция резолюции должна быть выполнена в следующую очередь.

В рассматриваемом здесь втором примере используется сколемизация и применяются выражения, которые не являются определенными. Это приводит к созданию немного более сложной структуры доказательства. На естественном языке эта задача формулируется, как описано ниже.

Каждого, кто любит всех животных, кто-то любит. Любого, кто убивает животных, никто не любит.

Джек любит всех животных. Кота по имени Тунец убил либо Джек, либо Любопытство.

Действительно ли этого кота убило Любопытство?

-Лтегкап(x) $\vee$ $\wedge$ Weapon(y) $\vee$ $\rightarrow$ Sells(x,yfz) $\vee$ $\wedge$ Hostile(z) $\vee$ Criminal(x) American(West) $\wedge$ Missile(x) $\vee$ Weapon(x) | Missile(Mj) $\wedge$ Missile(x) $\vee$ $\rightarrow$ Owns(Nono,x) $\vee$ Selb(Westjc,Nono) | Missile(Mj) Owns(Nono,Mj) -лЕпеты(x,America) $\vee$ Hostile(x) -Criminal(West) -Jmerican(West) $\vee$ - $\wedge$ Weapon(y) $\vee$ $\wedge$ Sells(West,y,z) $\vee$ $\wedge$ Hostile(z) - $\wedge$ Weapon(y) $\vee$ $\wedge$ Sells(West,y,z) $\vee$ $\wedge$ Hostile(z) $\wedge$ -yMssile(y) $\vee$ $\wedge$ Sells(West,y,z) $\vee$ $\wedge$ Hostile(z) -Sells(West,Mt,z) $\vee$ $\wedge$ Hostile(z) $\wedge$ - $\wedge$ Missile(Mj) $\vee$ -,Owns(Nono,Mj) $\vee$ -,Hostile(Nono) $\wedge$ 40wns(Nono,Ml) $\vee$ -yHostile(Nono) $\wedge$ - $\wedge$ Hostile(Nono) -,Enemy(Nono,America) $\wedge$ Enemy(Nono,America) Рис. 9.7. Процедура доказательства с помощью резолюции того, что полковник Уэст совершил преступление Вначале представим в логике первого порядка первоначальные высказывания, некоторые фоновые знания и отрицаемую цель G: • [$\exists y$ Loves (y, x)] [$\forall z$ -iLoves (z,x)] A. $\forall x$ [$\forall y$ Animal (y) $\Rightarrow$ Loves(x,y)] B. $\forall x$ [$\exists y$ Anijnal(y) $\wedge$ Kills{x,y)] :

C. $\forall x$ Animal (x) $\Rightarrow$ Loves {Jack, x) D. Kills{Jack,Tuna) $\vee$ Kills{Curiosity, Tuna) E. Ca t (Tima) F. $\forall x$ Cat(x) $\Rightarrow$ A.nijnal(x) -.G. -.Kills (Curiosity, Tuna) Затем применим процедуру преобразования, чтобы преобразовать каждое высказывание в форму CNF:

A1. Animal(F(x)) $\vee$ Loves(G(x),x) A2 . —iLoves {x, F{x)) $\vee$ Loves {G(x) , x) B. —iAni/nal (y) $\vee$ -ifCills (x,y) $\vee$ -iLoves (z, x) C. -iAnimal(x) $\vee$ Loves {Jack, x) D. Kills{Jack,Tuna) $\vee$ Kills(Curiosity, Tuna) E. Cat{Tuna) F. -iCat(x) $\vee$ Animal (x) -iG. - $\rightarrow$ Kills{Curiosity, Tuna) Доказательство с помощью метода резолюции того, что кота убило Любопытство, приведено на рис. 9.8. На естественном языке это доказательство может быть перефразировано, как показано ниже.

Предположим, что кота Тунца убило не Любопытство. Мы знаем, что это сделал либо Джек, либо Любопытство; в таком случае это должен был сделать Джек. Итак, Тунец — кот, а коты — животные, поэтому Тунец — животное. Любого, кто убивает животное, никто не любит, поэтому мы делаем вывод, что никто не любит Джека. С другой стороны, Джек любит всех животных, поэтому кто-то его любит; таким образом, возникает противоречие. Это означает, что кота убило Любопытство.

| CatjTuna) | [-Catjx) vAnimaljx) \ \ КИЩак, Tuna) v KillsjCuriosity, Типа\ \ ^KillsjCuriosity, Типа)] Рис. 9.8. Процедура доказательства с помощью резолюции того, что кота убило Любопытство.

Обратите внимание на то, что при выводе выражения Loves (G ('Jack;, Jackj использовалась факторизация Такое доказательство действительно отвечает на вопрос "Действительно ли этого кота убило Любопытство?", но часто требуется найти ответ на более общие вопросы, такие как "Кто убил кота?" Резолюция позволяет это сделать, но для получения ответа требует немного больше работы. Данная цель может быть представлена в виде выражения $3w \text{ Kills}(w, \text{Типа})$, которое после его отрицания принимает в форме CNF вид $\neg i\text{Kills}(w, \text{Типа})$. Повторяя доказательство, показанное на рис. 9.8, с новой отрицаемой целью, мы получим аналогичное дерево доказательства, но с подстановкой $\{w/ \text{Curio si ty}\}$ в одном из этапов. Поэтому в данном случае для поиска того, кто убил кота, достаточно проследить за связываниями, которые применяются к переменным запроса в этом доказательстве.

К сожалению, в методе резолюции могут вырабатываться $\wedge$ неконструктивные доказательства для существующих целей. Например, в выражении $\neg i\text{Kills}(w, \text{Типа})$ после применения операции резолюции удаляется взаимно дополнительный литерал, входящий в состав выражения $\text{Kills}(\text{Jack}, \text{Типа}) \vee \text{Kills}(\text{Curiosity}, \text{Типа})$, с получением выражения $\text{Kills}(\text{Jack}, \text{Типа})$, к которому снова применяется операция резолюции с использованием выражения $\neg i\text{Kills}(w, \text{Типа})$, что приводит к получению пустого выражения. Обратите внимание на то, что в этом доказательстве переменная w имеет два различных связывания, а правило резолюции сообщает нам, что кто-то действительно убил кота Тунца — либо Джек, либо Любопытство. Но в этом для нас нет ничего нового! Одно из решений данной проблемы состоит в том, чтобы регламентировать допустимые этапы резолюции так, чтобы переменные запроса могли быть связанными только один раз в каждом конкретном доказательстве; в таком случае можно будет предусмотреть применение перехода с возвратом по всем возможным связываниям. Еще одно решение состоит в добавлении специального $\wedge$ литерала ответа к отрицаемой цели, которая принимает вид $\neg \text{Kills}(w, \text{Tuna}) \vee \text{Answer}(w)$. Теперь процесс резолюции вырабатывает ответ каждый раз, когда формируется выражение, содержащее только единственный литерал ответа. Для доказательства, приведенного на рис. 9.8, таковым является выражение $\text{Answer}(\text{Curiosity})$. Неконструктивное доказательство привело бы к выработке выражения $\text{Answer}(\text{Curiosity}) \vee \text{Answer}(\text{Jack})$, которое не может рассматриваться как ответ и отбрасывается.

Полнота резолюции В настоящем разделе приведено доказательство полноты резолюции. Это доказательство может пропустить без ущерба для дальнейшего понимания текста любой читатель, который готов принять его на веру.

Мы покажем, что резолюция обеспечивает $\wedge$ полноту опровержения (refutation completeness), а это означает, что если множество высказываний является невыполнимым, то резолюция всегда позволяет прийти к противоречию. Резолюцию нельзя использовать для выработки всех логических следствий из множества высказываний, но она может применяться для подтверждения того, что данное конкретное высказывание следует из множества высказываний. Поэтому резолюция может служить для поиска всех ответов на данный конкретный вопрос с помощью метода отрицаемой цели, который был описан выше в настоящей главе.

Примем как истинное такое утверждение, что любое высказывание в логике первого порядка (без использования равенства) может быть перезаписано в виде множества выражений в форме CNF. Это можно доказать по индукции на основе анализа формы высказывания, применяя в качестве базового случая атомарные высказывания [336]. Поэтому наша цель состоит в том, чтобы доказать следующее: $\wedge$ если S — невыполнимое множество выражений, то применение к S конечного количества этапов резолюции приведет к противоречию.

Схема нашего доказательства повторяет первоначальное доказательство, приведенное Робинсоном, с некоторыми упрощениями, которые были внесены Генезеретом и Нильссоном [537]. Основная структура этого доказательства показана на рис. 9.9; оно осуществляется, как описано ниже.

Любое множество предложений S представимо в форме импликационных выражений I . Допустим, что множество S невыполнимо и представлено в форме импликационных выражений I . Некоторое множество U базовых экземпляров невыполнимо. Резолюция позволяет найти противоречие в S' . В этом состоит доказательство по методу резолюции противоречия в U . Рис. 9.9. Структура доказательства полноты резолюции 1. Вначале отметим, что если множество выражений S невыполнимо, то существует такое конкретное множество базовых экземпляров выражений S , что это множество также невыполнимо (теорема Эрбрана).

Теорема Эрбрана Базовая теорема резолюции Лемма поднятия

2. Затем прибегнем к базовой теореме резолюции (ground resolution theorem), приведенной в главе 7, в которой утверждается, что пропозициональная резолюция является полной для базовых высказываний.

3. После этого воспользуемся леммой поднятия, чтобы показать, что для любого доказательства по методу пропозициональной резолюции, в котором применяется множество базовых высказываний, существует соответствующее доказательство по методу резолюции первого порядка с использованием высказываний в логике первого порядка, из которых были получены базовые высказывания.

Теорема Гёделя о неполноте Немного дополнив язык логики первого порядка для обеспечения возможности применять схему математической индукции в арифметике, Гёдель сумел показать в своей теореме о неполноте, что существуют истинные арифметические высказывания, которые не могут быть доказаны.

Полное доказательство этой теоремы о неполноте немного выходит за рамки настоящей книги, поскольку в своем непосредственном виде оно занимает не меньше 30 страниц, но мы здесь приведем его набросок. Начнем с логической теории чисел. В этой теории существует единственная константа, 0, и единственная функция, S (функция определения преемника). В намеченной модели интерпретации этой теории $S(0)$ обозначает 1, $S(S(0))$ обозначает 2 и т.д., поэтому в рассматриваемом языке имеются имена для всех натуральных чисел. Кроме того, словарь языка включает функциональные символы $+$, $\times$ и E^x (возведение в степень), а также обычное множество логических связок и кванторов. Прежде всего, следует отметить, что множество высказываний, которые могут быть записаны на этом языке, может быть пронумеровано. (Для этого достаточно представить себе, что определен алфавитный порядок символов, а затем в алфавитном порядке расположено каждое из множеств высказываний с длиной 1, 2 и т.д.) Затем можно обозначить каждое высказывание a уникальным натуральным числом $\#a$ (которое называется гёделевским номером). Это — самый важный момент доказательства; в нем утверждается, что теория чисел включает отдельное имя для каждого из ее собственных высказываний. Аналогичным образом, с помощью гёделевского номера $G(P)$ можно пронумеровать каждое возможное доказательство P , поскольку любое доказательство — это просто конечная последовательность высказываний.

Теперь предположим, что имеется рекурсивно перечислимое множество A высказываний, которые представляют собой истинные утверждения о натуральных числах. Напомним, что высказывания из множества A можно именовать с помощью заданного множества целых чисел, поэтому можно представить себе, что на нашем языке записывается высказывание $a(j, A)$ такого рода: $\forall i$ — не гёделевский номер доказательства высказывания, гёделевским номером которого является j , где в доказательстве используются только предпосылки из множества A .

Затем допустим, что a представляет собой высказывание $a(\#a, A)$, т.е. высказывание, в котором утверждается его собственная недоказуемость из A (Утверждение о том, что такое высказывание всегда существует, является истинным, но его нельзя назвать полностью очевидным.) Теперь применим следующий остроумный довод: предположим, что высказывание a доказуемо из A ; в таком случае высказывание a ложно (поскольку в высказывании a утверждается, что оно не может быть доказано).

Но это означает, что имеется некоторое ложное высказывание, которое доказуемо из A , поэтому A не может состоять только из истинных высказываний, а это противоречит нашей предпосылке. Поэтому высказывание a не доказуемо из A . Но

именно это и утверждает о самом себе высказывание a , а это означает, что c — истинное высказывание.

Итак, мы доказали (и сэкономили 29 с половиной страниц), что для любого множества истинных высказываний теории чисел и, в частности, для любого множества базовых аксиом существуют другие истинные высказывания, которые не могут быть доказаны из этих аксиом. Из этого, кроме всего прочего, следует, что мы никогда не сможем доказать все теоремы математики в пределах любой конкретной системы аксиом. Очевидно, что это открытие имело для математики очень важное значение. Значимость этого открытия для искусственного интеллекта была предметом широких обсуждений, начиная с размышлений самого Гёделя. Мы вступим в эти дебаты в главе 26.

Для того чтобы выполнить первый этап доказательства, нам потребуются три новых понятия, описанных ниже.

- $\wedge$ Универсум Эрбрана. Если S — множество выражений, то $\mathcal{U}_S$, универсум Эрбрана для множества S , представляет собой множество всех базовых термов, которые могут быть сформированы из следующего: а) функциональные символы из множества S , если они имеются; б) константные символы из множества S , если они имеются; если они отсутствуют, то константный символ A .

Например, если множество S содержит только выражение $\neg P(x, F(x, A)) \vee \neg \langle 2(x, A) \vee R(x, v) \rangle$, то $\mathcal{U}_S$ представляет собой следующее бесконечное множество базовых термов: $\{A, B, F(A, A), F(A, B), F(B, A), F(B, B), F(A, F(A, A)), \dots\}$.

- $\wedge$ Насыщение. Если S — множество выражений, а P — множество базовых термов, то $P(S)$, насыщение S по отношению к P , представляет собой множество всех базовых выражений, полученное путем применения всех возможных совместимых подстановок базовых термов из P вместо переменных в S .
- $\wedge$ База Эрбрана. Насыщение множества выражений S по отношению к его универсуму Эрбрана называется базой Эрбрана множества S и записывается как $\#_S(S)$. Например, если S содержит только приведенное выше выражение, то $\#_S(S)$ представляет собой следующее бесконечное множество выражений:

$\{\neg P(A, F(A, A)) \vee \neg Q(A, A) \vee R(A, B), \neg P(B, F(B, A)) \vee \neg Q(B, A) \vee Y(?, ?), \neg P(F(A, A), F(F(A, A), A)) \vee \neg Q(F(A, A), A) \vee J(F(A, A), B), \neg P(F(A, B), F(F(A, B), A)) \vee \neg Q(F(A, B), A) \vee \wedge (F(A)^A), \dots\}$ Эти определения позволяют сформулировать одну из форм $\wedge$ теоремы Эрбрана[650]:

Если множество выражений s является невыполнимым, то существует конечное подмножество $H_s(S)$, которое также является невыполнимым.

Допустим, что S'' — конечное подмножество базовых высказываний. Теперь можно прибегнуть к базовой теореме резолюции (с. 311), чтобы показать, что резолюционное замыкание $RC(S')$ содержит пустое выражение. Это означает, что

доведение до конца процесса пропозициональной резолюции применительно к S_1 приводит к противоречию.

Теперь, после определения того, что всегда существует доказательство по методу резолюции, в котором применяется некоторое конечное подмножество базы Эрбрана множества S , на следующем этапе необходимо показать, что существует доказательство по методу резолюции, в котором используются выражения из самого множества S , которые не обязательно являются базовыми выражениями. Начнем с рассмотрения одного приложения правила резолюции. Из базовой леммы Робинсона следует приведенный ниже факт.

Допустим, что d и c_2 — два выражения без общих переменных, а d' и c_2' — базовые экземпляры d и c_2 . Если c — резольвента d' и c_2' , то существует выражение c , такое, что, во-первых, C — резольвента d и c_2 , и, во-вторых, C — базовый экземпляр C .

Это утверждение называется леммой поднятия (lifting lemma), поскольку оно позволяет поднять любой этап доказательства от базовых выражений к общим выражениям в логике первого порядка. Для того чтобы доказать свою основную лемму поднятия, Робинсону пришлось изобрести унификацию и определить все свойства наиболее общих унификаторов. Мы здесь не будем повторять доказательство Робинсона, а просто проиллюстрируем применение этой леммы следующим образом: $d = \neg P(x, F(x, A)) \vee \neg Q(x, A) \vee R(x, B)$ $d' = \neg I(N(G(y), z) \vee P(H(y), z))$ $C' = \neg P(H\{B\}, F(H(B), A)) \vee \neg Q(H\{B\}, A) \vee R(H(B), B)$ $C_2' = \neg N(G\{B\}, F(H(B), A)) \vee P(H\{B\}, F(H(B), A))$ $C = \neg N(G\{B\}, F(H(B), A)) \vee \neg Q(H(B), A) \vee R(H(B), B)$ $C = \neg N(G(y), F(H(y), A)) \vee \neg Q(H(y), A) \vee R(H(y), B)$ Очевидно, что C — действительно базовый экземпляр выражения C . Вообще говоря, для того чтобы выражения C' и C_2' имели какие-либо резольвенты, они должны быть получены путем предварительного применения к выражениям s и c_2 наиболее общего унификатора для пары взаимно дополнительных литералов в C' и C_2' . Из леммы поднятия можно легко получить аналогичное, приведенное ниже утверждение о любой последовательности применений правила резолюции.

Для любого выражения C в резолюционном замыкании множества выражений S' существует выражение c в резолюционном замыкании множества выражений S , такое, что C — базовый экземпляр выражения c и логический вывод c имеет такую же длину, как и логический вывод C .

Из этого факта следует, что если в резолюционном замыкании множества выражений S' появляется пустое выражение, оно должно также появиться в резолюционном замыкании множества выражений S . Это связано с тем, что пустое выражение не может быть базовым экземпляром любого другого выражения. Подведем итог: мы показали, что если множество выражений S невыполнимо, то для него существует конечная процедура логического вывода пустого выражения с помощью правила резолюции.

Поднятие способа доказательства теорем от базовых выражений к выражениям первого порядка обеспечивает огромное увеличение мощности доказательства. Это увеличение связано с тем фактом, что теперь в доказательстве первого порядка конкретизация переменных может выполняться только по мере того, как это потребуется для самого доказательства, тогда как в методах с использованием базовых выражений приходилось исследовать огромное количество произвольных конкретизаций.

Учет отношения равенства Ни в одном из методов логического вывода, описанных до сих пор в этой главе, не учитывалось отношение равенства. Для решения этой задачи может быть принято три различных подхода. Первый подход состоит в аксиоматизации равенства — включении в базу знаний высказываний, касающихся отношения равенства. При этом необходимо описать, что отношение равенства является рефлексивным, симметричным и транзитивным, а также сформулировать утверждение, что мы можем в любом предикате или функции заменять равные литералы равными. Таким образом, требуются три базовые аксиомы и еще по одной аксиоме для каждого предиката и функции, как показано ниже.

$\forall x \ x = x \quad \forall x, y \ x = y \Rightarrow y = x \quad \forall x, y, z \ x = y \wedge y = z \Rightarrow x = z \quad \forall x, y \ x = y \Rightarrow (P_i(x) \Leftrightarrow P_i(y))$
 $\forall x, y \ x = y \Rightarrow (P_2(x) \Leftrightarrow P_2(y)) \quad \forall w, x, y, z \ w = y \wedge X = z \Rightarrow \{F_1\{w, x\} = F_1\{y, z\}\} \quad \forall w, x, y, z$
 $w = y \wedge x = z \Rightarrow (F_2(w, x) = F_2\{y, z\})$ При наличии в базе знаний таких высказываний любая стандартная процедура логического вывода, такая как резолюция, позволяет решать задачи, требующие формирования рассуждений с учетом отношения равенства, например находить решения математических уравнений.

Еще один способ учета отношения равенства состоит в использовании дополнительного правила логического вывода. В простейшем правиле, правиле демодуляции, берется единичное выражение $x=y$, после чего терм y подставляется вместо любого терма, который унифицируется с термом x в каком-то другом выражении. Более формально эту идею можно представить, как описано ниже. • $\wedge$ Демодуляция. Для любых термов x, y и z , где $\text{Unify}(x, z) \neq 0$ и $m_n[z]$ — литерал, содержащий z , справедливо следующее:

$x = y, m_i \vee \dots \vee \text{дл}[r] \quad L?1 \vee \dots \vee 27?n[\text{Subst } @, y] \quad]$ Демодуляция обычно используется для упрощения выражений с помощью коллекции утверждений, таких как $x+0=x$, $xL=x$ и т.д. Это правило может быть также дополнено, чтобы можно было учитывать неединичные выражения, в которых появляется литерал со знаком равенства, как показано ниже. • $\wedge$ Парамодуляция. Для любых термов x, y и z , где $\text{Unify}(x, z) \neq 0$, справедливо следующее: $li \vee \dots \vee lk \vee x = y, \tau \setminus \vee \dots \vee m_n[z] \quad \text{Subst}@, ^i \vee \dots \vee /k \vee m_i \vee \dots \vee \text{тп}[y])$ В отличие от демодуляции, парамодуляция позволяет получить полную процедуру логического вывода для логики первого порядка с отношением равенства.

В третьем подходе формирование логических рассуждений с учетом равенства полностью осуществляется с помощью расширенного алгоритма унификации. Это означает, что термы рассматриваются как унифицируемые, если можно доказать, что они становятся равными при некоторой подстановке; здесь выражение "можно доказать" допускает включение в определенном объеме рассуждений о равенстве.

Например, термы $1 + 2$ и $2 + 1$ обычно не рассматриваются как унифицируемые, но алгоритм унификации, в котором известно, что $x + y = y + x$, способен унифицировать их с помощью пустой подстановки. ^ Унификация с учетом равенства (equational unification) такого рода может выполняться с помощью эффективных алгоритмов, разработанных с учетом данных конкретных используемых аксиом (коммутативность, ассоциативность и т.д.), а не с помощью явного логического вывода на основе этих аксиом. Программы автоматического доказательства теорем с использованием этого метода очень близки к системам логического программирования в ограничениях, описанным в разделе 9.4.

Стратегии резолюции Как известно, повторные применения правила логического вывода на основе резолюции позволяют в конечном итоге найти доказательство, если оно существует, а в этом подразделе рассматриваются стратегии, позволяющие находить доказательства не методом перебора, а более эффективно.

Преимущественное использование единичных выражений В этой стратегии преимущество отдается таким операциям резолюции, в которых одним из высказываний является единственный литерал (известный также как единичное выражение — unit clause). В основе этой стратегии лежит такая идея, что если осуществляются попытки получения пустого выражения, то может оказаться целесообразным отдавать предпочтение таким операциям логического вывода, в которых вырабатываются более короткие выражения. Применение операции резолюции к единичному высказыванию (такому как p) в сочетании с любым другим высказыванием (таким как $\neg p \vee r$) всегда приводит к получению выражения (в данном случае r), более короткого, чем это другое высказывание. Когда стратегия с преимущественным использованием единичных выражений была впервые оп-

робована в пропозициональном логическом выводе в 1964 году, она привела к резкому ускорению работы, обеспечив возможность доказывать теоремы, с которыми не удавалось справиться без использования этого метода предпочтения. Тем не менее метод предпочтения единичных выражений, отдельно взятый, не позволяет уменьшить коэффициент ветвления в задачах средних размеров до такой степени, чтобы можно было обеспечить возможность их решения с помощью резолюции.

Несмотря на это, он представляет собой полезный эвристический метод, который может успешно использоваться в сочетании с другими стратегиями. $\wedge$ Единичная резолюция (unit resolution) — это ограниченная форма резолюции, в которой на каждом этапе резолюции должно участвовать единичное выражение.

В общем случае метод единичной резолюции является неполным, но становится полным при его применении к хорновским базам знаний. Процесс доказательства по методу единичной резолюции применительно к хорновским базам знаний напоминает прямой логический вывод.

Множество поддержки Применение метода предпочтений, в котором в первую очередь осуществляется попытка выполнить определенные операции резолюции, вполне оправдано, но, вообще говоря, более эффективный метод может быть основан на том, что следует попытаться полностью устранить некоторые потенциальные этапы резолюции. В стратегии с использованием множества поддержки выполняется именно это. Применение данной стратегии начинается с выявления подмножества высказываний, называемого $\wedge$ множеством поддержки (set of support). На каждом этапе резолюции высказывание из множества поддержки комбинируется с другим высказыванием, а резольвента добавляется к множеству поддержки. Если множество поддержки является небольшим по сравнению со всей базой знаний, это позволяет резко сократить пространство поиска.

При использовании этого подхода необходимо соблюдать осторожность, поскольку при неправильном выборе множества поддержки алгоритм может стать неполным. Однако, если множество поддержки s будет выбрано так, чтобы оставшиеся высказывания, вместе взятые, оставались выполнимыми, то резолюция с помощью множества поддержки становится полной. Общепринятый подход, основанный на предположении, что первоначальная база знаний является непротиворечивой, состоит в том, чтобы в качестве множества поддержки применялся отрицаемый запрос. (В конце концов, если база знаний не является непротиворечивой, то сам факт, что запрос является следствием из нее, становится избыточным, поскольку из противоречия можно доказать все, что угодно.) Стратегия с использованием множества поддержки имеет дополнительное преимущество в том, что в ней часто вырабатываются деревья доказательства, легко доступные для понимания людей, поскольку само формирование доказательства осуществляется целенаправленно.

Резолюция с входными высказываниями В стратегии $\wedge$ резолюции с входными высказываниями (input resolution) на каждом этапе резолюции комбинируется одно из входных высказываний (из базы знаний или запроса) с некоторым другим высказыванием. В доказательстве, показанном на рис. 9.7, использовались только этапы резолюции с входными высказываниями и поэтому дерево доказательства имело характерную форму в виде единого "хребта"

с отдельными высказываниями, комбинирующимися с этим хребтом. Очевидно, что пространство деревьев доказательства такой формы меньше по сравнению с пространством всех возможных графов доказательства. В хорновских базах знаний как своего рода стратегия резолюции с входными высказываниями может рассматриваться правило отсечения, поскольку при использовании этого правила некоторая импликация из первоначальной базы знаний комбинируется с некоторыми другими высказываниями. Таким образом, нет ничего удивительного в том, что метод резолюции с входными высказываниями является полным применительно к базам знаний, которые находятся в хорновской форме, но в общем случае он неполон.

Стратегия $\wedge$ линейной резолюции (linear resolution) представляет собой небольшое обобщение, в котором допускается применять в одной операции резолюции высказывания p и Q , если P находится в первоначальной базе знаний или P является предком O в дереве доказательства. Метод линейной резолюции является полным.

Обобщение $\vee$ методе $\wedge$ обобщения (subsumption) устраняются все высказывания, которые обобщаются некоторым существующим высказыванием из базы знаний (т.е. являются более конкретными по сравнению с ним). Например, если в базе знаний есть высказывание $P(x)$, то нет смысла вводить в нее высказывание $P(A)$ и еще меньше смысла вводить $P(A) \vee Q(B)$. Обобщение позволяет поддерживать небольшие размеры базы знаний и тем самым ограничивать размеры пространства поиска.

Средства автоматического доказательства теорем Средства автоматического доказательства теорем (называемые также средствами автоматизированного формирования рассуждений) отличаются от языков логического программирования в двух отношениях. Во-первых, большинство языков логического программирования поддерживает только хорновские выражения, тогда как средства автоматического доказательства теорем поддерживают полную логику первого порядка. Во-вторых, в программах на таком типичном языке логического программирования, как Prolog, переплетаются логика и управление. Например, выбор программистом выражения $A : - B, C$ вместо $A : - C, B$ может повлиять на выполнение программы. С другой стороны, в большинстве средств автоматического доказательства теорем синтаксическая форма, выбранная для высказываний, не влияет на результаты. Для средств автоматического доказательства теорем все еще требуется управляющая информация, чтобы они могли функционировать эффективно, но эта информация обычно хранится отдельно от базы знаний, а не входит в состав самого представления знаний. Большинство исследований в области средств автоматического доказательства теорем посвящено поиску стратегий управления, которые приводят к общему повышению эффективности, а не только к увеличению быстродействия.

Проект одного из средств автоматического доказательства теорем В этом разделе описана программа автоматического доказательства теорем Otter (Organized Techniques for Theorem-proving and Effective Research) [1018]; в этом описании особое внимание будет уделено применяемой в ней стратегии управления.

Подготавливая любую задачу для программы Otter, пользователь должен разделить знания на четыре описанные ниже части.

- Множество выражений, известное как множество поддержки (или *sos* — set of support), в котором определяются важные факты о данной задаче. На каждом этапе резолюции операция резолюции применяется к одному из элементов множества поддержки и к другой аксиоме, поэтому поиск сосредоточивается на множестве поддержки.
- Множество полезных аксиом (*usable axiom*), которое выходит за пределы множества поддержки. Эти аксиомы предоставляют фоновые знания о проблемной области. Определение границы между тем, что должно войти в состав задачи (и поэтому в множество *sos*) и что относится к фоновым знаниям (и поэтому должно войти в число полезных аксиом), передается на усмотрение пользователя.
- Множество уравнений, известных как правила перезаписи (*rewrites*), или демодуляторы (*demodulators*). Хотя демодуляторы представляют собой уравнения, они всегда применяются в направлении слева направо, поэтому определяют каноническую форму, в которой должны быть представлены все упрощенные термы. Например, демодулятор $x+0=x$ указывает, что любой терм в форме $x+0$ должен быть заменен термом x .
- Множество параметров и выражений, который определяет стратегию управления. В частности, пользователь задает эвристическую функцию для управления поиском и функцию фильтрации для устранения некоторых подцелей как не представляющих интереса.

Программа Otter действует по принципу постоянного применения правила резолюции к одному из элементов множества поддержки и к одной из полезных аксиом. В отличие от системы Prolog, в этой программе используется определенная форма поиска по первому наилучшему совпадению. Ее эвристическая функция измеряет "вес" каждого выражения с учетом того, что наиболее предпочтительными являются выражения с наименьшими весами. Задача точной формулировки эвристической функции возлагается на пользователя, но, вообще говоря, вес любого выражения должен коррелировать с его размером или сложностью. Единичные выражения оцениваются как имеющие наименьший вес, поэтому такой метод поиска может рассматриваться как обобщение стратегии с преимущественным использованием единичных выражений. На каждом этапе программа Otter перемещает выражение "с наименьшим весом" из множества поддержки в список полезных аксиом и добавляет в множество поддержки некоторые непосредственные следствия применения операции резолюции к выражению с наименьшим весом и к элементам списка полезных аксиом. Программа Otter

останавливается, если обнаруживает противоречие или если возникает такая ситуация, что в множестве поддержки не остается больше выражений. Алгоритм работы этой программы показан более подробно в листинге 9.5.

Листинг 9.5. набросок структуры программы автоматического доказательства теорем Otter.

Эвристическое управление применяется при выборе выражения "с наименьшим весом" и в функции Filter, которая устраняет из рассмотрения такие выражения, которые не представляют интереса procedure Otter(sos, usable) inputs: sos, множество поддержки - выражения, определяющие решаемую задачу (глобальная переменная)

usable, множество фоновых знаний, которые потенциально могут быть релевантными для данной задачи repeat clause < элемент множества sos с наименьшим весом переместить выражение clause из множества sos в множество usable Process(Infer(clause, usable), sos) until sos = [] or обнаружится опровержение function Infer(clause, usable) returns множество выражений clauses применить правило резолюции к выражению clause и каждому элементу множества usable вернуть полученное множество clauses после применения функции Filter procedure Process(candidates, sos) for each clause in candidates do clause <— Simplify(clause) выполнить слияние идентичных литералов отбросить выражение clause, если оно представляет собой тавтологию sos <— [clause \ sos] if clause не имеет литералов, то обнаружено опровержение if clause имеет один литерал, то искать единичное опровержение Расширение системы Prolog Еще один способ создания средства автоматического доказательства теорем состоит в том, чтобы начать с компилятора Prolog и дополнить его в целях получения непротиворечивого и полного средства формирования рассуждений для полной логики первого порядка, Именно этот подход был принят при создании программы PTTP (Prolog Technology Theorem Prover) [1463]. Как описано ниже, программа PTTP включает пять существенных дополнений к системе Prolog, позволяющих восстановить полноту и выразительность алгоритма обратного логического вывода. • В процедуру унификации снова вводится проверка вхождения для того, чтобы эта процедура стала непротиворечивой. • Поиск в глубину заменяется поиском с итеративным углублением. Это позволяет добиться того, чтобы стратегия поиска стала полной, а увеличение продолжительности поиска измерялось лишь постоянной зависимостью от времени. • Разрешается применение отрицаемых литералов (таких как $\neg P(x)$). В этой реализации имеется две отдельные процедуры; в одной из них предпринимается попытка доказать P , а в другой — доказать $\neg P$. • Выражение с n атомами хранится в виде n различных правил. Например, при наличии в базе знаний выражения $A \leq B \wedge C$ должно быть также предусмотрено хранение в ней этого выражения, представленного как $\neg B \leq C \wedge \neg A$ и как

-пС $\leq$ В л —Л. Применение такого метода, известного под названием $\wedge$ блокирование (locking), означает, что текущая цель требует унификации только с головой каждого выражения, и вместе с тем позволяет должным образом учитывать отрицание. • Логический вывод сделан полным (даже для нехорновских выражений) путем добавления правила резолюции с линейным входным выражением: если текущая цель унифицируется с отрицанием одной из целей в стеке, то данная цель может рассматриваться как решенная. В этом состоит один из способов рассуждения от противного. Предположим, что первоначальной целью было высказывание Р и что эта цель свелась в результате применения ряда этапов логического вывода к цели $\neg P$. Тем самым установлено, что $\neg P \equiv P$, а это выражение логически эквивалентно Р.

Несмотря на эти изменения, программа РТТР сохраняет свойства, благодаря которым обеспечивается высокое быстродействие системы Prolog. Операции унификации все еще осуществляются посредством непосредственной модификации переменных, а отмена связывания выполняется путем разгрузки контрольного стека во время возврата. Стратегия поиска все еще основана на резолюции с применением входных выражений, а это означает, что в каждой операции резолюции участвует одно из выражений, содержащихся в первоначальной формулировке задачи (а не какое-то производное выражение). Такой подход позволяет осуществить компиляцию всех выражений из первоначальной формулировки задачи.

Основным недостатком программы РТТР является то, что пользователь должен отказаться от любых попыток взять на себя управление поиском решений. Каждое правило логического вывода в этой системе используется и в его первоначальной, и в контрапозитивной форме. Это может привести к выполнению таких операций поиска, которые противоречат здравому смыслу. Например, рассмотрим следующее правило:

$(f(x,y) = f(a,b)) \leq (x = a) \wedge (y = b)$ Если бы оно рассматривалось как правило Prolog, то применялся бы разумный способ доказательства того, что два терма f равны. Но в системе РТТР должно быть также сформировано контрапозитивное правило:

$(x \neq a) \leq (f(x,y) \neq f(a,b)) \wedge (y = b)$ По-видимому, попытка доказать, что любые два терма, x и a, являются разными, привела бы к непроизводительным затратам ресурсов.

Применение средств автоматического доказательства теорем в качестве помощников До сих пор в этой книге любая система формирования рассуждений рассматривалась как независимый агент, который должен был принимать решения и действовать самостоятельно. Еще одно направление использования средств автоматического доказательства теорем состоит в том, что они должны служить в

качестве помощников, предоставляя консультации, скажем, математикам. При эксплуатации подобных систем в режиме помощи математик действует в роли руководителя, очерчивая стратегию определения того, что должно быть сделано в следующую очередь, а затем передавая системе автоматического доказательства теорем просьбу проработать все детали. Это позволяет в определенной степени устранить проблему полураз-

решимости, поскольку руководитель научной разработки может отменить запрос и опробовать другой подход, если поиск ответа на запрос потребовал слишком много времени. Любая система автоматического доказательства теорем может также действовать в качестве $\wedge$ средства проверки доказательства; при ее использовании в таком режиме доказательство предоставляется человеком в виде ряда довольно крупных этапов; отдельные операции логического вывода, которые требуются для того, чтобы показать, что каждый из этих этапов является непротиворечивым, определяются системой.

В частности, $\wedge$ сократовское средство формирования рассуждений (socratic reasoner) представляет собой такую систему автоматического доказательства теорем, в которой функция Ask является неполной, но эта система всегда позволяет прийти к определенному решению, если ей будет задан правильный ряд вопросов. Таким образом, сократовские средства формирования рассуждений становятся хорошими помощниками, при условии, что есть руководитель, способный составить правильный ряд вызовов функции Ask. Одной из сократовских систем формирования рассуждений для математиков является Ontic [1005].

Области практического использования средств автоматического доказательства теорем Средства автоматического доказательства теорем позволили получить новейшие математические результаты. Программа SAM (Semi-Automated Mathematics) была первой программой, с помощью которой удалось доказать одну из лемм в теории решеток [602]. Кроме того, программа Aura позволила найти ответ на многие открытые вопросы в нескольких областях математики [1621]. Программа автоматического доказательства теорем Бойера-Мура [165] применялась и дорабатывалась в течение многих лет, и ею воспользовался Натараджан Шанкар для получения первого полностью строгого формального доказательства теоремы Гёделя о неполноте [1393].

Одним из самых строгих средств автоматического доказательства теорем является программа Otter; она использовалась для решения некоторых открытых задач в области комбинаторной логики. Наиболее известные из них касаются "&. алгебры Роббинса. В 1933 году Герберт Роббинс предложил простое множество аксиом, которые, на первый взгляд, могли служить для определения булевой алгебры, но ни одного доказательства этой гипотезы найти не удавалось (несмотря на напряженную работу нескольких выдающихся математиков, включая самого Альфреда Тарского).

Наконец, 10 октября 1996 года после восьми дней вычислений программа EQP (одна из версий программы Otter) нашла такое доказательство [1019].

Средства автоматического доказательства теорем могут также применяться для решения задач, связанных с $\wedge$ проверкой и $\wedge$ синтезом как аппаратных, так и программных средств, поскольку для обеих этих проблемных областей могут быть предусмотрены правильные варианты аксиоматизации. Поэтому исследования доказательства теорем проводятся не только в искусственном интеллекте, но и в таких областях, как проектирование аппаратных средств, языки программирования и разработка программного обеспечения. В случае программного обеспечения аксиомы определяют свойства каждого синтаксического элемента языка программирования. (Процесс формирования рассуждений о программах полностью аналогичен процессу формирования рассуждений о действиях в ситуационном исчислении.) Программный алгоритм проверяется путем демонстрации того, что его выходы соответствуют спецификациям для всех входов. Таким образом были проверены алго-

ритм шифрования RSA с открытым ключом и алгоритм согласования строк БойераМура [166]. В случае аппаратного обеспечения аксиомы описывают способы взаимодействия сигналов и элементов схемы (один из примеров приведен в главе 8).

Программой Aura [1610] был проверен проект 16-битового сумматора. Системы формирования логических рассуждений, предназначенные специально для проверки аппаратных средств, оказались способными проверять целые процессоры, включая свойства синхронизации этих процессоров [1455].

Формальный синтез алгоритмов был одним из первых направлений использования средств автоматического доказательства теорем, как и было намечено Корделлом Грином [591], который опирался на идеи, высказанные ранее Саймоном [1416].

Общий замысел состоит в том, что должна быть доказана теорема с утверждением, что "существует программа p , удовлетворяющая некоторой спецификации". Если удастся ограничить это доказательство конструктивной формой, то появляется возможность извлечь из результатов доказательства требуемую программу. Хотя полностью автоматизированный $\wedge$ дедуктивный синтез, как стало называться это направление, еще не осуществим применительно к программированию задач общего назначения, дедуктивный синтез, управляемый вручную, оказался успешным при проектировании нескольких новейших и сложнейших алгоритмов. Кроме того, активной областью исследования является синтез программ специального назначения.

В области синтеза аппаратных средств программа автоматического доказательства теорем Aura применялась для проектирования схем, оказавшихся более

компактными по сравнению с разработанными во всех предыдущих проектах [1609]. Для многих проектов логических схем достаточно применить пропозициональную логику, поскольку множество интересующих высказываний является фиксированным благодаря конечным размерам множества схемных элементов. В наши дни применение пропозиционального логического вывода в аппаратном синтезе является стандартным методом, имеющим много крупномасштабных областей использования (см., например, работу Новика и др. [1149]).

Те же методы теперь начинают также применяться для проверки программного обеспечения с помощью таких систем, как программа проверки моделей Spin [672].

Например, с ее помощью была проверена до и после полета программа управления космическим аппаратом Remote Agent [633].

9.6. Резюме В этой главе приведен анализ логического вывода в логике первого порядка и многих алгоритмов его выполнения. • В первом подходе используется правило логического вывода для конкретизации кванторов в целях преобразования задачи логического вывода в форму пропозициональной логики. Как правило, этот подход характеризуется очень низким быстродействием. • Использование унификации для выявления подходящих подстановок для переменных позволяет устранить этап конкретизации в доказательствах первого порядка, в результате чего этот процесс становится гораздо более эффективным.

В поднятой версии правила отделения унификация применяется для получения естественного и мощного правила логического вывода — обобщенного правила отделения. В алгоритмах прямого логического вывода и обратного логического вывода это правило применяется к множествам определенных выражений.

Обобщенное правило отделения является полным применительно к определенным выражениям, но проблема логического следствия остается полуразрешимой. Для программ Datalog, состоящих из определенных выражений без функций, проблема логического следствия разрешима.

Прямой логический вывод используется в дедуктивных базах данных, где он может сочетаться с реляционными операциями баз данных. Он также применяется в продукционных системах, которые обеспечивают эффективное обновление при наличии очень больших наборов правил.

Прямой логический вывод для программ Datalog является полным и выполняется за время, определяемое полиномиальной зависимостью.

Обратный логический вывод используется в системах логического программирования, таких как Prolog, в которых реализована сложная технология компиляции для обеспечения очень быстрого логического вывода.

Недостатками обратного логического вывода являются излишние этапы логического вывода и бесконечные циклы; эти недостатки можно устранить путем запоминания.

Обобщенное правило логического вывода на основе резолюции позволяет создать полную систему доказательства для логики первого порядка с использованием баз знаний в конъюнктивной нормальной форме.

Существует несколько стратегий сокращения пространства поиска для систем с резолюцией без потери полноты. Эффективные средства автоматического доказательства теорем на основе резолюции использовались для доказательства интересных математических теорем, а также для проверки и синтеза программных и аппаратных средств.

Библиографические и исторические заметки Логический вывод широко исследовался древнегреческими математиками.

Аристотель тщательно исследовал один из типов логического вывода, называемый $\wedge$ силлогизмом, который представляет собой своего рода правило логического вывода.

Силлогизмы Аристотеля включали элементы логики первого порядка, такие как кванторы, но были ограничены унарными предикатами. Силлогизмы классифицировались по "фигурам" и "модусам", в зависимости от порядка термов (которые следовало бы назвать предикатами) в высказываниях и от степени общности (которую теперь принято интерпретировать с помощью кванторов), применяемой к каждому терму, а также с учетом того, является ли каждый терм отрицаемым. Наиболее фундаментальным силлогизмом является тот, который относится к первой фигуре первого модуса:

Все S суть M. Все M суть P. Следовательно, все S суть P.

Аристотель пытался доказать истинность других силлогизмов, "приводя" их к силлогизмам первой фигуры. Описание того, в чем должно состоять такое "приведение", оказалось гораздо менее точным по сравнению с описанием, в котором были охарактеризованы сами фигуры и модусы силлогизмов.

Готтлоб Фреге, который разработал полную логику первого порядка в 1879 году, основал свою систему логического вывода на большой коллекции логически правильных схем и единственном правиле логического вывода — правиле отделения (Modus Ponens). Фреге воспользовался тем фактом, что результат применения любого правила логического вывода в форме "Из P вывести ?)" можно моделировать путем применения к высказыванию P правила отделения наряду с логически правильной схемой $P \Rightarrow Q$. Такой "аксиоматический" стиль представления знаний, в котором наряду с правилом отделения использовался целый ряд логически правильных схем, был принят на вооружение после Фреге

многими логиками; наиболее замечательным является то, что этот стиль использовался в основополагающей книге *Principia Mathematica* [1584].

В подходе на основе натуральной дедукции (*natural deduction*), который был введен Герхардом Генценом [541] и Станиславом Яськовским [725], основное внимание было сосредоточено не на аксиоматических системах, а на правилах логического вывода. Натуральная дедукция получила название "натуральной", поскольку в ней не требуется преобразование в нормальную форму (неудобную для восприятия человеком), а также в связи с тем, что в ней применяются такие правила логического вывода, которые кажутся естественными для людей. Правитц [1235] посвятил описанию натуральной дедукции целую книгу. Галье [515] применил подход Гентцена для разъяснения теоретических основ автоматизированной дедукции.

Крайне важным этапом в разработке глубокого математического анализа логики первого порядка явилось изобретение формы представления в виде импликационных выражений (*clausal form*). Уайтхед и Рассел [1584] описали так называемые правила прохождения (*rules of passage*) (фактически этот термин принадлежит Эрбрану [650]), которые используются для перемещения кванторов в переднюю часть формул. Торальфом Сколемом [1424] были достаточно своевременно предложены сколемовские константы и сколемовские функции. Общая процедура сколемизации, наряду с важным понятием универсума Эрбрана, описана в [1425].

Крайне важную роль в разработке автоматизированных методов формирования рассуждений, как до, так и после введения Робинсоном правила резолюции, играет теорема Эрбрана, названная в честь французского логика Жака Эрбрана [650]. Это отражается в применяемом нами термине "универсум Эрбрана", а не "универсум Сколема", даже несмотря на то, что это понятие в действительности было открыто Сколемом. Кроме того, Эрбран может считаться изобретателем операции унификации. Гёдель [565], опираясь на идеи Сколема и Эрбрана, сумел показать, что для логики первого порядка имеется полная процедура доказательства. Алан Тьюринг [1518] и Алонсо Черч [255] практически одновременно продемонстрировали, используя очень разные доказательства, что задача определения общезначимости в логике первого порядка не имеет решения. В превосходной книге Эндертон [438] все эти результаты описаны в строгой, но труднодоступной для понимания манере.

Хотя Маккарти [1009] предложил использовать логику первого порядка для представления знаний и формирования рассуждений в искусственном интеллекте, первые подобные системы были разработаны логиками, заинтересованными в полу-

нии средств автоматического доказательства математических теорем. Впервые применение метода пропозиционализации и теоремы Эрбрана предложено Абрахамом Робинсоном, а Гилмор [556] написал первую программу, основанную на этом подходе. Дэвис и Патнем [336] применили форму представления в виде импликационных выражений и разработали программу, в которой предпринимались попытки поиска противоречий путем подстановки элементов универсума Эрбрана вместо переменных для получения базовых выражений, а затем поиска пропозициональных противоречивостей среди этих базовых выражений. Правиц [1234] разработал ключевую идею, позволяющую использовать для управления процессом поиска тенденцию к обнаружению пропозициональных противоречивостей и вырабатывать термы из универсума Эрбрана, только если это необходимо для определения пропозициональной противоречивости. После дальнейшей разработки этой идеи другими исследователями Дж.Э. Робинсон (который не связан родством с Абрахамом Робинсоном) пришел к созданию метода резолюции [1298]. Примерно в то же время советским исследователем С. Масловым [994], [995] на основе немного иных принципов был разработан так называемый инверсный метод, который характеризуется некоторыми вычислительными преимуществами над пропозиционализацией. Метод соединения Вольфганга Бибеля [123] может рассматриваться как расширение этого подхода.

После разработки метода резолюции исследования в области логического вывода первого порядка стали развиваться в нескольких разных направлениях. В искусственном интеллекте метод резолюции применялся для создания систем поиска ответов на вопросы Корделлом Грином и Бертрамом Рафаэлем [593]. Несколько менее формальный подход был принят Карлом Хьюиттом [651]. Разработанный им язык Planner, хотя и не был полностью реализован, явился предшественником логического программирования и включал директивы для прямого и обратного логического вывода и для отрицания как недостижения цели. А подмножество этого языка, известное как Micro-Planner [1475], было реализовано и использовалось в системе понимания естественного языка Shrdlu [1601]. В ранних реализациях систем искусственного интеллекта большие усилия направлялись на разработку структур данных, которые должны были обеспечить эффективную выборку фактов; эти работы описаны в книгах по программированию для искусственного интеллекта [240], [479], [1148].

В начале 1970-х годов в искусственном интеллекте полностью утвердился метод прямого логического вывода как легко доступная пониманию альтернатива методу резолюции. Прямой логический вывод использовался в самых различных системах, начиная от программы автоматического доказательства геометрических теорем Невинса [1123] и заканчивая экспертной системой R1 для разработки конфигурации компьютеров VAX [1026]. Приложения искусственного интеллекта обычно

охватывают большое количество правил, поэтому было важно разработать эффективную технологию согласования с правилами, особенно для инкрементных обновлений.

Для поддержки таких приложений была разработана технология продукционных систем. Язык продукционных систем Ops-5 [197], [482] использовался для экспертной системы R1 и для когнитивной архитектуры Soar [880]. В язык Ops-5 был включен процесс согласования с помощью rete-алгоритма [483]. Архитектура Soar, позволяющая вырабатывать новые правила для кэширования результатов предыдущих вычислений, способна создавать очень большие множества правил; например, в системе TacAir-Soar, предназначенной для управления тренажером, моделирующим самолет-истребитель [743], количество правил превышало один миллион. Язык

CLIPS [1626] продукционных систем на основе языка C, разработанный в NASA, обеспечивал лучшую интеграцию с другими программными, аппаратными и сенсорными системами и использовался для автоматизации космической станции и разработки нескольких военных приложений.

Большой вклад в понимание особенностей прямого логического вывода внесли также работы в области исследований, известной как дедуктивные базы данных. Исследования в этой области начались с симпозиума, организованного в Тулузе в 1977 году Джеком Минкером, который собрал вместе специалистов в области логического вывода и систем баз данных [514]. В опубликованном сравнительно недавно историческом обзоре [1264] сказано: "Дедуктивные системы [баз данных] были попыткой адаптировать язык Prolog, воплощающий видение мира с «малыми данными», к миру «больших данных»". Таким образом, цель разработок в этой области состоит в объединении технологии баз данных, которая предназначена для выборки больших множеств фактов, с технологией логического вывода на основе языка Prolog, в которой обычно осуществляется выборка одновременно только одного факта. К числу работ в области дедуктивных баз данных относятся [228] и [1525].

Важная работа, выполненная Чандрой и Нарелом [231], а также Ульманом [1524], привела к признанию языка Datalog в качестве стандартного языка для дедуктивных баз данных. Кроме того, стал стандартным "восходящий" логический вывод, или прямой логический вывод, отчасти потому, что данный метод позволяет избежать проблем, обусловленных незавершаемыми и избыточными вычислениями, которые возникают при обратном логическом выводе, и отчасти потому, что имеет более естественную реализацию в терминах основных операций реляционной базы данных.

Разработка метода магических множеств для перезаписи правил Бансильоном и др. [67] позволила воспользоваться в прямом логическом выводе преимуществами целенаправленности, свойственными обратному логическому выводу. Методы табулированного логического программирования (с. 1), вступившие в конкурентную борьбу с другими методами, заимствовали преимущества динамического программирования от прямого логического вывода.

Большой вклад в понимание сложностей логического вывода внесло сообщество пользователей дедуктивных баз данных. Чандра и Мерлин [232] впервые показали, что задача согласования единственного нерекурсивного правила (в терминологии баз данных — $\wedge$ конъюнктивного запроса) может оказаться NP-трудной. Купер и Варди [870] предложили использовать понятие " $\wedge$ сложности данных (т.е. сложности как функции размера базы данных, в которой размер правила рассматривается как постоянный) в качестве подходящего критерия эффективности получения ответов на запросы. Готтлоб и др. [586] обсудили связь между конъюнктивными запросами и задачами удовлетворения ограничений, показав, как можно использовать способ декомпозиции гипердерева для оптимизации процесса согласования.

Как уже упоминалось выше, процедуры обратного логического вывода, применяемые для логического вывода, впервые появились в разработанном Хьюиттом языке Planner [651]. Но логическое программирование как таковое развивалось независимо от этого направления разработок. Ограниченная форма линейной резолюции, называемая "Sk SL-резолюцией, была разработана Ковальским и Кюнером [853] на основе метода устранения моделей Лавленда [947]; после применения этого метода к определенным выражениям он принял вид метода $\wedge$ SLD-резолюции, который предоставил возможность осуществлять интерпретацию определенных выражений как про-

грамм [849-851]. Между тем в 1972 году французский исследователь Ален Колмерор разработал и реализовал Prolog в целях синтаксического анализа текста на естественном языке; первоначально выражения Prolog предназначались для использования в качестве правил контекстно-свободной грамматики [285], [1311]. Основная часть теоретических основ логического программирования разработана Ковальским в сотрудничестве с Колмерором. Семантическое определение с использованием наименьших фиксированных точек предложено Ван Эмденом и Ковальским [1530].

Ковальский и Коэн [274], [852] подготовили хорошие исторические обзоры истоков языка Prolog. Теоретический анализ основ языка Prolog и других языков логического программирования приведен в книге Foundations of Logic Programming [940].

Эффективные компиляторы Prolog главным образом основаны на модели абстрактной машины Уоррена (Warren Abstract Machine — WAM), разработанной

Дэвидом Г.Д. Уорреном [1556]. Ван Рой [1536] показал, что благодаря применению дополнительных методов организации работы компилятора, таких как логический вывод типов, программы Prolog становятся способными конкурировать по быстродействию с программами С. Рассчитанный на 10 лет исследовательский проект создания компьютера пятого поколения, который был развернут в Японии в 1982 году, опирался полностью на язык Prolog, применяемый в качестве средства разработки интеллектуальных систем.

Методы предотвращения нежелательного заикливания в рекурсивных логических программах были разработаны независимо Смитом и др. [1434], а также Тамаки и Сато [1487]. Кроме того, последняя статья включала данные о методе запоминания, предназначенном для логических программ, который интенсивно разрабатывался в качестве метода табулированного логического программирования Дэвидом С.

Уорреном. Свифт и Уоррен [1483] показали, как дополнить машину WAM для обеспечения табуляции, что позволяет добиться быстродействия программ Datalog, превышающего на порядок быстродействие дедуктивных систем баз данных с прямым логическим выводом.

Первые теоретические работы по логическому программированию в ограничениях были выполнены Джаффаром и Лассе [722]. Джаффар и др. [723] разработали систему CLP(R) для обработки ограничений с действительными значениями. В [724] приведено описание обобщенной машины WAM, на основе которой создана машина CLAM (Constraint Logic Abstract Machine — абстрактная машина логики ограничений) для разработки спецификаций различных реализаций систем CLP. В [10] описан сложный язык Life, в котором методы CLP сочетаются с функциональным программированием и формированием рассуждений в логике наследования. В [820] описан перспективный проект использования логического программирования в ограничениях в качестве основы архитектуры управления в реальном времени, которая может применяться для создания полностью автоматических средств вождения самолетов (автопилотов).

Объем литературы по логическому программированию и языку Prolog очень велик. Одной из первых книг по логическому программированию явилась книга Logic for Problem Solving [851]. Языку Prolog посвящены, в частности, книги [175], [270] и [1405]. Превосходный обзор тематики CLP приведен в [987]. До его закрытия в 2000 году официальным журналом для публикаций в этой области был Journal of Logic Programming; теперь вместо него выпускается журнал Theory and Practice of Logic Programming. К числу основных конференций по логическому программированию относятся International Conference on Logic Programming (ICLP) и International Logic Programming Symposium (ILPS).

Исследования в области автоматического доказательства математических теорем начались еще до того, как были впервые разработаны полные логические системы первого порядка. В разработанной Гербертом Гелернтером программе Geometry Theorem Prover [532] использовались методы эвристического поиска в сочетании с диаграммами для отсека ложных подцелей; с помощью этой программы удалось обосновать некоторые весьма сложные результаты математических исследований в области евклидовой геометрии. Но с тех пор взаимодействие таких научных направлений, как автоматическое доказательство теорем и искусственный интеллект, не слишком велико.

На первых порах основные усилия ученых были сосредоточены на проблемах полноты. Вслед за появлением оригинальной статьи Робинсона в работах [1619] и [1620] были предложены правила демодуляции и парамодуляции для формирования рассуждений с учетом отношения равенства. Эти правила были также разработаны независимо в контексте систем перезаписи термов [811]. Внедрение средств формирования рассуждений с учетом отношения равенства в алгоритм унификации было осуществлено Гордоном Плоткиным [1217]; применение таких средств было также предусмотрено в языке Qlisp [1339]. В [752] приведен обзор средств унификации с учетом отношения равенства на основе процедур перезаписи термов. Эффективные алгоритмы для стандартной унификации были разработаны Мартелли и Монтанари [989], а также Патерсоном и Вегманом [1181].

Кроме средств формирования рассуждений с учетом отношения равенства, в программы автоматического доказательства теорем были включены всевозможные процедуры принятия решений специального назначения. В [1120] предложена получившая широкое распространение схема интеграции подобных процедур в общую схему формирования рассуждений; к другим методам относятся "резолюция теории" Стикеля [1462] и "специальные отношения" Манна и Валдингера [978].

Для метода резолюции был предложен целый ряд стратегий управления, начиная со стратегии предпочтения единичного выражения [1616]. В [1617] была предложена стратегия с использованием множества поддержки, которая позволяет обеспечить определенную целенаправленность резолюции. Линейная резолюция впервые была предложена в [948]. В [537, глава 5] приведен краткий, но исчерпывающий анализ всего разнообразия стратегий управления.

В [602] описана одна из первых программ автоматического доказательства теорем, Sam, которая позволила решить одну из открытых проблем в теории решеток.

В [1621] приведен краткий обзор того, какой вклад был внесен с помощью программы автоматического доказательства теорем Aura в решение открытых проблем в различных областях математики и логики. Это описание продолжено в [1018], где перечислены достижения программы Otter, преемника программы Aura,

в решении открытых проблем. В [1563] описана программа Spass— одна из сильнейших современных программ автоматического доказательства теорем. Основным справочником по программе автоматического доказательства теорем Бойера—Мура является книга *A Computational Logic* [165]. В [1463] рассматривается система PTTP (Prolog Technology Theorem Prover), в которой сочетаются преимущества компиляции Prolog с полнотой устранения моделей [947]. Еще одной широко применяемой программой автоматического доказательства теорем, основанной на этом подходе, является SETHEO [915]; она способна выполнять несколько миллионов логических выводов

в секунду на рабочих станциях модели 2000. Эффективной программой автоматического доказательства теорем, реализованной всего лишь в 25 строках на языке Prolog, является LeanTaP [91].

Одни из первых работ в области автоматизированного синтеза программ были выполнены Саймоном [1416], Грином [591], а также Манна и Валдингером [976].

В трансформационной системе Бурстолла и Дарлингтона [211] используется формирование рассуждений с учетом отношения равенства для синтеза рекурсивных программ. Одной из самых сильных современных систем является Kids [1435], [1436]; она действует в качестве помощника эксперта. В [979] приведено учебное введение с описанием современного состояния дел в этой области, в котором основное внимание уделено описанию собственного дедуктивного подхода этих авторов. В книге *Automating Software Design* [954] собрано множество статей из этой области. Обзор примеров использования логики в проектировании аппаратных средств приведен в [791]; в [267] рассматривается применение метода проверки по модели для диагностирования аппаратных средств.

Хорошим справочником по темам полноты и неразрешимости является книга *Computability and Logic* [150]. Многие ранние статьи в области математической логики можно найти в книге *From Frege to Godel: A Source Book in Mathematical Logic* [1532].

Официальным журналом для публикаций в области чистой математической логики (в отличие от автоматизированного дедуктивного логического вывода) является *The Journal of Symbolic Logic*. К числу учебников, посвященных автоматизированному дедуктивному логическому выводу, относятся классическая книга *Symbolic Logic and Mechanical Theorem Proving* [233], а также более новые работы [124], [776] и [1618].

Антология *Automation of Reasoning* [1408] включает много важных ранних статей по автоматизированному дедуктивному логическому выводу. Другие исторические обзоры приведены в [206] и [949]. Основным журналом для публикаций в области автоматического доказательства теорем является *Journal of Automated Reasoning*, а главной конференцией — ежегодно проводимая конференция *Conference on*

Automated Deduction (CADE). Кроме того, исследования в области автоматического доказательства теорем тесно связаны с работами по использованию логики при анализе программ и языков программирования, которым посвящена основная конференция Logic in Computer Science.

Упражнения 9.1. Докажите на основании главных логических принципов, что процедура конкретизации с помощью квантора всеобщности является непротиворечивой и что процедура конкретизации с помощью квантора существования позволяет получить базу знаний, эквивалентную с точки зрения логического вывода.

9.2. Представляется вполне обоснованным утверждение, что из отдельного факта Likes (Jerry, IceCream) можно вывести высказывание $\exists x \text{ Likes } \{x, \text{IceCream}\}$. Запишите общее правило логического вывода, правило $\wedge$ введения квантора существования, позволяющее узаконить такой логический вывод. Тщательно сформулируйте условия, которым должны удовлетворять переменные и термы, участвующие в этом выводе.

9.3. Предположим, что база знаний содержит только одно высказывание, $\exists x \text{ AsHighAs}\{x, \text{Everest}\}$. Какие из следующих фактов являются действительными результатами применения правила конкретизации с помощью квантора существования? а) $\text{AsHighAs}\{\text{Everest}, \text{Everest}\}$. б) $\text{AsHighAs}\{\text{Kilimanjaro}, \text{Everest}\}$. в) $\text{AsHighAs}\{\text{Kilimanjaro}, \text{Everest}\}$ и $\text{AsHighAs}\{\text{BenNevis}, \text{Everest}\}$ (после двух применений).

9.4. Для каждой приведенной ниже пары атомарных высказываний укажите наиболее общий унификатор, если он существует. а) $P\{A, B, B\}$, $P\{x, y, z\}$. б) $Q\{y, G\{A, B\}\}$, $Q\{G\{x, x\}, y\}$. в) $\text{Older}\{\text{Father}\{y\}, y\}$, $\text{Older}\{\text{Father}\{x\}, \text{John}\}$. г) $\text{Knows}\{\text{Father}\{y\}, y\}$, $\text{Knows}\{x, x\}$.

9.5. Рассмотрите решетки обобщения, приведенные на рис. 9.1. а) Составьте решетку для высказывания $\text{Employs}\{\text{Mother}\{\text{John}\}, \text{Father}\{\text{Richard}\}\}$. б) Составьте решетку для высказывания $\text{Employs}(\text{IBM}, y)$ ("Компания IBM является нанимателем для всех"). Не забудьте включить запрос любого рода, который унифицируется с этим высказыванием. в) Предположим, что функция Store индексирует каждое высказывание под каждым узлом в его решетке обобщения. Объясните, как должна работать функция Fetch, если некоторые из этих высказываний содержат переменные; воспользуйтесь в качестве примера высказываниями, приведенными в упр. 9.5, а и 9.5, б, а также запросом $\text{Employs}\{x, \text{Father}(x)\}$.

9.6. Предположим, что в логическую базу данных помещена часть данных переписи населения США с указанием возраста, города проживания, даты рождения и имени матери каждого лица, с использованием номеров карточек социального страхования в качестве идентифицирующих констант для каждого лица.

Таким образом, например, возраст Джорджа задается выражением AgeD4365-1282,56). Какая из приведенных ниже схем индексации S1-S5 позволяет эффективно находить ответы на каждый из запросов Q1-Q4 (при условии, что применяется обычный метод обратного логического вывода)? • Схемы индексации. • S1. Индекс для каждого атомарного термина в каждой позиции. • S2. Индекс для каждого первого параметра. • S3. Индекс для каждого атомарного термина предиката. • S4. Индекс для каждой комбинации предиката и первого параметра. • S5. Индекс для каждой комбинации предиката и второго параметра и индекс для каждого первого (нестандартного) параметра. • Запросы. • Q1. Ade{443-44-4321,x). • Q2. ResidesIn{x,Houston).

• Q3. Mother (x, y). • Q4.Age(x,34) л ResidesIn(xf TinyTownUSA).

9.7. Можно было бы предположить, что стандартизация раз и навсегда отличий всех высказываний в базе знаний позволяет избежать проблемы конфликта переменных при унификации в процессе обратного логического вывода. Покажите, что для некоторых высказываний этот подход не применим. (Подсказка.

Рассмотрите высказывание, одна часть которого унифицируется с другой.) 9.8. Объясните, как записать любую конкретную формулировку задачи 3-SAT произвольного размера с использованием единственного определенного выражения первого порядка и не больше 30 базовых фактов.

9.9. Запишите логические представления для приведенных ниже высказываний, применимые для использования с обобщенным правилом отделения. а) Лошади, коровы и свиньи — млекопитающие. б) Рожденный лошастью — лошадь. в) Блюбёрд — лошадь. г) Блюбёрд — родитель Чарли. д) Отношения "быть рожденным" и "быть родителем" — обратные. е) Каждое млекопитающее имеет родителя.

9.10. В этом упражнении для получения ответов на вопросы с помощью алгоритма обратного логического вывода используются высказывания, записанные при решении упр. 9.9. а) Нарисуйте дерево доказательства, сформированное исчерпывающим алгоритмом обратного логического вывода для запроса $\exists h$ Horse(h) ("Существует некоторая лошадь"), в котором выражения согласуются в указанном порядке. б) Какие особенности этой проблемной области вы обнаружили? в) Какое количество решений для h фактически следует из ваших высказываний? г) Можете ли вы предложить способ поиска всех этих решений? (Подсказка.

Вам может потребоваться обратиться к [1434].) 9.11. Одной из известных детских английских загадок является следующая:

"Brothers and sisters have I none, but that man's father is my father's son" (Братьев и сестер у меня нет, но отец этого человека — сын моего отца). С человек, о котором

говорится в загадке. Вы можете применять любые методы логического вывода, описанные в этой главе. Почему, по вашему мнению, эту загадку трудно отгадать сразу?

9.12. Проследите за выполнением алгоритма обратного логического вывода, приведенного в листинге 9.3, при его применении для решения задачи доказательства преступления. Покажите, какую последовательность значений принимает переменная *goals*, и расположите эти значения в виде дерева.

9.13. Приведенный ниже код Prolog определяет предикат *p*:

`P(X,[X|Y]). P(X,[Y Z]) :- P(X,Z).`

- а) Покажите деревья доказательства и решения для запросов `P(A/[1,2/3])` и `P(B, [1,A,3])`.
- б) Какую стандартную операцию со списками представляет предикат *P*?

9.14. Si) В этом упражнении рассматривается применение сортировки в языке Prolog.

- а) Напишите выражения Prolog, которые определяют предикат `sorted (L)`, принимающий истинное значение тогда и только тогда, когда список *L* отсортирован в возрастающем порядке.
- б) Напишите на языке Prolog определение предиката `perm(L, M)`, который принимает истинное значение тогда и только тогда, когда *L* — перестановка *M*.
- в) Определите предикат `sort (L,M)` (*M* — отсортированная версия *L*) с использованием предикатов `perm` и `sorted`.
- г) Применяйте предикат `sort` ко все более длинным и длинным спискам, пока вам это не надоест. Какова временная сложность вашей программы?
- д) Реализуйте на языке Prolog более быстрый алгоритм сортировки, такой как сортировка вставкой (`insert sort`) или быстрая сортировка (`quicksort`).

9.15. Si) В этом упражнении рассматривается рекурсивное применение правил перезаписи с использованием логического программирования. Правилom перезаписи (или демодулятором, в терминологии программы Otter) является уравнение с указанным направлением применения. Например, правило перезаписи `x+0->x` указывает, что любое выражение, которое согласуется с `x+0`, должно заменяться выражением *x*. Средства применения правил перезаписи составляют центральную часть систем формирования рассуждений с учетом отношения равенства. Для представления правил перезаписи мы будем использовать предикат `rewrite (X, Y)`. Например, приведенное выше правило перезаписи может быть представлено как `rewrite (X+0, X)`. Некоторые термы являются примитивными и не могут подвергаться дальнейшим упрощениям, поэтому мы будем использовать запись `primitive @` для указания на то, что 0 — примитивный терм.

- а) Запишите определение предиката `simplify (X,Y)`, который принимает истинное значение, если *Y* — упрощенная версия *X*, т.е. к каким-либо подвыражениям *Y* больше не применимы какие-либо правила перезаписи.
- б) Запишите коллекцию правил перезаписи для упрощения выражений, в которых применяются арифметические

операторы, и примените ваш алгоритм упрощения к некоторым примерам выражений. в) Запишите коллекцию правил перезаписи для символического дифференцирования и примените их наряду с определенными вами правилами упрощения для дифференцирования и упрощения выражений, в которых есть арифметические выражения, включая возведение в степень.

9.16. В этом упражнении рассматривается реализация алгоритмов поиска на языке Prolog. Предположим, что предикат `successor (X,Y)` принимает истинное значение, если состояние `Y` является преемником состояния `X`, и что предикат `goal (X)` принимает истинное значение, если `X` — целевое состояние.

Запишите определение для предиката `solve (X,P)`, который означает, что `P` — путь (список состояний), начинающийся от `X`, оканчивающийся в целевом

состоянии и состоящий из последовательности допустимых шагов, которые определены предикатом `successor`. Вы обнаружите, что простейшим способом решения этой задачи является поиск в глубину. Насколько легко будет ввести эвристическое управление поиском?

9.17. Как можно воспользоваться методом резолюции для демонстрации того, что некоторое высказывание является общезначимым? Невыполнимым?

9.18. Из высказывания "Лошади — животные" следует, что "голова лошади — голова животного". Продемонстрируйте, что этот логический вывод является допустимым, выполнив приведенные ниже этапы. а) Преобразуйте предпосылку и вывод этого высказывания в язык логики первого порядка. Воспользуйтесь тремя предикатами: `HeadOf(hfx)` (который означает, что "`h` — голова `x`"), `Horse (x)` и `Animal (x)`. б) Примените отрицание к заключению и преобразуйте предпосылку и отрицаемое заключение в конъюнктивную нормальную форму. в) Воспользуйтесь правилом резолюции, чтобы показать, что заключение следует из предпосылки.

9.19. Ниже приведены два высказывания на языке логики первого порядка.

$A. \forall x \exists y (x > y)$ $B. \exists y \forall x (x > y)$ а) Допустим, что переменные пробегают по всем натуральным числам $0, 1, 2, \dots$, и что предикат $>$ означает "больше или равно". При использовании такой интерпретации переведите высказывания A и B на естественный язык. б) Является ли высказывание A истинным при этой интерпретации? в) Является ли высказывание B истинным при этой интерпретации? г) Является ли B логическим следствием A ? д) Является ли A логическим следствием B ? е) С использованием правила резолюции попытайтесь доказать, что A следует из B . Сделайте эту попытку, даже если вы считаете, что A не следует логически из B ; продолжайте свои усилия до тех пор, пока доказательство не оборвется и вы не сможете продолжать дальше (поскольку оно оборвалось). Покажите унифицирующую подстановку для каждого этапа резолюции. Если доказательство

окончилось неудачей, точно объясните, где, как и почему оно оборвалось. ж) А теперь попытайтесь доказать, что в следует из А.

9.20. Метод резолюции способен вырабатывать неконструктивные доказательства для запросов с переменными, поэтому приходится вводить специальные механизмы для извлечения только определенных ответов. Объясните, почему такая проблема не возникает при использовании баз знаний, содержащих только определенные выражения.

9.21. В этой главе было указано, что метод резолюции не может использоваться для формирования всех логических следствий из некоторого множества высказываний. Позволяет ли какой-то другой алгоритм решить эту задачу?

Глава 10. Представление знаний

10 Представление знаний В этой главе показано, как использовать логику первого порядка для представления наиболее важных аспектов реального мира, таких как действия, пространство, время, мыслимые события, а в качестве примера описано осуществление покупок.

В предыдущих трех главах была описана технология агентов, основанных на знаниях: синтаксис, семантика и теория доказательства для пропозициональной логики и логики первого порядка, а также реализация агентов, в которых применяются эти варианты логики. В данной главе рассматривается вопрос о том, какое информационное наполнение следует поместить в базу знаний такого агента, т.е. как представлять факты о мире.

В разделе 10.1 описывается идея общей онтологии, которая позволяет организовать все, что существует в мире, в виде иерархии категорий. В разделе 10.2 рассматриваются основные категории объектов, веществ и мер. В разделе 10.3 обсуждаются способы представления для действий, которые играют центральную роль в создании агентов, основанных на знаниях, а также раскрывается более общее понятие событий, или пространственно-временных фрагментов. В разделе 10.4 излагаются знания об убеждениях, а в разделе 10.5 все эти знания переводятся в контекст среды осуществления покупок в Internet. В разделах 10.6 и 10.7 рассматриваются специализированные системы формирования рассуждений для представления неопределенных и изменяющихся знаний.

10.1. Онтологическая инженерия В "учебных", экспериментальных проблемных областях выбор представления не столь важен; для них несложно найти подходящий словарь. С другой стороны, для таких сложных проблемных областей, как осуществление покупок в Internet или управление роботом в изменяющейся физической среде, требуются более общие и гибкие способы представления. В настоящей главе показано, как создать такие представления, сосредоточиваясь на

общих понятиях (таких как действия, время, физические объекты и убеждения), которые проявляются во многих разных про-

блемных областях. Способы представления таких абстрактных понятий иногда называют $\wedge$ онтологической инженерией; процесс применения этих способов связан с процессом инженерии знаний, описанным в разделе 8.4, но разворачивается в больших масштабах.

Попытка формально представить все, что существует в мире, была бы бесперспективной. Безусловно, нам не требуется фактически составлять полное описание всего, что существует в мире (требуемый для этого объем изложения был бы слишком велик даже для учебника с 1000 страниц), но мы можем оставить свободные места, в которые будут укладываться новые знания о любой проблемной области.

Например, мы определим, что подразумевается под понятием физический объект, а описания подробных сведений о различных типах объектов (роботах, телевизорах, книгах или о чем-то другом) оставим на будущее. Такая общая инфраструктура понятий называется $\wedge$ верхней онтологией, поскольку принято соглашение составлять схемы онтологии, помещая общие понятия в верхней части, а более конкретные понятия — под ними, как показано на рис. 10.1.

Anything AbstractObjects GeneralizedEvents Sets Numbers RepresentationalObjects Interval Places PhysicalObjects Processes Categories Sentences Measurements Moments Things Stuff Times Weights Animals Agents Solid Liquid Gas V Humans Рис. 10.1. Верхняя онтология мира, в которой показаны темы, рассматриваемые ниже в данной главе. Каждая дуга указывает на нижнее понятие, которое является уточнением верхнего. Прежде чем приступить к более подробному обсуждению онтологии, необходимо высказать одно важное предостережение. Мы решили использовать логику первого порядка для обсуждения содержания и организации знаний. Но некоторые аспекты реального мира трудно представить в логике первого порядка. Основная сложность состоит в том, что почти все обобщения имеют исключения или соблюдаются только до определенной степени. Например, хотя правило "помидоры имеют красный цвет" является достаточно удобным, некоторые помидоры имеют зеленый, желтый или оранжевый цвет. Аналогичные исключения можно указать применительно почти ко всем общим утверждениям из этой главы. Способность учитывать исключения и неопределенность является чрезвычайно важной, но она ортогональна задаче понимания общей онтологии. По этой причине мы отложим обсуждение исключений до раздела 10.6, а более общей темы неопределенной информации — до главы 13.

Для чего может применяться верхняя онтология? Еще раз рассмотрим онтологию электронных схем, описанную в разделе 8.4. В ней принято большое количество упрощающих допущений. Например, полностью исключено понятие времени.

Сигналы являются постоянными и не распространяются. Структура схемы остается неизменной. А если бы потребовалось сделать эту онтологию более общей, то нужно было бы учитывать, какие значения сигналы имеют в конкретные моменты времени, а также вводить данные о длине проводников и задержках распространения. Это позволило бы моделировать временнь/е свойства схемы, и такое моделирование действительно часто осуществляется проектировщиками схем. Кроме того, можно было бы ввести более широкий перечень интересующих классов электронных элементов, например, описывая технологию (ТТЛ, МОП, КМОП и т.д.), а также спецификацию ввода-вывода. Если же нам потребовалось бы обсуждать надежность или проводить диагностику, то необходимо было бы учесть возможность неконтролируемого изменения структуры схемы или свойств электронных элементов. Чтобы учесть паразитные емкости, нужно было бы перейти от чисто топологического представления связей к более реалистичному описанию геометрических свойств.

Рассматривая мир вампуса, можно прийти к аналогичным выводам. Хотя в описание этого мира включено время, оно имеет очень простую структуру: в мире ничего не происходит, кроме действий агента, а все изменения осуществляются мгновенно. Более общая онтология, которая лучше подходила бы для описания реального мира, должна была бы позволить синхронным изменениям распространяться во времени. Кроме того, для представления информации о том, в каких квадратах имеются ямы, использовался предикат *Pit*. Но можно было бы разрешить применять ямы разных типов, определив несколько объектов, принадлежащих к конкретному классу ям, каждый из которых характеризуется разными свойствами. Кроме того, в этом мире могла быть предусмотрена возможность обеспечить существование других животных, кроме вампусов. В таком случае не было бы возможности точно определять вид встретившегося животного из доступных результатов восприятия, поэтому потребовалось бы разработать биологическую таксономию для мира вампуса, позволяющую агенту прогнозировать оптимальное поведение на основании скупой информации.

При использовании любой онтологии специального назначения существует возможность вносить изменения, подобные этим, для перехода к большей общности.

В таком случае возникает очевидный вопрос — сходятся ли все эти онтологии к некоторой онтологии общего назначения? По истечении целых столетий философских и математических исследований на этот вопрос был дан лишь один ответ — "Возможно". В данном разделе представлена одна из таких версий,

воплощающая в себя синтез идей, собранных в течение многих веков. Онтологии общего назначения обладают двумя описанными ниже основными особенностями, которые отличают их от коллекций отдельных онтологий специального назначения.

- Любая онтология общего назначения должна быть в большей или меньшей степени применимой для любой специализированной проблемной области (с добавлением аксиом конкретной проблемной области). Это означает, что в ней по мере возможности не следует ни детализировать, ни скрывать какие-либо вопросы конкретного представления.

- В любой проблемной области, характеризующейся достаточно высокой значимостью, различные области знаний должны быть унифицированы, поскольку процессы формирования рассуждений и решения задач могут включать сразу несколько направлений одновременно. Например, в системе ремонта электронных схем работа необходимо формировать рассуждения и о самих схемах, с точки зрения состояния электрических соединений и физической компоновки, и о времени, поскольку последнее требуется для анализа временных свойств схем и оценки необходимых затрат труда на ремонт.

Поэтому высказывания с описанием времени должны быть применимыми для комбинирования с высказываниями, описывающими пространственную компоновку, и одинаково хорошо представлять не только наносекунды и минуты, но и ангстремы и метры.

После представления в этой главе общей онтологии мы применим ее для описания проблемной области осуществления покупок в Internet. Эта проблемная область чрезвычайно хорошо подходит для упражнений в области онтологии, а также оставляет для читателя большой простор, чтобы он мог сам создавать творческие представления знаний. Достаточно представить себе, например, что агент для осуществления покупок в Internet должен знать о бесчисленном множестве тем и авторов, чтобы покупать книги на узле Amazon.com, владеть информацией о всевозможных сортах пищевых продуктов, чтобы покупать бакалейные товары на узле Peapod.com, а также иметь сведения обо всем, что можно найти на дешевой распродаже, чтобы не упустить выгодные сделки на узле Ebay.com¹.

10.2. Категории и объекты Крайне важной частью любого способа представления знаний является классификация объектов по n категориям. Хотя взаимодействие с миром происходит на уровне отдельных объектов, с n формирование рассуждений в основном происходит на уровне категорий. Например, покупатель может иметь перед собой цель купить хотя бы какой-то баскетбольный мяч, а не конкретный баскетбольный мяч, допустим, обозначенный номером BB9. Кроме того, категории позволяют многое предсказывать в отношении объектов после того, как эти объекты вошли в состав классификации. При этом делается заключение о наличии некоторых объектов на основании сенсорных входных данных, определяется принадлежность

к категории из воспринятых свойств объектов, а затем информация о категории используется для составления прогнозов, касающихся этих объектов. Например, на основании того, что некоторый объект имеет зеленый цвет, полосатую поверхность, большой размер и круглую форму, можно сделать вывод, что это — арбуз, а исходя из этого заключить, что данный объект подходит для использования в качестве сладкого десерта.

Для представления категорий в логике первого порядка могут применяться два основных способа: представление с помощью предикатов или с помощью объектов. Это означает, что для этого можно либо применить предикат, такой как Basketball (b),¹ Авторы заранее приносят извинения, если по независящим от них обстоятельствам некоторые из упомянутых здесь оперативных магазинов уже не будут функционировать к тому времени, как эта книга попадет в руки читателя.

либо овестить всю категорию баскетбольных мячей в виде некоторого объекта — множества баскетбольных мячей Basketballs. После этого можно сформировать высказывание Member (b, Basketballs) (которое мы будем сокращенно записывать как $b \in \text{Basketballs}$) в качестве утверждения, что b — элемент категории баскетбольных мячей. Высказывание Subset {Basketballs, Balls} (сокращенно обозначаемое как Basketballs $\subset$ Balls) может применяться в качестве утверждения, что Basketballs — подкатегория, или подмножество мячей Balls. Поэтому любую категорию можно рассматривать как множество элементов или считать ее более сложным объектом и полагать, будто просто так оказалось, что для этого объекта определены отношения Member и Subset.

Категории служат для организации и упрощения базы знаний с помощью $\wedge$ наследования. Если известно, что все экземпляры категории Food (пища) съедобны, и сформулировано утверждение, что Fruit (фрукты) — это подкласс класса Food, а Apples (яблоки) — подкласс класса Fruit, то становится известно, что каждое яблоко съедобно. Это определение формулируется таким образом, что отдельные яблоки наследуют свойства съедобности, в данном случае в силу своей принадлежности к категории Food.

Отношения между классами и подклассами позволяют организовывать категории в виде некоторой $\wedge$ таксономии, или таксономической иерархии. Явно заданные таксономии использовались в прикладных науках в течение многих столетий.

Например, предметом систематической биологии является создание таксономии для всех существующих и исчезнувших видов; в библиографии разработана таксономия всех областей знаний, закодированная в виде десятичной системы Дьюи; а налоговые органы и другие правительственные организации разработали обширные таксономии профессий и коммерческих товаров. Кроме того, таксономия представляет собой важный аспект общих повседневных знаний.

С другой стороны, логика первого порядка позволяет легко формулировать факты о категориях, либо связывая объекты с категориями, либо применяя кванторы к их элементам, как описано ниже. • Любой объект — элемент некоторой категории, например:

BV9 e Basketballs • Любая категория — подкласс другой категории, например:

Basketballs C Balls • Все элементы категории имеют некоторые свойства, например: $x \in \text{Basketballs} \Rightarrow \text{Round}(x)$ • Элементы категории могут быть распознаны по некоторым свойствам, например:

$\text{Orange}(x) \wedge \text{Round}(x) \wedge \text{Diameter}(x) = "9.5" \wedge x \in \text{Balls} \Rightarrow x \in \text{Basketballs}$ • Вся категория в целом имеет некоторые свойства, например:

$\text{Dogs} \in \text{DomesticatedSpecies}$ Обратите внимание на то, что Dogs (Собаки) — и категория, и элемент категории DomesticatedSpecies (Домашние животные), поэтому последняя должна быть

категорией категорий. Могут даже существовать категории категорий категорий, но область их применения не так уж велика.

Хотя отношения между подклассами и классами, а также между элементами и множествами являются для категорий наиболее важными, необходимо также иметь возможность формулировать отношения между категориями, которые не являются подклассами друг друга. Например, если будет отмечено, что Males (Самцы) и Females (Самки) — подклассы класса Animals (Животные), то этим не будет сказано, что ни один самец не может одновременно быть самкой. Две или несколько категорий являются $\wedge$ непересекающимися, если они не имеют общих элементов.

И даже если известно, что категории самцов и самок не пересекаются, на этом основании нельзя утверждать, что животное, не являющееся самцом, должно быть самкой, если не сформулировано дополнительное утверждение, что самцы и самки образуют $\wedge$ исчерпывающую декомпозицию категории животных.

Непересекающуюся исчерпывающую декомпозицию принято называть $\wedge$ сегментацией. Эти три понятия иллюстрируются в приведенных ниже примерах.

Disjoint ({Animals, Vegetables}) ExhaustiveDecompositioni{Americans, Canadians,Mexicans}, Northernericans) Partitioni{Males,Females),Animals) (Обратите внимание на то, что исчерпывающая декомпозиция ExhaustiveDecomposition североамериканцев NorthAmeri cans не является сегментацией Parti tion, поскольку некоторые люди имеют двойное гражданство.) Эти три предиката определены следующим образом:

$\text{Disjoint } \{s\} \Leftrightarrow (\forall c_1, c_2 \in s \wedge c_1 \neq c_2 \Rightarrow \text{Intersection}(c_1, c_2) = \{\})$
 $\text{ExhaustiveDecompositionis, c} \Leftrightarrow (\forall i \in c \Rightarrow \exists c_2 \in s \wedge i \in c_2) \text{ Partition}(s, c) \Leftrightarrow$

Disjoint (s) л ExhaustiveDecompositionis, c) Категории могут быть также определены путем указания необходимых и достаточных условий принадлежности к ним. Например, холостяк — это неженатый взрослый мужчина: $x \in \text{Bachelors} \Leftrightarrow \text{Unmarried}(x) \wedge x \in \text{Adults} \wedge x \in \text{Males}$ Как описано во врезке "Естественные разновидности", строгие логические определения для категорий либо не всегда возможны, либо не всегда необходимы.

Физическая композиция Идея о том, что один объект может составлять часть другого, для нас весьма привычна. Нос — это часть лица, Румыния — часть Европы, а данная глава — часть настоящей книги. Для указания на то, что одна вещь является частью другой, используется общее отношение PartOf. Объекты могут группироваться в иерархии PartOf, напоминающие иерархию Subset (подмножество), например, как показано ниже.

PartOf (Bucharest, Romania) PartOf(Romania,EasternEurope) PartOf(EasternEurope, Europe) PartOf{Europe,Earth}

Отношение PartOf является транзитивным и рефлексивным, т.е. для него справедливы следующие высказывания:

$\text{PartOf}(x,y) \wedge \text{PartOf}(y,z) \Rightarrow \text{PartOf}(x,z)$ $\text{PartOf}(x,x)$ Поэтому можно сделать вывод, что $\text{PartOf}\{\text{Bucharest}, \text{Earth}\}$.

Категории $\wedge$ составных объектов часто характеризуются структурными отношениями между частями. Например, любое двуногое имеет две и только две ноги, прикрепленные к телу:

$\text{Biped}(a) \Rightarrow \exists l_1, l_2/b \text{ Leg}(l_i) \wedge \text{Body}(b) \wedge \text{PartOf}(l_1, a) \wedge \text{PartOf}(l_2, a) \wedge \text{PartOf}(b, a) \wedge \text{Attached}(l_1, b) \wedge \text{Attached}(l_2, b) \wedge l_1 \neq l_2 \wedge [\forall l_3 \text{ Leg}(h) \wedge \text{PartOf}(l_3, a) \Rightarrow (l_3 = l_1 \vee l_3 = l_2)]$ Применяемая здесь система обозначения понятия "две и только две" является довольно громоздкой; мы были вынуждены указать, что ног две, что ноги не являются одинаковыми и что если кто-то из двуногих будет утверждать, что у него есть третья нога, она в конечном итоге окажется одной из его двух ног. В разделе 10.6 будет показано, что формальная система, называемая описательной логикой, позволяет проще представить ограничения типа "две и только две".

Для категорий может быть определено отношение PartPartition, аналогичное отношению Partition (см. упр. 10.6). Любой объект состоит из частей, принадлежащих к его категориям PartPartition, и может рассматриваться как получающий некоторые свойства от собственных частей. Например, масса составного объекта — это сумма масс его частей. Обратите внимание на то, что это утверждение не распространяется на категории, не имеющие массы, даже если эти категории состоят из элементов, обладающих массой.

Кроме того, целесообразно определить составные объекты, имеющие различные части, но не имеющие конкретной структуры. Например, может потребоваться сформулировать утверждение: "Яблоки в этой сумке весят два килограмма". Может возникнуть соблазн приписать этот вес множеству яблок в сумке, но это было бы ошибкой, поскольку множество — это абстрактное математическое понятие, которое имеет элементы, но не имеет веса. Вместо этого необходимо ввести новое понятие, которое мы будем именовать $\wedge$ совокупностью (BunchOf). Например, если три яблока обозначены как Apple1, Apple2 и Apple3, то выражение BunchOf ({Apple1, Apple2, Apple3}) обозначает составной объект, частями (а не элементами) которого являются три яблока. Затем эта совокупность может использоваться как обычный, хотя и не структурированный объект. Обратите внимание на то, что BunchOf ({x}) = x. Кроме того, BunchOf (Apples) — это составной объект, состоящий из всех яблок, но его не следует путать с объектом Apples — категорией или множеством всех яблок.

Определим понятие BunchOf в терминах отношения PartOf. Очевидно, что каждый элемент множества s — это часть объекта BunchOf (s):

$\forall x \forall s \Rightarrow \text{PartOf}(x, \text{BunchOf}(s))$

Более того, BunchOf(s) — это наименьший объект, удовлетворяющий данному условию. Иными словами, BunchOf(s) должен быть частью любого объекта, который включает все элементы множества s в качестве части:

$\forall y [\forall x (x \in s \Rightarrow \text{PartOf}(x, y)) \Rightarrow \text{PartOf}(\text{BunchOf}(s), y)]$ Эти аксиомы представляют собой пример общего метода, называемого $\wedge$ логической минимизацией, с помощью которого любой объект может быть определен как наименьший объект, удовлетворяющий определенным условиям.

Естественные разновидности Некоторые категории имеют строгие определения, например, некоторый объект рассматривается как треугольник тогда и только тогда, когда он является многоугольником с тремя сторонами. С другой стороны, большинство категорий в реальном мире не имеют четких определений; они называются категориями $\wedge$ естественных разновидностей. Например, помидоры, как правило, имеют красный цвет; их форма приближается к шарообразной; там, где в верхней части был черенок, остается углубление; диаметр их составляет примерно от пяти до десяти сантиметров; кожица тонкая, но прочная; внутри находятся мякоть, семена и сок. Тем не менее существуют экземпляры, не совсем подходящие под это описание: некоторые помидоры — оранжевые, незрелые помидоры — зеленые, бывают помидоры больше или меньше среднего, а все помидоры-сливки одинаково малы. Вместо наличия полного определения помидоров мы имеем дело с набором характеристик, позволяющих выявлять объекты, которые, безусловно, представляют собой типичные помидоры, но могут не позволить отличить их от других объектов.

(Может быть, скоро появятся даже помидоры, покрытые пушком, как персики?) В связи с этим перед любым логическим агентом возникает проблема.

Агент не может быть уверен в том, что некоторый объект, ставший предметом его восприятия, действительно является помидором, и даже если бы он был уверен в этом, то не смог бы с уверенностью утверждать, какими свойствами типичных помидоров обладает данный объект. Эта проблема является неизбежным следствием того, что агенту приходится действовать в частично наблюдаемых вариантах среды.

Один из полезных подходов состоит в том, что нужно отделить сведения, истинные для всех экземпляров некоторой категории, от сведений, истинных только для типичных экземпляров. Поэтому, кроме категории Tomatoes, необходимо также предусмотреть категорию Typical (Tomatoes). Здесь функция Typical отображает категорию на подкласс, который содержит только типичные экземпляры:

Typical (c) с c Основной объем знаний об естественных разновидностях фактически относится к их типичным экземплярам: $x \in \text{Typical (Tomatoes)} \Rightarrow \text{Red}(x) \vee \text{Round}(x)$
Таким образом, мы получаем возможность формулировать полезные факты о категориях без точных определений.

Сложности подготовки точных определений для большинства естественных категорий были глубоко исследованы Витгенштейном [1608] в его книге Philosophical Investigations. Он использовал пример с играми, чтобы показать, что элементы любой категории обладают, скорее, "семейным сходством", а не необходимыми и достаточными для их классификации характеристиками.

Полезность понятия строгого определения была также поставлена под сомнение Квайном [1252], который указал, что небесспорно даже определение "холостяка" как неженатого взрослого мужчины. Например, вряд ли является корректным такое утверждение, что "Папа Римский — холостяк". Хотя это утверждение, строго говоря, не является ложным, сам способ употребления в нем несовместимых слов, безусловно, не заслуживает одобрения, поскольку вызывает у слушателя нежелательные ассоциации. Возможно, указанную проблему можно было бы решить, проводя различия между логическими определениями, подходящими для внутреннего представления знаний, и более утонченными критериями, касающимися корректного лингвистического словоупотребления. Последней цели можно было бы достичь, "фильтруя" утверждения, полученные после достижения первой. Кроме того, возможно, что неудачные результаты при лингвистической проверке правильности словоупотребления могут использоваться в качестве обратной связи для модификации внутренних определений, чтобы такая фильтрация стала ненужной.

Меры И в научных, и в обыденных теориях мира объекты имеют высоту, массу, стоимость и т.д. Значения, применяемые для оценки этих свойств, называются $\wedge$ мерами. Обычные количественные меры можно представить довольно легко. Предположим, что вселенная включает абстрактные "объекты мер", такие как длина, которая представляет собой длину следующего отрезка прямой: I 1. Этот объект можно назвать отрезком с длиной 1,5 дюйма, или 3,81 сантиметра. Поэтому одна и та же длина может иметь в естественном языке разные имена. С точки зрения логики такую особенность можно учесть, комбинируя $\wedge$ функцию единиц измерения с числом (альтернативная схема рассматривается в упр. 10.8). Если приведенному выше отрезку прямой присвоено имя L1? то можно записать следующее:

Length(Li) = InchesA.5) = CentimetersC.81) Преобразование из одних единиц измерения в другие осуществляется путем приравнивания кратных значений одной единицы другой:

CentimetersB.54 x d) = Inches(d) Аналогичные аксиомы могут быть записаны для фунтов и килограммов, секунд и суток, долларов и центов. Меры могут использоваться для описания объектов следующим образом:

Diameter(Basketball $\wedge$) = Inches(9.5) ListPrice(Basketballs) = \$A9) d e Days => Duration {d) = Hours B4) Обратите внимание на то, что \$ A) — это не долларовая купюра! Некто может иметь две долларовые купюры, но существует только один объект с именем \$ A).

Следует также отметить, что объекты Inches@) и Centimeters@) обозначают одну и ту же нулевую длину, но не идентичны другим нулевым мерам, таким как Seconds@).

Простые, количественные меры можно представить легко, а проблема представления других мер становится более сложной, поскольку для них не предусмотрена согласованная шкала значений. Упражнения характеризуются сложностью, десерты — тонким вкусом, а стихи — красотой, но эти качества не могут быть обозначены числами. Можно было бы, стремясь к полной описуемости всего, отказаться от этих свойств как бесполезных с точки зрения логических рассуждений, или, что еще хуже, попытаться оценить красоту по числовой шкале. Но такое решение было бы грубой ошибкой, поскольку оно отнюдь не является необходимым. Наиболее важным свойством мер являются не конкретные числовые значения, а тот факт, что меры могут быть упорядочены.

Хотя многие меры не являются числами, все еще остается возможность сравнивать их с помощью некоторого символа упорядочения, такого как >. Например, можно предположить, что упражнения, составленные Норвигом, — более трудные, чем составленные Расселом, и что сложные упражнения не позволяют добиться высоких оценок, как показано ниже. e-i e Exercises л e2 e Exercises л Wrote(Norvig, e±) л Wrote (Russell, e2) => Difficulty{ex) > Difficulty{e2) e± G Exercises л e2 e Exercises л Difficulty(e±)

$\text{Difficulty}(e_2) \Rightarrow \text{ExpectedScore}(e_1) < \text{ExpectedScore}(e_2)$ Этого достаточно, чтобы можно было принять решение о том, какие упражнения следует выбирать на экзаменах, даже несмотря на то, что для оценки их сложности не использовались какие-либо числовые значения (тем не менее для этого необходимо иметь возможность определить, кем составлены те или иные упражнения).

Такого рода монотонные связи между мерами образуют основу для качественной физики — одного из направлений искусственного интеллекта, в котором изучаются способы формирования рассуждений о физических системах без погружения в числовые расчеты и подробные уравнения. Качественная физика обсуждается в разделе с историческими замечками.

Вещества и объекты Реальный мир может рассматриваться как составленный из примитивных объектов (частиц) и сложных объектов, состоящих из примитивных. Проведение рассуждений на уровне больших объектов, таких как яблоки и автомобили, позволяет преодолеть сложности, связанные с учетом бесконечных количеств примитивных объектов, отдельно взятых. Тем не менее существует значительная часть реальности, которая, по-видимому, не поддается никаким явным попыткам ^ индивидуализации — деления на отдельные объекты. Мы присвоим этой части мира универсальное имя — ^вещество. Например, предположим, что перед исследователем находится немного масла и муравьед. Аналитик может утверждать, что перед ним один муравьед, но не в состоянии указать какое-то явное количество "масло-объектов", поскольку любая часть масло-объекта — это также масло-объект, по крайней мере, до тех пор, пока не удастся дойти действительно до очень мелких его частей. В этом и состоит основное различие между веще-

ствами и объектами. Две половинки муравьеда, разрезанного пополам, уже нельзя, к сожалению, назвать двумя муравьедами.

Во многих языках, в том числе в английском, проводится четкое различие между веществами и объектами. Например, в нем можно употребить неопределенный артикль со словом "муравьед" — "an aardvark", но нигде, кроме претенциозных калифорнийских ресторанов, не приходится слышать выражение "a butter", обозначающее порцию масла. Лингвисты проводят различие между ^ исчисляемыми существительными, такими как муравьеды, ямы и теоремы, и ^ неисчисляемыми существительными, такими как масло, вода и энергия. Разработано несколько конкурирующих онтологий, которые претендуют на то, что в них учитывается это различие. В данной главе будет описана только одна из них, а остальные рассматриваются в разделе с историческими замечками.

Чтобы правильно представить понятие вещества, начнем с очевидных соображений. Нам необходимо иметь в своей онтологии в качестве объектов, по меньшей мере, достаточно большие "куски" вещества, чтобы описать способы

взаимодействия с этими объектами. Например, в качестве куска масла можно было бы рассматривать то масло, которое осталось на столе от вчерашнего ужина; его можно собрать, взвесить, продать или проделать какие-то другие действия. В этом смысле кусок масла представляет собой объект, полностью аналогичный муравьеде. Назовем его Butter3. Кроме того, определим категорию Butter. Неформально всеми его элементами будут такие объекты, о которых можно сказать "Это — масло", включая Butter3. Если не учитывать некоторые предостережения об очень малых частях вещества, которые мы пока не рассматриваем, то любая часть масло-объекта также представляет собой масло-объект: $x \in \text{Butter} \wedge \text{PartOf}(y, x) \Rightarrow y \in \text{Butter}$ Теперь мы можем утверждать, что масло плавится при температуре около 30 градусов по Цельсию: $x \in \text{Butter} \Rightarrow \text{MeltingPoint}(x, \text{CentigradeC0})$ Продолжая его описание, можно сказать, что масло — желтое, имеет плотность меньше воды, размягчается при комнатной температуре, содержит много жиров и т.д. С другой стороны, масло не имеет конкретного размера, формы или веса. Могут быть определены более специализированные категории масла, такие как UnsaltedButter (Несоленое масло), которые также представляют собой разновидность вещества. С другой стороны, категория PoundOfButter (Фунт масла), которая включает в качестве элементов все масло-объекты, весящие один фунт, не является веществом! Если мы разрежем фунт масла пополам, то, к сожалению, не получим два фунта масла.

В этом анализе фактически обнаружилось следующее: существуют некоторые свойства, которые являются $\wedge$ внутренними, — они принадлежат к каждой части объекта, а не к самому объекту в целом. Разрезав кусок вещества пополам, получим две части, обладающие одним и тем же набором внутренних свойств, таких как плотность, точка кипения, запах, цвет, принадлежность и т.д. С другой стороны, $\wedge$ внешние свойства прямо противоположны — вес, длина, форма, назначение и пр. не сохраняются после деления на части.

Поэтому тот класс объектов, который включает в свое определение только внутренние свойства, обозначается качественными, или неисчисляемыми существительными, а класс, который включает в свое определение только внешние свойства, обозначается количественными, или исчисляемыми существительными. Категория Stuff — это наиболее общая категория вещества, в которой не задаются какие-либо внутренние свойства, а категория Thing — это наиболее общая категория различных объектов, в которой не задаются какие-либо внешние свойства. Все физические объекты принадлежат к обеим категориям, поэтому эти категории коэкстенсивны (равнообъемны) — они относятся к одним и тем же сущностям.

10.3. Действия, ситуации и события Формирование рассуждений о результатах действий является центральным звеном в функционировании агента, основанного на знаниях. В главе 7 приведены примеры пропозициональных высказываний, описывающих, как действия влияют на мир вампуса, например, в уравнении 7.3 на

с. 325 указано, как изменяется местонахождение агента в результате движения вперед. Одним из недостатков пропозициональной логики является необходимость иметь отдельную копию описания действия для каждого момента времени, в который может быть выполнено это действие.

В данном разделе описан метод представления с использованием логики первого порядка, позволяющий избежать возникновения этой проблемы.

Онтология ситуационного исчисления Один из очевидных способов предотвращения необходимости создавать многочисленные копии аксиом состоит в том, чтобы применить квантификацию по времени и использовать примерно такие высказывания: " $\forall t$ (для всех t) то-то и то-то является полученным в момент времени $t+1$ результатом выполнения такого-то действия в момент времени t ". Но в этом разделе вместо применения таких явных обозначений времени, как $t+1$, мы сосредоточимся на описании ситуаций, которые обозначают состояния, возникающие в результате выполнения действий. Такой подход, называемый $\wedge$ ситуационным исчислением, предусматривает использование описанной ниже онтологии.

- Как и в главе 8, действия представляют собой логические термы, такие как Forward и Turn (Right). В данном разделе предполагается, что среда содержит только одного агента. (Если количество агентов больше одного, может быть вставлен дополнительный параметр для указания на то, какой агент выполняет это действие.)
- $\wedge$ Ситуации представляют собой логические термы, состоящие из начальной ситуации (обычно называемой sQ) и всех ситуаций, которые создаются в результате применения некоторого действия в некоторой ситуации. Функция Result $\{a, s\}$ (иногда называемая Do) обозначает ситуацию, возникающую в результате выполнения действия a в ситуации s . Эта идея иллюстрируется на рис. 10.2.

Result(Тит(Right), Result(Forward, sQ)) Turn(Right) Result(Forward, sQ) Forward Рис. 10.2. Иллюстрация к понятию ситуации. В ситуационном исчислении каждая ситуация (кроме s_0) представляет собой результат некоторого действия

- $\wedge$ Флюентными называются функции и предикаты, которые изменяются от одной ситуации к другой, такие как местонахождение агента или наличие живого вампуса. Этот термин происходит от английского слова fluent, о котором в словаре сказано, что оно обозначает нечто текучее, как жидкость. В данном контексте термином "флюентный" обозначается все, что перетекает, или изменяется от одной ситуации к другой. В соответствии с общепринятым соглашением ситуация — это всегда последний параметр флюентного высказывания. Например, во флюентном высказывании $\neg \text{Holding}(\text{Glf } s_0)$ сообщается о том, что агент не владеет золотом G1 в начальной ситуации s_0 , а в высказывании Age (Wumpus, s_0) обозначается возраст вампуса в ситуации s_0 .
- Допускается также использовать вневременные, или неизменные предикаты и функции. В качестве соответствующих примеров можно указать предикат GoldiGx) и функцию LeftLegOf (Wumpus).

Кроме единственных действий, имеет также смысл рассуждать о последовательностях действий. Мы можем определять результаты последовательностей действий в терминах результатов отдельных действий. Прежде всего необходимо отметить, что выполнение пустой последовательности оставляет ситуацию неизменной:

$\text{Result}([], s) = s$ Выполнение непустой последовательности равносильно выполнению первого действия, а затем выполнению остальной ее части в результирующей ситуации:

$\text{Result}([a \backslash \text{seq}] , s) = \text{Result}(\text{seq}, \text{Result}(a, s))$

Агент, действующий на основе ситуационного исчисления, должен быть способен определять итог некоторой конкретной последовательности действий посредством логического вывода; в этом состоит задача $\wedge$ проектирования (нахождения проекции действий на будущее). При наличии подходящего конструктивного алгоритма логического вывода он должен также обладать способностью находить последовательность действий, позволяющую достичь желаемого эффекта; в этом состоит задача $\wedge$ планирования.

Мы будем использовать пример из модифицированной версии мира вампуса, в которой можно не задумываться об ориентации агента и в которой агент может переходить с помощью действия Go из одного места в соседнее. Предположим, что агент находится в квадрате [1,1], а золото — в квадрате [1,2]. Задача состоит в том, чтобы перенести золото в квадрат [1,1]. Флюэнтными предикатами являются At (o,x, s) и Holdingio, s). В таком случае первоначальная база знаний может включать следующее описание:

At (Agent, [1,1] , So) $\wedge$ At (Glt [1, 2] , So) Но этого описания совершенно недостаточно, поскольку в нем не сказано о том, какие предикаты не являются истинными в состоянии S0. (Дальнейшее обсуждение этой темы продолжается на с. 484.) Полное описание состоит в следующем:

$\text{At}(o,x,S0) \iff [(o = \text{Agent} \wedge x = [1,1]) \vee (o = G \pm \wedge x = [1,2])] \wedge \neg \text{Holdingio}, So)$ Необходимо также указать, что G $\pm$ — золото и что квадраты [1,1] и [1,2] являются соседними:

$\text{GoldiGx} \wedge \text{Adjacent}([1,1],[1,2]) \wedge \text{Adjacent}([1,2],[1,1])$ Неплохо было бы иметь возможность доказать, что агент достигнет цели, перейдя в квадрат [1,2], схватив золото и возвратившись в квадрат [1,1], следующим образом:

$\text{At}(\text{Glt } [1,1] , \text{Result} ([\text{Go}([1,1] , [1,2]) , \text{Grab}(\text{Gx}) , \text{Go}([1, 2] , [1,1])] , So))$ Но гораздо интереснее составить план приобретения агентом золота и решать эту задачу, получая ответ на показанный ниже запрос, который словесно формулируется таким образом: "Какая последовательность действий seq приведет к тому, что золото окажется в квадрате [1,1]?"

$\exists \text{seq } \text{At}(\text{Gi}, [1,1], \text{Result}(\text{seq}, \text{So}))$ Рассмотрим, что должно войти в базу знаний, с помощью которой можно было бы получать ответы на такие запросы.

Описание действий в ситуационном исчислении В простейшей версии ситуационного исчисления каждое действие описывается с помощью двух аксиом: $\wedge$ аксиомы возможности, которая указывает, существует или не существует возможность выполнить действие, и $\wedge$ аксиомы результата, которая указывает, что произойдет после выполнения возможного действия. Мы будем использовать высказывание $\text{Poss}(a, s)$ для обозначения того, что возможно выполнить действие a в ситуации s . Эти аксиомы имеют показанную ниже форму.

Аксиома возможности: $\text{Предусловия} \Rightarrow \text{Poss}(a, s)$ Аксиома результата:

$\text{Poss}(a, s) \Rightarrow \text{Изменения}$, представляющие собой результат выполнения действия Представим эти аксиомы для модифицированного мира вампуса. Чтобы можно было сократить соответствующие высказывания, мы будем исключать кванторы всеобщности, область действия которых распространяется на все высказывание.

Предполагается, что переменная s пробегает по всем ситуациям, a — по действиям, o — по объектам (включая агентов), d — по состояниям владения и не владения золотом, x и y — по местонахождениям.

Приведенные ниже аксиомы возможности для этого состояния мира указывают, что агент может переходить из одного местонахождения в другое, соседнее местонахождение, хватать кучу золота в текущем местонахождении и отпускать золото, которое он держит.

$\text{At}(\text{Agent}, x, s) \wedge \text{Adjacent}\{x, y\} \Rightarrow \text{Poss}(\text{Go}\{x, y\}, s) \wedge \text{Gold}(g) \wedge \text{At}\{\text{Agent}, x, s\} \wedge \text{At}(g, x, s) \Rightarrow \text{Poss}\{\text{Grab}(g), s\} \wedge \text{Holding}(g, s) \Rightarrow \text{Poss}\{\text{Release}(g), s\}$ А в приведенных ниже аксиомах результата утверждается, что если действие возможно, то некоторые (флюентные) свойства будут иметь место в ситуации, возникающей в итоге выполнения данного действия. Переход из квадрата x в квадрат y приводит к пребыванию агента в квадрате y , схватывание золота приводит к владению золотом, а отпускание золота приводит к тому, что агент им больше не владеет.

$\text{Poss}(\text{Go}\{x, y\}, s) \Rightarrow \text{At}\{\text{Agent}, y, \text{Result}\{\text{Go}\{x, y\}, s\}\} \wedge \text{Poss}(\text{Grab}(g), s) \Rightarrow \text{Holding}(g, \text{Result}\{\text{Grab}(g), s\}) \wedge \text{Poss}\{\text{Release}(g), s\} \Rightarrow \neg \text{Holding}(g, \text{Result}\{\text{Release}(g), s\})$ Сможем ли мы теперь доказать, что наш маленький план позволяет достичь цели, сформулировав все эти аксиомы? К сожалению, нет! Вначале все идет прекрасно; действие $\text{Go}([1,1], [1,2])$ действительно возможно в ситуации S_0 и аксиома результата для Go позволяет заключить, что агент достиг квадрата $[1,2]$:

$\text{At}\{\text{Agent}, [1,2], \text{Result}\{\text{Go}\{[1,1], [1,2]\}, S_0\}$ Теперь рассмотрим действие $\text{Grab}\{G_x\}$. Необходимо показать, что оно возможно в новой ситуации, т.е. возможно следующее:

$At(Glt [1,2], Result (Go \{ [1, 1] , [1,2] \} , So))$ К сожалению, в базе знаний нет ничего, что позволило бы обосновать такое заключение. Интуитивно можно понять, что действие агента Go не должно оказывать влияния на местонахождение золота, поэтому золото должно было все время находиться в квадрате [1,2], даже когда агент был в ситуации S0. (Ж* Проблема состоит в том, что аксиомы результата указывают, что изменилось, но не указывают, что осталось неизменным.

Проблема представления всего того, что остается неизменным, называется ^проблемой окружения (frame problem). Мы должны найти эффективное решение проблемы окружения, поскольку в реальном мире почти все и почти всегда остается неизменным. В каждом действии затрагивается лишь крошечная часть всех флюентных высказываний.

Один из подходов состоит в том, что должны быть записаны явные ^ аксиомы окружения, которые указывают, что остается неизменным. Например, после перемещения агента все другие объекты остаются на месте, если только он не берет эти объекты:

$At(o, x, s) \wedge (o \neq Agent) \wedge \neg Holding(o, s) \Rightarrow At(o, x, Result (Go(y, z), s))$ Если имеется F флюентных предикатов и A действий, то потребуется $O(AF)$ аксиом окружения. С другой стороны, если бы каждое действие имело самое большее E результатов, где E, как правило, намного меньше, чем F, то необходимо было бы иметь возможность представлять, что происходит, с помощью гораздо меньшей базы знаний, с размером $O(AE)$. В этом и состоит ^ проблема представительного окружения (representational frame problem). Тесно связанная с ней ^ проблема выводимого окружения (inferential frame problem) заключается в том, что нужно проектировать результаты t-шаговой последовательности действий за время $O(Et)$, а не за время $O(Ft)$ или $O(AEt)$, чтобы действовать успешно. Мы будем решать каждую из этих проблем по очереди. Но даже после этого останется еще одна проблема, связанная с тем, что должны быть обеспечены все необходимые условия для успешного выполнения любого действия. Например, действие Go окончится неудачей, если в ходе его выполнения агент погибнет. В этом состоит ^ проблема спецификации (qualification problem), полное решение которой еще не найдено.

Решение проблемы представительного окружения Для решения проблемы представительного окружения достаточно лишь немного изменить точку зрения на то, как следует записывать аксиомы. Вместо регистрации результатов каждого действия мы будем рассматривать, как каждый флюентный предикат развивается во времени². Применяемые при этом аксиомы называются ^ аксиомами состояния-преемника. Они имеют следующую форму:

Аксиома состояния-преемника: Действие возможно $\Rightarrow$ (Флюентное высказывание является истинным в результирующем состоянии $\Leftarrow$ Ф Оно стало истинным в

результате действия v оно было истинным и прежде, а действие оставило его неизменным) Пояснив, что мы не рассматриваем невозможные действия, хотим обратить внимание читателя на то, что в приведенном выше определении используется логическая связка $\Leftrightarrow$, а не $\Rightarrow$. Таким образом, в этой аксиоме указано, что данное флюентное высказывание будет истинным тогда и только тогда, когда выполняется его правая часть. Иными словами, истинностное значение каждого флюентного высказывания в следующем состоянии определено как функция действия и истин² По сути именно этот подход был принят при создании агента на основе логической схемы в главе 7. И действительно, такие аксиомы, как уравнения 7.4 и 7.5, могут рассматриваться как аксиомы состояния-преемника.

ностного значения в текущем состоянии. Это означает, что следующее состояние полностью задано текущим состоянием и поэтому нет необходимости использовать дополнительные аксиомы окружения.

В аксиоме состояния-преемника для местонахождения агента утверждается, что агент находится в квадрате y после выполнения некоторого действия, либо если это действие было возможным и заключалось в перемещении в квадрат y , либо если агент уже находился в квадрате y и его действие не заключалось в том, чтобы перемещаться куда-то в другое место:

$$\text{Poss}(a, s) \Rightarrow (\text{At}(\text{Agent}, y, \text{Result}(a, s)) \Leftrightarrow a = \text{Go}(x, y) \vee (\text{At}(\text{Agent}, y, s) \wedge a \neq \text{Go}(y, z)))$$
 В аксиоме для флюентного предиката *Holding* утверждается, что агент владеет золотом d после выполнения некоторого действия, если этим действием было схватывание золота d и такое схватывание было возможно или если агент уже владел золотом d и выполненное агентом действие не заключалось в отпускании золота:

$$\text{Poss}(a, s) \Rightarrow (\text{Holding}(g, \text{Result}(a, s)) \Leftrightarrow a = \text{Grab}(g) \vee (\text{Holding}(g, s) \wedge a \neq \text{Release}(g)))$$
 «3е Аксиомы состояния-преемника позволяют решить проблему представительного окружения, поскольку общий размер этих аксиом измеряется величиной в $O(AE)$ литералов: каждый из E результатов каждого из A действий упоминается один и только один раз. Литералы распределяются по F разным аксиомам, поэтому аксиомы имеют средний размер AE/F .

Внимательный читатель должен был заметить, что в этих аксиомах участвует флюентное высказывание *At*, касающееся агента, но не золота, поэтому мы все еще не можем доказать, что приведенный выше трехшаговый план позволяет достичь цели — переноса золота в квадрат [1,1]. Мы должны указать, что $\wedge$ неявным результатом перемещения агента из квадрата x в квадрат y является то, что переместится также все золото, которое он несет (а также все муравьи, которые заползли в это золото, все пылинки и бактерии на этих муравьях и т.д.). Учет таких неявных результатов связан с решением так называемой $\wedge$ проблемы

распространения последствий (ramification problem). Более подробно мы рассмотрим эту проблему позже, но для данной конкретной проблемной области ее можно решить, дописав более общую аксиому состояния-преемника для высказывания At. В новой аксиоме, которая обобщает предыдущую версию, утверждается, что объект o находится в квадрате y, либо если агент перешел в квадрат y и o — это или агент, или то, что держит агент, либо если объект o уже находился в квадрате y и агент никуда не переходил, притом что o является или агентом, или тем, что держит агент.

$\text{Poss}(a, s) \Rightarrow \text{At}(o, y, \text{Result}(a, s)) \Leftrightarrow (a - \text{Go}(x, y) \wedge (o = \text{Agent} \vee \text{Holding}(o, s))) \vee (\text{At}(o, y, s) \wedge \neg \exists z (y \Phi z \wedge a - \text{Go}(y, z) \wedge (o = \text{Agent} \vee \text{Holding}(o, s))))$ Возникает еще одна формальная сложность: процесс логического вывода, в котором используются эти аксиомы, должен позволять доказывать неравенства. Про-

стейшего рода неравенство задается между константами, например $\text{Agent} \wedge G \pm$.

Общая семантика логики первого порядка допускает, чтобы разные константы ссылались на один и тот же объект, поэтому база знаний должна включать какую-то аксиому, позволяющую предотвратить такую ситуацию. В $\wedge$ аксиоме уникальных имен (unique names axiom) провозглашается неравенство для каждой пары констант в базе знаний. Если же это условие подразумевается в программе автоматического доказательства теорем, а не записывается в базу знаний, то называется предположением об уникальности имен (unique names assumption). Необходимо также сформулировать условие неравенства между термами действий, например, утверждение о том, что $\text{Go}([1,1]/[1,2])$ — это действие, отличное от $\text{Go}([1,2], [1,1])$ или $\text{Grab}(G1)$.

Прежде всего необходимо указать, что каждый тип действия является различным, т.е., например, что действие Go — это не действие Grab. Для каждой пары имен действий AiB необходимо иметь:

$A(x_1, \dots, x_m) \Phi B(y_1, \dots, y_n)$ Затем требуется указать, что два терма действия с одним и тем же именем действия ссылаются на одно и то же действие тогда и только тогда, когда все участвующие в них объекты являются одинаковыми:

$A(X_1, \dots, X_m) = A(Y_1, \dots, Y_n) \Leftrightarrow X_i = Y_i \wedge \dots \wedge X_m = Y_n$ Все эти аксиомы, вместе взятые, называются $\wedge$ аксиомами уникального действия (unique action axiom). Такая комбинация описания начального состояния, аксиом состояния-преемника, аксиом уникальности имен и аксиом уникального действия является достаточной для доказательства того, что предлагаемый план позволяет достичь цели.

Решение проблемы выводимого окружения Аксиомы состояния-преемника позволяют решить проблему представительного окружения, но не проблему выводимого окружения. Рассмотрим t-шаговый план p, такой, что $\text{St} = \text{Result}(p, S0)$.

Чтобы определить, какие флюентные высказывания являются истинными в состоянии St , необходимо рассмотреть каждую из F аксиом окружения в каждом из t временных шагов. Поскольку аксиомы имеют средний размер AE/F , это соответствует объему работы по логическому выводу, измеряемому величиной $O(AEt)$.

Основной объем этой работы связан с копированием флюентных высказываний в неизменном виде из описания одной ситуации в описание следующей.

Для решения проблемы выводимого окружения можно воспользоваться следующими двумя подходами. Во-первых, можно отбросить ситуационное исчисление и изобрести новую формальную систему для записи аксиом. Указанная задача была решена в таких формальных системах, как исчисление флюентных высказываний (fluent calculus). Во-вторых, можно изменить механизм логического вывода таким образом, чтобы в нем аксиомы окружения обрабатывались более эффективно. На то, что должна существовать такая возможность, указывает сам факт, что трудоемкость этого простого подхода оценивается величиной $O(AEt)$; но почему она должна зависеть от количества действий A , если точно известно, какое именно действие выполняется в каждом временном шаге? Чтобы определить, как можно улучшить состояние дел, вначале рассмотрим формат аксиом окружения:

$$\text{Poss}(a,s) \Rightarrow \text{Fi}(\text{Result}(a,s)) \Leftrightarrow (a = A_1 \vee a = A_2 \dots) \vee \text{Fi}(s) \wedge (a \wedge A_3) \wedge (a \Phi A_4) \dots$$

Таким образом, в каждой аксиоме рассматриваются несколько действий, которые могут сделать это флюентное высказывание истинным, и несколько действий, которые могут сделать его ложным. Мы можем формализовать такие истинностные преобразования, введя предикат $\text{PosEffect}(a, FL)$, означающий, что в результате действия a флюентное высказывание Fi становится истинным, и предикат $\text{NegEffect}(a, Fi)$, означающий, что после выполнения действия a флюентное высказывание Fi становится ложным. Это означает, что приведенную выше систему аксиом можно переписать следующим образом:

$$\text{Poss}(a,s) \Rightarrow \text{Fi}(\text{Result}(a,s)) \Leftrightarrow \text{PosEffect}(a,Fi) \vee [\text{Fi}(s) \wedge \neg \text{NegEffect}(a, Fi)] \wedge \text{PosEffect}(A_1, Fi) \wedge \text{PosEffect}(A_2, Fi) \wedge \text{NegEffect}(A_3, Fi) \wedge \text{NegEffect}(A_4, Fi)$$

Ответ на вопрос о том, может ли такая операция формирования системы аксиом быть выполнена автоматически, зависит от точного формата аксиом окружения. Для того чтобы обеспечить использование подобных аксиом в эффективной процедуре логического вывода, необходимо провести три описанных ниже преобразования.

1. Проиндексировать предикаты PosEffect и NegEffect по их первому параметру таким образом, чтобы после получения информации о действии, которое произошло в момент времени t , можно было бы найти его результаты за время $O(A)$.

2. Проиндексировать аксиомы таким образом, чтобы после определения того, что F_i представляет собой результат какого-то действия, мы могли найти аксиому для F_i за время $O(A)$. В таком случае не придется даже рассматривать аксиомы, касающиеся тех флюентных высказываний, которые не являются результатом данного действия.

3. Представить каждую ситуацию как. предыдущую ситуацию плюс некоторая дельта. Таким образом, если ничего не изменяется от одного шага к другому, то не нужно вообще выполнять никакой работы. При использовании старого подхода требовалось выполнять объем работы $O(F)$, вырабатывая утверждение для каждого флюентного высказывания F_i ($\text{Result}(a, s)$) из предыдущих $F_i(s)$ утверждений.

Поэтому в каждом временном шаге мы рассматриваем текущее действие, собираем данные о его результатах и обновляем множество истинных флюентных высказываний. Среднее количество таких обновлений в каждом временном шаге соответствует E , поэтому общая сложность измеряется величиной $O(Ft)$. В этом и состоит решение проблемы выводимого окружения.

Исчисление времени и событий Ситуационное исчисление вполне себя оправдывает, если существует единственный агент, выполняющий мгновенные, дискретные действия, а если действия имеют продолжительность и могут накладываться друг на друга, то ситуационное исчисление становится довольно громоздким. Поэтому мы будем рассматривать данные темы с помощью альтернативной формальной системы, известной под названием 3c исчисление событий (event calculus), которая основана на точках во времени, а не на ситуациях. (Термины "событие" и "действие" могут использоваться как взаимозаменяемые. Неформально выражаясь, "событие" обозначает более широкий класс действий, включая те, в которых нет явного агента. С описанием такого класса проще справиться с помощью исчисления событий, а не ситуационного исчисления.) В исчислении событий флюентные высказывания становятся истинными или ложными в определенных точках во времени, а не в ситуациях, притом что само исчисление предназначено для того, чтобы с его помощью можно было формировать рассуждения, касающиеся интервалов времени. В аксиоме исчисления событий указано, что флюентное высказывание является истинным в какой-то момент времени, если действие, описываемое этим высказыванием, было инициировано некоторым событием в некоторый момент времени в прошлом и еще не закончено под влиянием какого-либо промежуточного события. Отношения *Initiates* и *Terminates* играют примерно такую же роль, как и отношение *Result* в ситуационном исчислении; отношение *Initiates*(e, f, t) означает, что возникновение события e во время t вызвало то, что флюентное высказывание f стало истинным, а отношение *Terminates*(w, f, t) означает, что высказывание f перестало быть истинным. Мы будем использовать предикат *Happens*(e, t) для обозначения того, что событие e произошло во время t , и предикат *Clipped*(f, t, t_2) для обозначения того, что

высказывание f перестало быть истинным под влиянием некоторого события, происшедшего в какое-то время между t и t_2 . Формально эта аксиома записывается следующим образом:

Аксиома исчисления событий: $T\{f, t_2\} \Leftrightarrow \exists e, t \text{ Happens}(e, t) \wedge \text{Initiates}(e, f, t) \wedge (t < t_2)$
 $\wedge \neg \text{Clipped}(f, t, t_2) \Leftrightarrow \exists e, t_i \text{ Happens}(e, t_i) \wedge \text{Terminates}(e, f, t_i) \wedge (t_i < t_2)$
 Это позволяет нам получить такие же функциональные возможности, как и с использованием ситуационного исчисления, в сочетании с возможностями формировать утверждения о точках и интервалах времени, что позволяет, например, сформулировать высказывание $\text{Happens}[\text{TurnOff}[\text{LightSwitch}^{\wedge}, 1:00]$, чтобы сообщить о выключении света точно, в 1:00.

Было разработано много дополнений к исчислению событий, чтобы можно было решать задачи с опосредованными событиями; событиями, имеющими продолжительность; событиями, происходящими одновременно; непрерывно изменяющимися событиями; недетерминированными результатами; причинно-следственными ограничениями и другими осложнениями. Мы вернемся к рассмотрению этих тем в следующем подразделе. Следует откровенно сказать, что в данный момент для большинства таких задач еще отсутствуют полностью удовлетворительные решения, но на пути к достижению этих решений не обнаружено каких-либо непреодолимых препятствий.

Обобщенные события До сих пор в этой главе рассматривались две основные концепции: действия и объекты, а теперь настало время перейти к описанию того, как эти концепции укладываются во всеобъемлющую онтологию, в которой и действия, и объекты могут трактоваться как аспекты физического универсума. Мы рассматриваем конкретный универсум как имеющий и пространственное, и временное измерения. Мир ватсона характеризуется пространственным компонентом, представленным в виде двухмерной решетки, а время в нем дискретно; мир, в котором существуют люди, имеет три пространственных измерения и одно временное³, причем все эти измерения непрерывны. $\wedge$ Обобщенное событие (generalized event) состоит из аспектов некоторого "пространственно-временного фрагмента" — части этого многомерного пространственно-временного универсума.

Эта абстракция позволяет обобщить большинство концепций, рассматривавшихся нами до сих пор, включая действия, местонахождения, интервалы времени, флюентные высказывания и физические объекты. Иллюстрация этой основной идеи приведена на рис. 10.3. Начиная с этого момента, мы будем использовать неуточненный термин "событие" для обозначения обобщенных событий.

"пространство" WorldWarII MTwentiethCentury Австралия т\ время Рис. 10.3. Обобщенные события. Универсум имеет три пространственных и одно временное измерения; на этом рисунке показано только одно пространственное измерение.

Все события являются частью, PartOf, этого универсума. Любое событие, такое как Вторая мировая война, WorldWarII, происходит в части пространства—времени с немного произвольными и изменяющимися во времени границами. Любой интервал Interval, такой как двадцатый век, TwentiethCentury, имеет фиксированную, ограниченную временную протяженность и максимальную пространственную протяженность, а любое место, Place, такое как Австралия, имеет примерно фиксированную пространственную протяженность и максимальную временную протяженность. 3 Некоторые физики, работающие в области теории суперструн (string theory), доказывают, что измерений — 10 или больше, а другие утверждают, что мир дискретен, но для формирования обыденных рассуждений вполне подходит представление в виде четырехмерного непрерывного пространства/времени.

Например, Вторая мировая война — это событие, которое имело место в различных точках пространства—времени, что обозначено на этом рисунке в виде серого пятна неправильной формы. Это событие можно разбить на ряд $\wedge$ подсобытий4:

SubEvent (BattleOfBritain, WorldWarII) Равным образом, Вторая мировая война — это подсобытие двадцатого столетия:

SubEvent (WorldWarII, TwentiethCentury) Двадцатое столетие— это интервал времени. Интервалы представляют собой фрагменты пространства—времени, которые включают все пространство между двумя точками во времени. Функция Period(e) обозначает наименьший интервал, включающий событие e, а функция Duration(i) — это продолжительность времени, занятая некоторым интервалом i, поэтому можно сформулировать высказывание $\text{Duration}(\text{Period}(\text{WorldWarII})) > \text{Years}(5)$.

Австралия — это место, т.е. фрагмент пространства—времени с некоторыми фиксированными пространственными границами. Границы могут изменяться во времени вследствие геологических или политических изменений. Для обозначения отношения с участием подсобытия, которое имеет место, если пространственная проекция одного события является частью PartOf другого события, используется предикат In:

In {Sydney, Australia} Функция Location(e) обозначает наименьшее место, которое включает событие e.

Как и объекты любого другого рода, события могут быть сгруппированы по категориям. Например, WorldWarII принадлежит к категории войн Wars. Чтобы сформулировать утверждение, что гражданская война происходила в Англии в 1640-х годах, можно привести следующее высказывание:

Зw w E CivilWars л SubEvent {w, 1640s) л In {Location{w) , England) Понятие категории событий позволяет найти ответ на вопрос, которого мы избегали при описании результатов действий в разделе 10.3: на что именно ссылаются такие логические термы, как Go ([1,1], [1,2])? Являются ли они событиями?

Ответ на этот вопрос отрицателен, что на первый взгляд может показаться неожиданным. Но в этом можно убедиться, рассмотрев план с двумя "идентичными" действиями, такой как следующий: [Go{[l,l],[l,2]),Go{[1,2], [1,1]),Go{[1,1], [1,2]) В этом плане выражение Go{ [1,1] , [1, 2]) не может быть именем события, поскольку в нем представлены два разных события, происходящих в разное время.

Вместо этого Go { [1,1] , [1, 2]) представляет собой имя категории событий — всех тех событий, в которых агент переходит из квадрата [1,1] в квадрат [1,2].

Приведенный выше трехшаговый план сообщает о том, что должны осуществиться экземпляры этих трех категорий событий.

Обратите внимание на то, что здесь мы впервые встретились с категориями, именованными с помощью сложных термов, а не просто константных символов. Это обстоятельство не должно стать источником новых затруднений, поскольку фактически мы 4 Обратите внимание на то, что отношение SubEvent (Подсобытие) — это частный случай отношения Par to f и также является транзитивным и рефлексивным.

можем использовать такую структуру параметров предикатов, которая является наиболее удобной. Устранение параметров позволяет создать более общую категорию:

Go{x,y) С GoTo(y) Go{x,y) с GoFrom(x) Аналогичным образом можно добавлять параметры для создания более конкретных категорий. Например, чтобы описать действия, осуществляемые другими агентами, можно добавить параметр с обозначением агента. Поэтому, чтобы сформулировать утверждение, что знаменитый математик Шанкар вчера прилетел из НьюЙорка в Нью-Дели, можно записать следующее:

Зе e e Fly(Shankar, NewYork, NewDelhi) л SubEvent (e, Yesterday) Формулы в таком виде применяются настолько часто, что мы создадим для таких формул сокращение E(c, i), которое будет означать, что элемент категории событий с является подсобытием события или интервала i:

E{c,i) <=> Зе e e ел SubEvent (e, i) Таким образом, переформулируем приведенное выше утверждение следующим образом:

E(Fly(Shankar, NewYork, NewDelhi) , Yesterday) Процессы События, рассматривавшиеся нами до сих пор, были тем, что принято называть "дискретными событиями— они имеют определенную структуру. Путешествие

Шанкара имело начало, середину и конец. Если бы он прервал свое путешествие на полпути, то связанное с этим событие было бы другим — не путешествием из Нью-Йорка в Нью-Дели, а, допустим, поездкой из Нью-Йорка в какой-то европейский город. С другой стороны, категория событий, обозначенная высказыванием $\text{Flying}(\text{Shankar})$, имеет другое качество. Если бы мы стали исследовать небольшой интервал полета Шанкара, скажем, третий двадцатиминутный отрезок времени (в течение которого он нетерпеливо ждал, когда ему принесут еще один пакетик арахиса), это событие все еще оставалось бы элементом события $\text{Flying}(\text{Shankar})$.

В действительности такое утверждение остается истинным для любого субинтервала. Категории событий, обладающих этим свойством, называются категориями $\wedge$ процессов (process) или категориями $\wedge$ вневременных событий (liquid event).

Любой субинтервал процесса также является элементом той же категории процессов.

Чтобы высказать мысль, что Шанкар, например, был вчера в полете в какой-то момент времени, можно использовать такую же систему обозначений, которая используется для дискретных событий:

$E(\text{Flying}(\text{Shankar}), \text{Yesterday})$ Кроме того, часто возникает необходимость сформулировать утверждение, что некоторый процесс происходил на протяжении всего определенного временного интервала, а не просто некоторого его субинтервала. Для этого используется предикат T следующим образом:

$T(\text{Working}(\text{Stuart}), \text{TodayLunchHour})$

Высказывание $t(c, i)$ обозначает, что некоторое событие типа c происходило точно в течение интервала i , т.е. что это событие началось и закончилось в то же время, что и сам этот интервал.

Различие между вневременными (liquid) и внутривременными (nonliquid) событиями полностью аналогично различию между веществами (или материей) и отдельными объектами. В действительности некоторые ученые называют типы вневременных событий "сЭч временными веществами (temporal substance), по аналогии с тем, что масло рассматривается как $\wedge$ пространственное вещество (spatial substance).

Концепция вневременных событий позволяет описывать не только процессы непрерывного изменения, но и процессы непрерывного "неизменения". Такие процессы часто называют $\wedge$ состояниями (state). Например, "Шанкар находится в Нью-Йорке" — это категория состояний, для которой нами принято обозначение $\text{In}(\text{Shankar}, \text{NewYork})$. Чтобы сформулировать утверждение, что Шанкар находился весь нынешний день в Нью-Йорке, можно записать следующее:

T(In (Shankar, NewYork) , Today) Для формирования более сложных высказываний о состояниях и событиях могут применяться сочетания примитивных высказываний. Этот подход называется $\wedge$ исчислением флюэнтных высказываний (fluent calculus). В исчислении флюэнтных высказываний используются комбинации флюэнтных высказываний, а не просто отдельные флюэнтные высказывания. Выше уже был показан способ представления события, состоящего из двух событий, происходящих одновременно, а именно функция $\text{Both}(e_1, e_2)$. В исчислении флюэнтных высказываний такое выражение обычно записывается сокращенно с помощью инфиксной системы обозначений: $e_1 \circ e_2$. Например, чтобы сформулировать утверждение, что некто шел и одновременно жевал резинку, можно записать следующее:

$\exists p, i (p \leq \text{People}) \wedge T(\text{Walk}(p) \circ \text{ChewGum}(p), i)$ Функция $\circ$ является коммутативной и ассоциативной, как и логическая конъюнкция.

Могут быть также определены аналоги дизъюнкции и отрицания, но необходимо соблюдать осторожность, поскольку существуют два одинаково приемлемых способа интерпретации дизъюнкции событий. Например, утверждение, что "в течение последних двух минут агент либо шел, либо жевал резинку", может означать, что агент либо выполнял одно или другое из этих действий в течение всего интервала времени, либо чередовал эти два действия. Для обозначения таких двух вариантов будут использоваться предикаты OneOfn $E_i \text{ then}$. Сложные события схематически показаны на рис. 10.4. а) б) в) Рис. 10.4. Схематическая иллюстрация сложных событий: одновременное протекание двух событий $T(\text{Both}(p, q), i)$, обозначаемое также как $T(p \circ q, i)$ (а); осуществление одного из двух событий $T(\text{OneOf}(p, q), i)$ (б); чередование двух событий $T(E_i \text{ then}(p, q), i)$ (в)

Интервалы Время — это важный фактор для любого агента, выполняющего действия, и поэтому в области представления временных интервалов был проведен большой объем исследовательских работ. В данном разделе рассматриваются интервалы двух типов: моменты времени и продолжительные интервалы. Различие между ними состоит в том, что только моменты времени имеют нулевую продолжительность:

$\text{Partition}(\{\text{Moments}, \text{ExtendedIntervals}\}, \text{Intervals}) \wedge \text{Moments} \Leftrightarrow \text{Duration}(i) = \text{Seconds}@i$ Затем необходимо ввести временную шкалу и связать точки на этой шкале с моментами времени, что позволяет сформулировать понятие абсолютных значений времени. Временная шкала выбирается произвольно; в данной книге время измеряется в секундах и используется соглашение, что момент времени в полночь (среднее время по Гринвичу) 1 января 1900 года имел значение времени 0.

Функции Start и End позволяют определить самый первый и самый последний моменты времени в интервале, а функция Time сообщает момент времени на

временной шкале, соответствующий текущему моменту. Функция Duration измеряет разность между временем окончания и временем начала. Примеры применения этих функций приведены ниже.

Interval(i) => Duration(i) = (Time{End(i)}-Time{Start(i)}) Time{Start(AD1900)} = Seconds@) Time{Start(AD2001)} = SecondsC187324800 Time{End(AD2001)} = SecondsC218860800 Duration(AD2001) = SecondsC1536000) Для того чтобы было проще читать эти числа, обозначающие количество секунд от начала отсчета, введем также функцию Date, которая принимает шесть параметров (часы, минуты, секунды, день, месяц и год) и возвращает точку во времени:

Time{Start(AD2001)} = Date{0,0,0,1,Jan,2001} Date@,20,21,24,l,1995) = SeconclsC000000000) Предикат Meet позволяет определить, равно ли время окончания первого интервала времени начала второго интервала; эти значения времени задаются в секундах.

Существует возможность определить такие предикаты, как Before, After, During и Overlap, исключительно в терминах предиката Meet, но более интуитивно понятными являются их определения в терминах точек на временной шкале (графическое представление этих предикатов приведено на рис. 10.5):

Meet(i,j) <=> Time (End(i)) = Time (Start (j)) Befored, j) <=> Time {End (i) } < Time{Start(j)} After (j, i) <=> Befored, j) Duringd, j) <=> Time{Start(j)} < Time{Start (i) } л Time{End(i)} < Time{End(j)} Overlap {i , j) <=> 3k During {k, i) л During (к, j) Например, чтобы сформулировать утверждение, что царствование Елизаветы II следовало за царствованием Георга VI, а царствование Элвиса Пресли совпадало с периодом 1950-х годов, можно записать следующее:

After(ReignOf{ElizabethII},ReignOf{GeorgeVI}) Overlap{Fif ties, ReignOf{Elvis})

Start(Fifties) = Start(AD1950) End(Fifties) = End(AD1959) Meet(iI) | 1 | 1 | 1 Before(iI) | 1 Afier(j,i) | During(iI) | : 1 Overlap(ij) | ; 1 OverlapHJ) | 1 1 Рис. 10.5. Предикаты, задаваемые на временных интервалах Флюентные высказывания и объекты Как было указано выше, физические объекты могут рассматриваться как обобщенные события в том смысле, что любой физический объект представляет собой фрагмент пространства—времени. Например, США можно рассматривать как событие, которое началось, скажем, в 1776 году в виде союза 13 штатов и все еще продолжается в наши дни как союз 50 штатов. Мы можем описать изменяющиеся свойства объекта USA с использованием флюентных высказываний, касающихся состояния.

Например, можно сформулировать утверждение, что в какой-то момент в 1999 году население США составляло 271 миллион человек:

E(Population(USA, 271000000), AD1999) Еще одним свойством объекта USA, которое изменяется через каждые четыре или восемь лет, если не учитывать непредвиденные происшествия, является смена президента этого государства. Можно было бы предложить описывать это свойство, условившись считать, что President (USA) — это логический терм, который обозначает различные объекты в разное время. К сожалению, это невозможно, поскольку в каждой конкретной структуре модели любой терм обозначает один и только один объект. (Терм President (USA, t) мог бы обозначать разные объекты в зависимости от значения t, но в принятой нами онтологии временн/е индексы рассматриваются отдельно от флюентных высказываний.) Единственная возможность состоит в том, чтобы считать, что President (USA) обозначает единственный объект, который в разное время состоит из разных людей. Это — объект, которым был Джордж Вашингтон с 1789 по 1796 гг., Джон Адаме с 1796 по 1800 гг. и т.д. (рис. 10.6).

Для того чтобы высказать утверждение, что Джордж Вашингтон был президентом в течение всего 1790 года, можно записать следующее:

T(President(USA) = GeorgeWashington, AD1190)

Рис. 10.6. Схематическое изображение объекта President (USA,) за первые 15 лет его существования Однако необходимо соблюдать осторожность. В этом высказывании знак = должен представлять собой функциональный символ, а не стандартный логический оператор.

Его интерпретация состоит не в том, что GeorgeWashington и President {USA} были логически идентичными в 1790 году; логическое равенство есть нечто, неизменное во времени. В данном случае логическое равенство существует между подсобытиями каждого объекта, которые определены на период 1790 года.

Кроме того, не следует путать физический объект GeorgeWashington с коллекцией атомов, из которых он состоял. Джордж Вашингтон не был логически равен какой-либо конкретной коллекции атомов, поскольку множество атомов, из которых он состоял, существенно изменялось во времени. Жизнь его была коротка, а каждый атом, отдельно взятый, имеет свою собственную, очень длинную продолжительность жизни. На какой-то период времени они пересеклись, и тогда этот временной срез периода существования данного атома стал частью (PartOf) Джорджа Вашингтона, после чего каждый из этих двух объектов направился по своему пути.

10.4. Мыслительные события и мыслимые объекты Агенты, которые были созданы в этой книге до сих пор, были уверены в истинности или ложности некоторых высказываний и обладали способностью выводиться логическим путем новые убеждения, но ни один из этих агентов не обладал какими-либо знаниями о самих убеждениях или о логическом выводе. Но в одноагентных проблемных областях

знания о собственных знаниях и процессах формирования рассуждений весьма полезны для управления логическим выводом. Например, если некто знает, что он ничего не знает о географии Румынии, то для него нет необходимости впустую затрачивать невероятные вычислительные усилия, чтобы рассчитать кратчайший путь от Арада до Бухареста. Вместо этого он может порассуждать о собственных знаниях, чтобы составить планы их расширения, например, купив карту Румынии. Кроме того, в мультиагентных проблемных областях для агента важно иметь возможность рассуждать о мыслимых состояниях других агентов. Например,

наилучший способ попасть в Бухарест вполне может знать румынский полицейский, поэтому агенту достаточно обратиться к нему за помощью.

По сути нам требуется модель мыслимых объектов, которые находятся в чьей-то голове (или базе знаний), и мыслительных процессов, которые манипулируют с этими мыслимыми объектами. Такая модель должна быть достоверной, но не требуется, чтобы она была слишком детализированной. Нам не нужно иметь способность предсказывать, сколько именно миллисекунд потребуется, чтобы какой-то конкретный агент пришел к нужному логическому заключению, или же прогнозировать, какие именно нейроны активизируются, когда исследуемое животное столкнется с конкретными визуальными стимулами. Для нас будет достаточно иметь возможность прийти к выводу, что румынский полицейский сообщит нам, как попасть в Бухарест, если будет знать дорогу и убедится в том, что мы действительно заблудились, а не подшучиваем над ним.

Формальная теория убеждений Начнем с изучения связей между агентами и "мыслимыми объектами", таких как Believes (Убежден), Knows (Знает) и Wants (Желает). Отношения такого рода называются $\wedge$ пропозициональными позициями (propositional attitude), поскольку они описывают позицию, которую может занять агент по отношению к некоторому высказыванию. Предположим, что Лойс в чем-то убеждена, т.е. Believes {Lois, x}. Какого рода объектом является x! Очевидно, что x не может быть логическим высказыванием.

Если утверждение, что Супермен летает (Flies {Superman}) — логическое высказывание, то нельзя составить выражение Believes {Lois, Flies {Superman} }, поскольку параметрами предикатов могут быть только термы (а не высказывания). Но если Flies — функция, то Flies{Superman} становится подходящим кандидатом на использование в качестве мыслимого объекта, а Believes может быть отношением между агентом и пропозициональным флюентным термом. Преобразование высказывания в объект называется $\wedge$ овеществлением (reificationM).

По-видимому, такой подход позволяет достичь желаемого — появления у агента способности рассуждать об убеждениях агентов. Но, к сожалению, при использовании этого подхода возникает определенная проблема: если киноактер

Кент Кларк и его герой Супермен — это одно и то же (т.е. Clark-Superman), то полет Кларка и полет Супермена — одна и та же категория событий, т.е. $\text{Flies}\{\text{Clark}\} = \text{Flies}\{\text{Superman}\}$. Поэтому мы обязаны сделать такой вывод: если Лойс уверена в том, что Супермен может летать, она должна быть также уверена в том, что Кларк может летать, даже если она не уверена в том, что Кларк — Супермен, как показано ниже. $\{\text{Superman} = \text{Clark}\} N \{\text{Believes}\{\text{Lois}, \text{Flies}\{\text{Superman}\}\} \Leftrightarrow \text{Believes}\{\text{Lois}, \text{Flies}\{\text{Clark}\}\}$) В определенном смысле такое утверждение недалеко от истины: Лойс убеждена в том, что есть некое лицо, которому некогда присвоили имя Кларк, и что это лицо может летать. Но есть еще один смысл, в котором приведенное выше высказывание является ложным: если Лойс задать вопрос: "Может ли Кларк летать?", она, безус5 Англоязычный вариант термина "овеществление", reification, происходит от латинского слова res. Джон Маккарти предложил термин-кальку "thingification", но последний так и не прижился.

ловно, ответит отрицательно. Овеществленные объекты и события вполне приемлемы для использования в первом смысле толкования предиката Believes, но во втором толковании необходимо овеществлять описания этих объектов и событий таким образом, чтобы Clark и Superman могли иметь разные описания (даже несмотря на то, что оба эти имени относятся к одному и тому же объекту).

Формально свойство высказывания, позволяющее свободно подставлять любой терм вместо равного терма, называется ^ ссылочной прозрачностью (referential transparency). В логике первого порядка свойством ссылочной прозрачности обладает каждое отношение. Но нам было бы желательно иметь возможность определять Believes (и другие пропозициональные позиции) как отношения, второй параметр которых является ссылочно ^ непрозрачным (opaque), т.е. таким, чтобы без изменения смысла нельзя было бы подставить равный терм вместо второго параметра.

Существуют два способа достижения этой цели. Первый из них состоит в использовании другой формы логики, называемой ^ модальной логикой (modal logic), в которой такие пропозициональные позиции, как Believes и Knows, становятся ^ модальными операторами, обладающими свойством ссылочной непрозрачности.

Этот подход рассматривается в разделе с историческими замечками. Второй подход, который будет применяться в данной главе, состоит в эффективном обеспечении непрозрачности в рамках ссылочно прозрачного языка с использованием ^ синтаксической теории мыслимых объектов. Это означает, что мыслимые объекты представляются константами — ^ строками. Результатом становится грубая модель базы знаний агента, которая представлена как состоящая из строк, соответствующих высказываниям, в истинности которых убежден агент. Строка — это сложный терм, обозначающий список символов, поэтому событие Flies (Clark) может быть

представлено в виде списка символов [F, 1, i, e, s, (, C, 1, a, г,к,)], которые мы будем сокращенно записывать в виде строки, заключенной в кавычки, "Flies{Clark} ".

Эта синтаксическая теория включает $\wedge$ аксиому об уникальности строк, в которой утверждается, что строки являются идентичными тогда и только тогда, когда они состоят из идентичных символов в одинаковой последовательности. Таким образом, даже если Clark= Superman, мы все равно имеем "Clark" $\neq$ "Superman".

Теперь достаточно лишь предусмотреть синтаксис, семантику и теорию доказательства для языка представления строк, точно так же, как это было сделано в главе 7. Различие состоит в том, что необходимо определить все эти компоненты в логике первого порядка. Начнем с определения Den (сокращение от denotatum — денотат, или обозначаемое) как функции, которая отображает некоторую строку на обозначаемый ею объект, и Name как функции, которая отображает объект на строку, представляющую собой имя константы, обозначающей этот объект. Например, денотатом строки "Clark" и строки "Superman" является объект, на который ссылается константный символ ManOfSteel (Человек из стали), а именем этого объекта в базе знаний может быть "Superman", "Clark" или какая-то другая константа, например "Хц":

Den("Clark") = ManOfSteel л Den("Superman") = ManOfSteel Name (ManOfSteel) = "Хц"

Следующий этап состоит в том, чтобы определить правила логического вывода для логических агентов. Например, может потребоваться сформулировать утверждение, что любой логический агент способен использовать правило отделения: если он

уверен в истинности высказываний $p \Rightarrow d$, то он будет также уверен в истинности d . Первая попытка записать эту аксиому может состоять в следующем:

LogicalAgent(a) л Believes(a,p) л Believes {a, "p => d"} => Believes {a, d} Но эта попытка неудачна, поскольку строка "p => d" содержит буквы 'p' и 'd', но не имеет ничего общего со строками, являющимися значениями переменных p и d. Правильная формулировка состоит в следующем:

LogicalAgent(a) л Believes {a,p} л Believes (a, Concat (p,"=>", d)) => Believes(a,d) где Concat— функция на строках, которая соединяет (конкатенирует) свои параметры. Мы будем сокращенно записывать операцию Concat (p, "=>", d) с помощью подчеркивания, как "p => d". Это означает, что вхождение x в строке должно быть $\wedge$ взятым без кавычек, или раскавыченным, т.е. мы должны подставить строковое значение переменной x. В языке Lisp такая операция выполняется с помощью оператора "запятая/обратная кавычка", а в языке Perl для этого используется интерполяция \$-переменной.

После введения других правил логического вывода, кроме правила отделения, мы получим возможность отвечать на вопросы в такой форме: "Если дано, что

логический агент знает такие-то предпосылки, может ли он прийти к такому-то заключению?" Кроме этих обычных правил логического вывода, требуются определенные правила, характерные для рассуждений об убеждениях. Например, следующее правило утверждает, что если логический агент в чем-то уверен, то он уверен, что он в этом уверен:

$\text{LogicalAgent}(a) \wedge \text{Believes}\{a,p\} \Rightarrow \text{Believes}\{a, \text{"Believes}\{\text{Name}(a), p\}\text{"}$ Теперь, согласно нашим аксиомам, любой агент может безошибочно вывести любое следствие из своих убеждений. Такое свойство агента называется $\wedge$ логическим всеведением (logical omniscience). Было сделано много попыток определить спецификации ограниченных рациональных агентов, которые способны выполнять ограниченное количество логических выводов за ограниченное время.

Ни одна из этих попыток не оказалась полностью успешной, но описанные выше формулировки позволяют получить в высшей степени обоснованный ряд предсказаний о возможностях ограниченных агентов.

Знания и убеждения Соотношение между знаниями и убеждениями широко исследовалось в философии.

Общепризнано, что знания — это убеждения, истинность которых подтвердилась. Таким образом, если вы в чем-то уверены по неоспоримо надежным причинам и если то, в чем вы уверены, действительно истинно, то вы это знаете. Условие, согласно которому причина должна быть "неоспоримо надежной", необходимо для предотвращения такой ситуации, что вы будете утверждать: "Я знаю, что эта монета всегда будет падать орлом вверх", и окажетесь правы лишь применительно к половине бросков монеты.

Допустим, что предикат $\text{Knows}(a,p)$ означает следующее: агент a знает, что высказывание p является истинным. Возможно также определить другие формы знаний. Например, ниже приведено определение "знаний о погоде".

$\text{KnowsWeather}\{a, p\} \Leftrightarrow \text{Knows}\{a,p\} \vee \text{Knows}\{a, \text{"—ip"}\}$ Продолжая приведенный выше пример, отметим: Лойс знает, может ли Кларк летать, если она либо знает, что Кларк может летать, либо знает, что он этого не может.

Понятие "знаний о чем-то" является более сложным. Например, может возникнуть соблазн утверждать, что агент знает номер телефона Боба, если есть какой-то объект x , в отношении которого агент знает, что $\text{PhoneNumber}\{\text{Bob}\} = x$. Но этого недостаточно, поскольку агент может знать, что Алиса и Боб имеют один и тот же номер (например, $\text{PhoneNumber}\{\text{Bob}\} = \text{PhoneNumber}\{\text{Alice}\}$), но если номер Алисы неизвестен, то эти знания не могут ему помочь. Лучшее определение понятия "знаний о чем-то" состоит в том, что агент должен знать о некотором x , что это — строка цифр и что это — номер Боба:

KnowsWhat{a, "PhoneNumber {b} ") <=> $\exists x$ Knows{a,"x = PhoneNumber(b) ") л х е DigitStrings Безусловно, применительно к другим вопросам необходимо иметь другие критерии определения того, какой ответ является приемлемым. Например, подходящим ответом на вопрос: "Какой город является столицей штата Нью-Йорк", является имя собственное, "Олбани", а не нечто подобное высказыванию: "Город, в котором находится резиденция губернатора штата". Для обеспечения этого необходимо оформить предикат KnowsWha t в виде трехместного отношения: он принимает в качестве параметра обозначение агента, строку, представляющую некоторый терм, и категорию, к которой должен принадлежать ответ. Например, для определения упомянутых выше знаний могут использоваться следующие формулировки:

KnowsWhat {Agent, " Capital {NewYork) " , ProperNames)
KnowsWhat{Agent,"PhoneNumber{Bob)",DigitStrings) Знания, время и действия В большинстве реальных ситуаций любому агенту приходится иметь дело с убеждениями (своими собственными или других агентов), которые изменяются во времени. Агенту также приходится составлять планы, касающиеся изменения своих собственных убеждений, такие как план покупки карты для определения способа попасть в Бухарест. Как и в случае других предикатов, мы можем овеществить предикат Believes и вести речь об убеждениях, которые имели место в течение определенного периода. Например, чтобы сформулировать высказывание "Лойс сегодня уверена в том, что Супермен может летать", мы будем записывать следующее:

T{Believes{Lois,"Flies{Superman)"),Today) Если объектом убеждения является высказывание, которое может изменяться во времени, и это можно описать, используя оператор T внутри строки. Например, Лойс может сегодня быть убеждена в том, что Супермен мог летать вчера:

T{Believes{Lois, "T{Flies{Superman), Yesterday)" , Today) После получения способа описания того, как убеждения меняются со временем, мы приобретаем возможность использовать всю механику исчисления событий для составления планов, касающихся убеждений. Действия могут включать ^ предусловия знаний и ^ результаты знаний. Например, предусловием действия по набору номера телефона некоторого лица является знание этого номера, а результатом действия по поиску номера становится знание этого номера. Например, последнее действие можно описать с использованием средств исчисления событий следующим образом:

Initiates(Lookup(a,"PhoneNumber{b)") KnowsWhat(a,"PhoneNumber{b)",DigitStrings), t)
Планы сбора и использования информации часто представляют с помощью сокращенных обозначений, известных под названием ^ переменных этапа прогона

(runtime variable), соглашения по использованию которых тесно связаны с соглашениями о применении переменных, взятых без кавычек, которые были описаны выше. Например, план поиска номера Боба, а затем набора этого номера может быть записан следующим образом: `[Lookup{Agent,"PhoneNumber{Bob}",n),Dial{n}]` Здесь `n` — это переменная этапа прогона, с которой будет связано значение путем выполнения действия `Lookup`, поэтому данная переменная может затем использоваться в действии `Dial`. Планы такого рода часто встречаются в частично наблюдаемых проблемных областях. Примеры таких планов рассматриваются в следующем разделе и в главе 12.

10.5. Мир покупок в Internet В данном разделе показано, как можно представить некоторые знания, касающиеся осуществления покупок в Internet. Мы создадим агента по исследованию возможности осуществления покупок (торгового агента), который помогает покупателю найти в Internet предложения по интересующим его товарам. Торговый агент получает описание товара от покупателя и выполняет задачу по подготовке списка Web-страниц, на которых данный товар предлагается на продажу. В некоторых случаях описание покупателем товара будет точным, таким как "Цифровая камера Coolpix 995", и тогда задача заключается в поиске магазина (магазинов) с наилучшим предложением. В других случаях описание может быть определено лишь частично, как, например, "Цифровая камера не дороже 300 долларов", и агенту придется сравнивать различные товары.

Средой существования торгового агента является вся система World Wide Web — не игрушечная моделированная среда, а та сложная, постоянно развивающаяся среда, которая используется каждый день миллионами людей. Результатами восприятия агента являются Web-страницы, но в отличие от человека — пользователя Web, который рассматривает страницы, отображенные в виде массива пикселей на экране, торговый агент воспринимает страницу в виде символьной строки, состоящей из обычных слов, чередующихся с командами форматирования на языке разметки HTML. В листинге 10.1 показаны Web-страница и соответствующая символьная строка HTML. Проблема восприятия для торгового агента состоит в извлечении полезной информации из восприятий такого рода.

Безусловно, что решить задачу восприятия Web-страниц проще по сравнению, скажем, с восприятием дорожной ситуации во время вождения такси в Каире. Тем не менее задача восприятия в Internet усложняется многими факторами. Web-страница, показанная в листинге 10.1, является очень простой по сравнению с реальными торговыми узлами, которые включают cookie-файлы, код Java, Javascript, Flash, протоколы защиты от роботов, плохо сформированный код HTML, звуковые файлы, фильмы и текст, который появляется только в составе графического изображения, например в формате JPEG. Агент, способный действовать в пределах всей Internet, должен быть почти таким же сложным, как и робот, способный

двигаться в реальном мире. Поэтому мы сосредоточимся на описании простого агента, в котором не учитывается большая часть этих осложнений.

Листинг 10.1. Web-страница типичного оперативного магазина в форме, воспринимаемой человеком — пользователем браузера (вверху), и соответствующая строка HTML, принимаемая браузером или воспринимаемая торговым агентом (внизу). В языке HTML символы между знаками < и > представляют собой директивы разметки, которые указывают, как должна отображаться страница. Например, строка <i>Select</i> означает, что нужно переключиться на курсивный шрифт, отобразить слово Select, а затем отменить использование курсивного шрифта. Идентификатор страницы, такой как http://gen-store.cam/books, называется унифицированным локатором ресурсов, или URL (Uniform Resource Locator). Разметка anchor означает, что должна быть создана гипертекстовая ссылка на локатор url с текстом анкера anchor Generic Online Store Select from our fine line of products: • Computers • Cameras • Books • Videos • Music <h1>Generic Online Store</h1> <i>Select</i> from our fine line of products: Computers Cameras Books Videos Music В первую очередь задача агента состоит в том, чтобы найти соответствующие товарные предложения (ниже будет показано, как выбрать наилучшие из искомых предложений). Допустим, что query — описание товара, которое вводит пользователь (например, "laptops" — портативные компьютеры); в таком случае некоторая страница представляет собой предложение, соответствующее запросу query, если страница действительно ему соответствует и содержит подходящее предложение.

Мы должны также следить за URL, связанным с этой страницей:

RelevantOffer{page, url, query) t=\$ Relevant (page, url, query) а Offer(page) Запросу будет соответствовать страница с обзором новейших портативных компьютеров высокого класса, но если она не предоставляет способ купить такой компьютер, то ее нельзя рассматривать как предложение. Например, можно утверждать, что страница является предложением, если на ней содержится слово "buy" (покупка) или "price" (цена) в какой-то ссылке HTML или в форме, относящейся к этой странице. Иными словами, если страница содержит строку в виде "<a ... buy", то является предложением; на ней может также применяться слово "price" вместо "buy", а вместо дескрип-

тора "a" использоваться дескриптор "form". Аксиомы, относящиеся к этим утверждениям, могут быть записаны следующим образом:

$Offer(page) \leq \rightarrow (InTag("a", str, page) \vee 1nTag\{ "form" , str, page\}) \wedge (Xl("buy" , str) \vee In ("price" , str)) InTagitag, str, page) \leq \rightarrow In\{ "<" + tag + str + "</" + tag, page\} In\{sub, str\} \leq \rightarrow \exists i \ str[i: i+Length(sub)] = sub$ Теперь необходимо найти соответствующие страницы. Применяемая для этого стратегия состоит в том, чтобы начать с начальной страницы некоторого оперативного магазина и рассмотреть все страницы, которых можно достичь, следуя по соответствующим ссылкам. Агент должен обладать знаниями о многих магазинах, например, в таком виде:

Amazon e OnlineStores л Homepage{Amazon, "amazon.com"} Ebay G OnlineStores л Homepage(Ebay, "ebay.com") GenStore G OnlineStores л Homepage{GenStore, "gen-store.com"} В этих магазинах товары классифицируются по категориям товаров, а на начальной странице предоставляются ссылки на основные категории товаров. К младшим категориям можно перейти, следуя по цепочке соответствующих ссылок, что позволяет в конечном итоге проложить путь к требуемым предложениям. Иными словами, страница соответствует запросу, если ее можно достичь, проследовав по цепочке соответствующих ссылок на категории от начальной страницы магазина, а затем пройдя еще по одной ссылке к предложению товара:

$Relevant \{page, url, query\} \leq \rightarrow \exists store, home \ store \ G \ OnlineStores \ л \ Homepage\{store, home\} \wedge \exists url2 \ RelevantChain\{home, url2, query\} \wedge Link\{url2, url\} \wedge page = GetPage\{url\}$ Здесь предикат $Link\{from, to\}$ означает, что имеется гиперссылка от URL from к URL to (см. упр. 10.13); чтобы определить, какая цепочка ссылок может рассматриваться как соответствующая запросу, Relevant Chain, необходимо следовать не просто по любым существующим гиперссылкам, а только по таким ссылкам, связанный с которыми текст анкера указывает, что эта ссылка соответствует данному запросу на приобретение товара. Для этого используется предикат $LinkText\{from, to, text\}$, который означает, что имеется ссылка между URL from и to, текстом анкера которой является text. Цепочка ссылок между двумя URL, start и end, соответствует описанию d, если текст анкера каждой ссылки представляет собой соответствующее имя категории для d. Существование самой цепочки определяется с помощью рекурсивной формулировки, в которой в качестве базового случая используется пустая цепочка ($start=end$):

$RelevantChain \{start, end, query\} \leq \$ \{start=end\} \vee \{ \exists u, text \ LinkText \{start, u, text\} \wedge RelevantCategoryName\{query, text\} \wedge RelevantChain\{u, end, query\} \}$ 6 Альтернативой стратегии, предусматривающей прохождение по ссылкам, является использование какой-то машины поиска в Internet; технология, лежащая в основе поиска в Internet и выборки информации, рассматривается в разделе 23.2.

Теперь необходимо определить, что подразумевается под высказыванием, будто текст text является соответствующим именем категории RelevantCategoryName для запроса query. Прежде всего необходимо связать строки с категориями, именуемыми с помощью этих строк. Такую задачу можно решить с использованием

предиката $Name\{s, c\}$, который сообщает, что строка s является именем категории c , например, чтобы можно было сформулировать утверждение, что $Name\{\text{"laptops"}, LaptopComputers\}$. Некоторые дополнительные примеры применения предиката $Name$ приведены в табл. 10.1, б. Затем определим понятие соответствия. Допустим, что запросом $query$ является "laptops" (Портативные компьютеры). В таком случае предикат $RelevantCategoryName\{query, text\}$ принимает истинное значение, когда выполняется одно из приведенных ниже условий.

Таблица 10.1. Примеры обозначений категорий Таксономия категорий товаров а) Слова, б) которыми обозначаются эти категории

Books	Products	Name
MusicRecordings	c Products	Name MusicCDs
c MusicRecordings	Name MusicTapes	c MusicRecordings
Name Electronics	c Products	Name DigitalCameras
c Electronics	Name StereoEquipment	c Electronics
Name Computers	c Electronics	Name LaptopComputers
c Computers	Name DesktopComputers	c Computers
Name "books", Books)	"music", MusicRecordings)	"CDs", MusicCDs)
"tapes", MusicTapes)	"electronics", Electronics)	"digital cameras", DigitalCameras)
"stereos", Stereo Equipment)	"computers", Computers)	"laptops", LaptopComputers)
"desktops", DesktopComputers)		

• Строки $text$ и $query$ именуют одну и ту же категорию, например "laptop computers" и "laptops". • В строке $text$ именуется над категория, такая как "computers". • В строке $text$ именуется подкатегория, такая как "ultralight notebooks".

Логическое определение предиката $RelevantCategoryName$ состоит в следующем:

$RelevantCategoryName(query, text) \iff \exists c_1, C_2 \text{ Name}(query, c_1) \wedge \text{Name}(text, C_2) \wedge (C_1 \subseteq C_2 \vee c_2 \subseteq C_1)$ A0.1) В противном случае текст анкера не соответствует запросу, поскольку в нем именуется категория, не соответствующая данному описанию, такая как "mainframe computers" (мэйнфреймы) или "lawn & garden" (газон и сад).

Поэтому, чтобы обладать способностью следовать по искомым ссылкам, необходимо иметь широкие знания об иерархии категорий товаров. Верхняя часть этой иерархии может выглядеть, как показано в табл. 10.1, а. Задача перечисления всех возможных товарных категорий неосуществима, поскольку у покупателя всегда могут

возникнуть какие-то новые пожелания, а производители всегда готовы предложить новые товары для их удовлетворения (возможно, скоро будут даже продаваться обогреватели для коленных чашечек?). Тем не менее онтология, состоящая примерно из тысячи категорий, должна оказаться очень полезным инструментом для большинства покупателей.

Кроме самой иерархии товаров, необходимо иметь богатый словарь имен для категорий. Жизнь была бы гораздо проще, если бы существовало взаимно однозначное соответствие между категориями и символьными строками, которые их именуют.

Мы уже сталкивались с проблемой синонимов — двух имен для одной и той же категории, таких как "laptop computers" and "laptops". Кроме того, существует также проблема омонимов — обозначения одним именем двух или нескольких разных категорий. Например, если в базу знаний, приведенную в табл. 10.1, #, будет добавлено следующее вполне допустимое высказывание, что словом "CDs" обозначаются депозитные сертификаты:

Name("CDs", CertificatesOfDeposit) то имя "CDs" будет относиться к двум различным категориям.

Наличие синонимов и омонимов может стать причиной значительного увеличения количества путей, по которым должен следовать агент, и иногда может затруднить определение того, действительно ли данная страница соответствует запросу.

Гораздо более серьезная проблема состоит в том, что пользователь может вводить очень широкий спектр описаний, а магазин может использовать не такие имена категорий, о которых знает агент. Например, в ссылке может быть указано "laptop", тогда как в базе знаний имеется только имя "laptops", или пользователь может ввести запрос: "компьютер, который помещается на откидном столике сидения экономического класса в самолете Boeing 737". Невозможно заранее перечислить все способы, с помощью которых могут присваиваться имена какой-то категории, поэтому в некоторых случаях агент должен проявлять способность проводить дополнительные рассуждения для определения того, соответствует ли рассматриваемый текст отношению Name. В наихудшем случае для этого требуется полное понимание естественного языка, но отложим эту тему до главы 22. На практике многое можно сделать с помощью нескольких простых правил, например, позволяющих согласовать слово "laptop" с категорией, названной как "laptops". В упр. 10.15 предлагается разработать множество таких правил после проведения некоторых исследований, касающихся оперативных магазинов.

Готовы ли мы применить некоторый алгоритм логического вывода для получения множества предложений, соответствующих нашему запросу, после того как составлены логические определения, описанные в предыдущих абзацах, и подготовлены приемлемые базы знаний о категориях товаров и соглашениях об именовании?

К сожалению, не совсем! Недостающим элементом является функция Get Page (url), которая ссылается на HTML-страницу, находящуюся по заданному URL. Агент не хранит в своей базе знаний содержимое страниц с каждым URL, а также не располагает явными правилами дедуктивного вывода того, каким должно быть это содержимое. Вместо этого можно предусмотреть, чтобы правильная процедура HTTP выполнялась каждый раз, когда какая-то подцель требует применения

функции выборки страницы GetPage. Благодаря этому машина логического вывода сталкивается с такой ситуацией, как если бы вся система Web находилась в базе знаний. В этом состоит пример применения общего метода, называемого $\wedge$ процедурным вложением, с помощью которого конкретные предикаты и функции могут выполняться с использованием методов специального назначения.

Сравнение коммерческих предложений Теперь предположим, что в результате применения процессов формирования рассуждений, описанных в предыдущем разделе, подготовлено множество страниц с предложениями, касающимися нашего запроса "laptops". Чтобы сравнить эти предложения, агент должен извлечь из страниц с предложениями соответствующую информацию о цене, быстродействии, объеме диска, весе и т.д. Применительно к реальным Web-страницам эта задача может оказаться сложной, по всем тем же причинам, которые уже упоминались выше. Общепринятый способ решения этой проблемы состоит в использовании программ, называемых $\wedge$ оболочками (wrapper), для извлечения информации из страницы. Технология извлечения информации будет описана в разделе 23.3. А пока предположим, что такие оболочки существуют и после получения доступа к странице и базе знаний они добавляют к базе знаний определенные утверждения. Как правило, для обработки страницы должна применяться определенная иерархия оболочек: наиболее общая оболочка извлекает информацию о датах и ценах, более конкретная извлекает данные об атрибутах товаров, относящихся к категории компьютеров, а в случае необходимости применяется оболочка, относящаяся к конкретному узлу, которая обладает сведениями о формате представления данных в конкретном магазине. Допустим, что имеется страница с узла genstore . com, на которой находится текст:

YVM ThinkBook 970. Our price: \$1449.00 сопровождаемый данными о различных технических характеристиках. В этом случае желательно иметь оболочку для извлечения примерно такой информации:

3lc, offer 1c e LaptopComputers л offer e ProductOffers л ScreenSize(1c, Inches(1A)) л ScreenTypedc, ColorLCD) л MemorySizedc, Megabytes E12)) л CPUSpeed{1c, GHzB.4)) л OfferedProduct(offer, 1c) л Storeioffer, GenStore) л URL{offer, "genstore.com/comps/34356.html") л Price(offer, \$D49)) л Date(offer, Today) Этот пример иллюстрирует некоторые проблемы, которые возникают с самого начала серьезных исследований в области инженерии знаний для коммерческих сделок. Например, обратите внимание на то, что цена является атрибутом предложения offer, а не самого товара. Это важно, поскольку предложение в каждом конкретном магазине может изменяться со дня на день, даже на один и тот же портативный компьютер, а для некоторых категорий (такие как дома и картины) предложения по приобретению одного и того же отдельного объекта могут даже сопровождаться другими одновременными предложениями по покупке сопутствующих товаров по

различным ценам. Возникают и многие другие осложнения, о которых еще не шла речь, например, возможность того, что цена будет зависеть от метода платежа и от прав покупателя на некоторые скидки. Так или иначе, но в этой области еще предстоит выполнить очень много интересной работы.

Последняя задача состоит в сравнении информации о предложениях, которая была извлечена из разных страниц. Например, рассмотрим следующие три предложения:

A: 2.4 GHz CPU, 512MB RAM, 80 GB disk, DVD, CDRW, \$1695 B: 2.0 GHz CPU, 1GB RAM, 120 GB disk, DVD, CDRW, \$1800 C: 2.2 GHz CPU, 512MB RAM, 80 GB disk, DVD, CDRW, \$1800 Предложение A доминирует над предложением C; это означает, что предлагаемый на узле A компьютер дешевле и имеет более высокое быстродействие, а во всем остальном два компьютера одинаковы. Вообще говоря, предложение X доминирует над Y, если x имеет лучшее значение по меньшей мере одного атрибута и не хуже по всем остальным атрибутам. Но ни A, ни B не доминируют друг над другом. Чтобы определить, какое из этих предложений лучше, необходимо знать, как покупатель оценивает быстродействие процессора и стоимость по отношению к объему памяти и диска.

Тема выбора предпочтений среди многочисленных атрибутов в целом рассматривается в разделе 16.4, а пока наш торговый агент просто возвращает список всех предложений, соответствующих описанию покупателя, над которыми не доминируют другие предложения. В данном примере ни одно предложение не доминирует ни над A, ни над B. Обратите внимание на то, что данный результат основан на предположении, что все предпочитают более низкие цены, более быстродействующие процессоры и больший объем памяти. Некоторые атрибуты, такие как размер экрана в ноутбуке, зависят от конкретных предпочтений пользователя (хочет ли он иметь большой экран или миниатюрный ноутбук); для решения вопроса о таких предпочтениях торговый агент должен получить ответ от пользователя.

Торговый агент, описанный в этом разделе, является весьма простым; для него возможны многие усовершенствования. Тем не менее этот агент обладает достаточными способностями, чтобы при наличии правильных знаний о конкретной проблемной области он мог действительно оказать помощь покупателю. Поскольку этот агент имеет декларативную конструкцию, его можно легко дополнить для использования в более сложных приложениях. Основной замысел данного раздела состоял в том, чтобы показать, что для агента, подобного этому, требуется определенное представление знаний (в частности, об иерархии товаров), а после приобретения некоторых знаний в такой форме уже не очень сложно создать все остальное, что требуется от агента, основанного на знаниях.

10.6. Системы формирования рассуждений о КАТЕГОРИЯХ Выше было показано, что категории являются основными строительными блоками любой крупномасштабной схемы представления знаний. А в данном разделе описаны системы, специально предназначенные для организации и проведения рассуждений, касающихся категорий. Существуют два тесно связанных семейства систем: семантические сети предоставляют графические средства, позволяющие визуально представить базу знаний, и эффективные алгоритмы для логического вывода сведений о любом объекте на основании его принадлежности к некоторой категории, а описательные логики предоставляют формальный язык для конструирования и

комбинирования определений категорий, а также эффективные алгоритмы для установления связей между категориями на уровне подмножеств и надмножеств.

Семантические сети В 1909 году Чарльз Пирс предложил графическую систему обозначений в виде узлов и дуг, получившую название $\wedge$ экзистенциальных графов, которую он назвал "логикой будущего". Так зародилось понятие семантических сетей. С того времени и начались продолжительные дебаты между приверженцами "чистой логики" и приверженцами "чисто семантических сетей". К сожалению, эти дебаты затмили тот факт, что семантические сети (по меньшей мере, те из них, в которых правильно определена семантика) представляют собой всего лишь одну из форм логики. Система обозначений для высказываний некоторых типов, предусмотренная в семантических сетях, часто является более удобной для человека, но если отбросить эти особенности, касающиеся "удобного интерфейса", то базовые понятия (объекты, отношения, кванторы и т.д.) окажутся теми же самыми.

Существует много вариантов семантических сетей, но все они способны представлять отдельные объекты, категории объектов и отношения между объектами. В типичной графической системе обозначений имена объектов или категорий изображаются в овалах или прямоугольниках, а связи между ними обозначаются с помощью дуг с метками.

Например, на рис. 10.7 показана связь MemberOf между Mary и FemalePersons, которая соответствует логическому утверждению $Mary \in FemalePersons$; аналогичным образом, связь SisterOf между Mary и John соответствует утверждению $SisterOf(Mary, John)$. Категории можно соединять с помощью связи SubsetOf и т.д. Рисовать эти овалы и стрелки так интересно, что часто можно забыть обо всем.

Например, мы знаем, что матерями людей являются особы женского пола, но можно ли нарисовать связь HasMother от категории Persons к категории FemalePersons? Ответ является отрицательным, поскольку HasMother — это связь между человеком и его матерью, а категории не имеют матерей⁷. По этой причине на рис. 10.7 используется специальное обозначение — связь с меткой в

прямоугольнике с двойным контуром. Эта связь соответствует следующему утверждению:

$\forall x \ x \in \text{Persons} \Rightarrow [\forall y \ \text{HasMother}(x,y) \Rightarrow y \in \text{FemalePersons}]$ Нам может также потребоваться сформулировать утверждение, что люди имеют две ноги, таким образом:

$\forall x \ x \in \text{Persons} \Rightarrow \text{Legs}(x, 2)$ Как и прежде, необходимо тщательно следить за тем, чтобы в этом высказывании не утверждалось, что ногами обладает категория; как показано на рис. 10.7, для 7 В некоторых ранних системах не была предусмотрена возможность проводить различие между свойствами элементов категории и свойствами всей категории в целом. Это могло приводить непосредственно ко многим несогласованностям, как указал Дрю Макдермотт в своей статье "Artificial Intelligence Meets Natural Stupidity" (Искусственный интеллект сталкивается с естественной глупостью) [1020]. Еще одной распространенной проблемой было использование связей isA и для отношений между подмножествами и множествами, и для отношений между элементами и подмножествами, в полном соответствии с грамматикой английского языка: "a cat is a mammal"

(Кошка — млекопитающее) и "Fifi is a cat" (Фифи — кошка). Дополнительная информация по этой теме приведена в упр. 10.25.

формулировки утверждений о свойствах каждого элемента категории используется связь, обозначенная меткой в прямоугольнике с одинарным контуром.

John У —> 1 Рис. 10.7. Семантическая сеть с четырьмя объектами (John, Mary, 1 и 2) и четырьмя категориями. Отношения обозначаются связями с метками Система обозначений с использованием семантических сетей обеспечивает значительные удобства при формировании рассуждений с учетом наследования такого типа, которые были впервые представлены в разделе 10.2. Например, благодаря тому, что Мэри является человеком, она наследует свойство иметь две ноги. Поэтому, чтобы узнать, сколько ног имеет Мэри, алгоритм наследования проходит по связи MemberOf от объекта Mary к категории, к которой она принадлежит, а затем проходит по связям SubsetOf вверх по иерархии до тех пор, пока не находит категорию, для которой имеется связь с обведенной прямоугольником меткой Legs, в данном случае категорию Persons. Простота и эффективность этого механизма логического вывода по сравнению с логическим доказательством теорем издавна была одной из основных привлекательных особенностей семантических сетей.

Наследование становится сложнее, если некоторый объект может принадлежать больше чем к одной категории или если категория может быть подмножеством больше чем одной отличной от нее категории; такая ситуация называется $\wedge$ множественным наследованием. В подобных случаях алгоритм наследования может находить два или несколько конфликтующих значений, представляющих собой

ответ на запрос. По этой причине множественное наследование запрещено в некоторых языках объектно-ориентированного программирования (Object-Oriented Programming— OOP), таких как язык Java, в котором используется наследование в иерархии классов. В семантических сетях множественное наследование обычно разрешено, но мы отложим обсуждение этой темы до раздела 10.7.

Еще одной общей формой логического вывода является использование $\wedge$ инверсных связей. Например, связь Has Sister является инверсной по отношению к SisterOf, а это означает, что имеет место следующее высказывание:

$\forall p, s \text{ HasSister}\{p, s\} \Leftrightarrow \text{SisterOf}(s, p)$ Это высказывание может быть подтверждено в семантической сети, если связи являются о вещественными, т.е. преобразованными в объекты в полном смысле этого понятия. Например, может быть предусмотрен объект SisterOf, соединен-

ный связью Inverse с объектом HasSister. После получения запроса, касающегося того, кто является сестрой (SisterOf) Джона, алгоритм логического вывода может обнаружить, что связь HasSister является инверсной по отношению к связи SisterOf, и поэтому ответить на данный запрос, проследовав по связи HasSister от объекта John к объекту Mary. При отсутствии этой инверсной информации могло бы потребоваться проверить каждое лицо женского пола для определения того, не имеет ли это лицо связь SisterOf, направленную к Джону. Это связано с тем, что в семантических сетях непосредственная индексация предусмотрена только для самих объектов, а также категорий и связей, исходящих из них; проводя сравнение с логикой первого порядка, можно отметить, что это аналогично индексации базы знаний только по первому параметру каждого предиката.

Читатель должен был обнаружить очевидный недостаток системы обозначений с помощью семантических сетей по сравнению с логикой первого порядка: тот факт, что между овалами проводятся связи, соединяющие два овала, говорит о том, что в семантических сетях могут быть представлены только бинарные отношения.

Например, высказывание Fly(Shankar, NewYork, NewDelhi, Yesterday) не может быть реализовано непосредственно в семантической сети. Тем не менее можно достичь эффекта реализации n -арных высказываний путем о вещественного самого рассматриваемого высказывания в виде события (см. раздел 10.3), принадлежащего к соответствующей категории событий. На рис. 10.8 показана структура семантической сети для указанного выше конкретного события. Обратите внимание на то, что ограничение, связанное с необходимостью применять только бинарные отношения, вынуждает создавать богатую онтологию о вещественных понятий; в действительности основная часть онтологии, разработанной в данной главе, происходила из систем с семантическими сетями.

Рис. 10.8. Фрагмент семантической сети, на которой показано представление логического утверждения $\text{Fly}(\text{Shankar}, \text{NewYork}, \text{NewDelhi}, \text{Yesterday})$. Овеществление высказываний дает возможность представить в системе обозначений семантической сети любое базовое, атомарное высказывание логики первого порядка, не содержащее функций. Реализация некоторых видов высказываний с кванторами всеобщности может осуществляться с помощью инверсных связей и применяемых к категориям стрелок с метками в одинарных и двойных прямоугольниках, но даже такие ухищрения отнюдь не позволяют приблизиться к возможностям полной логики первого порядка. В этой системе обозначений недостает всех таких выразительных возможностей, как отрицание, дизъюнкция, вложенные функциональные символы и кванторы существования. В последнее время появилась возможность расширить эту систему обозначений, чтобы сделать ее эквивалентной

логике первого порядка (как при использовании экзистенциальных графов Пирса или секционированных семантических сетей Хендрикса [647]), но это приводит к исчезновению одного из основных преимуществ семантических сетей — простоты и прозрачности процессов логического вывода. Простая система обозначений позволяет проектировщикам создавать большие сети и все еще иметь полное представление о том, какие запросы должны быть эффективными, поскольку, во-первых, можно легко визуально представить, через какие шаги должна пройти процедура логического вывода, и, во-вторых, в некоторых случаях язык запросов настолько прост, что в нем невозможно сформулировать сложные, запутанные запросы. Для тех случаев, когда обнаруживается, что выразительная мощь слишком ограничена, во многих системах семантических сетей предусмотрены процедурные вложения, позволяющие компенсировать эти недостатки. Процедурное вложение — это метод, с помощью которого при выполнении запроса, касающегося некоторого отношения (а иногда и при вводе в базу знаний некоторого утверждения), осуществляется вызов специальной процедуры (предназначенной лишь для обработки этого отношения), а не общего алгоритма логического вывода.

Одной из самых важных характерных особенностей семантических сетей является их способность представлять $\wedge$ заданные по умолчанию значения для категорий.

Внимательно изучив рис. 10.7, можно обнаружить, что Джон имеет одну ногу, несмотря на тот факт, что он — человек, а все люди имеют две ноги. В строго логической базе знаний такое сочетание фактов было бы противоречивым, но в семантической сети утверждение, что все люди имеют две ноги, обладает лишь статусом значения, применяемого по умолчанию; т.е. применительно к любому человеку предполагается, что он имеет две ноги, если этому не противоречит более конкретная информация. Принудительное применение семантики заданных по умолчанию значений осуществляется естественным образом с помощью алгоритма

наследования, поскольку этот алгоритм следует по связям, направленным вверх от самого объекта (в данном случае Джона), и останавливается, как только находит искомое значение. Принято говорить, что заданное по умолчанию значение $\wedge$ перекрывается более конкретным значением. Обратите внимание на то, что можно было бы также перекрыть информацию о заданном по умолчанию количестве ног, создав категорию одноногих людей `OneLeggedPersons`, подмножество категории `Persons`, элементом которого является `John`.

Можно было бы также сохранить строго логическую семантику для этой сети, сформировав следующее высказывание, что утверждение `Legs` для `Persons` содержит исключение для `John`:

$\forall x \in G \text{ Persons } \neg (x = \text{John} \Rightarrow \text{Legs}(x, 2))$ При наличии фиксированной сети такое дополнение было бы семантически адекватным, но стало бы гораздо менее кратким по сравнению с самой сетевой системой обозначений, если бы и без этого в сети было бы много исключений. А в сети, которая обновляется путем добавления дополнительных утверждений, такой подход оказался бы полностью неприемлемым, поскольку фактически потребовалось бы указывать, что к исключениям относятся все люди, в отношении которых еще не известно, имеют ли они две или только одну ногу. В разделе 10.7 приведены дополнительные сведения об этой проблеме и о формировании рассуждений в логике умолчаний в целом.

Описательные логики Синтаксис логики первого порядка предназначен для упрощения процедуры формирования высказываний об объектах, а $\wedge$ описательные логики представляют собой системы обозначений, которые предназначены для упрощения процедуры описания определений и свойств категорий. Системы описательной логики развились из семантических сетей в ответ на необходимость формализовать тот смысл, который несут в себе сети, сохранив вместе с тем акцент на использование таксономической структуры в качестве принципа организации.

Основные задачи логического вывода для описательных логик сводятся к $\wedge$ обобщению (проверке того, является ли одна категория подмножеством другой путем сравнения их определений) и $\wedge$ классификации (определению принадлежности некоторого объекта к какой-то категории). В некоторых системах предусматривается также проверка непротиворечивости категории, т.е. того, являются ли выполнимыми критерии принадлежности к категории с точки зрения логики.

Типичным языком описательной логики является `Classic` [155]. Синтаксис описаний `Classic` показан в листинге 10.2. Например, чтобы сформулировать утверждение, что холостяками называют неженатых взрослых мужчин, можно записать следующее:

`Bachelor = And{Unmarried, Adult, Male}` Эквивалент этого утверждения в логике первого порядка выглядел бы так:

Bachelor(x) <=> Unmarried(x) л Adult (x) л Ma1e(x) Листинг 10.2. Синтаксис описаний в подмножестве языка Classic Concept —> Thing | ConceptName | And {Concept, ...} | All{RoleName,Concept} | AtLeast(Integer,RoleName) | AtMost(Integer,RoleName) | Fills {RoleName, IndividualName,...} j SameAs{Path, Path} | OneOf {IndividuaiName, ...} Path —> [RoleName, ...] Следует отметить, что в этом языке описательной логики фактически разрешается непосредственно выполнять логические операции с предикатами, что исключает необходимость в первую очередь создавать высказывания, которые должны быть соединены связками. Любое описание на языке Classic может быть сформулировано и в логике первого порядка, но некоторые описания Classic формулируются проще. Например, чтобы описать множество мужчин, имеющих трех сыновей, из которых все безработны и женаты на врачах, и, самое большее, двух дочерей, из которых все являются преподавателями на кафедрах физики или математики, можно записать следующее:

8 Следует отметить, что этот язык не позволяет просто формулировать утверждение, что одно понятие (или категория) является подмножеством другого. Это требование введено сознательно: отношение обобщения между категориями должно быть выведено логическим путем из некоторых аспектов описаний категорий. Если этого не удастся сделать, то в описаниях имеются какие-то упущения.

And (Man,AtLeastC,Son),AtMostB,Daughter), All{Son,And{Unemployed, Married,All(Spouse,Doctor))), All{Daughter,And{Professor,Fills{Department, Physics,Math}})) Перевод этого высказывания на язык логики первого порядка оставляем читателю в качестве упражнения.

По-видимому, одной из наиболее важных характерных особенностей описательных логик является сделанный в них акцент на осуществимости логического вывода.

Решение любого экземпляра проблемы осуществляется путем его описания, а затем выполнения запроса, касающегося того, является ли этот экземпляр обобщением одной из нескольких возможных категорий решений. В стандартных системах логики первого порядка предсказание времени выработки решения часто оказывается невозможным, а пользователю чаще всего самому приходится разрабатывать представление, позволяющее исключать множества высказываний, которые, по-видимому, вынудят систему выполнять вычисления в течение нескольких недель, чтобы решить задачу. С другой стороны, в описательных логиках все направлено на обеспечение того, чтобы проблема проверки обобщения могла быть решена за время, полиномиально зависящее от размера описаний⁹.

На первый взгляд такое свойство описательных логик может показаться удивительным, пока не станет очевидно, что в процессе формулировки задачи может быть достигнут только один из двух безуспешных результатов: либо задача окажется настолько сложной, что ее описание вообще невозможно будет

сформулировать, либо она потребует экспоненциально большого описания! Тем не менее анализ осуществимости логического вывода позволяет пролить свет на то, какого рода конструкции вызывают проблемы, и поэтому помочь пользователю понять, какие следствия вытекают из использования различных представлений. Например, в описательных логиках обычно не используются такие отношения, как отрицание и дизъюнкция.

Дело в том, что каждое из этих отношений вынуждает логические системы первого порядка для обеспечения полноты проходить через этап анализа вариантов, который может потенциально характеризоваться экспоненциальной сложностью. По той же причине эти отношения исключены из языка Prolog. В языке Classic допускается использовать только ограниченную форму дизъюнкции в конструкциях Fills и OneOf, которые допускают выполнение дизъюнкции по явно заданным объектам, а не по их описаниям. Если бы было разрешено использовать дизъюнктивные описания, то вложенные определения могли бы легко привести к появлению экспоненциального количества альтернативных путей, по которым одна категория могла бы обобщать другую.

10.7. Формирование рассуждений с использованием информации, заданной по умолчанию В предыдущем разделе был приведен простой пример утверждения с заданной по умолчанию информацией о состоянии: люди имеют две ноги. Это заданное по умолчанию значение можно переопределить с помощью более конкретной информации, 9 В принципе, язык Classic обеспечивает эффективную проверку обобщения, но в худшем случае время прогона может стать экспоненциальным.

например, такой, что Длинный Джон Сильвер имеет одну ногу. Кроме того, было показано, что механизм наследования в семантических сетях предоставляет простой и естественный способ переопределения значений, заданных по умолчанию. В этом разделе проводится более общее исследование заданной по умолчанию информации, с тем чтобы можно было понять семантику умолчаний, а не просто создать какой-то процедурный механизм.

Открытые и закрытые миры Предположим, что вы просматриваете доску объявлений университетской кафедры компьютерных наук и видите сообщение со словами: "Студентам будут предложены следующие курсы лекций: CS 101, CS 102, CS 106, EE 101". Итак, сколько курсов здесь предлагается? Если вы ответите "четыре", то поступите в соответствии с принципами работы типичной системы баз данных. После ввода информации в реляционную базу данных с помощью оператора, эквивалентного следующим четырем утверждениям:

Course {CS, 101), CourseiCS, 102), Course(CS, 106), Course{EE, 101) A0.2) запрос на языке SQL select count (*) from Course возвратит 4. С другой стороны, логическая система

первого порядка, скорее всего, ответит примерно в том смысле, что количество предлагаемых курсов должно находиться в пределах от одного до бесконечности, но только не быть равно "четырем". Причина этого состоит в том, что эти утверждения Course не отрицают возможности, что могут также предлагаться другие не упомянутые здесь курсы, а также не позволяют сделать вывод, что все указанные курсы отличаются друг от друга.

Этот пример показывает, что системы баз данных и соглашения, неявно применяемые людьми при общении, отличаются от логики первого порядка по меньшей мере в двух аспектах. Во-первых, в языках баз данных (и в языках общения людей) предполагается, что предоставленная информация является полной, поэтому предполагается, что базовые атомарные высказывания, не обозначенные как истинные, являются ложными. Такое предположение называется $\wedge$ предположением о замкнутом мире, или CWA (Closed-World Assumption). Во-вторых, обычно предполагается, что разные имена относятся к различным объектам. В этом состоит предположение об уникальности имен, или UNA (Unique Names Assumption), которое было впервые описано в контексте имен действий в разделе 10.3.

В логике первого порядка такие соглашения не приняты, и поэтому необходимо их определять явно. Чтобы сформулировать высказывание, что предлагаются четыре и только четыре разных курса, необходимо записать следующее:

$\text{Course}(d, n) \Leftrightarrow [d, n] = [05, 101] \vee [d, n] = [CS, 102] \vee [d, n] = [CS, 106] \vee [d, n] = [?, 101]$

A0.3) Уравнение 10.3 называется $\wedge$ дополнением10 уравнения 10.2. Вообще говоря, такое дополнение должно содержать определение для каждого предиката (высказывание в форме "тогда и только тогда"), а каждое определение должно содержать по одному 10 Иногда это дополнение называют "дополнением Кларка" в честь предложившего его Кейта Кларка.

дизъюнкту для каждого определенного выражения, головой которого является этот предикат11. Как правило, такое дополнение создается, как описано ниже.

1. Собрать все выражения с одним и тем же названием предиката (p) и с одной и той же аргументностью (л).

2. Преобразовать каждое выражение в $\wedge$ нормальную форму Кларка, например, заменить выражение $\text{P}(t_1, \dots, t_n) \leftarrow \text{Body}$ где t_i — термины, выражением $\text{P}(v_1, \dots, v_n) \leftarrow \exists w_1 \dots w_n [v_1 \# \dots, v_n] = [\text{tlt} \dots, \text{tn}] \wedge \text{Body}$ где V_i — вновь введенные переменные, а $w^{\wedge}$ — переменные, которые присутствуют в первоначальном выражении. При этом следует использовать одно и то же множество v_L для каждого выражения. Это приводит к получению такого множества выражений:

$P(V_i, j, V_n) \leftarrow B_i \vee P(V_i / j / V_n) \wedge B_k$ 3. Скомбинировать их вместе в одно большое дизъюнктивное выражение:

$P(v_i, \dots, v_n) \leftarrow B_i \vee \dots \vee B_k$ 4. Сформировать завершение, заменяя оператор $\leftarrow$ оператором эквивалентности:

$P(v_i, \dots, v_n) \Leftrightarrow B_i \vee \dots \vee B_k$ Пример дополнения Кларка для базы знаний, включающей и базовые факты, и правила, приведен в табл. 10.2. Чтобы ввести в действие предположение об уникальности имен, достаточно сформировать дополнение Кларка для отношения равенства, где единственными известными фактами являются $CS=CS$, $101 = 101$ и т.д.

Оставляем эту задачу читателю в качестве упражнения.

Таблица 10.2. Дополнение Кларка для множества хорновских выражений. В первоначальной хорновской программе (слева) четыре курса перечислены явно, а также содержатся утверждения, что проводятся курсы по математике с целочисленными номерами от 101 до 130, а для каждого курса CS по компьютерным наукам в серии 100 (для студентов) имеется соответствующий курс в серии 200 (для аспирантов). В дополнении Кларка (справа) указано, что других курсов больше нет. С помощью этого дополнения и предположения об уникальности имен (а также очевидного определения предиката Integer) можно получить желаемое заключение, что имеется точно 36 курсов: 30 курсов по математике и 6 курсов по компьютерным наукам. Хорновские выражения Дополнение Кларка

$Course(CS, 101)$
 $Course(d, n) \Leftrightarrow [d, n] = [05, 101] \vee [d, n] = [CS, 102] \vee [d, n] = [CS, 106] \vee [d, n] = [CS, 106] \vee [d, n] = [EE, 101] \vee [d, n] = [EE, 101]$
 $Course(EE, i) \leftarrow Integer(i) \wedge 3 \leq i \leq 10$
 $Integer(i) \wedge Integer(i)$

11 Следует отметить, что эти выражения также представляют собой одну из разновидностей аксиом состояния-преемника, приведенных в разделе 10.3.

Окончание табл. 10.2 Хорновские выражения Дополнение Кларка

$101 < 2 \leq 130$
 $Course(CS, 101) \Leftrightarrow \exists t [d, n] = [CS, t+100] \wedge Course(CS, t) \wedge 100 < t \leq 130$
 $Course(CS, m) \wedge 100 < m \leq 200 \wedge \forall n (n < 200 \rightarrow \neg Course(CS, n))$
 Предположение о замкнутом мире позволяет найти минимальную модель отношения. Это означает, что в данном примере можно найти модель отношения Course с наименьшим количеством элементов. Минимальная модель отношения Course, соответствующая уравнению 10.2, имеет четыре элемента; при меньшем количестве элементов возникало бы противоречие. Для хорновских баз знаний всегда имеется уникальная минимальная модель. Следует отметить, что с учетом предположения об уникальности имен это утверждение распространяется также и на отношение равенства, поскольку каждый терм равен только самому себе. Как ни парадоксально, это означает также, что минимальные модели являются одновременно и максимальными, в том смысле, что включают максимально возможное количество объектов.

Было бы допустимо взять любую хорновскую программу, сформировать дополнение Кларка и передать полученный результат в программу автоматического доказательства теорем для выполнения логического вывода. Но обычно является более эффективным использование механизма логического вывода специального назначения, такого как Prolog, поскольку в этом механизме логического вывода уже учитываются предположения о замкнутом мире и уникальности имен.

Те, кому приходится использовать предположение о замкнутом мире, должны соблюдать осторожность при выборе способа формируемых рассуждений.

Например, при использовании базы данных с результатами переписи населения было бы резонно принимать предположение о замкнутом мире при формировании рассуждений о текущем населении городов, но было бы неправильно делать вывод, что в будущем не родится ни один ребенок, лишь на основании того, что база данных не содержит записей, касающихся будущих дней рождения. Благодаря CWA база данных становится полной в том смысле, что позволяет дать либо положительный, либо отрицательный ответ на каждый атомарный запрос; если же какие-то факты не заданы в этой базе данных изначально (например, о будущих рождениях), то CWA не может применяться. Более сложная система представления знаний может дать пользователю возможность указывать правила, по которым следует применять предположение о замкнутом мире.

Отрицание как недостижение цели и устойчивая семантика модели Как было показано в главах 7 и 9, базы знаний в форме хорновских выражений имеют желаемые вычислительные свойства. Однако во многих приложениях довольно сложно выполнить выдвигаемое при этом требование, чтобы каждый литерал в теле выражения был положительным. Например, может потребоваться сформулировать утверждение: "На прогулку можно выходить, если нет дождя", не будучи вынужденным изобретать такие предикаты, как NotRaining. В этом разделе рассматривается дополнение к хорновским выражениям в форме явного отрицания, которое

основано на идее использования $\wedge$ отрицания как недостижения цели. Эта идея состоит в том, что истинность отрицательного литерала $\text{not } P$ может быть "доказана" просто на основании того, что попытка доказательства P окончилась неудачей.

В этом заключается одна из форм рассуждений по умолчанию, тесно связанная с предположением о замкнутом мире: предполагается, что нечто является ложным, если нельзя доказать, что оно истинно. Для того чтобы можно было отличить отрицание как недостижение цели от логического оператора "-", для обозначения первого используется оператор "not".

В языке Prolog допускается применять оператор `not` в теле выражения.

Например, рассмотрим следующую программу Prolog: IDEdrive $\leftarrow$ Drive л not SCSIdrive.

SCSIdrive $\leftarrow$ Drive л not IDEdrive. SCSIcontroller $\leftarrow$ SCSIdrive.

Drive. A0.4) Первое правило указывает, что если в компьютере имеется жесткий диск, не являющийся диском SCSI, то это должен быть диск IDE. Согласно второму правилу, если это — не диск IDE, то он должен быть диском SCSI. В третьем правиле утверждается, что из наличия в компьютере диска SCSI следует наличие в нем контроллера SCSI, а четвертое утверждает, что в компьютере обязательно должен быть жесткий диск. Эта программа имеет следующие две минимальные модели:

$M_1 = \{\text{Drive, IDEdrive}\}$ $M_2 = \{\text{Drive, SCSIdrive, SCSIcontroller}\}$ Очевидно, что минимальные модели не позволяют выразить намеченную семантику программ с отрицанием как недостижением цели. Рассмотрим следующую программу:

$P \leftarrow \neg Q$. A0.5) Эта программа имеет две минимальные модели, $\{P\}$ и $\{Q\}$. С точки зрения логики первого порядка в этом есть смысл, поскольку выражение $P \leftarrow \neg Q$ эквивалентно $P \vee Q$. Но с точки зрения языка Prolog это сомнительно: если терм Q ни разу не появляется в этой программе слева от стрелки, то как же он может стать следствием?

Альтернативный подход состоит в использовании идеи $\wedge$ стабильной модели, которая представляет собой такую минимальную модель, что каждый атом в модели имеет $\wedge$ обоснование: правило, голова которого является атомом, а каждый литерал в теле выполняется. Формально модель M определяется как стабильная модель программы Y , если M — уникальная минимальная модель $\wedge$ сокращения Y по отношению к M . Сокращение программы Y определяется путем предварительного удаления из Y любого правила, которое имеет в теле литерал $\text{not } A$, где A имеется в модели, а затем удаления всех отрицательных литералов в оставшихся правилах. Поскольку после этого сокращение программы Y становится списком хорновских выражений, оно должно иметь уникальную минимальную модель.

Сокращением программы $P \leftarrow \text{not } 0$ по отношению к $\{P\}$ является P , и это сокращение имеет минимальную модель $\{P\}$. Поэтому $\{P\}$ — стабильная модель.

Сокращение по отношению к $\{?\}$ представляет собой пустую программу, и это сокращение имеет минимальную модель $\{\}$. Поэтому $\{Q\}$ — это нестабильная модель, поскольку Q не имеет обоснования в уравнении 10.5. В качестве еще одного примера можно привести сокращение уравнения 10.4 по отношению к m_1 , как показано ниже.

IDEdrive $\leftarrow$ Drive. SCSIcontroller $\leftarrow$ SCSIdrive. Drive.

Эта программа имеет минимальную модель m_1 ? поэтому m_1 — стабильная модель. $\wedge$ Программирование множества ответов — это разновидность логического программирования с отрицанием как недостижением цели, которая функционирует на основе преобразования логической программы в базовую форму с последующим поиском стабильных моделей (называемых также $\wedge$ множествами ответов) с использованием методов проверки пропозициональной модели. Таким образом, программирование множества ответов разработано на основе и системы Prolog, и быстродействующих алгоритмов автоматического доказательства выполнимости пропозициональных высказываний, таких как WalkSAT. И действительно, программирование множества ответов было успешно применено для решения задач планирования, по такому же принципу, как применялись программы автоматического доказательства выполнимости пропозициональных высказываний. Преимуществом планировщиков на основе множества ответов над другими планировщиками является их большая гибкость: операторы и ограничения планирования могут быть выражены в виде логических программ и не привязаны к жесткому формату конкретной формальной структуры планирования.

Недостаток методов планирования на основе множества ответов является таким же, как и в других пропозициональных методах: если количество объектов в универсуме очень велико, то может возникать экспоненциальное замедление.

Логика косвенного описания и логика умолчания Выше приводились два примера, в которых с виду естественные процессы формирования рассуждений нарушают свойство монотонности логики, которое было доказано¹² в главе 7. В первом примере свойство, наследуемое всеми элементами категории в семантической сети, может быть переопределено с использованием более конкретной информации, касающейся подкатегории. Во втором примере отрицаемые литералы, выведенные на основании предположения о замкнутом мире, могут быть переопределены путем добавления положительных литералов.

Простой самоанализ показывает, что такие нарушения монотонности широко распространены и в обыденных рассуждениях. Создается впечатление, что люди часто не переходят, а "перепрыгивают" к заключениям. Например, увидев автомобиль, припаркованный на улице, человек обычно предполагает, что у этого автомобиля четыре колеса, даже если видит три. (Если вы чувствуете, что существование четвертого колеса действительно находится под сомнением, продумайте также вопрос о том, действительно ли являются настоящими три видимых колеса или они заменены изображениями на картонках, а сам автомобиль стоит на подставках.) И в самом деле, теория вероятностей позволяет уверенно прийти к заключению, что вероятность существования четвертого колеса весьма велика (хотя и не равна единице¹² Напомним, что условие монотонности требует, чтобы все высказывания, которые следуют из базы знаний KB, по-прежнему

следовали бы из нее после добавления новых высказываний. Это означает, что если $\text{кв } N \text{ а}$, то $\text{кв } \text{л } 3 \neq \text{а}$.

це), но большинству людей мысль о том, что у автомобиля нет четвертого колеса, даже не приходит в голову, если им не представляются новые факты, позволяющие изменить свое мнение. Таким образом, создается впечатление, что вывод о наличии у наблюдаемого автомобиля четырех колес достигается по умолчанию, если нет каких-либо причин поставить этот вывод под сомнение. Если же поступают новые факты (например, становится очевидно, что владелец катит колесо, а сам автомобиль поднят домкратом), то первоначальный вывод может быть отброшен. Принято говорить, что в рассуждениях такого рода проявляется $\wedge$ немонотонность, поскольку множество убеждений не возрастает монотонно со временем, по мере поступления новых фактов. Для того чтобы можно было описывать подобное поведение, были разработаны $\wedge$ немонотонные логики, в которых используются модифицированные определения понятий истинности и логического следствия. В данном разделе рассматриваются две такие логики, для изучения которых были проведены обширные исследования: логика косвенного описания и логика умолчания. $\wedge$ Косвенное описание (circumscription) может рассматриваться как более мощная и точная версия предположения о замкнутом мире. Его идея состоит в том, что должны быть заданы конкретные предикаты, в отношении которых предполагается, что они являются "настолько ложными, насколько это возможно", т.е. ложными для всех объектов, за исключением тех, для которых эти предикаты заведомо истинны.

Например, предположим, что необходимо ввести в базу знаний применяемое по умолчанию правило, что птицы летают. Для этого введем предикат, скажем $\text{Abnormal1}(x)$, и запишем следующее:

$\text{Bird}(x) \text{ л } \neg \text{Abnormal1}(x) \Rightarrow \text{Flies}(x)$ Если будет указано, что для предиката Abnormal1 должно применяться косвенное описание, то программа формирования рассуждений на основе косвенных описаний получает право предполагать, что $\neg \text{Abnormal1}(x)$, если не известно, что $\text{Abnormal1}(x)$ является истинным. Это позволяет выводиться заключение $\text{Flies}(\text{Tweety})$ из предпосылки $\text{Bird}(\text{Tweety})$, но такое заключение становится недействительным, если в базу знаний вводится утверждение $\text{Abnormal1}(\text{Tweety})$.

Косвенное описание может рассматриваться как один из примеров логики $\wedge$ предпочтения моделей (model preference). В подобных логиках высказывание следует из базы знаний (со статусом, заданным по умолчанию), если оно истинно во всех предпочтительных моделях базы знаний; в этом данное требование отличается от требования истинности во всех моделях в классической логике. С точки зрения косвенного описания одна модель является предпочтительной по отношению к другой, если в ней имеется меньшее количество аномальных объектов¹³.

Рассмотрим, как эта идея может применяться в контексте множественного наследования в семантических сетях. Стандартный пример, демонстрирующий проблемы множественного наследования, называется "парадоксом Никсона". Этот пример основан на том наблюдении, что Ричард Никсон был одновременно и квакером (поэтому по 13 С точки зрения предположения о замкнутом мире одна модель является предпочтительной по отношению к другой, если в ней имеется меньше истинных атомов, т.е. предпочтительными моделями являются минимальные модели. Существует естественная связь между предположением о замкнутом мире и базами знаний с определенными выражениями, поскольку фиксированная точка, достигаемая в процессе прямого логического вывода в такой базе знаний, представляет собой уникальную минимальную модель (см. с. 315).

умолчанию пацифистом), и республиканцем (поэтому по умолчанию не пацифистом). Эту ситуацию можно описать следующим образом:

Republican {Nixon} л Quaker (Nixon) Republican (x) л —\Abnormal2 (x) => —\Pacifist (x)
 Quaker (x) л —\Abnormal3 (x) => Pacifist(x) Если применяется косвенное описание для предикатов Abnormal2 и Abnormal3, то возникают две предпочтительные модели: в одной из них истинны выражения Abnormal2 {Nixon} и Pacifist {Nixon}, а в другой — Abnormal3 {Nixon} и —Pacifist {Nixon}. Таким образом, программа формирования рассуждений на основе косвенных описаний остается в полном неведении в отношении того, является ли Никсон пацифистом. При желании можно дополнительно ввести утверждение, что религиозные убеждения имеют приоритет над политическими убеждениями; для этого можно воспользоваться формализмом, называемым $\wedge$ косвенным описанием с приоритетами (prioritized circumscription), чтобы отдать предпочтение моделям, в которых минимизируется предикат Abnormal3.

Ъ. Логика умолчания (default logic) — это формальная система, в которой могут быть записаны $\wedge$ применяемые по умолчанию правила, применяемые для вывода непротиворечивых немонотонных заключений. Заданное по умолчанию правило выглядит примерно таким образом:

Bird(x) : Flies(x) / Flies(x) Это правило означает, что если выражение Bird(x) является истинным, а выражение Flies (x) не противоречит базе знаний, то вывод Flies (x) может быть сделан по умолчанию. В общем случае заданное по умолчанию правило выглядит следующим образом:

P : J₁, ..., J_n / C где P называется предпосылкой, C — заключением, а J_i представляют собой обоснования; если можно доказать, что любое из них ложно, то нельзя вывести заключение.

Любая переменная, которая появляется в J_i или C, должна также находиться и в P.

Пример с парадоксом Никсона может быть представлен в логике умолчаний с помощью одного факта и двух заданных по умолчанию правил следующим образом:

Republican (Nixon) ∧ Quaker (Nixon) Republican(x) : ¬Pacifist(x) ∨ ¬Pacifist (x) Quaker(x) : Pacifist(x) ∨ Pacifist(x) Для интерпретации того, что означают заданные по умолчанию правила, определим понятие $\wedge$ расширения теории умолчаний как максимального множества следствий из этой теории. Таким образом, расширение S состоит из первоначально известных фактов и множества заключений, полученных на основе заданных по умолчанию правил, таких, что из S нельзя больше вывести дополнительные заключения, а обоснования всех сделанных по умолчанию заключений в S не противоречат S.

Как и в случае предпочтительных моделей, в логике косвенного описания существуют два возможных расширения для парадокса Никсона: согласно одному из них он является пацифистом, а согласно другому — нет. Имеются также схемы с приоритетами, в которых определенные заданные по умолчанию правила могут получать преимущества над другими, что позволяет разрешать некоторые противоречия.

С 1980 года, когда впервые были предложены немонотонные логики, был достигнут большой прогресс в понимании их математических свойств. А начиная с последней половины 1990-х годов появились практические системы, основанные на логическом программировании, которые продемонстрировали перспективность их использования в качестве инструментальных средств представления знаний. Тем не менее остаются нерешенными некоторые вопросы. Например, если высказывание:

"Автомобили имеют четыре колеса" является ложным, то что влечет за собой наличие такого высказывания в некоторой базе знаний? Каковым является наиболее приемлемое множество заданных по умолчанию правил? Если нет возможности принять отдельно для каждого правила решение о том, должно ли оно находиться в нашей базе знаний, то налицо серьезная проблема отсутствия модульности.

Наконец, как могут использоваться для принятия решений определенные убеждения, имеющие заданный по умолчанию статус? По-видимому, эта проблема является наиболее сложной для систем формирования рассуждений по умолчанию. Принятие решений часто связано с поиском компромиссов, и поэтому необходимо сравнивать силу убеждений в отношении предполагаемых результатов различных действий.

В тех случаях, когда решения одного и того же типа должны быть приняты повторно, появляется возможность интерпретировать заданные по умолчанию правила как высказывания с "пороговой вероятностью". Например, заданное по умолчанию правило "Тормоза в моем автомобиле всегда в порядке" фактически означает "Если нет другой информации, то вероятность, что тормоза в моем автомобиле в порядке,

достаточно велика, чтобы оптимальным решением для меня был выезд на дорогу без их проверки". Если контекст принятия решения изменяется (например, если приходится спускать тяжело груженный самосвал вниз по крутой горной дороге), такое заданное по умолчанию правило внезапно становится неприемлемым, даже если нет новых фактов, говорящих о том, что тормоза неисправны. Подобные соображения привели некоторых исследователей к выводу, что необходимо продумать, как внедрить средства формирования рассуждений по умолчанию в теорию вероятностей.

10.8. Системы поддержки истинности В предыдущем разделе было показано, что многие логические выводы, полученные с помощью той или иной системы представления знаний, могут иметь лишь некоторый заданный по умолчанию статус, а не быть абсолютно достоверными.

Поэтому некоторые из таких полученных логическим путем фактов неизбежно оказываются ложными и должны быть отброшены на основании новой информации. Этот процесс называется ^пересмотром убеждений (belief revision)14. Предположим, что база знаний KB содержит высказывание P (возможно, заключение, сформированное по умолчанию с помощью алгоритма прямого логического вывода, или, возможно, просто неверное утверждение) и требуется выполнить операцию Tell (KB, -iP).

14 Пересмотр убеждений часто противопоставляется обновлению убеждений (belief update), которое происходит, когда осуществляется пересмотр базы знаний для того, чтобы она отражала какое-то изменение в мире, а не добавление новой информации о неизменном мире. В обновлении убеждений объединяется пересмотр убеждений с рассуждениями о времени и изменениях; процесс обновлений связан также с процессом фильтрации, описанным в главе 15.

Но для предотвращения возникновения противоречия необходимо вначале выполнить операцию Retract {KB, P}. На первый взгляд в этом нет ничего сложного. Но если на основании P были выведены какие-то дополнительные высказывания и внесены в базу знаний, то возникают проблемы. Например, импликация $P \Rightarrow Q$ могла использоваться для внесения в базу знаний высказывания Q. Очевидное "решение"

(извлечение всех высказываний, которые следуют из P) неприемлемо, поскольку подобные высказывания могут иметь другие обоснования, помимо P. Например, если в базе знаний имеются также высказывания R и $R \Rightarrow O$, то O в конечном итоге вообще не следует удалять. ^ Системы поддержки истинности, или TMS (Truth Maintenance System), предназначены именно для того, чтобы можно было проще справиться с подобными осложнениями.

Один из очень простых подходов к созданию системы поддержки истинности состоит в том, чтобы следить за порядком, в котором высказывания вводятся в базу знаний, путем присваивания им номеров от $P_{\pm}$ до P_n . После того как формируется вызов $\text{Retract}(XB, P_i)$, система возвращается к состоянию, непосредственно предшествующему добавлению высказывания P_i ? удаляя тем самым и p_i и любые результаты логического вывода, полученные на основании $p_{\pm}$. После этого могут быть снова добавлены высказывания от p_{i+1} до P_n . Такая организация работы является простой и гарантирует, что база знаний всегда будет оставаться непротиворечивой, но для извлечения PL требуется извлечение и повторная вставка $n-i$ высказываний, а также отмена и повторное выполнение всех логических выводов, вытекающих из этих высказываний. Для систем, в которые происходит добавление многих фактов (таких как крупные коммерческие базы данных), указанный подход является практически не применимым.

Более эффективный подход состоит в создании системы поддержки истинности на основе обоснований, или системы $\wedge$ JTMS (Justification-Based Truth Maintenance System). В системе JTMS к каждому высказыванию в базе знаний прилагается аннотация в виде Эч обоснования, состоящего из множества высказываний, на основании которых было выведено это высказывание. Например, если база знаний уже содержит высказывание $P \Rightarrow Q$, то операция $\text{Tell}(P)$ вызовет добавление Q с обоснованием $\{P, P \Rightarrow Q\}$. Вообще говоря, высказывание может иметь любое количество обоснований. Обоснования используются для обеспечения эффективного извлечения. После выполнения вызова $\text{Retract}(P)$ система JTMS удалит такие и только такие высказывания, для которых P является элементом каждого обоснования.

Поэтому, если высказывание Q имеет единственное обоснование $\{P, P \Rightarrow Q\}$, оно будет удалено; если имеет также дополнительное обоснование $\{P, P \vee R \Rightarrow Q\}$, оно также будет удалено; но если, кроме этого, имеет обоснование $\{R, P \vee R \Rightarrow Q\}$, оно будет сохранено. Таким образом, время, требуемое для извлечения высказывания P , зависит только от количества высказываний, полученных на основании P , а не от количества других высказываний, добавленных после того, как P было введено в базу знаний.

В системе JTMS предполагается, что высказывания, которые уже когда-то рассматривались, по-видимому, будут рассматриваться снова, поэтому вместо полного удаления из базы знаний некоторого высказывания после того, как оно теряет все обоснования, это высказывание просто отмечается как находящееся вне базы знаний. Если же какое-то последующее утверждение восстанавливает одно из обоснований, то это высказывание снова отмечается как находящееся внутри базы.

Благодаря этому система JTMS позволяет сохранить все цепочки логического вывода, которые в ней используются, и не нуждается в повторном выводе высказываний после того, как некоторое обоснование вновь становится действительным.

Кроме успешного исключения из базы знаний неправильной информации, системы поддержки истинности могут также использоваться для ускорения анализа многочисленных гипотетических ситуаций. Предположим, например, что Олимпийский комитет Румынии выбирает площадки для проведения соревнований по плаванию, легкой атлетике и конному спорту для олимпийских игр 2048 года, которые должны проводиться в Румынии. Например, допустим, что первой гипотезой является следующая: Site {Swimming, Pi test}, Site {Athletics, Bucharest} и Site {Equestrian, Arad}. В таком случае необходимо провести большой объем рассуждений, чтобы определить логические следствия, а значит и целесообразность этого выбора. Если же вместо этого потребуется рассмотреть вариант Site {Athletics, Sibiu}, то такая система TMS позволит избавиться от необходимости начинать всю эту работу с нуля. Вместо этого достаточно будет просто извлечь гипотезу Site {Athletics, Bucharest} и ввести Site {Athletics, Sibiu}, после чего система TMS возьмет на себя весь необходимый пересмотр. А цепочки логического вывода, выработанные на основании выбора площадки в Бухаресте, могут повторно использоваться для Сибиу, при условии, что логические заключения остаются теми же самыми.

Особенно эффективное переключение контекста между гипотетическими мирами обеспечивает система поддержки истинности на основе предположения, или $\wedge$ ATMS (Assumption-based Truth Maintenance System), которая предназначена именно для этой цели. В системе JTMS средства сопровождения обоснований позволяют быстро переходить от одного состояния к другому, выполнив лишь небольшой объем извлечений и вставок, но в них в любой момент времени представлено только одно состояние. А в системе ATMS представлены все состояния, которые когда-либо рассматривались одновременно. Это означает, что в системе JTMS каждое высказывание обозначается как находящееся вне или внутри базы знаний, тогда как в системе ATMS для каждого высказывания отслеживается, какие предположения могли бы вынудить это высказывание стать истинным. Иными словами, каждое высказывание имеет метку, состоящую из множества множеств предположений. Это высказывание становится истинным только в том случае, если истинными являются все предположения в одном из множеств предположений.

Системы поддержки истинности предоставляют также механизм выработки 3ч объяснений. Формально объяснением высказывания p является такое множество высказываний E , что из E следует P . Если уже известно, что высказывания E истинны, то E просто предоставляет достаточную базу для доказательства того, что p также

является таковым. Но объяснения могут также включать "&". предположения — высказывания, в отношении которых неизвестно, являются ли они истинными, но которые были бы достаточными, чтобы доказать истинность P , если бы они были истинными. Например, некто может не иметь достаточной информации, чтобы доказать, что двигатель его автомобиль не запустится, но обоснованное объяснение может включать предположение, что аккумулятор разряжен. Это позволяет объяснить наблюдаемое неправильное поведение двигателя, в сочетании со знаниями о том, как

работает двигатель автомобиля. В большинстве случаев предпочтительным является минимальное объяснение E под этим подразумевается, что не существует строгое подмножество E , которое также было бы объяснением. Система ATMS может сформировать объяснения для проблемы "двигатель автомобиля не запускается", делая предположения (такие как "в карбюратор попал бензин" или "разряжен аккумулятор") в любом желательном для пользователя порядке, даже если некоторые предположения противоречат друг другу. После этого достаточно посмотреть на метку, предусмотренную для высказывания "двигатель автомобиля не запускается", чтобы ознакомиться с множествами предположений, которыми могло бы оправдываться это высказывание.

Точные алгоритмы, используемые для реализации систем поддержки истинности, являются довольно сложными, и в этой главе они не рассматриваются.

Вычислительная сложность задачи поддержки истинности является, по меньшей мере, такой же, какая характерна для пропозиционального логического вывода, т.е. NP-трудной.

Поэтому не следует рассчитывать на то, что подход на основе поддержки истинности окажется панацеей. Однако при их продуманном использовании системы TMS могут обеспечить существенное повышение способности логических систем действовать в условиях применения сложных гипотез и вариантов среды.

10.9. Резюме Эта глава оказалась наиболее подробной по сравнению со всеми предыдущими главами данной книги. Описывая детали того, какие способы используются для представления различных знаний, авторы надеялись дать читателю возможность понять, как создаются реальные базы знаний. Основные идеи, высказанные в этой главе, перечислены ниже. • Для крупномасштабного представления знаний требуется онтология общего назначения, позволяющая организовать и связать воедино различные специализированные области знаний. • Онтология общего назначения должна охватывать широкий круг знаний и быть способной, в принципе, представить любую проблемную область. • В главе описана верхняя онтология, основанная на категориях и исчислении событий. Здесь рассматривались структурированные объекты, время и пространство, изменения,

процессы, вещества и убеждения. • Действия, события и время могут быть представлены либо с помощью ситуационного исчисления, либо с применением более выразительных средств представления, таких как исчисление событий и исчисление флюентных высказываний. Такие представления дают возможность агенту составлять планы с помощью логического вывода. • Мыслительные состояния агентов могут быть представлены строками, которые описывают их убеждения. • В главе представлен подробный анализ проблемной области совершения покупок в Internet, исследована общая онтология и показано, как знания проблемной области могут использоваться торговым агентом.

• Описаны системы представления общего назначения, такие как семантические сети и описательные логики, позволяющие организовывать иерархии категорий. С этими системами связана важная форма логического вывода, называемая наследованием, позволяющая определять логическим путем свойства объектов на основании данных об их принадлежности к категориям. • Предположение о замкнутом мире, будучи реализованным в логических программах, предоставляет простой способ избежать необходимости задавать большие объемы отрицательной информации. Такую информацию проще всего интерпретировать как заданную по умолчанию, которая может быть переопределена с помощью дополнительной информации. • В целом для предоставления возможности осуществлять формирование рассуждений по умолчанию предназначены немонотонные логики, такие как логика косвенного описания и логика умолчания. Применение программирования множества ответов позволяет ускорить немонотонный логический вывод, во многом аналогично тому, как использование алгоритма WalkSAT ускоряет пропозициональный логический вывод. • Системы поддержки истинности позволяют эффективно осуществлять обновления и пересмотры баз знаний.

Библиографические и исторические заметки Многие исследователи вполне обосновано утверждают [186], что исследования в области формального представления знаний начались с классических работ индийских теоретиков, посвященных грамматике шастрических текстов на санскрите, которые относятся к первому тысячелетию до н.э. На Западе к числу самых первых примеров таких исследований относится использование определений терминов в трудах древнегреческих математиков. Безусловно, как форма представления знаний может также рассматриваться разработка технической терминологии в любой области.

На первых порах дискуссии на тему представления в искусственном интеллекте в основном сосредоточивались на "представлении задач", а не на "представлении знаний" (см., например, приведенное Амарелем [24] обсуждение задачи с миссионерами и каннибалами). В 1970-х годах основной акцент в искусственном интеллекте был сделан на разработке "экспертных систем" (называемых также

"системами, основанными на знаниях"), которые были способны при наличии соответствующих знаний в проблемной области достичь или превзойти производительность людей-экспертов при решении узко определенных задач. Например, даже первая экспертная система, Dendral [458], [936], интерпретировала выходные данные массового спектрометра (прибора, применяемого для анализа структуры органических химических веществ) столь же точно, как и опытные химики. Хотя успех системы Dendral позволил сообществу исследователей по искусственному интеллекту убедиться в важности представления знаний, формальные средства представления, используемые в Dendral, были в высшей степени специализированными и относящимися только к данной проблемной области химии. Со временем у исследователей появилось стремление к стандартизации формальных средств представления знаний и онтологии, что позволило бы упростить процесс создания новых экспертных систем.

В ходе решения этой проблемы им пришлось проникнуть на территорию, которая до сих пор исследовалась только философами, изучающими проблемы науки и языка.

Принятое в искусственном интеллекте требование, согласно которому предлагаемые теории должны "работать", привело к более быстрому и глубокому прогрессу в решении проблем по сравнению с той ситуацией, когда эти проблемы рассматривались как принадлежащие исключительно к области философии (хотя такой подход время от времени приводил к "повторному изобретению колеса").

Первые попытки создания всеобъемлющих таксономии, или классификаций, относятся к временам древности. Применения правильных схем классификации и категоризации настоятельно требовал Аристотель (384-322 до н.э.). В Органон, коллекцию трудов Аристотеля по логике, собранную учениками Аристотеля после его смерти, включен трактат под названием Категории, в котором Аристотель попытался создать то, что мы теперь называем верхней онтологией. Он также ввел понятия родов и видов для классификации нижнего уровня, хотя и не в их современном, сугубо биологическом смысле. Современная система биологической классификации, включая использование "биномиальной номенклатуры" (в формальном смысле слова — классификации по родам и видам), была предложена шведским биологом Карлом Линнеем (1707-1778). Проблемы, связанные с естественными разновидностями и неточными границами категорий, кроме многих других работ, рассматривались в [882], [1252], [1372] и [1608].

В последнее время продолжает расти интерес к крупномасштабным онтологиям.

В проекте CYC [908], [910] была разработана верхняя онтология с 6000 понятий, состоящая из 60 000 фактов, а также лицензирована гораздо более крупная глобальная онтология. Организацией ШЕЕ был создан подкомитет P1600.1,

получивший название "Рабочей группы по стандарту верхней онтологии" (Standard Upper Ontology Working Group), а организация Open Mind Initiative привлекла к работе больше 7000 пользователей Internet, собрав свыше 400 000 фактов об обыденных понятиях. В среде Web появились такие стандарты, как RDF, XML и Semantic Web [ПО], хотя они еще не нашли широкого распространения. Много интересных статей в области общих и специализированных онтологии публикуется по материалам конференции Formal Ontology in Information Systems (FOIS).

Таксономия, используемая в этой главе, разработана авторами данной книги и частично основана на опыте их участия в проекте CYC, а также на работах Хванга, Шуберта [714] и Дэвиса [1332]. Воодушевляющее описание общего проекта представления обыденных знаний приведено в книге Хейса The Naive Physics Manifesto [634], [637].

Проблемы представления времени, изменений, действий и событий широко исследовались не только в искусственном интеллекте, но и в философии и теоретических компьютерных науках. К числу наиболее развитых направлений относится временная логика, представляющая собой специализированную логику, в которой каждая модель описывает полную траекторию развития процесса во времени (обычно либо линейную, либо ветвящуюся), а не просто статическую реляционную структуру. Эта логика включает модальные операторы, применяемые к формулам; $\Box$ означает "р будет истинным во все времена в будущем", а $\Diamond$ означает "р станет истинным в некоторый момент в будущем". Исследования в области временной логики были впервые начаты Аристотелем, а также представителями мегарской и стоической школ в древней Греции. В современной истории развития науки применение формального исчисления для проведения рассужде-

ний о времени было впервые предложено Финдлеем [468], но считается, что наибольшее влияние оказала работа Артура Прайора [1239]. К учебникам по временной логике относятся [1283] и [1529].

Специалисты в области теоретических компьютерных наук уже давно стремились формально описать свойства программ, рассматриваемых как последовательности вычислительных действий. Бурстолл [210] выдвинул идею использования модальных операторов для формирования рассуждений о компьютерных программах.

Вскоре после этого Воган Пратт [1233] разработал динамическую логику, в которой модальные операторы обозначают результаты выполнения программ или другие действия (см. также [620]). Например, в динамической логике, если a — имя программы, то " $[a]p$ " означает "р должно быть истинным во всех состояниях мира, ставших результатом выполнения программы a в текущем состоянии мира", а " $\langle a \rangle p$ "

означает "р должно быть истинным по меньшей мере в одном из состояний мира, ставшим результатом выполнения программы а в текущем состоянии мира".

Динамическая логика была применена для фактического анализа программ Фишером и Ладнером [472]. В [1218] предложена идея использования классической временной логики для формирования рассуждений о программах.

Временная логика предусматривает введение времени непосредственно в теорию моделей языка, а методы представления времени, применяемые в искусственном интеллекте, как правило, предусматривали явное включение аксиом об интервалах времени и событиях в базу знаний, поэтому время не получало особого статуса в этой логике. Кроме того, данный подход в некоторых случаях обеспечивает большую ясность и гибкость. К тому же знания о времени, выраженные в логике первого порядка, могут быть более легко интегрированы с другими знаниями, накопленными в рассматриваемой проблемной области.

Первой попыткой представления времени и действий в искусственном интеллекте стало ситуационное исчисление, предложенное Джоном Маккарти [1010]. Первой системой искусственного интеллекта, в которой широко применялись рассуждения общего назначения о действиях в логике первого порядка, была система QA3 [592].

Ковальский [851] развил идею о веществе высказываний в рамках ситуационного исчисления.

Проблема окружения была впервые описана в [1013]. Многие исследователи считали эту проблему неразрешимой в логике первого порядка, и она стала стимулом для проведения большого объема исследований в области немонокотных логик.

Многие философы, начиная от Дрейфуса [414] и заканчивая Крокеттом [310], указывали, что проблема окружения является одним из симптомов неизбежного краха всего направления работ по созданию искусственного интеллекта. Частичное решение проблемы представительного окружения с использованием аксиом состояния-преемника было предложено Рэем Рейтером [1277]; истоки решения проблемы выводимого окружения прослеживаются в [671], в которой были предложены методы, получившие название исчисления флюентных высказываний [1504].

Описание, приведенное в настоящей главе, частично основано на результатах анализа, изложенных в [931] и [1504]. Книги Шенахана [1391] и Рейтера [1279] содержат полное, современное изложение вопросов формирования рассуждений о действиях в ситуационном исчислении.

Частичное разрешение проблемы окружения снова пробудило стремление к использованию декларативного подхода для формирования рассуждений о действиях,

что привело к созданию многих систем планирования специального назначения, начиная с первой половины 1970-х годов (см. главу 11). В исследованиях, проводимых в рамках "8. когнитивной робототехники, был достигнут большой прогресс в области разработки средств логического представления действий и времени. В языке Golog используется полная выразительная мощь логического программирования для описания действий и планов [917], к тому же этот язык был дополнен, для того чтобы в нем можно было представлять параллельные действия [550], стохастические варианты среды [160] и результаты восприятия [1278].

Для создания средств представления непрерывного времени было предложено использовать исчисление событий, разработанное Ковальским и Серготом [854], а в дальнейшем появилось еще несколько вариантов исчисления событий [1341].

В [1392] представлен хороший краткий обзор. Джеймс Аллен предложил использовать для этой же цели временн/е интервалы [15], [16], указывая, что интервалы являются гораздо более естественным средством формирования рассуждений о продолжительных и одновременных событиях, чем ситуации. В [877], [878] впервые предложены "вогнутые" временн/е интервалы (интервалы с перерывами; по сути объединения обычных "выпуклых" временных интервалов) и для представления времени применены математические методы абстрактной алгебры. Аллен [17] систематически исследовал широкий спектр методов, которые могут применяться для представления времени. Шохем [1403] описал процедуру о веществления событий и предложил использовать для этой цели разработанную им новейшую схему. Между онтологией на основе событий, приведенной в данной главе, и анализом событий, выполненным философом Дональдом Давидсоном [328], имеются весьма важные аналогии. К тому же типу относятся хронологии (history), предложенные в работе Патрика Хейса [636]; аналогичной разновидностью представления во многом является также онтология вневременных событий (liquid event).

Проблемы онтологического статуса веществ имеют длинную историю. Платон считал, что вещества — это абстрактные сущности, полностью отличающиеся от физических объектов; с его точки зрения следовало бы сказать, что кусок масла сделан из масла, `MadeOf (Butter3, Butter)`, а не что кусок масла является элементом множества масла, `Butter3 e Butter`. Такая идея ведет к созданию иерархии веществ, в которой, например, несоленое масло `UnsaltedButter` является более конкретно определенным веществом, чем само масло `Butter`. Научная позиция, принятая в этой главе, согласно которой вещества представляют собой категорию объектов, была обоснована Ричардом Монтегю [1072]. Кроме того, эта позиция была принята и в

проекте СYC. В [293] на эту позицию предпринята серьезная, но не настолько уж неотразимая атака. Альтернативный подход, упомянутый в данной главе, согласно которому масло представляет собой единственный объект, состоящий из всех маслоподобных объектов во вселенной, был первоначально предложен польским логиком Лесьневским [913]. В разработанной им "&. мереологии (это название происходит от греческого слова, обозначающего "часть") используется отношение "часть-целое" в качестве замены математической теории множеств, в целях устранения таких абстрактных сущностей, как множества. Более удобное для чтения изложение этих идей приведено в [911], а в книге Гудмана *The Structure of Appearance* [579] эти идеи применяются для решения различных проблем в области представления знаний. Хотя в некоторых аспектах мереологический подход является весьма громоздким (например, в нем требуется отдельный механизм наследования, осно-

ванный на отношениях "часть-целое"), он получил поддержку Квайна [1253]. Гарри Бант [207] провел широкий анализ перспектив использования этого подхода в области представления знаний.

Мыслимые объекты и мыслительные состояния были предметом интенсивных исследований в области философии и искусственного интеллекта. ^ Модальная логика представляет собой классический метод формирования рассуждений о знаниях, применяемый в философии. В модальной логике логика первого порядка дополняется модальными операторами, такими как *B* (от *believes*— убежден) и *K* (от *knows* — знает), которые принимают в качестве своих параметров высказывания, а не термы. В теории доказательств для модальной логики жестко регламентируются подстановки в пределах модальных контекстов, что позволяет обеспечить ссылочную непрозрачность. Модальная логика знаний была предложена Яакко Хинтикой [654]. Саул Крипке [859] определил семантику модальной логики знаний в терминах возможных миров. Грубо говоря, мир для агента является возможным, если не противоречит всему, что знает агент. На основании такого подхода могут быть разработаны правила логического вывода с использованием оператора *K*. Роберт К.

Мур связал модальную логику знаний с таким стилем формирования рассуждений о знаниях, который позволяет непосредственно ссылаться на возможные миры в логике первого порядка [1080], [1081]. Модальная логика может на первый взгляд показаться устрашающе запутанной областью науки, но она нашла очень важные применения в области формирования рассуждений об информации в распределенных компьютерных системах. В книге Фейгина и др. *Reasoning about Knowledge* [449] приведено исчерпывающее введение в этот модальный подход. Приложения теории знаний в искусственном интеллекте, экономике и

распределенных системах обсуждаются на проводимой один раз в два года конференции Theoretical Aspects of Reasoning About Knowledge (TARK).

Синтаксическая теория мыслимых объектов была впервые глубоко исследована Капланом и Монтегю [770], которые показали, что эта теория может приводить к парадоксам, если при использовании ее средств не соблюдается чрезвычайная осторожность. Поскольку такая теория позволяет создавать естественные модели физических конфигураций компьютера или мозга в терминах убеждений, она в последние годы получила широкое распространение в области искусственного интеллекта.

В [604] и [833] эта теория использовалась для описания машин логического вывода с ограниченной мощностью, а Моргенштерн [1086] показал, как можно ее использовать для описания предусловий знаний в планировании. Методы планирования таких действий, как наблюдение, описанные в главе 12, основаны на этой синтаксической теории. Эрни Дэвис [332] дал превосходное сравнение синтаксической и модальной теорий знания.

Греческий философ Порфирий (ок. 234—305 н.э) в своих комментариях к трактату Аристотеля Категории продемонстрировал то, что может рассматриваться как первая семантическая сеть. Чарльз С. Пирс [1199] разработал экзистенциальные графы, которые могут рассматриваться как первое формальное определение семантической сети с использованием современной логики. Инициатором исследований по семантическим сетям, проводимых в рамках искусственного интеллекта, был Росс Квиллиан [1251], основным стимулом для которого был интерес к человеческой памяти и языковой обработке. В важной статье Марвина Минского [1053] представлена одна из версий семантических сетей, основанная на использовании

так называемых фреймов; фреймы служили для представления объектов или категорий и характеризовались наличием атрибутов и отношений с другими объектами или категориями. Хотя эта статья послужила важной причиной пробуждения интереса к области представления знаний как таковой, она подверглась критике за то, что в ней просто под другим углом были изложены идеи статей [132], [320], разработанные ранее в объектно-ориентированном программировании, такие как наследование и использование заданных по умолчанию значений. Однако еще не совсем ясно, в какой степени на указанные статьи по объектно-ориентированному программированию, в свою очередь, повлияли еще более ранние работы в области искусственного интеллекта, посвященные семантическим сетям.

Проблемы семантики приобрели особую остроту применительно к семантическим сетям, разработанным Квиллианом (и теми, кто стал последователем

предложенного им подхода), в связи с тем, что в них использовались вездесущие и весьма неопределенные "связи IS-A", а также применительно к другим ранним формальным системам представления знаний, таким как Merlin [1079], с ее загадочными операциями "flat" и "cover". Знаменитая статья Вудса "What's In a Link?" [1613] привлекла внимание исследователей в области искусственного интеллекта к тому, что в формальных системах представления знаний должна быть точно определена семантика. Брачман [167] провел исследования по этой проблеме и предложил некоторые решения. Патрик Хейс в своей книге The Logic of Frames [635] провел еще более глубокие исследования и сформулировал утверждение, что "так называемые «фреймы» по большей части представляют собой просто новые синтаксические обозначения для фрагментов логики первого порядка". Дрю Макдермотт в своей работе Tarskian Semantics, or, No Notation without Denotation! [1022] доказывал, что модельнотеоретический подход к семантике, используемый в логике первого порядка, должен быть распространен на все формальные системы представления знаний. Однако эта идея является внутренне противоречивой; замечательно то, что сам Макдермотт пересмотрел свою позицию в работе L Critique of Pure Reason [1023]. Проявлением нового подхода стала NetI [451], сложная система семантической сети, в которой связи IS-A (называемые связями "виртуальной копии", или VC — virtual copy) были основаны в большей степени на понятии характеристик "наследования" систем фреймов или объектно-ориентированных языков программирования, чем на отношении подмножества, и были определены гораздо более точно по сравнению со связями, применявшимися Квиллианом в эпоху, предшествовавшую появлению работ Вудса.

Система NetI привлекла особое внимание, поскольку предназначалась для реализации в параллельном аппаратном обеспечении для преодоления сложностей выборки информации из больших семантических сетей. Дэвид Турецкий [1512] подверг понятие наследования строгому математическому анализу. В [1381] обсуждаются сложности наследования с исключениями и показано, что в большинстве формулировок задача представления наследования является NP-полной.

Разработка описательных логик представляет собой один из наиболее современных этапов в длинной цепи исследований, нацеленных на поиск полезных подмножеств логики первого порядка, для которой задача логического вывода является осуществимой с помощью вычислительных средств. Гектор Левеск и Рон Брачман [916] показали, что неразрешимость логического вывода в основном возникает из-за использования определенных логических конструкций (особенно отличаются этим и оторые варианты использования дизъюнкции и отрицания). На основе системы

KL-One [1362] был разработан целый ряд систем, в проектах которых применены результаты теоретического анализа сложности; к числу наиболее известных из них относятся Krypton [168] и Classic [155]. Полученные при этом результаты продемонстрировали заметное повышение скорости логического вывода и позволили гораздо лучше понять зависимости между сложностью и выразительностью в системах формирования рассуждений. Общий итог современного состояния дел в этой области подведен в [215]. Протестуя против этой тенденции к упрощению, Дойл и Петил [411] доказывали, что ограничение выразительности языка либо исключает возможность решения некоторых проблем, либо побуждает пользователя обходить ограничения языка с помощью нелогических средств.

Все три основные формальные системы, предназначенные для использования в немонотонном логическом выводе, — косвенное описание [1012], логика умолчания [1276] и модальная немонотонная логика [1025] — были предложены в одном специальном выпуске *AI Journal*. Программирование множества ответов может рассматриваться как расширение понятия отрицания как недостижения цели или как уточнение понятия косвенного описания; основополагающая теория семантики стабильных моделей была предложена в [533], а ведущими системами программирования множества ответов являются DLV [432] и Smodels [1138]. Пример с дисковым накопителем взят из руководства пользователя Smodels [1484]. В [929] обсуждается использование программирования множества ответов для планирования. В [184] приведен хороший краткий обзор различных подходов к использованию немонотонной логики. В [262] рассматривается подход к логическому программированию, основанный на отрицании как недостижении цели, и анализируется дополнение Кларка. Ван Эмден и Ковальский [1530] показали, что каждая программа Prolog без отрицаний имеет уникальную минимальную модель. Последние годы характеризуются повторным пробуждением интереса к приложениям немонотонных логик в крупномасштабных системах представления знаний. По-видимому, первой системой, достигшей коммерческого успеха, является система Benlq для обработки данных опросов, касающихся страховых льгот; эта система представляет собой приложение системы немонотонного наследования [1087]. В [929] обсуждается применение программирования множества ответов в планировании. Целый ряд систем формирования немонотонных рассуждений, основанных на логическом программировании, описан в трудах конференции *Logic Programming and Nonmonotonic Reasoning (LPNMR)*.

Исследования в области систем поддержки истинности начались с создания систем TMS [409] и RUP [1003], которые по существу представляли собой системы JTMS. Подход на основе систем ATMS был описан в ряде статей Йохана де Клеера [345-347]. В работе *Building Problem Solvers* [479] подробно объяснено, как могут

применяться системы TMS в приложениях искусственного интеллекта. В [1117] показано, что использование эффективной системы TMS обеспечило возможность планирования операций космического корабля агентства NASA в реальном времени.

По очевидным причинам в данной главе подробно не рассматривается каждая область представления знаний. Ниже описаны три основные темы, которые в ней не представлены. • ^ Качественная физика. Это подобласть представления знаний, которая главным образом относится к формированию логической, нечисловой теории фи-

зических объектов и процессов. Этот термин был предложен Йоханом де Клеером [344], хотя вполне можно считать, что это направление исследований началось с создания Фалманом [450] программы Build— развитого планировщика для построения сложных башен из блоков. В процессе проектирования этой программы Фалман открыл, что основная часть усилий (по его оценке, 80%) уходит на моделирование физических свойств мира блоков в целях вычисления устойчивости различных субфрагментов структур блоков, а не на само планирование как таковое. Он сформулировал набросок гипотетического процесса, подобного проведению рассуждений в рамках наивной физики, для объяснения причин того, почему дети младшего возраста способны решать задачи, подобные рассматриваемым в программе Build, без обращения к быстродействующей арифметике с плавающей точкой, которая используется в физическом моделировании программой Build. Хейс [636] применил "истории" (четырёхмерные фрагменты пространства-времени, аналогичные событиям Дэвидсона) для построения довольно сложной наивной физической теории жидкостей. Хейсу впервые удалось доказать в рамках искусственного интеллекта, что ванна с заткнутым сливом в конечном итоге переполнится, если вода из крана будет продолжать бежать, и что человек, упавший в озеро, весь вымокнет. В [349] и [478] описаны попытки создать нечто подобное общей теории физического мира на основе качественных абстракций физических уравнений. За последние годы качественная физика была разработана до такой степени, что с ее помощью стало возможным анализировать сложные физические системы, разнообразие которых весьма впечатляет [1340], [1631].

Качественные методы использовались для разработки новейших проектов часов, дворников для ветрового стекла и шестиногих шагающих машин [1470], [1472]. Хорошим введением в эту область является сборник статей Readings in Qualitative Reasoning about Physical Systems [1570]. • ^ Пространственные рассуждения. Рассуждения, необходимые для навигации в мире вампуса и в мире совершения покупок, являются тривиальными по сравнению с теми, что требуются в богатой пространственной структуре реального мира. Результаты самых первых серьезных попыток описать обыденные рассуждения о пространстве приведены в работах

Эрнеста Дэвиса [331], [332]. В исчислении региональных связей Кона и др. [280] поддерживается определенная форма качественных пространственных рассуждений; эти исследования привели к созданию географических информационных систем нового типа. Как и с помощью качественной физики, агент, так сказать, многое может сделать без обращения к полному метрическому представлению. А если такое представление становится необходимым, то можно воспользоваться методами, разработанными в робототехнике (глава 25). • ^ Психологические рассуждения. Психологические рассуждения касаются проблемы разработки для искусственных агентов работоспособных психологических моделей, которые могли бы использоваться агентами для формирования рассуждений о себе и о других агентах. Такие рассуждения часто основаны на так называемой народной психологии (folk psychology) — теории, претендующей на понимание того, какие принципы используются людьми в рассуждениях о себе и других людях. Когда исследователи в области искусст-

венного интеллекта снабжают своих искусственных агентов психологическими теориями для формирования рассуждений о других агентах, эти теории часто основаны на описании исследователями проектов самих этих логических агентов. Поэтому в настоящее время наиболее продуктивны такие исследования в области психологических рассуждений в контексте понимания естественного языка, где наибольшую важность имеет предугадывание намерений говорящего.

Наиболее современными источниками сведений о работах в этих областях являются труды международной конференции Principles of Knowledge Representation and Reasoning. В работах Readings in Knowledge Representation [169] и Formal Theories of the Commonsense World [665] приведены превосходные антологии по представлению знаний; в первой в большей степени обсуждаются статьи, имевшие историческое значение в языках и формальных системах представления, а последняя посвящена накоплению самих знаний. В [332], [1448] и [1458] приведены учебные введения в проблематику представления знаний.

Упражнения 10.1. Запишите высказывание, позволяющее определить результаты действия Shoot в мире вампуса. Опишите эти результаты применительно к вампусу, но не забудьте, что после выстрела у агента больше нет стрелы.

10.2. С помощью ситуационного исчисления запишите аксиому, чтобы связать время 0 с ситуацией S0, и еще одну аксиому, чтобы связать время t с любой ситуацией, которая развилась из S0 в результате последовательности из t действий.

10.3. В данном упражнении рассматривается проблема планирования маршрута для робота, чтобы он мог добраться из одного города в другой. Основным действием, предпринимаемым роботом, является действие Go(x, y), которое позволяет ему добраться из города x в город y, если между этими городами есть прямой путь.

Предикат $\text{DirectRoute}(x,y)$ принимает истинное значение тогда и только тогда, когда есть прямой путь из x в y , вы можете принять предположение, что все соответствующие факты уже находятся в базе знаний (см. карту на с. 114). Робот начинает движение с Арада и должен достичь Бухареста. а) Запишите подходящее логическое описание начальной ситуации для робота. б) Запишите подходящий логический запрос, решения которого могут служить описаниями возможных путей к цели. в) Запишите высказывание, описывающее действие Go. г) Теперь предположим, что, следуя по прямому пути между двумя городами, агент тратит топливо, количество которого равно расстоянию между этими городами. Робот начинает движение с полной загрузкой топлива.

Дополните разработанное вами представление с учетом этих соображений. Предложенное вами описание действий должно быть таковым, чтобы сформулированный вами ранее запрос все еще приводил к получению осуществимых планов.

д) Опишите начальную ситуацию и запишите новое правило (или правила), определяющее действие Go. е) Теперь допустим, что в некоторых узлах маршрута имеются также бензозаправочные станции, в которых робот может пополнить свой топливный бак. Дополните разработанное вами представление и запишите все правила, необходимые для описания бензозаправочных станций, включая действие Fillup (Заправка).

10.4. Исследуйте способы дополнения исчисления событий, позволяющие учитывать в нем одновременные события. Возможно ли избежать комбинаторного взрыва количества аксиом?

10.5. Представьте приведенные ниже семь высказываний, используя и дополняя средства представления, описанные в этой главе. а) Вода находится в жидком состоянии при температуре от 0 до 100 градусов Цельсия. б) Вода кипит при температуре 100 градусов Цельсия. в) Вода в бутылке для воды, принадлежащей Джону, замерзла. г) Минеральная вода "Перрье" — это тоже вода. д) Джон держит в своей бутылке для воды минеральную воду "Перрье". е) Все жидкости имеют определенную температуру замерзания. ж) Литр воды весит больше, чем литр алкоголя.

Теперь повторите это упражнение с использованием представления, основанного на метеорологическом подходе, в котором, например, Water — это объект, содержащий в качестве частей всю воду в мире.

10.6. Запишите определения для следующих предикатов: а) ExhaustivePartDecomposition б) PartPartition в) PartwiseDisjoint Эти определения должны быть аналогичными определениям для предикатов ExhaustiveDecomposition, Partition и Disjoint. Является ли, по вашему мнению, общезначимым выражение $\text{PartPartition}\{s, \text{BunchOf}$

(s)) ? Если да, докажите его общезначимость; в противном случае приведите контрпример и определите достаточные условия, при которых это выражение становится истинным.

10.7. Запишите множество высказываний, которое позволяет рассчитать цену отдельного помидора (или другого объекта), если указана цена за килограмм. Дополните эту теорию, чтобы иметь возможность рассчитать цену пакета помидоров.

10.8. Одна из альтернативных схем представления мер предусматривает применение функции единиц к абстрактному объекту длины. В такой схеме следует писать $\text{Inches}(\text{Length}(L \pm)) = 1.5$. Как выглядит эта схема по сравнению с той, что описана в данной главе? Заслуживают внимания такие вопросы, как аксиомы преобразования, имена для абстрактных количеств (такие как 0 долларов") и сравнения абстрактных мер в различных единицах (например, 0 дюймов больше 50 сантиметров").

10.9. Разработайте средство представления для курсов обмена валют, которое допускает ежесуточные колебания курсов.

10.10. Это упражнение касается связей между категориями событий и интервалами времени, в которые они происходят. а) Определите предикат $T(cf\ i)$ в терминах отношений During kg . б) Дайте точное объяснение причин, по которым не требуются две различные системы обозначений для описания конъюнктивных категорий событий. в) Дайте формальное определение для предикатов $T\{\text{OneOf}(p,q),i\}$ и $T(\text{Either}(p,q), i)$. г) Объясните, почему целесообразно иметь две формы отрицания событий, аналогичные двум формам дизъюнкции событий. Назовите их *Not* и *Never* и дайте им формальные определения.

10.11. Определите предикат *Fixed*, где $\text{Fixed}(\text{Location } (x))$ означает, что местонахождение объекта *x* постоянно во времени.

10.12. Определите предикаты *Before*, *After*, *During* и *Overlap* с использованием предиката *Meet* и функции *Start* и *End*, но не функции *Time* или предиката $<$.

10.13. В разделе 10.5 предикаты *Link* и *LinkText* использовались для описания связей между Web-страницами. Запишите определения для *Link* и *LinkText*, используя, кроме всего прочего, предикаты *Has* и *GetPage*.

10.14. Одна из составляющих процесса совершения покупок, которая не рассматривалась в этой главе, касается проверки совместимости товаров. Например, если клиент заказывает компьютер, будет ли этот компьютер совместимым с имеющимися у него периферийными устройствами? Если же оформляется заказ на цифровую камеру, то будут ли к ней прилагаться подходящие плата памяти и аккумулятор? Разработайте базу знаний, позволяющую принимать

решение о том, является ли множество товаров совместимым, которая может использоваться для предоставления консультаций, касающихся замены или приобретения дополнительных товаров, если имеющиеся товары являются несовместимыми. Убедитесь в том, что эта база знаний может применяться по меньшей мере с одной номенклатурой товаров и что ее несложно дополнить для использования с другими номенклатурами.

10.15. Введите правила, позволяющие расширить определение предиката Name (s, c) так, чтобы строка, подобная "laptop computer", согласовывалась с соответствующей категорией имен, применяемых в самых различных магазинах. Попытайтесь добиться того, чтобы предложенное вами определение стало наиболее общим.

Проверьте его, изучив десять оперативных магазинов и ознакомившись с тем, какие имена в них присвоены трем различным категориям. Например, применительно к категории портативных компьютеров авторы обнаружили имена "Notebooks", "Laptops", "Notebook Computers", "Notebook", "Laptops and Notebooks" и "Notebook PCs". Некоторые из них могут быть описаны с помощью явно заданных фактов Name, а для других могут применяться правила образования множественного числа, оформления перечислений и т.д.

10.16. Полное решение проблемы неточных соответствий описаниям, сформулированным покупателем при совершении покупок, является очень сложным и требует применения полного спектра средств обработки естественного языка и методов выборки информации (см. главы 22 и 23). Одним из небольших шагов в этом направлении является предоставление пользователю возможности задавать минимальные и максимальные значения для различных атрибутов. Допустим, что мы требуем от покупателя, чтобы он использовал следующую грамматику для формулировки описаний товаров:

Description → Category [Connector Modifier]* Connector → "with" | "and" | ", "c" | "u"

Modifier → Attribute \ Attribute Op Value Op → "=" | ">" | "<"

Здесь Category обозначает категорию товара, Attribute— это некоторая характеристика, такая как "частота процессора" или "цена", а Value— желаемое значение этой характеристики. Поэтому запрос "компьютер с частотой процессора по меньшей мере 2,5 ГГц по цене меньше \$1000" должен быть выражен так: "компьютер с частотой процессора > 2,5 ГГц и ценой < \$1000".

Реализуйте торгового агента, который принимает описания на этом языке.

10.17. В нашем описании процесса осуществления покупок в Internet упущен наиболее важный этап — фактическая покупка товара. Предоставьте формальное логическое описание этапа покупки с использованием исчисления событий.

Иначе говоря, определите последовательность событий, которая происходит, когда покупатель приобретает товар по кредитной карточке, а затем в конечном итоге получает выставленный ему счет и ему вручается товар.

10.18. Опишите событие, в котором один объект обменивается на какой-то другой.

Опишите процесс покупки как разновидность обмена, в которой одним из объектов, участвующих в обмене, является некоторая денежная сумма.

10.19. Ш В двух предыдущих упражнениях подразумевается использование довольно примитивного понятия владения. Например, покупатель приступает к приобретению товара, овладев денежными купюрами. Такое описание процесса покупки становится неадекватным, если, например, деньги покупателя хранятся в банке, поскольку у него на руках больше нет какой-то конкретной совокупности денежных купюр, которыми он может немедленно распорядиться. Ситуация становится еще более сложной, если учесть возможность займа, сдачи в аренду, взятия в аренду и передачи в залог. Исследуйте различные обыденные и юридические понятия владения и предложите схему, с помощью которой их можно представить формально.

10.20. Вы должны разработать систему предоставления консультаций студентам отделения компьютерных наук в части того, какие курсы они должны пройти в течение некоторого продолжительного периода, чтобы выполнить условия учебной программы (исходите из тех требований, которые предъявляются в вашем учебном заведении). Прежде всего подготовьте словарь, позволяющий представить всю информацию, а затем представьте эту информацию; после этого примените подходящий запрос к системе, которая должна возвра-

тить в качестве решения приемлемую программу обучения. Вы должны предусмотреть в определенной степени необходимость учитывать подготовку отдельных студентов в том смысле, что система должна спрашивать, какие основные или эквивалентные курсы уже прошел студент, и не формировать программы, в которых повторяются эти курсы.

Предложите способы усовершенствования этой системы, например, для того, чтобы в ней учитывались сведения о предпочтениях студента, рабочей нагрузке, о том, какие преподаватели ему нравятся или не нравятся, и т.д.

Рассматривая каждую разновидность знаний, объясните, как она может быть представлена логически. Способна ли ваша система легко учитывать эту информацию и находить наилучшую программу обучения для студента?

10.21. На рис. 10.1 показаны верхние уровни иерархии для всех понятий. Дополните ее, чтобы включить максимально возможное количество реальных категорий.

Удобным способом выполнения этого требования является описание всего, с чем приходится сталкиваться в повседневной жизни. К этому относятся объекты и события. Начните с утреннего пробуждения и последовательно учитывайте все, что вы видите, чего касаетесь, что делаете и о чем думаете.

Например, случайно выбранный день может свестись к цепочке: музыка, новости, молоко, ходьба, поездка, заправка топливом, вестибюль университета, ковер, разговор, профессор Фейтмен, цыпленок под острым соусом, обожженный язык, 7 долларов, солнце, дневная газета и т.д.

Вы должны подготовить единственную схему иерархии (на большом листе бумаги), а также список объектов и категорий с отношениями, которым соответствуют элементы каждой категории. Каждый объект должен находиться в некоторой категории, а каждая категория должна относиться к этой иерархии.

10.22. (Адаптировано на основании примера Дуга Лената (Doug Lenat).) Ваша задача состоит в том, чтобы представить в логической форме достаточный объем знаний, позволяющий ответить на ряд вопросов о следующем простом высказывании:

Вчера Джон отправился в супермаркет North Berkeley Safeway и купил два фунта помидоров и фунт говяжьего фарша.

Начните с попытки представить содержимое этого высказывания в виде ряда утверждений. Вы должны записать высказывания, которые имеют простую логическую структуру (например, утверждения, согласно которым объекты имеют определенные свойства, связаны определенными отношениями, и все объекты, удовлетворяющие некоторому свойству, удовлетворяют также какому-то другому свойству). Для начала может помочь получение ответов на приведенные ниже вопросы. • Какие классы, объекты и отношения вам потребуются? Есть ли у них родительские объекты, сестринские объекты и т.д.? (Кроме всего прочего, вам может потребоваться учитывать упорядочение событий и временное упорядочение.) • В каком месте эти объекты должны войти в более общую иерархию? • Каковы ограничения и взаимосвязи между ними? • Насколько подробно следует определять каждое из различных понятий?

Созданная вами база знаний должна предоставлять возможность найти ответ на список вопросов, который будет вскоре приведен. Некоторые вопросы касаются материала, явно сформулированного в этом кратком рассказе, но для большинства из них требуется, чтобы тот, кто на них отвечает, обладал другими фоновыми знаниями — умел читать между строк. Вам придется вспомнить о том, какого рода товары бывают в супермаркете, что связано с покупкой выбранных товаров, для чего могут использоваться покупки, и т.д. Попытайтесь сделать применяемое вами представление настолько общим, насколько это возможно. Приведем тривиальный пример: не следует вводить в базу знаний высказывание: "Люди покупают продукты

в супермаркете Safeway", поскольку это не позволяет учесть существование тех, кто покупает продукты в другом супермаркете. Нет смысла вводить информацию о том, что "Джо готовит спагетти с помидорами и говяжьим фаршем", поскольку она вообще не позволит сделать какие-либо полезные выводы. Кроме того, не превращайте вопросы в ответы; например, в вопросе 10.22, в сказано: "Купил ли Джон какие-либо мясные продукты?", а не "Купил ли Джон фунт говяжьего фарша?"

Наметьте цепочки рассуждений, которые позволили бы ответить на эти вопросы.

В процессе этого вам несомненно потребуется создать дополнительные понятия, ввести дополнительные утверждения и т.д. Если это возможно, воспользуйтесь какой-то системой формирования логических рассуждений, чтобы продемонстрировать достаточность вашей базы знаний. Многие из составленных вами высказываний в реальности могут оказаться лишь приближенно правильными, но не следует об этом слишком беспокоиться; идея состоит в том, чтобы извлечь обыденные знания (на уровне здравого смысла), которые, в принципе, позволяют вам ответить на эти вопросы. Задача составления действительно полного ответа на многие из этих вопросов является чрезвычайно трудной и, по-видимому, выходит за рамки современных средств представления знаний. Но вы должны быть способны составить непротиворечивое множество аксиом для получения ответов на поставленный здесь ограниченный круг вопросов. а) Джон — ребенок или взрослый? [Взрослый] б) У Джона теперь имеется не меньше двух помидоров? [Да] в) Купил ли Джон какие-либо мясные продукты? [Да] г) Если бы Мэри покупала помидоры в то же время, что и Джон, увидел бы он ее? [Да] д) Сделаны ли помидоры в супермаркете? [Нет] е) Что Джон собирается сделать с помидорами? [Съесть их] ж) В супермаркете Safeway продаются дезодоранты? [Да] з) Джон принес какие-то деньги в супермаркет? [Да] и) Стал ли Джон иметь меньше денег после поездки в супермаркет? [Да] 10.23. Внесите необходимые дополнения или изменения в вашу базу знаний, разработанную в предыдущем упражнении, так, чтобы можно было отвечать на вопросы, которые следуют из этих знаний. Покажите, что на них действительно можно найти ответы с помощью этой базы знаний, и включите в ваш отчет описание исправлений, объяснение того, почему они потребовались, указание, являются ли эти исправления мелкими или крупными, и т.д.

а) Находились ли другие люди в супермаркете Safeway в то время, как там был Джон? [Да — работники супермаркета!] б) Является ли Джон вегетарианцем? [Нет] в) Кому принадлежит дезодорант в супермаркете Safeway? [Корпорации Safeway] г) Имеется ли у Джона унция говяжьего фарша? [Да] д) Имеется ли бензин в расположенной рядом бензозаправочной станции Shell? [Да] е) Имеются ли помидоры в багажнике автомобиля Джона? [Да] 10.24. Напомним, что информация о наследовании в семантических сетях может быть представлена логически с помощью подходящих импликационных высказываний. В данном упражнении рассматривается

эффективность использования подобных высказываний для представления информации о наследовании. а) Рассмотрим содержание каталога подержанных автомобилей, такого как Kelly's Blue Book, например, сведения о том, что микроавтобусы типа 1973 Dodge Van стоят 575 долларов. Предположим, что вся эта информация (об 11 тысячах моделей) закодирована в виде логических правил, как было предложено в данной главе. Запишите три таких правила, включая правило, касающееся микроавтобусов типа 1973 Dodge Van. Как вы сможете воспользоваться этими правилами, чтобы найти стоимость конкретного автомобиля (например, модели JB, которая относится к типу 1973 Dodge Van) при наличии такого средства автоматического доказательства теорем по методу обратного логического вывода, как Prolog? б) Сравните временную эффективность использования для решения этой задачи метода обратного логического вывода и метода наследования, используемого в семантических сетях. в) Объясните, как метод прямого логического вывода позволяет использовать систему на основе логики для эффективного решения той же задачи, при условии, что база знаний содержит только 11 тысяч правил, касающихся цен. г) Опишите ситуацию, в которой ни прямой, ни обратный логический вывод с помощью этих правил не позволяет эффективно выполнить запрос о цене на какой-то отдельный автомобиль. д) Можете ли вы предложить решение, позволяющее эффективно отвечать на запросы этого типа во всех вариантах логических систем? {Подсказка.

Напомним, что два автомобиля одной и той же категории имеют одинаковую цену.) 10.25. Можно было бы предположить, что не требуется подчеркивать в семантических сетях синтаксические различия между связями с метками, не заключенными в прямоугольники, и связями с метками в одинарном прямоугольнике, поскольку связи с метками в одинарном прямоугольнике всегда закреплены за категориями; а в алгоритме наследования, допустим, можно было бы просто принять допущение, что связь с меткой без прямоугольника, закрепленная за некоторой категорией, предназначена для применения ко всем элементам этой категории. Покажите, что такое утверждение является ошибочным, и приведите примеры ошибок, которые могут возникнуть из-за этого.

(empty page - no text in the layer)